



C S C I 8314

Spring 2023

SPARSE MATRIX COMPUTATIONS

***Class time* : MW 9:45 – 11:00am**
***Room* : Ackerman Hall 211**
***Instructor* : Yousef Saad**

August 27, 2023

About this class: Objectives

Set 1 An introduction to sparse matrices and sparse matrix computations.

- Sparse matrices;
- Sparse matrix direct methods ;
- Graph theory viewpoint; graph theory methods;

Set 2 Iterative methods and eigenvalue problems

- Iterative methods for linear systems
- Algorithms for sparse eigenvalue problems and the SVD
- Possibly: nonlinear equations

Set 3 Applications of sparse matrix techniques

- Applications of graphs; Graph Laplaceans; Networks ...;
- Standard Applications (PDEs, ..)
- Applications in machine learning
- Data-related applications
- Other instances of sparse matrix techniques

➤ Please fill out (now if you can)

This survey

Short link url:

<https://forms.gle/i5MCBg3X289JMAHd8>

Who is in this class today?

- Out of 20 [as of Tuesday] - registered
 - 5 in Computer Science
 - 5 in Aerospace Engineering
 - 2 Electrical Engineering
 - 2 Civil Engineering
 - 2 Chemical Engineering/ Materials Science
 - 2 Mathematics
 - 1 Statistics
 - 1 Industrial & Systems Eng.

Logistics:

- Lecture notes and minimal information will be located here:
[8314 at cselabs class web-sites](#)

URL:

<https://www-users.cselabs.umn.edu/classes/Spring-2023/csci8314>

[also follow: 'teaching' at www.cs.umn.edu: /saad]

- There you will find :
 - Lecture notes, Schedule of assignments/ tests, class info
- Canvas will contain the rest of the information: assignments, grades, etc.

About lecture notes:

- Lecture notes (like this first set) will be posted on the class web-site – usually before the lecture.
- Review them to get some understanding if possible before class.
- Read the relevant section (s) in the texts or provided references
- Lecture note sets are grouped by topics rather than by lecture.
- In the notes the symbol  indicates suggested easy exercises or questions – often [not always] done in class.
- Also: occasional practice exercises posted

Matlab, Python-Numpy, etc..

- Important to use either Matlab (mostly) or Python to quickly illustrate and test algorithms.
- Scripts in either matlab or python will be posted in the 'matlab' section of the class web-site.
- Also: matlab or python demos seen in class will be posted

Roadmap – [subject to itinerary change!]

Part 1

1. Sparse matrices;
2. Graph representations;
3. Sparse direct methods for linear systems;

Part 2

4. Iterative methods for linear systems ;
5. Projection methods and Krylov subspace methods;
6. Eigenvalue problems;

Part 3

7. Back to Graphs; Paths in graphs; Markov Chains;
8. Graph centrality;
9. Graph Laplaceans and applications; Clustering;
10. Graph embeddings.

CSCI 8314: SPARSE MATRIX COMPUTATIONS

GENERAL INTRODUCTION

- *General introduction - a little history*
- *Motivation*
- *Resources*
- *What will this course cover*
- *Examples of problems leading to sparse matrix computations*

Historical Perspective: Focus of numerical linear algebra

➤ Linear algebra took many direction changes in the past

1940s–1950s: Major issue: flutter problem in aerospace engineering
→ eigenvalue problem [cf. Olga Taussky Todd] → LR, QR, .. → ‘EISPACK’

1960s: Problems related to the power grid promoted what we would call today general sparse matrix techniques

1970s– Automotive, Aerospace, ...: Computational Fluid Dynamics (CFD)

Late 1980s: Thrust on parallel matrix computations.

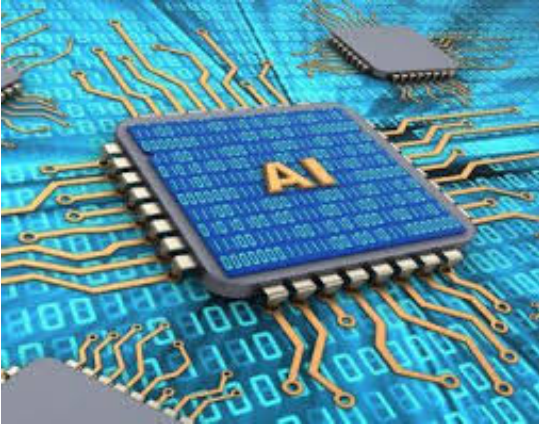
Late 1990s: Spur of interest in “financial computing”

Current: Machine Learning

Solution of PDEs (e.g., Fluid Dynamics) and problems in mechanical eng. (e.g. structures) major force behind numerical linear algebra algorithms in the past few decades.

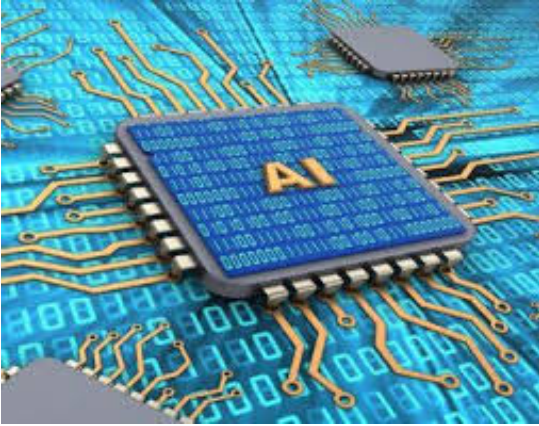
- Strong new forces are now reshaping the field today: Applications related to the use of “data”
- Machine learning is appearing in unexpected places:
 - design of materials
 - machine learning in geophysics
 - self-driving cars, ..
 -

Big impact on the economy



- New economy driven by Google, Facebook, Netflix, Amazon, Twitter, Ali-Baba, Tencent, ..., and even the big department stores (Walmart, ...)
- Huge impact on **Jobs**

Big impact on the economy



- New economy driven by Google, Facebook, Netflix, Amazon, Twitter, Ali-Baba, Tencent, ..., and even the big department stores (Walmart, ...)
- Huge impact on **Jobs**

- Old leaders - e.g., Mining; Car companies; Aerospace; Manufacturing; offer little growth
 - Some instances of renewal driven by new technologies [e.g. Tesla]



- Look at what you are doing under new lenses: DATA

Sparse matrices: a brief history

Sparse matrices have been identified as important early on – terminology is quite old. Gauss defined the first method for such systems in 1823. Varga used explicitly the term 'sparse' in his 1962 book on iterative methods.

<https://www-users.cs.umn.edu/~saad/PDF/icerm2018.pdf>

- Special techniques used for sparse problems coming from Partial Differential Equations
- One has to wait until to the 1960s to see the birth of the general methodology available today
- Graphs introduced as tools for sparse Gaussian elimination in 1961 [Seymour Parter]

- Early work on reordering for banded systems, envelope methods
- Various reordering techniques for general sparse matrices introduced.
- Minimal degree ordering [Markowitz - 1957] ...
- ... later used in Harwell MA28 code [Duff] - released in 1977.
- Tinney-Walker Minimal degree ordering for power systems [1967]
- Nested Dissection [A. George, 1973]
- SPARSPAK [commercial code, Univ. Waterloo]
- Elimination trees, symbolic factorization, ...

History: development of iterative methods

- 1950s up to 1970s : focus on “relaxation” methods
- Development of ‘modern’ iterative methods took off in the mid-70s. but...
- ... The main ingredients were in place earlier [late 40s, early 50s: Lanczos; Arnoldi ; Hestenes (a local!) and Stiefel;]
- The next big advance was the push of ‘preconditioning’: in effect a way of combining iterative and (approximate) direct methods – [Meijerink and Van der Vorst, 1977]

<https://www-users.cs.umn.edu/~saad/PDF/NIST75th.pdf>

History: eigenvalue problems

- Another parallel branch was followed in sparse techniques for large eigenvalue problems.
- A big problem in 1950s and 1960s : flutter of airplane wings.. This leads to a large (sparse) eigenvalue problem
- Overlap between methods for linear systems and eigenvalue problems [Lanczos, Arnoldi]

Resources

- SuiteSparse site (Formerly : Florida collection)

<https://sparse.tamu.edu/>

- SPARSKIT, etc. [SPARSKIT = old written in Fortran. + more recent 'solvers']

<http://www.cs.umn.edu/~saad/software>

Resources – continued

Books: on sparse direct methods.

- Book by Tim Davis [SIAM, 2006] see syllabus for info
- An old reference [Still a great book]: Alan George and Joseph W-H Liu, **Computer Solution of Large Sparse Positive Definite Systems**, Prentice-Hall, 1981.
- Of interest, mostly for reference:
 - I. S. Duff and A. M. Erisman and J. K. Reid, **Direct Methods for Sparse Matrices**, Clarendon press, Oxford, 1986; 2nd edition: 2017.
 - Some coverage in Golub and van Loan [John Hopinks, 4th Ed., Chap. 10 to end]

BACKGROUND: PROBLEMS LEADING TO SPARSE MATRICES

Background: Examples leading to sparse matrices

- The classical: CFD, electrical networks,
- ... and the modern:
 - Graph algorithms and tools (Sparse graphs, graph coarsening, graphs and sparse methods). ..
 - Dimension reduction methods; Graph embeddings;
 - Specific machine learning algorithms; unsupervised/ supervised learning;
 - Deep learning;
 - Network analysis
 - ...

Example: Fluid flow

Physical Model



Nonlinear PDEs



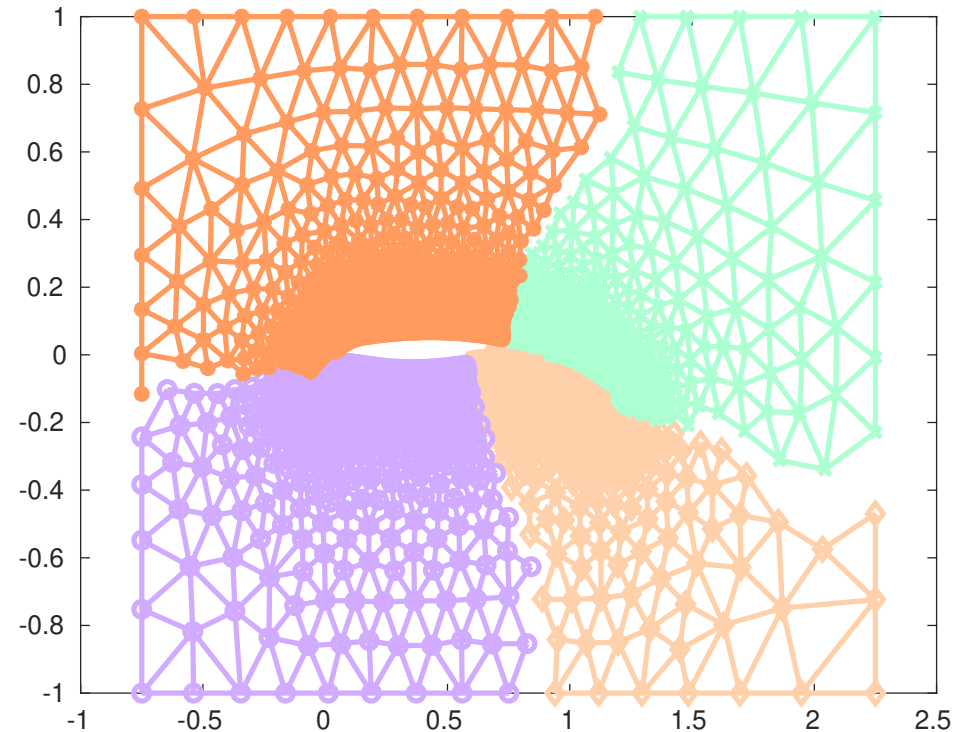
Discretization



Linearization (Newton)

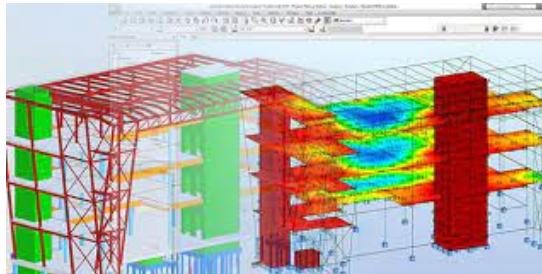
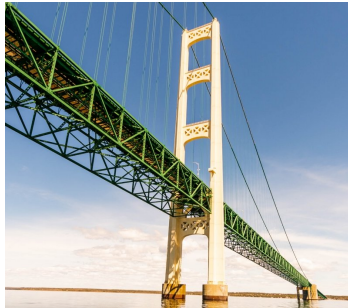
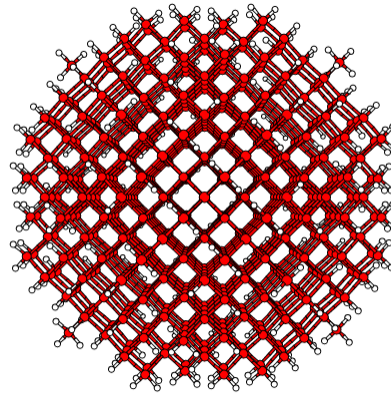
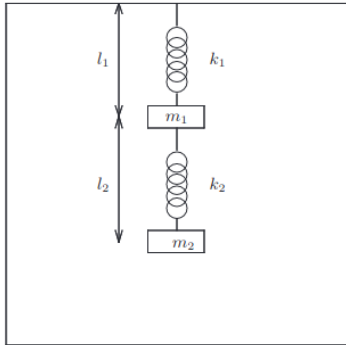


Sparse Linear Systems $Ax = b$



Example: Eigenvalue Problems

- Many applications require the computation of a few eigenvalues + associated eigenvectors of a matrix A

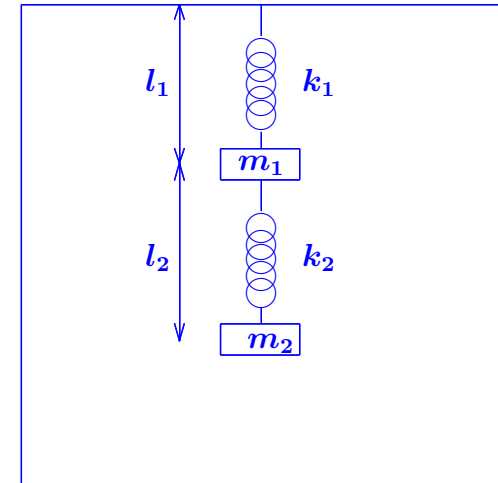


- Structural Engineering – (Goal: frequency response)
- Electronic structure calculations [Schrödinger equation..] – Quantum chemistry
- Stability analysis [e.g., electrical networks, mechanical system,..]
- ...

Example: Vibrations

- Vibrations in mechanical systems. See: www.cs.umn.edu/~saad/eig_book_2ndEd.pdf

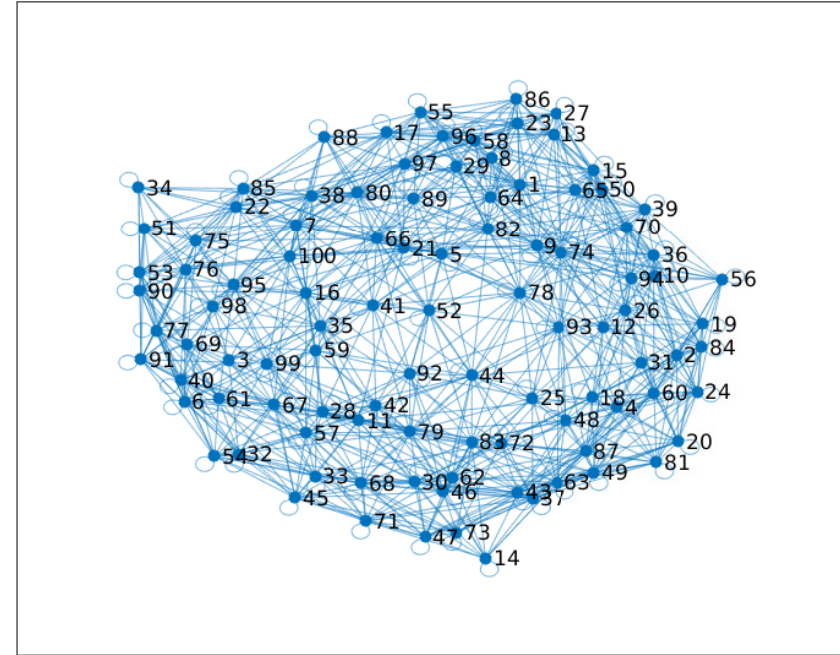
Problem: Determine the vibration modes of the mechanical system [to avoid resonance]. See details in Chapter 10 (sec. 10.2) of above reference.



- Problem type: Eigenvalue Problem

Example: Google Rank (pagerank)

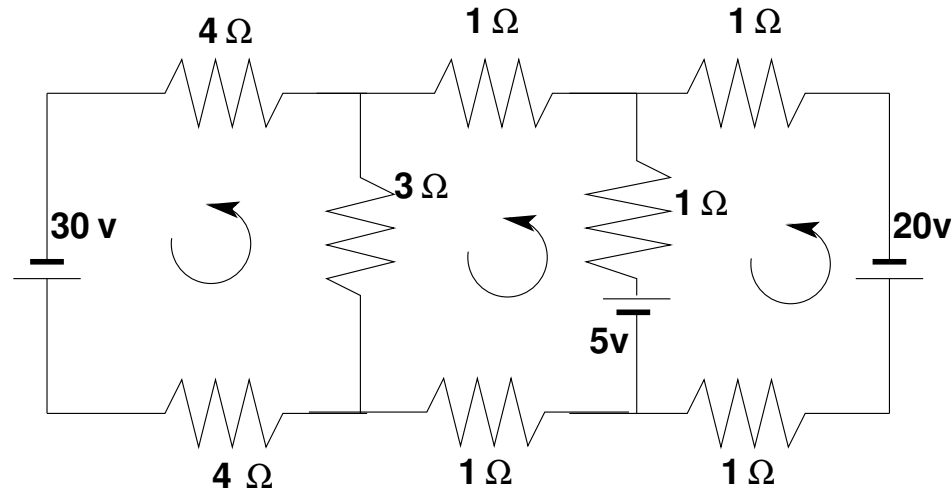
If one were to do a random walk from web page to web page, following each link on a given web page at random with equal likelihood, which are the pages to be encountered this way most often?



➤ Problem type: (homogeneous) Linear system. Eigenvector problem.

Example: Power networks

- Electrical circuits .. [Kirchhoff's voltage Law]



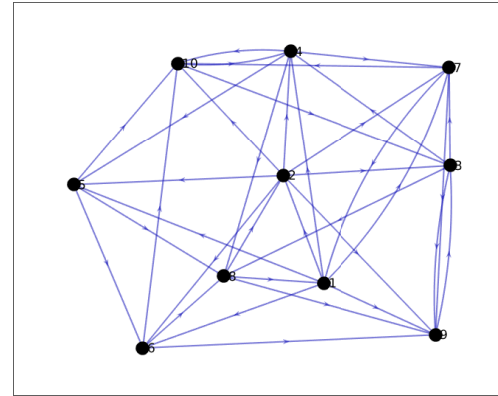
Problem: Determine the loop currents in a an electrical circuit - using Kirchhoff's Law ($V = RI$)

- Problem: Sparse Linear Systems [at the origin Sparse Direct Methods]

Example: Economics/ Marketing/ Social Networks

- Given: an influence graph G : g_{ij} = strength of influence of j over i
- Goal: charge member i price p_i in order to maximize profit
- Utility for member i : [x_i = consumption of i]

$$u_i = ax_i - bx_i^2 + \sum_{j \neq i} g_{ij}x_j - p_i x_i$$



- 1: ‘Monopolist’ fixes prices; 2: agent i fixes consumption x_i

Result: Optimal pricing proportional to **Bonacich** centrality:

$(I - \alpha G)^{-1} \mathbb{1}$ where $\alpha = \frac{1}{2b}$ [Candogan et al., 2012 + many refs.]

- 'centrality' defines a measure of importance of a node (or an edge) in a graph
- Many other ideas of centrality in graphs [degree centrality, betweenness centrality, closeness centrality, ...]
- Important application: Social Network Analysis

Example: Method of least-squares

- First use of least squares by Gauss, in early 1800's:

A planet follows an elliptical orbit according to $ay^2 + bxy + cx + dy + e = x^2$ in cartesian coordinates. Given a set of noisy observations of (x, y) positions, compute a, b, c, d, e , and use to predict future positions of the planet. This least squares problem is nearly rank-deficient and hence very sensitive to perturbations in the observations.

- Problem type: Least-Squares system

Read Wikipedia's article on planet ceres:

[http://en.wikipedia.org/wiki/Ceres_\(dwarf_planet\)](http://en.wikipedia.org/wiki/Ceres_(dwarf_planet))

Example: Dynamical systems and epidemiology

A set of variables that fill a vector y are governed by the equation

$$\frac{dy}{dt} = Ay$$

Determine $y(t)$ for $t > 0$, given $y(0)$ [called ‘orbit’ of y]

➤ Problem type: (Linear) system of ordinary differential equations.

Solution:

$$y(t) = e^{tA}y(0)$$

➤ Involves exponential of A [think Taylor series], i.e., a **matrix function**

- This is the simplest form of dynamical systems (linear).
- Consider the slightly more complex system:

$$\frac{dy}{dt} = A(y)y$$

- Nonlinear. Requires 'integration scheme'.

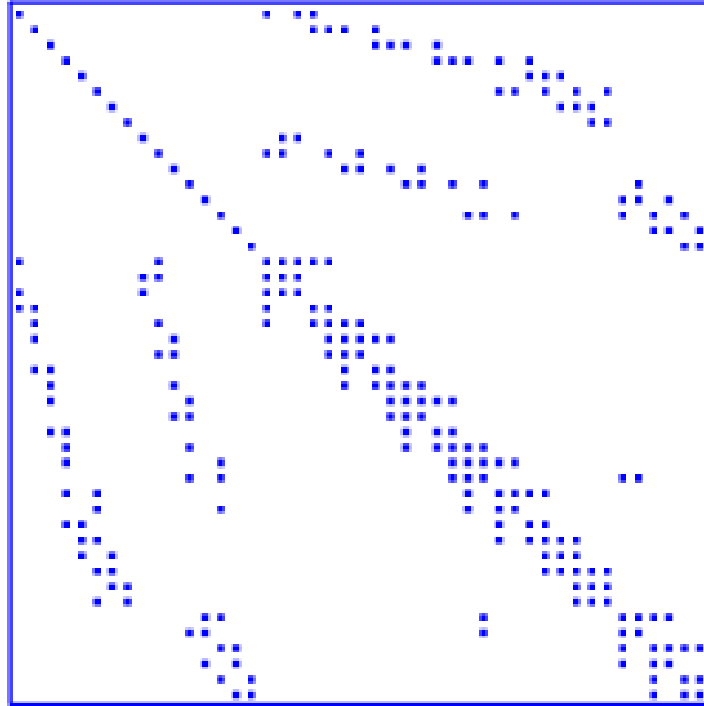
General Problems in Numerical Linear Algebra (dense & sparse)

- Linear systems: $Ax = b$. Often: A is large and sparse
- Least-squares problems $\min \|b - Ax\|_2$
- Eigenvalue problem $Ax = \lambda x$. Several variations -
- SVD .. and
- ... Low-rank approximation
- Tensors and low-rank tensor approximation
- Matrix equations: Sylvester, Lyapunov, Riccati, ..
- Nonlinear equations – acceleration methods
- Matrix functions and applications
- Many many more ...

SPARSE MATRICES

- *Introduction to sparse matrices*
- *See Chap. 3 of text*
- *See also the various sparse matrix sites*
- *Sparse matrices in matlab*

What are sparse matrices?



Pattern of a small sparse matrix

- Vague definition: matrix with few nonzero entries
- For all practical purposes: an $m \times n$ matrix is sparse if it has $O(\min(m, n))$ nonzero entries.
- This means roughly a constant number of nonzero entries per row and column -
- This definition excludes a large class of matrices that have $O(\log(n))$ nonzero entries per row.
- Other definitions use a slow growth of nonzero entries with respect to n or m .

“..matrices that allow special techniques to take advantage of the large number of zero elements.” (J. Wilkinson)

A few applications which lead to sparse matrices:

Structural Engineering, Computational Fluid Dynamics, Reservoir simulation, Electrical Networks, optimization, Google Page rank, information retrieval (LSI), circuit simulation, device simulation,

Goal of Sparse Matrix Techniques

- To perform standard matrix computations economically i.e., without storing the zeros of the matrix.

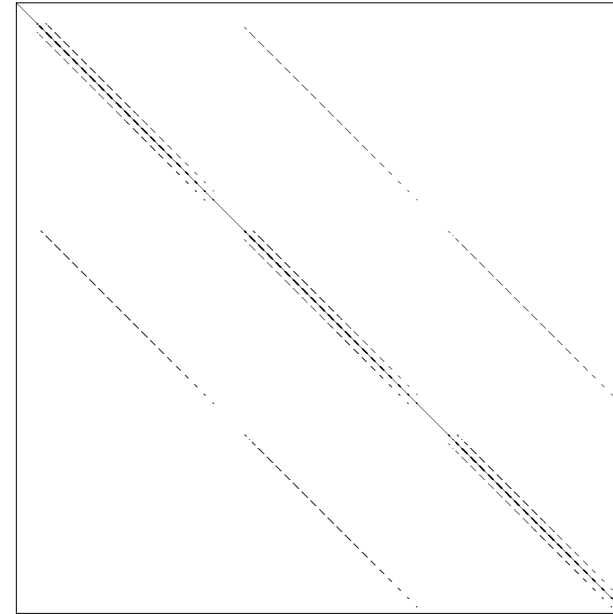
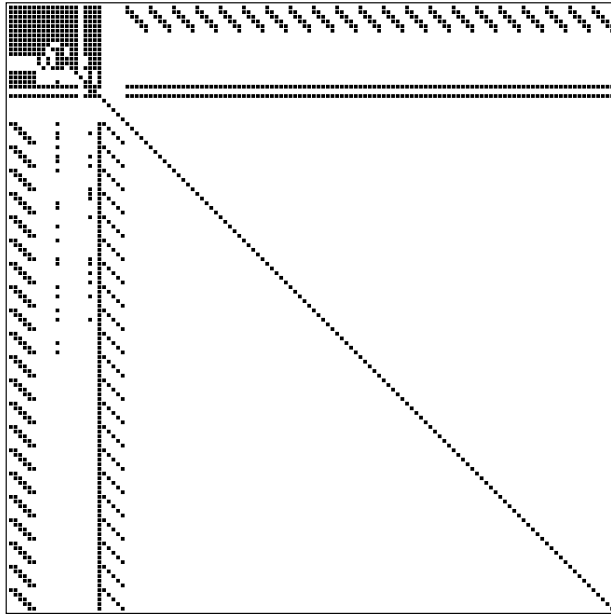
Example: To add two square dense matrices of size n requires $O(n^2)$ operations. To add two sparse matrices A and B requires $O(nnz(A) + nnz(B))$ where $nnz(X) =$ number of nonzero elements of a matrix X .

- For typical Finite Element /Finite difference matrices, number of nonzero elements is $O(n)$.

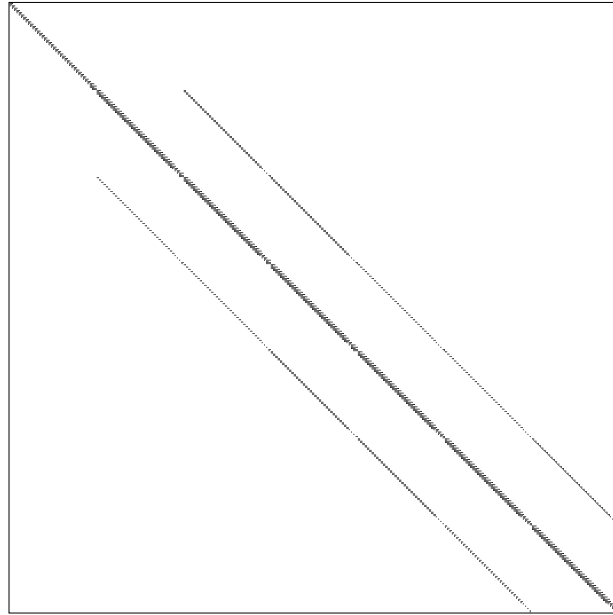
Remark: A^{-1} is usually dense, but L and U in the LU factorization may be reasonably sparse (if a good technique is used)

-  2 Look up Cayley-Hamilton's theorem if you do not know about it.
-  3 Show that the inverse of a matrix (when it exists) can be expressed as a polynomial of A , where the polynomial is of degree $\leq n - 1$.
-  4 When is the degree $< n - 1$? [Hint: look-up minimal polynomial of a matrix]
-  5 What is the pattern of the inverse of a tridiagonal matrix? a bidiagonal matrix?

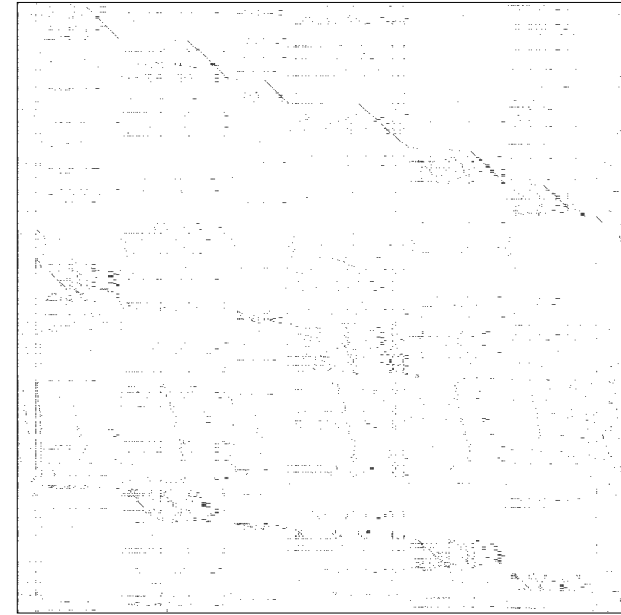
Nonzero patterns of a few sparse matrices



ARC130: Unsymmetric matrix from laser problem. a.r.curtis, oct 1974 SHERMAN5: fully implicit black oil simulator 16 by 23 by 3 grid, 3 unk



PORES3: Unsymmetric MATRIX FROM PORES

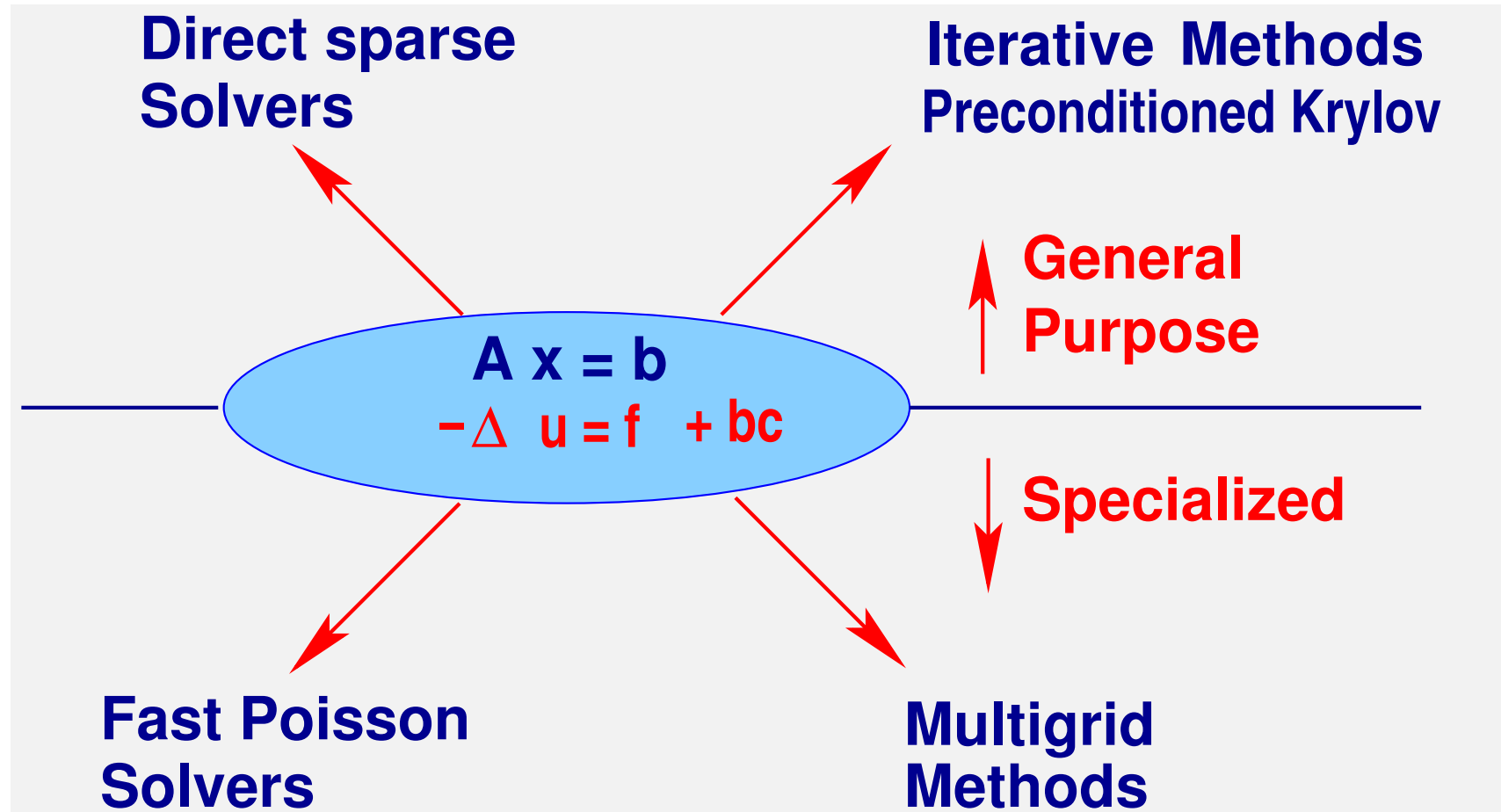


BP_1000: UNSYMMETRIC BASIS FROM LP PROBLEM BP

Types of sparse matrices

- Two types of matrices: structured (e.g. Sherman5) and **unstructured** (e.g. BP_1000)
- The matrices PORES3 and SHERMAN5 are from Oil Reservoir Simulation. Often: 3 unknowns per mesh point (Oil , Water saturations, Pressure). Structured matrices.
- 40 years ago reservoir simulators used rectangular grids.
- Modern simulators: Finer, more complex physics ➤ harder and larger systems. Also: unstructured matrices
- A naive but representative challenge problem: $100 \times 100 \times 100$ grid + about 10 unknowns per grid point ➤ $N \approx 10^7$, and $nnz \approx 7 \times 10^8$.

Solving sparse linear systems: existing methods



Two types of methods for general systems:

- Direct methods : based on sparse Gaussian elimination, sparse Cholesky,...
- Iterative methods: compute a sequence of iterates which converge to the solution - preconditioned Krylov methods..

Remark:

These two classes of methods have always been in competition.

- 40 years ago solving a system with $n = 10,000$ was a challenge
- Now you can solve this in a fraction of a second on a laptop.

- Sparse direct methods made huge gains in efficiency. As a result they are very competitive for 2-D problems.
- 3-D problems lead to more challenging systems [inherent to the underlying graph]

Difficulty:

- No robust 'black-box' iterative solvers.
 - At issue: Robustness in conflict with efficiency.
- Iterative methods are starting to use some of the tools of direct solvers to gain 'robustness'

Consensus:

1. Direct solvers are often preferred for two-dimensional problems (robust and not too expensive).
2. Direct methods loose ground to iterative techniques for three-dimensional problems, and problems with a large degree of freedom per grid point,

Sparse matrices in matlab

- Matlab supports sparse matrices to some extent.
- Can define sparse objects by conversion

```
A = sparse(X) ; X = full(A)
```

- Can show pattern

```
spy(X)
```

- Define the analogues of `ones`, `eye`:

```
speye(n,m) ,      spones(pattern)
```

- A few reorderings functions provided.. [will be studied in detail later]

```
symrcm, symamd, colamd, colperm
```

- Random sparse matrix generator:

```
sprand(S) or sprand(m,n, density)
```

(also `texttsprandn(...)`)

- Diagonal extractor-generator utility:

```
spdiags(A) , spdiags(B,d,m,n)
```

- Other important functions:

```
spalloc(..) , find(..)
```


Graph Representations of Sparse Matrices

- Graph theory is a fundamental tool in sparse matrix techniques.

DEFINITION. A graph G is defined as a pair of sets $G = (V, E)$ with $E \subset V \times V$. So G represents a binary relation. The graph is **undirected** if the binary relation is reflexive. It is **directed** otherwise. V is the vertex set and E is the edge set.

Example: Given the numbers 5, 3, 9, 15, 16, show the two graphs representing the relations

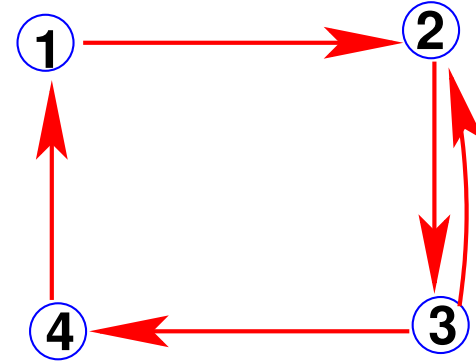
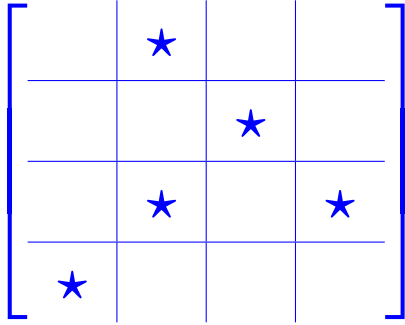
R1: Either $x < y$ or y divides x .

R2: x and y are congruent modulo 3. [$\text{mod}(x,3) = \text{mod}(y,3)$]

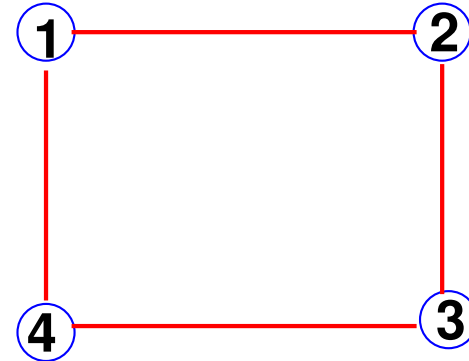
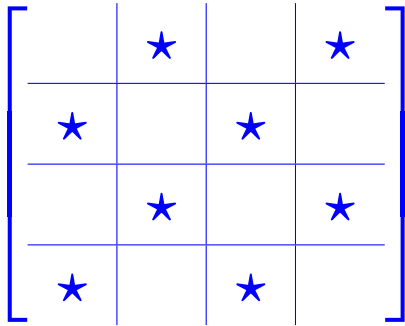
- Adjacency Graph $G = (V, E)$ of an $n \times n$ matrix A :
- Vertices $V = \{1, 2, \dots, n\}$.
 - Edges $E = \{(i, j) | a_{ij} \neq 0\}$.
- Often self-loops (i, i) are not represented [because they are always there]
- Graph is **undirected** if the matrix has a symmetric structure:

$$a_{ij} \neq 0 \quad \text{iff} \quad a_{ji} \neq 0.$$

Example: (directed graph)



Example: (undirected graph)



 6 Graph of a tridiagonal matrix? Of a dense matrix?

 7 Adjacency graph of:

$$A = \begin{bmatrix} \star & \star & & & \star & \\ \star & \star & \star & & & \star \\ & \star & \star & & & \\ & & & \star & \star & \\ \star & & & \star & \star & \star \\ & \star & & & \star & \star \end{bmatrix} ?$$

 8 Recall what a star graph is. Show a matrix whose graph is a star graph. Consider two situations: Case when center node is labeled first and case when it is labeled last.

- Note: Matlab now has a `graph` function.
- `G = graph(A)` creates adjacency graph from A
- G is a matlab class/
- `G.Nodes` will show the vertices of G
- `G.Edges` will show its edges.
- `plot(G)` will show a representation of the graph



Do the following:

- Load the matrix 'Bmat.mat' located in the class web-site (see 'matlab' folder)
- Visualize pattern (`spy(B)`) + find: Number of nonzero elements, size, ...
- Generate graph - without self-edges:

```
G = graph(B, 'OmitSelfLoops')
```

- Plot the graph –
- \$1M question: Any idea on how this plot is generated?

DISCRETIZATION OF PARTIAL DIFFERENTIAL EQUATIONS

Goal: to show how partial differential lead to sparse linear systems

- *See Chap. 2 of text*
- *Finite difference methods*
- *Finite elements*
- *Assembled and unassembled finite element matrices*

Why study discretized PDEs?

- One of the most important sources of sparse linear systems
- Will help understand the structures of the problem and their connections with “meshes” in 2-D or 3-D space
- Also: iterative methods are often formulated for the PDE directly – instead of a discretized (sparse) system.

NOTE: Useful to have an idea of how Finite Difference matrices are generated. For Finite Elements: goal is to unravel the related sparse computations to which they lead.

A typical numerical simulation

Physical Problem →

Nonlinear PDEs →

Discretization →

Linearization (Newton) →

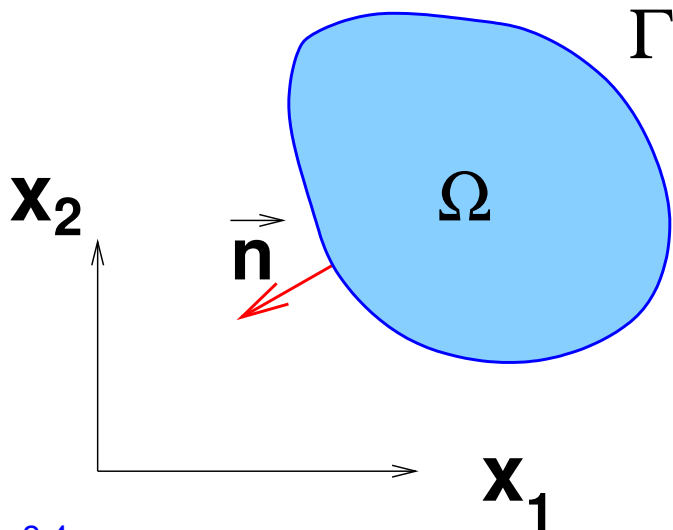
Sequence of Sparse Linear Systems $Ax = b$

Example: discretized Poisson equation

➤ Common Partial Differential Equation (PDE) :

$$\frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2} = f, \text{ for } x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \text{ in } \Omega$$

where Ω = bounded, open domain in $\mathbb{R}^2$



➤ + boundary conditions:

Dirichlet: $u(x) = \phi(x)$

Neumann: $\frac{\partial u}{\partial \vec{n}}(x) = 0$

Cauchy: $\frac{\partial u}{\partial \vec{n}} + \alpha(x)u = \gamma$

- $\Delta = \frac{\partial^2}{\partial x_1^2} + \frac{\partial^2}{\partial x_2^2}$ is the **Laplace** operator or **Laplacean**
- How to approximate the Poisson problem shown above?
- Answer: discretize, i.e., replace continuum with discrete set.
- Then approximate Laplacean using this discretization
- Many types of discretizations.. will briefly cover Finite Differences (FD) and Finite Elements (FEM)

Finite Differences: Basic approximations

- Formulas derived from Taylor series expansion:

$$u(x + h) = u(x) + h \frac{du}{dx} + \frac{h^2}{2} \frac{d^2u}{dx^2} + \frac{h^3}{6} \frac{d^3u}{dx^3} + \frac{h^4}{24} \frac{d^4u}{dx^4}(\xi)$$

Discretization of PDEs - Basic approximations

- Simplest scheme: forward difference

$$\begin{aligned}\frac{du}{dx} &= \frac{u(x+h) - u(x)}{h} - \frac{h}{2} \frac{d^2u(x)}{dx^2} + O(h^2) \\ &\approx \frac{u(x+h) - u(x)}{h}\end{aligned}$$

- Centered differences for second derivative:

$$\frac{d^2u(x)}{dx^2} = \frac{u(x+h) - 2u(x) + u(x-h))}{h^2} - \frac{h^2}{12} \frac{d^4u(\xi)}{dx^4},$$

where $\xi_- \leq \xi \leq \xi_+$.

Notation:

$$\delta^+ u(x) = u(x+h) - u(x)$$

$$\delta^- u(x) = u(x) - u(x-h)$$

➤ Operations of the type: $\frac{d}{dx} \left[a(x) \frac{d}{dx} \right]$ are common [in-homogeneous media].

➤ The following is a second order approximation:

$$\begin{aligned} \frac{d}{dx} \left[a(x) \frac{du}{dx} \right] &= \frac{1}{h^2} \delta^+ \left(a_{i-\frac{1}{2}} \delta^- u \right) + O(h^2) \\ &\approx \frac{a_{i+\frac{1}{2}}(u_{i+1} - u_i) - a_{i-\frac{1}{2}}(u_i - u_{i-1})}{h^2} \end{aligned}$$

 1 Show that $\delta^+ \left(a_{i-\frac{1}{2}} \delta^- u \right) = \delta^- \left(a_{i+\frac{1}{2}} \delta^+ u \right)$

Finite Differences for 2-D Problems

Consider the simple problem,

$$-\left(\frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2}\right) = f \quad \text{in } \Omega \quad (1)$$

$$u = 0 \quad \text{on } \Gamma \quad (2)$$

$\Omega = \text{rectangle } (0, l_1) \times (0, l_2)$ and Γ its boundary.

Discretize uniformly :

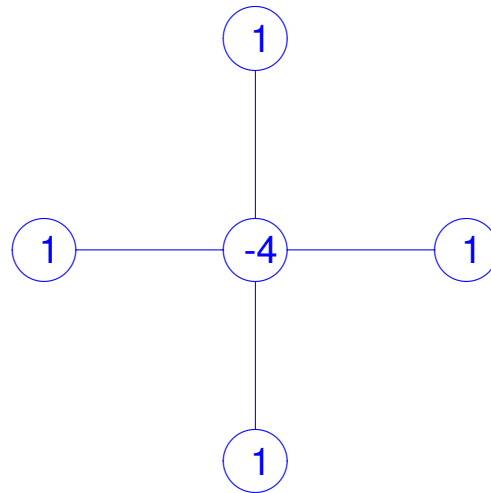
$$\begin{aligned} x_{1,i} &= i \times h_1 \quad i = 0, \dots, n_1 + 1 & h_1 &= \frac{l_1}{n_1 + 1} \\ x_{2,j} &= j \times h_2 \quad j = 0, \dots, n_2 + 1 & h_2 &= \frac{l_2}{n_2 + 1} \end{aligned}$$

Finite Difference Scheme for the Laplacean

- Use centered differences for $\frac{\partial^2}{\partial x_1^2}$ and $\frac{\partial^2}{\partial x_2^2}$ - with mesh sizes $h_1 = h_2 = h$:

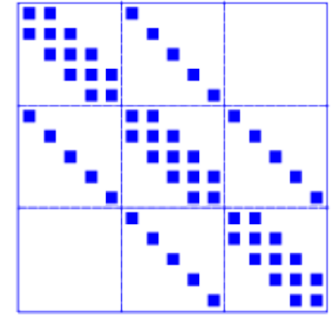
$$\Delta u(x) \approx \frac{1}{h^2} [u(x_1 + h, x_2) + u(x_1 - h, x_2) + u(x_1, x_2 + h) + u(x_1, x_2 - h) - 4u(x_1, x_2)]$$

The 5-point 'stencil:'



The resulting matrix has the following block structure:

$$A = \frac{1}{h^2} \begin{bmatrix} B & -I & \\ -I & B & -I \\ & -I & B \end{bmatrix} \rightarrow$$



Case: 7×5 grid

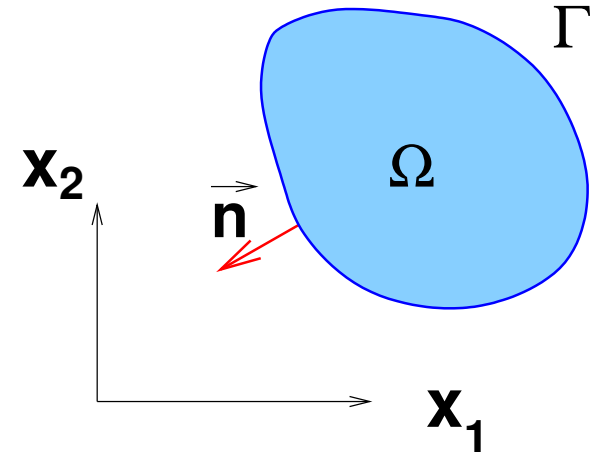
With

$$B = \begin{bmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & -1 & 4 & -1 & \\ & & -1 & 4 & -1 \\ & & & -1 & 4 \end{bmatrix}.$$

Finite Element Method (FEM): a quick overview

Background: Green's formula

$$\int_{\Omega} \nabla v \cdot \nabla u \, dx = - \int_{\Omega} v \Delta u \, dx + \int_{\Gamma} v \frac{\partial u}{\partial \vec{n}} \, ds.$$



➤ The dot indicates a dot product of two vectors.

➤ ∇ = gradient operator. In 2-D: $\nabla u = \begin{pmatrix} \frac{\partial u}{\partial x_1} \\ \frac{\partial u}{\partial x_2} \end{pmatrix},$

➤ Δu = Laplacean of u

➤ $\vec{n}$ is the unit vector that is normal to Γ and directed outwards.

➤ Frechet derivative:

$$\frac{\partial u}{\partial \vec{v}}(x) = \lim_{h \rightarrow 0} \frac{u(x + h\vec{v}) - u(x)}{h}$$

➤ Green's formula generalizes the usual formula for integration by parts

➤ Define

$$a(u, v) \equiv \int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} \left(\frac{\partial u}{\partial x_1} \frac{\partial v}{\partial x_1} + \frac{\partial u}{\partial x_2} \frac{\partial v}{\partial x_2} \right) dx$$
$$(f, v) \equiv \int_{\Omega} f v \, dx.$$

➤ With Dirichlet BC, the integral on the boundary in Green's formula vanishes →

$$a(u, v) = -(\Delta u, v).$$

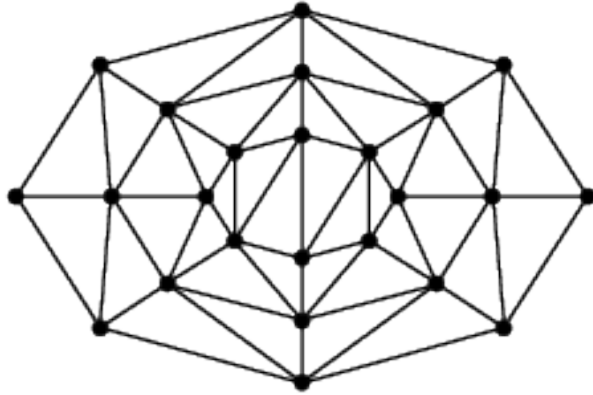
- Suppose we want to solve $-\Delta u = f$ in Ω + Dirichlet BC
- Weak formulation of the original problem: select a subspace of reference V of L^2 and then solve

$$\text{Find } u \in V \text{ such that } \underbrace{a(u, v)}_{=-(\Delta u, v)} = (f, v), \quad \forall v \in V$$

- Finite Element method solves this weak problem...
- ... by discretization

- The original domain is approximated by the union Ω_h of m triangles K_i ,

Triangulation of Ω :



$$\Omega_h = \bigcup_{i=1}^m K_i.$$

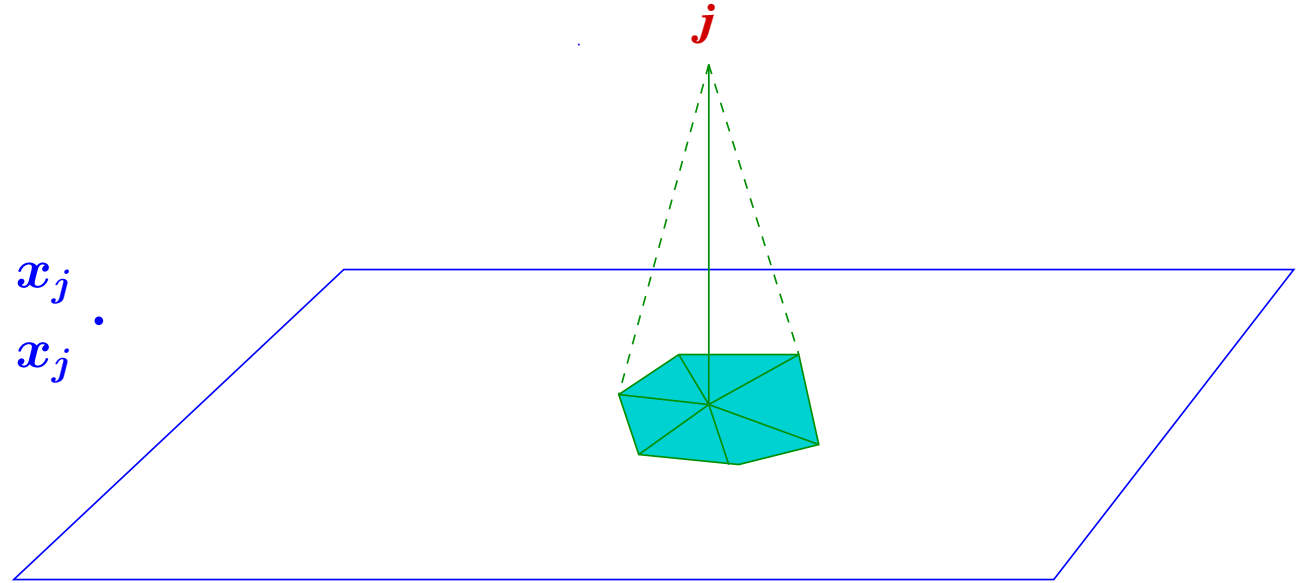
- Some restrictions on angles, edges, etc..

$$V_h = \{ \phi \mid \phi|_{\Omega_h} \in \mathcal{C}^0, \quad \phi|_{\Gamma_h} = 0, \quad \phi|_{K_j} \text{ linear } \forall j \}$$

- $\mathcal{C}^0$ = set of *continuous* functions
- $\phi|_X$ == restriction of ϕ to the subset X
- Let $x_j, j = 1, \dots, n$, be the nodes of the triangulation

- Can define a (unique) 'hat' function ϕ_j in V_h associated with each x_j s.t.:

$$\phi_j(x_i) = \delta_{ij} = \begin{cases} 1 & \text{if } x_i = x_j \\ 0 & \text{if } x_i \neq x_j \end{cases}.$$



- Each function u of V_h can be expressed as

$$u(x) = \sum_{j=1}^n \xi_j \phi_j(x). \quad (*)$$

- FEM approximation $\equiv$ Galerkin condition for functions in V_h :

Find $u \in V_h$ such that $a(u, v) = (f, v), \quad \forall v \in V_h$

- Express u in the basis $\{\phi_j\}$ (see *), then substitute above. Result:

- Linear system $\sum_{j=1}^n \alpha_{ij} \xi_j = \beta_i$ where: $\alpha_{ij} = a(\phi_j, \phi_i), \quad \beta_i = (f, \phi_i).$

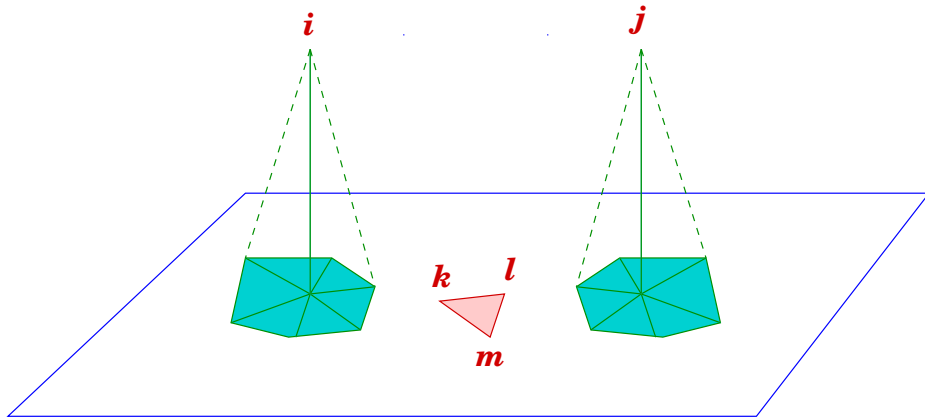
The above equations form a linear system of equations

$$Ax = b$$

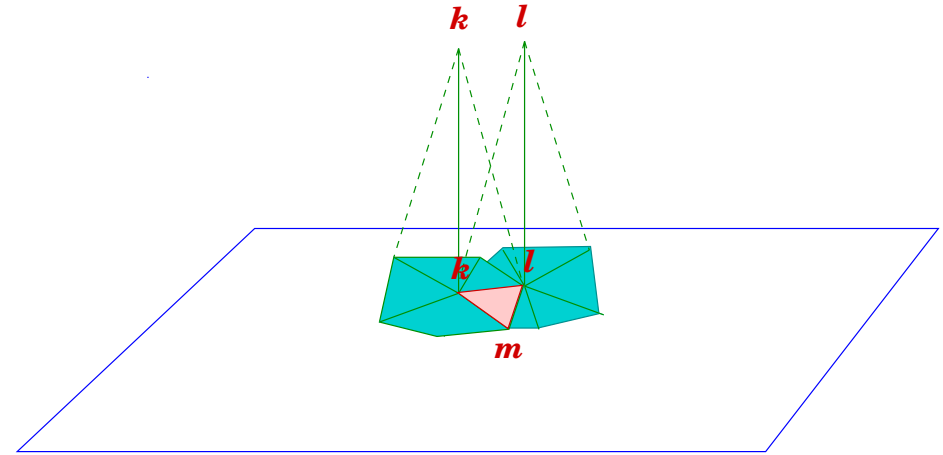
- A is *Symmetric Positive Definite*

 2 Prove it

The Assembly Process: Illustration



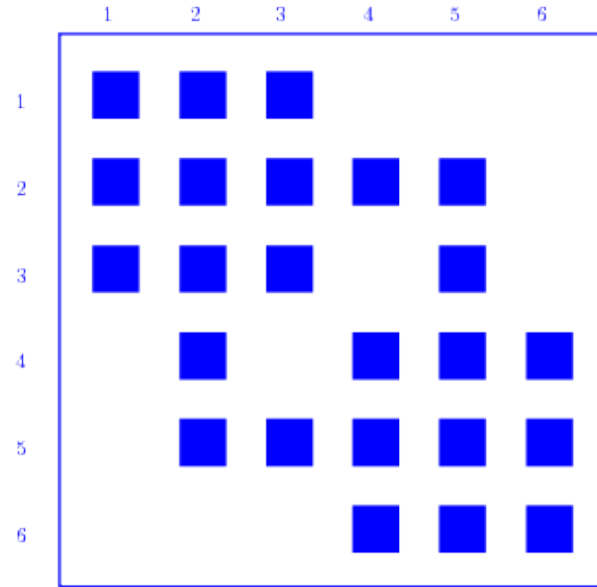
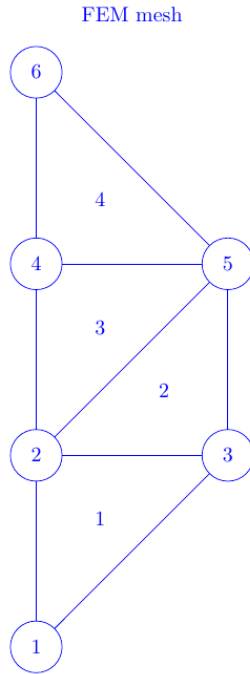
If triangle $K \notin$ support domains of both ϕ_i and ϕ_j then $a_K(\phi_i, \phi_j) = 0$



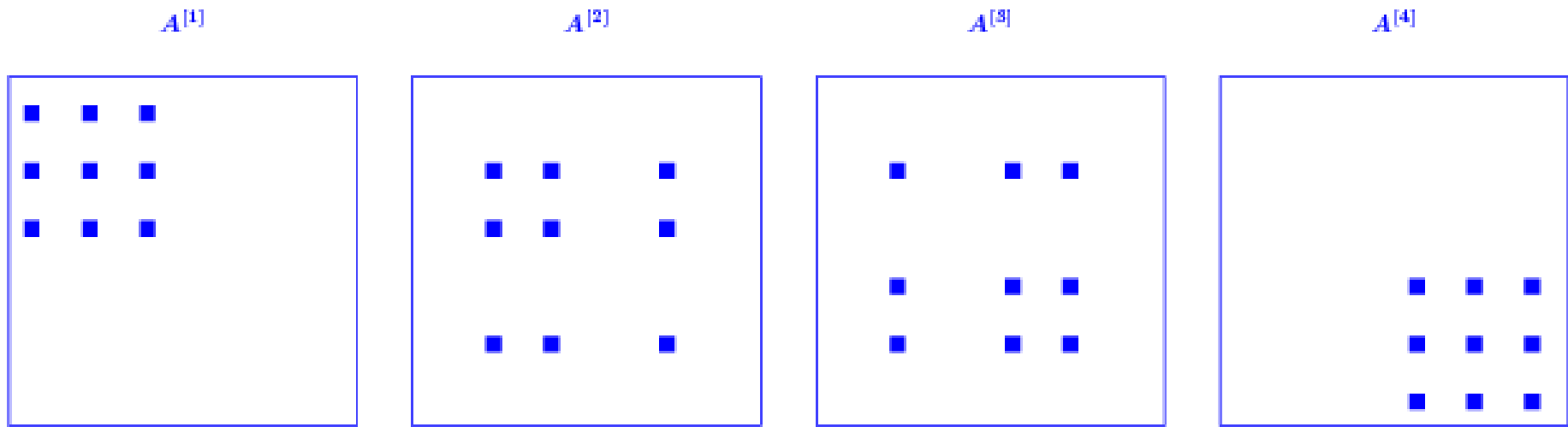
If triangle $K \in$ *both* nonzero domains of ϕ_i and ϕ_j then $a_K(\phi_i, \phi_j) \neq 0$

► So: $a_K(\phi_i, \phi_j) \neq 0$ iff $i \in \{k, l, m\}$ and $j \in \{k, l, m\}$.

The Assembly Process



Small finite element mesh and pattern of the corresponding assembled matrix



Element matrices $A^{[e]}$, $e = 1, \dots, 4$ for FEM mesh shown above

- Each element contributes a 3×3 submatrix $A^{[e]}$ (spread out)
- Can also use the matrix in un-assembled form - To multiply a vector by A for example we can do:

$$y = Ax = \sum_{e=1}^{nel} A^{[e]}x = \sum_{e=1}^{nel} P_e A_{K_e} (P_e^T x)$$

- Can be computed using the element matrices A_{K_e} - no need to assemble
- The product $P_e^T x$ gathers x data associated with the e -element into a 3-vector consistent with the ordering of the matrix A_{K_e} .
- Advantage: some simplification in process
- Disadvantage: cost (memory + computations).

Resources: A few matlab scripts

- These (and others) will be posted in the matlab folder of class web-site

```
>> help fd3d
```

```
function A = fd3d(nx,ny,nz,alpx,alpy,alpz,dshift)
```

```
NOTE nx and ny must be > 1 -- nz can be == 1.
```

```
5- or 7-point block-Diffusion/conv. matrix. with
```

- A stripped-down version is `lap2D(nx,ny)`

```
>> help mark
```

```
[A] = mark(m)
```

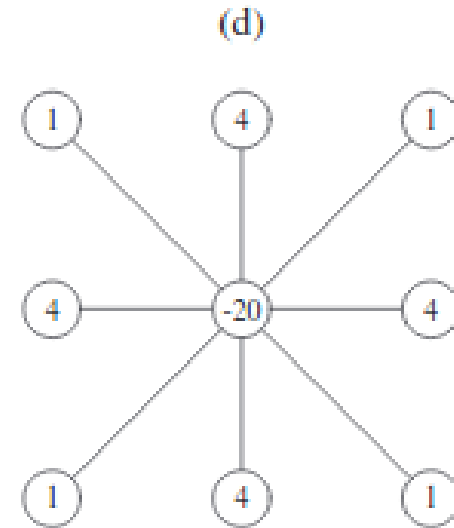
```
generates a Markov chain matrix for a random walk
```

```
on a triangular grid. A is sparse of size  $n=m*(m+1)/2$ 
```

3 Explore A few useful matlab functions

- * `kron`
- * `gplot` for plotting graphs
- * `reshape` for going from say 1-D to 2-D or 3-D arrays

 4 Write a script to generate a 9-point discretization of the Laplacean.



WORKING WITH SPARSE MATRICES

- *See Chap. 3 of text*
- *Data structures for storing sparse matrices*
- *How these are implemented*
- *Basic operation: Sparse matrix by dense vector product*

Data Structures for sparse matrices

➤ The use of a proper data structures is critical to achieving good performance.

 1 Generate a symmetric sparse matrix A in matlab and time the operations of accessing (only) all entries by columns and then by rows. Observations?

➤ Many data structures; sometimes unnecessary variants.

➤ These variants are more useful in the context of iterative methods

➤ Basic linear algebra kernels (e.g., matrix-vector products) depend on data structures.

Some Common Data Structures (from SPARSKIT)

DNS

Dense

BND

Linpack Banded

COO

Coordinate

CSR

Compressed Sparse Row

CSC

Compressed Sparse Column

MSR

Modified CSR

ELL

Ellpack-Itpack

DIA

Diagonal

BSR

Block Sparse Row

SSK

Symmetric Skyline

NSK

Nonsymmetric Skyline

JAD

Jagged Diagonal

➤ Most common (and important): CSR (/ CSC), COO

The coordinate format (COO)

$$A = \begin{pmatrix} 1. & 0. & 0. & 2. & 0. \\ 3. & 4. & 0. & 5. & 0. \\ 6. & 0. & 7. & 8. & 9. \\ 0. & 0. & 10. & 11. & 0. \\ 0. & 0. & 0. & 0. & 12. \end{pmatrix}$$

- Simplest data structure -
- Often used as 'entry' format in packages
- Variant used in matlab and matrix market
- Also known as 'triplet format'
- Note: order of entries is arbitrary [in matlab: sorted by columns]

AA	JR	JC
12.	5	5
9.	3	5
7.	3	3
5.	2	4
1.	1	1
2.	1	4
11.	4	4
3.	2	1
6.	3	1
4.	2	2
8.	3	4
10.	4	3

Compressed Sparse Row (CSR) format

$$A = \begin{pmatrix} 12. & 0. & 0. & 11. & 0. \\ 10. & 9. & 0. & 8. & 0. \\ 7. & 0. & 6. & 5. & 4. \\ 0. & 0. & 3. & 2. & 0. \\ 0. & 0. & 0. & 0. & 1. \end{pmatrix}$$

- IA(j) points to beginning of row j in arrays AA, JA
- Related formats: Compressed Sparse Column format, Modified Sparse Row format (MSR).

AA	JA	IA
12	1	1
11	4	
10	1	3
9	2	
8	4	6
7	1	
6	3	10
5	4	
4	5	12
3	3	
2	4	13
1	5	

- Used mainly in Fortran & portable codes – what about C?

CSR (CSC) format - C-style

* CSR: Collection of pointers of rows & array of row lengths

```
typedef struct SpaFmt {  
/*-----  
| C-style CSR format - used internally  
| for all matrices in CSR/CSC format  
|-----*/  
    int n;          /* size of matrix */  
    int *nzcount;    /* length of each row */  
    int **ja;        /* to store column indices */  
    double **ma;     /* to store nonzero entries */  
} SparMat;
```

aa[i][*] == entries of i-th row (col.);

ja[i][*] == col. (row) indices,

nzcount[i] == number of nonzero elmts in row (col.) i

Data structure used in Csparse

[T. Davis' SuiteSparse code]

```
typedef struct cs_sparse
{ /* matrix in compressed-column or triplet form */
  int nzmax ; /* maximum number of entries */
  int m ; /* number of rows */
  int n ; /* number of columns */
  int *p ; /* column pointers (size n+1) or
            col indices (size nzmax) */
  int *i ; /* row indices, size nzmax */
  double *x ; /* numerical values, size nzmax */
  int nz ; /* # of entries in triplet matrix,
            -1 for compressed-col */
} cs ;
```

- Can be used for CSR, CSC, and COO (triplet) storage
- Easy to use from Fortran

The Diagonal (DIA) format

$$A = \begin{pmatrix} 1. & 0. & 2. & 0. & 0. \\ 3. & 4. & 0. & 5. & 0. \\ 0. & 6. & 7. & 0. & 8. \\ 0. & 0. & 9. & 10. & 0. \\ 0. & 0. & 0. & 11. & 12. \end{pmatrix}$$

$$DA = \begin{array}{|c|c|c|} \hline * & 1. & 2. \\ \hline 3. & 4. & 5. \\ \hline 6. & 7. & 8. \\ \hline 9. & 10. & * \\ \hline 11 & 12. & * \\ \hline \end{array}$$

$$IOFF = \boxed{-1 \ 0 \ 2}$$

The Ellpack-Itpack format

$$A = \begin{pmatrix} 1. & 0. & 2. & 0. & 0. \\ 3. & 4. & 0. & 5. & 0. \\ 0. & 6. & 7. & 0. & 8. \\ 0. & 0. & 9. & 10. & 0. \\ 0. & 0. & 0. & 11. & 12. \end{pmatrix}$$

$$AC = \begin{array}{|c|c|c|} \hline 1. & 2. & 0. \\ \hline 3. & 4. & 5. \\ \hline 6. & 7. & 8. \\ \hline 9. & 10. & 0. \\ \hline 11 & 12. & 0. \\ \hline \end{array}$$

$$JC = \begin{array}{|c|c|c|} \hline 1 & 3 & 1 \\ \hline 1 & 2 & 4 \\ \hline 2 & 3 & 5 \\ \hline 3 & 4 & 4 \\ \hline 4 & 5 & 5 \\ \hline \end{array}$$

Block matrices

$$A = \left(\begin{array}{cc|cc|cc} 1. & 2. & 0. & 0. & 3. & 4. \\ 5. & 6. & 0. & 0. & 7. & 8. \\ \hline 0. & 0. & 9. & 10. & 11. & 12. \\ 0. & 0. & 13. & 14. & 15. & 16. \\ \hline 17. & 18. & 0. & 0. & 20. & 21. \\ 22. & 23. & 0. & 0. & 24. & 25. \end{array} \right)$$

$$AA = \begin{array}{|cc|cc|cc|} \hline 1. & 3. & 9. & 11. & 17. & 20. \\ 5. & 7. & 15. & 13. & 22. & 24. \\ \hline 2. & 4. & 10. & 12. & 18. & 21. \\ 6. & 8. & 14. & 16. & 23. & 25. \\ \hline \end{array}$$

$$JA = \boxed{1 \ 5 \ 3 \ 5 \ 1 \ 5}$$

$$IA = \boxed{1 \ 3 \ 5 \ 7}$$

- Columns of AA hold 2 x 2 blocks. JA(k) = col. index of (1,1) entries of k-th block. FORTRAN: declare as AA(2,2,6)

- Can also store the blocks row-wise in AA.

AA =

1.	5.	2.	6.
3.	7.	4.	8.
9.	15.	10.	14.
11.	13.	12.	16.
17.	22.	18.	23.
20.	24.	21.	25.

JA =

1	5	3	5	1	5
---	---	---	---	---	---

IA =

1	3	5	7
---	---	---	---

- In other words AA is simply transposed



What are the advantages and disadvantages of each scheme?

- Block formats are important in many applications..
- Also valuable: block structure with variable block size.

Sparse matrices – data structure in C

➤ Recall:

```
typedef struct SpaFmt {  
/*-----  
| C-style CSR format - used internally  
| for all matrices in CSR format  
|-----*/  
    int n;  
    int *nzcount; /* length of each row */  
    int **ja; /* to store column indices */  
    double **ma; /* to store nonzero entries */  
} CsMat, *csptr;
```

➤ Can store rows of a matrix (CSR) – or its columns (CSC)

➤ Q: How to perform the operation $y = A * x$ in each case?

Matvec – row version

```
void matvec( csptr mata, double *x, double *y )
{
    int i, k, *ki;
    double *kr;
    for (i=0; i<mata->n; i++) {
        y[i] = 0.0;
        kr = mata->ma[i];
        ki = mata->ja[i];
        for (k=0; k<mata->nzcount[i]; k++)
            y[i] += kr[k] * x[ki[k]];
    }
}
```

► Uses sparse dot products (**sparse SDOTS**)

 3 Operation count

Matvec – Column version

```
void matvecC( csptr mata, double *x, double *y )
{
    int n = mata->n, i, k, *ki;
    double *kr;
    for (i=0; i<n; i++)
        y[i] = 0.0;
    for (i=0; i<n; i++) {
        kr = mata->ma[i];
        ki = mata->ja[i];
        for (k=0; k<mata->nzcount[i]; k++)
            y[ki[k]] += kr[k] * x[i];
    }
}
```

➤ Uses sparse vector combinations (sparse **SAXPY**)



Operation count

➤ Using the CS data structure from Suite-Sparse:

```
int cs_gaxpy (cs *A, double *x, double *y) {
    int p, j, n, *Ap, *Ai;
    n = A->n; Ap = A->p; Ai = A->i; Ax = A->x;
    for (j=0; j<n; j++) {
        for (p=Ap[j]; p<Ap[j+1];p++)
            y[Ai[p]] += Ax[p]*x[j];
    }
    return(1)
}
```

Matvec – row version - FORTRAN

```
      subroutine amux (n, x, y, a, ja, ia)
      real*8  x(*), y(*), a(*), t
      integer n, ja(*), ia(*), i, k
c----- row loop
      do 100 i = 1,n
c----- inner product of row i with vector x
          t = 0.0d0
          do 99 k=ia(i), ia(i+1)-1
              t = t + a(k)*x(ja(k))
          99      continue
          y(i) = t
      100  continue
      return
      end
```

Matvec – column version - FORTRAN

```
      subroutine atmux (n, x, y, a, ja, ia)
      real*8 x(*), y(*), a(*)
      integer n, ia(*), ja(*)
      integer i, k
c----- set y to zero
      do 1 i=1,n
          y(i) = 0.0
1      continue
c----- column loop
      do 100 i = 1,n
c----- sparse saxpy
          do 99 k=ia(i), ia(i+1)-1
              y(ja(k)) = y(ja(k)) + x(i)*a(k)
          99      continue
      100      continue
c
      return
      end
```

Sparse matrices in matlab

-  5 Generate a tridiagonal matrix T
-  6 Convert T to sparse format
-  7 See how you can generate this sparse matrix directly using `sparse`
-  8 See how you can use `spconvert` to achieve the same result
-  9 What can you observe about the way the triplets of a sparse matrix are ordered?
-  10 Important for performance: `spalloc`. See the difference between

`A = sparse(m,n)` **and** `A = spalloc(m,n,nzmax)`

Sparse matrices in Python

- Must load *scipy.sparse* : For example, *import scipy.sparse as sp*
- Difference with matlab: Must explicitly specify a matrix format. [→ a matrix is not just 'sparse' it is *csr_sparse* for example]
- Formats available:
 - *bsr_matrix* - Block Sparse Row matrix
 - *coo_matrix* - A sparse matrix in COOrdinate format
 - *csc_matrix* - Compressed Sparse Column matrix
 - *csr_matrix* - Compressed Sparse Row matrix
 - *dia_matrix* - Sparse matrix with DIAGONAL storage
 - *dok_matrix* - Dictionary Of Keys based sparse matrix
 - *lil_matrix* - Row-based list of lists sparse matrix

- Can get info on a matrix by accessing fields like: *S.nnz* or *S.shape*
- Also for CSR: *S.data*, *S.indices*, and *S.indptr*, give the arrays *AA*, *JA*, *IA*, seen earlier (vals., col., ptr)
- Convert to another format: e.g., *S.tocsc()*
- Equivalent of `full` on matlab: *A = S.todense()*
- Inverse operation: [dense to sparse (csr)] *S = sp.csr_matrix(A)*

 11 Create a 6×6 tridiagonal matrix (dense mode) and convert it to a *csr* matrix.

 12 Try to understand the *dok*, *lil* formats.

BACKGROUND: A Brief Introduction to Graph Theory

- *General definitions; Representations;*
- *Graph Traversals;*
- *Topological sort;*

Graphs – definitions & representations

- Graph theory is a fundamental tool in sparse matrix techniques.

DEFINITION. A graph G is defined as a pair of sets $G = (V, E)$ with $E \subset V \times V$. So G represents a binary relation. The graph is **undirected** if the binary relation is symmetric. It is **directed** otherwise. V is the vertex set and E is the edge set.

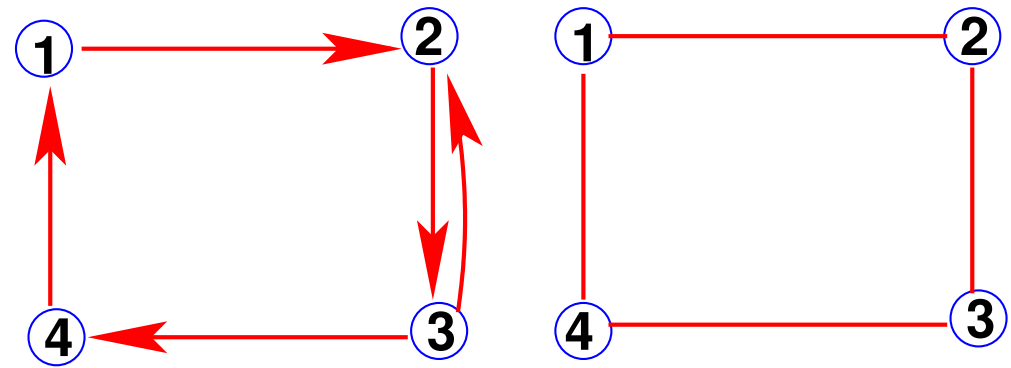
If R is a binary relation between elements in V then, we can represent it by a graph $G = (V, E)$ as follows:

$$(u, v) \in E \leftrightarrow u R v$$

Undirected graph $\leftrightarrow$ symmetric relation

Left: (1 R 2); (4 R 1); (2 R 3); (3 R 2); (3 R 4)

Right: (1 R 2); (2 R 3); (3 R 4); (4 R 1)



 1 Given the numbers 5, 3, 9, 15, 16, show the two graphs representing the relations

R1: Either $x < y$ or y divides x .

R2: x and y are congruent modulo 3. [$\text{mod}(x,3) = \text{mod}(y,3)$]

➤ $|E| \leq |V|^2$. For undirected graphs: $|E| \leq |V|(|V| + 1)/2$.

➤ A sparse graph is one for which $|E| \ll |V|^2$.

Graphs – Examples and applications

1. Airport connection system: (a) R (b) if there is a non-stop flight from (a) to (b).
2. Highway system;
3. Computer Networks;
4. Electrical circuits;
5. Traffic Flow;
6. Social Networks;
7. Sparse matrices;
- ...

Basic Terminology & notation:

- If $(u, v) \in E$, then v is **adjacent** to u . The edge (u, v) is **incident** to u and v .
- If the graph is directed, then (u, v) is an **outgoing** edge from u and **incoming** edge to v
- $Adj(i) = \{j | j \text{ adjacent to } i\}$
- The **degree** of a vertex v is the number of edges incident to v . Can also define the **indegree** and **outdegree**. (Sometimes self-edge $i \rightarrow i$ omitted)
- $|S|$ is the cardinality of set S [so $|Adj(i)| == \deg(i)$]
- A **subgraph** $G' = (V', E')$ of G is a graph with $V' \subset V$ and $E' \subset E$.

Representations of Graphs

- A graph is nothing but a collection of vertices (indices from 1 to n), each with a set of its adjacent vertices [in effect a 'sparse matrix without values']
- Therefore, can use any of the sparse matrix storage formats - omit the real values arrays.

Adjacency matrix Assume $V = \{1, 2, \dots, n\}$. Then the **adjacency matrix** of $G = (V, E)$ is the $n \times n$ matrix, with entries:

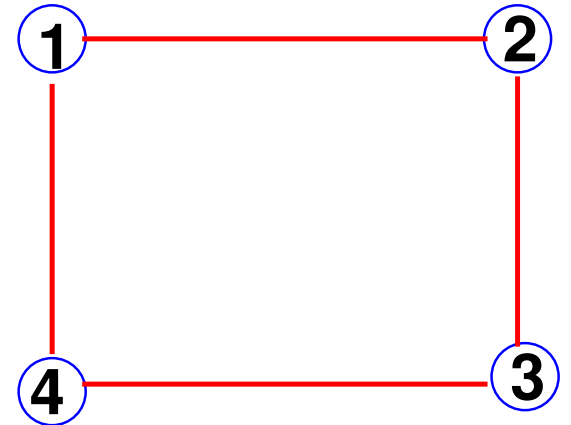
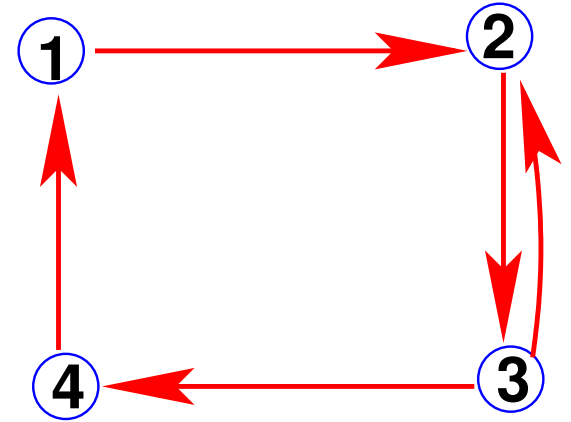
$$a_{i,j} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{Otherwise} \end{cases}$$

Representations of Graphs (cont.)

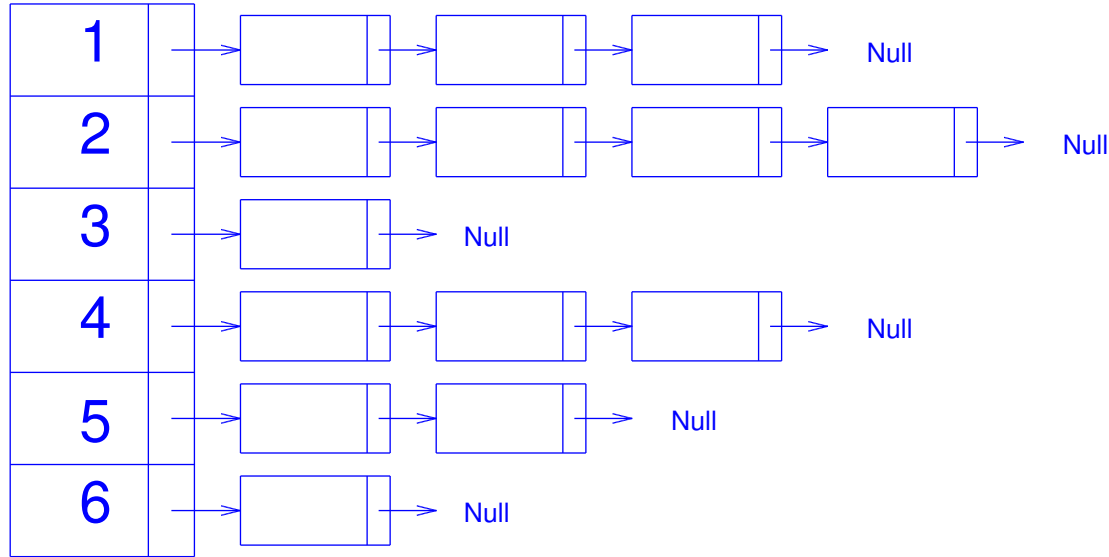
Example:

$$\begin{bmatrix} & 1 & & \\ & & 1 & \\ 1 & & & \\ & 1 & & 1 \end{bmatrix}$$

$$\begin{bmatrix} & 1 & & 1 \\ 1 & & 1 & \\ & 1 & & 1 \\ 1 & & 1 & \end{bmatrix}$$



Dynamic representation: Linked lists



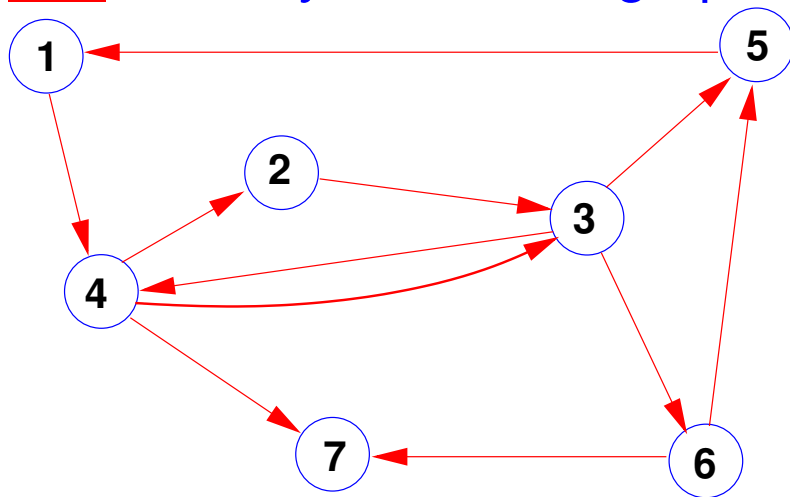
- An array of linked lists. A linked list associated with vertex i , contains all the vertices adjacent to vertex i .
- General and concise for 'sparse graphs' (the most practical situations) - but not economical for use in sparse matrix methods

More terminology & notation

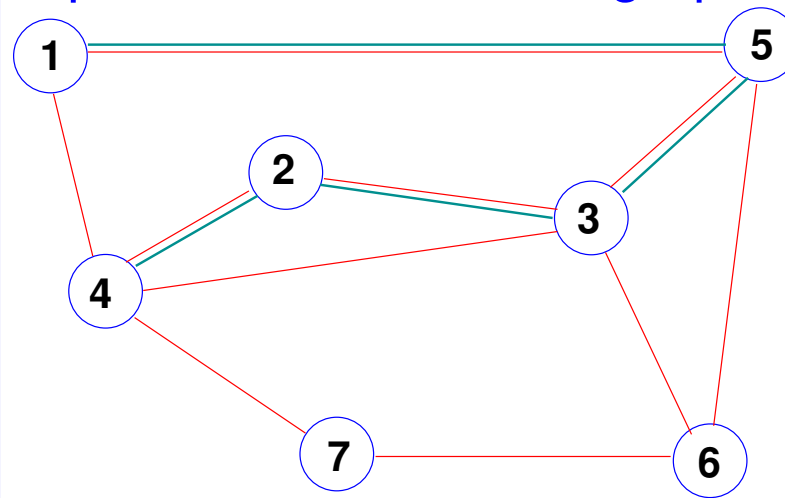
- For a given $Y \subset X$, the **section** graph of Y is the subgraph $G_Y = (Y, E(Y))$ where $E(Y) = \{(x, y) \in E \mid x \in Y, y \text{ in } Y\}$
- A section graph is a **clique** if all the nodes in the subgraph are pairwise adjacent ($\rightarrow$ dense block in matrix)
- A **path** is a sequence of vertices $w_0, w_1, \dots, w_k$ such that $(w_i, w_{i+1}) \in E$ for $i = 0, \dots, k - 1$.
- The **length** of the path $w_0, w_1, \dots, w_k$ is k (# of edges in the path)
- A **cycle** is a closed path, i.e., a path with $w_k = w_0$.
- A graph is **acyclic** if it has no cycles.



2 Find cycles in this graph:

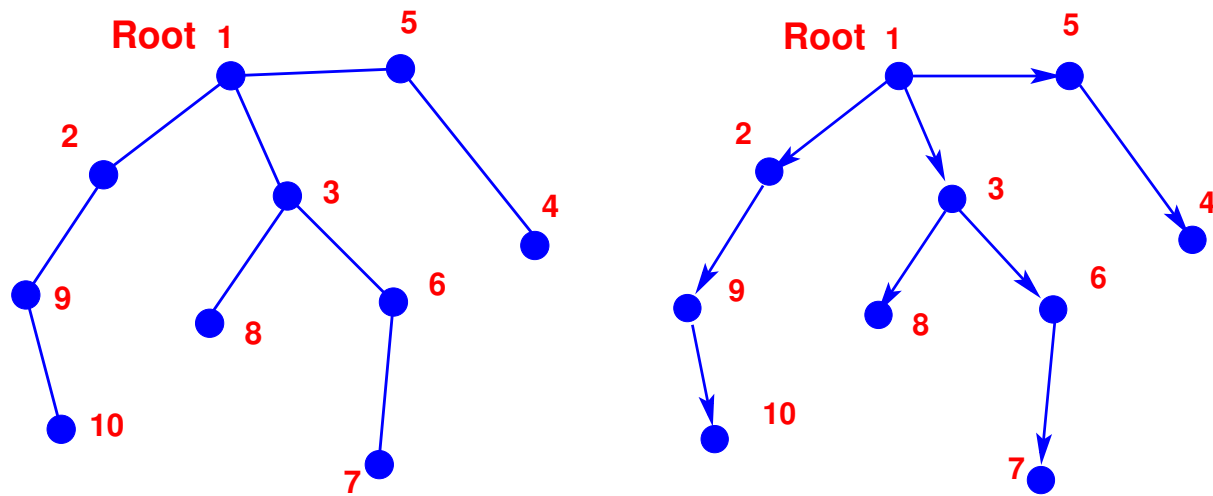


A path in an undirected graph



- A path $w_0, \dots, w_k$ is **simple** if the vertices $w_0, \dots, w_k$ are distinct (except that we may have $w_0 = w_k$ for cycles).
- An **undirected** graph is **connected** if there is path from every vertex to every other vertex.
- A **digraph** with the same property is said to be **strongly connected**

- The **undirected form** of a directed graph the undirected graph obtained by removing the directions of all the edges.
- Another term used “**symmetrized**” form -
- A directed graph whose undirected form is connected is said to be **weakly connected** or **connected**.
- **Tree** = a graph whose undirected form, i.e., symmetrized form, is acyclic & connected
- Forest = a collection of trees
- In a **rooted tree** one specific vertex is designated as a root.
- Root determines orientation of the tree edges in parent-child relation



- Parent-Child relation: immediate neighbors of root are children. Root is their parent. Recursively define children-parents
- In example: v_3 is parent of v_6, v_8 and v_6, v_8 are children of v_3 .
- Nodes that have no children are **leaves**. In example: v_{10}, v_7, v_8, v_4
- Descendent, ancestors, ...

Tree traversals

- Tree traversal is a process of visiting all vertices in a tree. Typically traversal starts at root.
- Want: systematic traversals of all nodes of tree – moving from a node to a child or parent
- **Preorder traversal:** Visit parent before children [recursively]

In example: $v_1, v_2, v_9, v_{10}, v_3, v_8, v_6, v_7, v_5, v_4$

- **Postorder traversal:** Visit children before parent [recursively]

In example : $v_{10}, v_9, v_2, v_8, v_7, v_6, v_3, v_4, v_5, v_1$

Graph Traversals – Depth First Search

- Issue: systematic way of visiting all nodes of a **general** graph
- Two basic methods: Breadth First Search (will see later) & ...

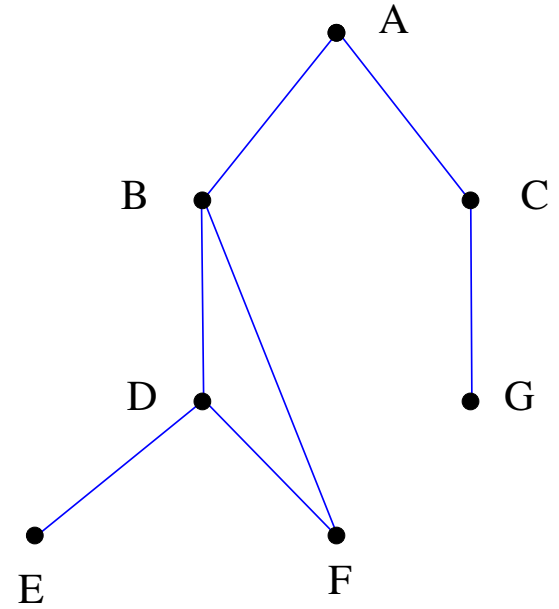
Algorithm $List = DFS(G, v)$ (DFS from v)

- Depth-First Search
 - Visit and Mark v ;
 - for all edges (v, w) do
 - if w is not marked then $List = DFS(G, w)$
 - Add v to top of list produced above
- If G is undirected and connected, all nodes will be visited
- If G is directed and strongly connected, all nodes will be visited

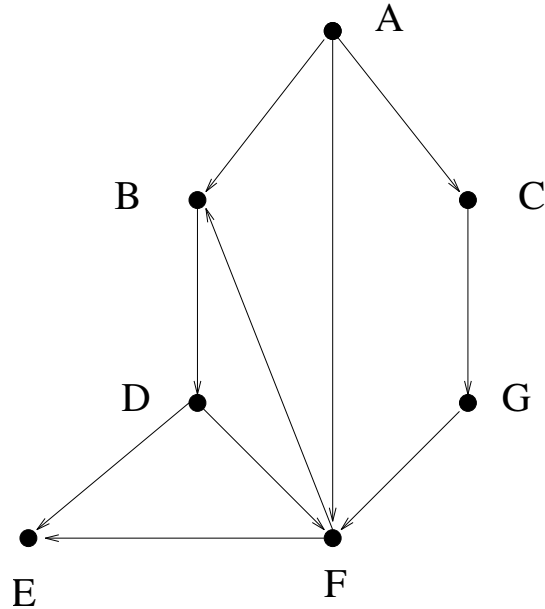
Depth First Search – undirected graph example

➤ Assume adjacent nodes are listed in alphabetical order.

 3 DFS traversal from A ?



Depth First Search – directed graph example



➤ Assume adjacent nodes are listed in alphabetical order.

 4 DFS traversal from A?

NOTE: We will now use a column-oriented graph representation:

$$j \rightarrow i \quad \text{if} \quad a_{ij} \neq 0$$


```

function [Lst, Mark] = dfs(u, A, Lst, Mark)
%% function [Lst, Mark] = dfs(u, A, Lst, Mark)
%% dfs from node u  -- Recursive
%%-----
[ii, jj, rr] = find(A(:,u));
Mark(u) = 1;
for k=1:length(ii)
    v = ii(k);
    if (~Mark(v))
        [Lst, Mark] = dfs(v, A, Lst, Mark);
    end
end
Lst = [u,Lst]

```

Depth-First-Search Tree: Consider the parent-child relation: v is a parent of u if u was visited from v in the depth first search algorithm. The (directed) graph resulting from this binary relation is a tree called the Depth-First-Search Tree. To describe tree: only need the parents list.

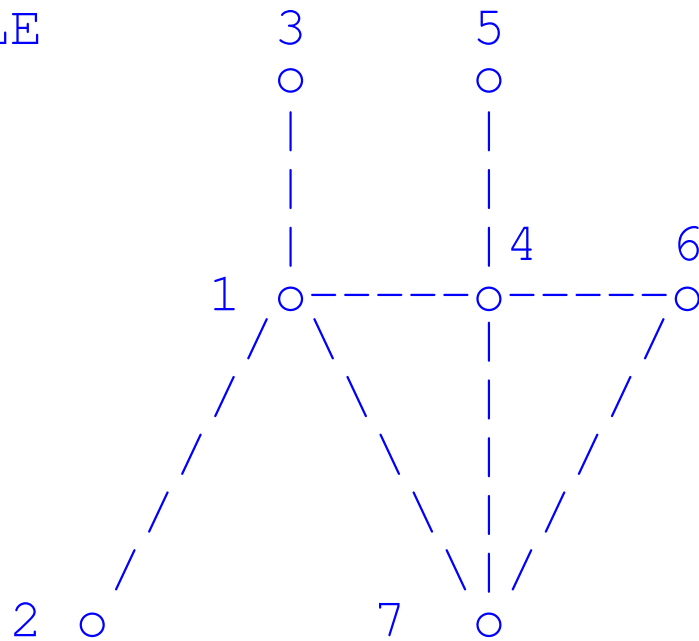
➤ To traverse all the graph we need a $\text{DFS}(v, G)$ from each node v that has not been visited yet – so add another loop. Refer to this as:

$\text{DFS}(G)$

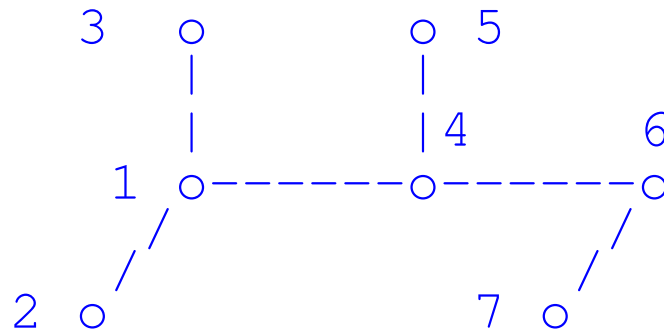
➤ When a new vertex is visited in DFS, some work is done. Example: we can build a stack of nodes visited to show order (reverse order: easier) in which the node is visited.

EXAMPLE

We assume adjacency list
is in increasing order.
[e.g: $\text{Adj}(4) = (1, 5, 6, 7)$]

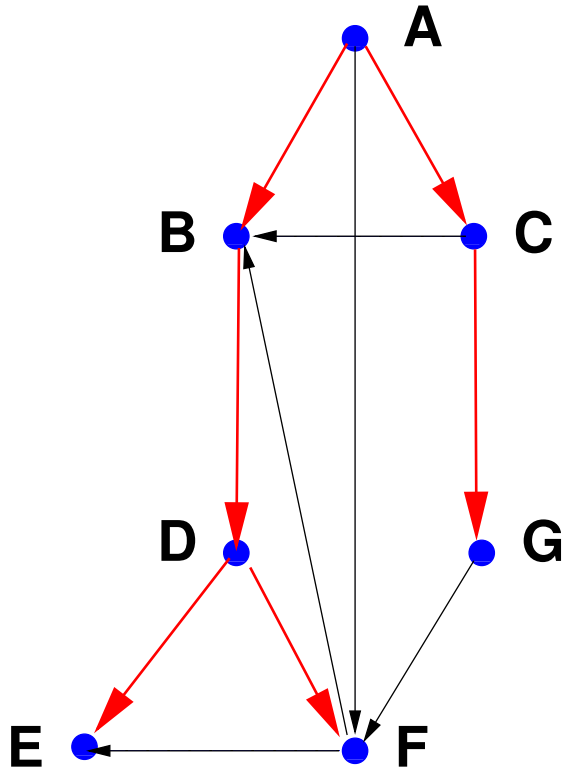


DFS traversal: 1 --> 2₁ --> 3₁ --> 4₁ --> 5₄ --> 6₄ --> 7₆
Parents list:



<----- Depth First
Search Tree

Back edges, forward edges, and cross edges



- Thick red lines: DFS traversal tree from A
- $A \rightarrow F$ is a Forward edge
- $F \rightarrow B$ is a Back edge
- $C \rightarrow B$ and $G \rightarrow F$ are Cross-edges.

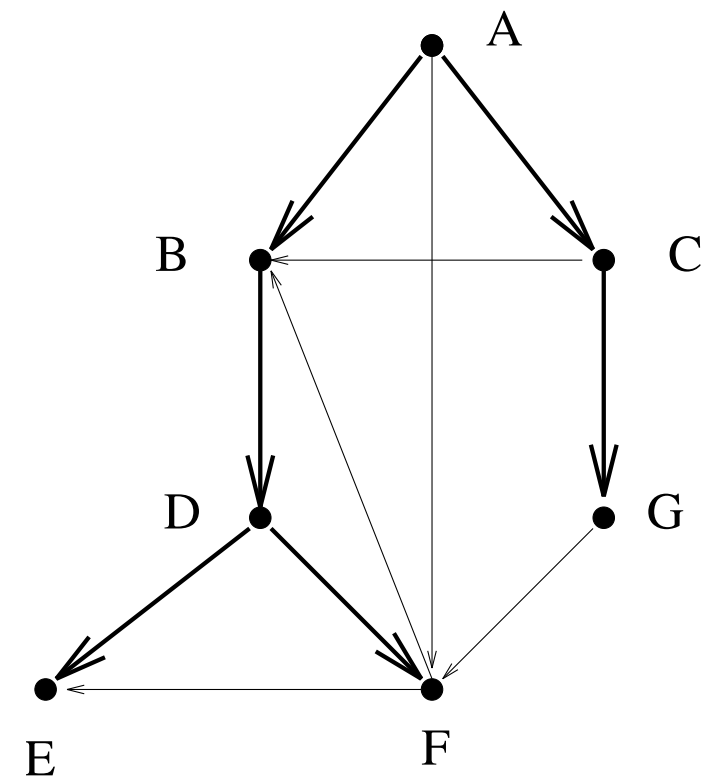
➤ Consider the 'List' produced by DFS.

Lst=[A, C, G, B, D, F, E]

➤ Order in list is important for some algorithms

➤ Notice: Label nodes in List from 1 to n . Then:

- Tree-edges / Forward edges : labels increase in $\rightarrow$
- Cross edges : labels in/de-crease in $\rightarrow$ [depends on labeling]
- Back-edges : labels decrease in $\rightarrow$



Properties of Depth First Search

- If G is a connected undirected (or strongly connected) graph, then each vertex will be visited once and each edge will be inspected at least once.
- Therefore, for a connected undirected graph, The cost of DFS is $O(|V| + |E|)$
- If the graph is undirected, then there are no cross-edges. (all non-tree edges are called 'back-edges')

Theorem: A directed graph is acyclic iff a DFS search of G yields no back-edges.

- Terminology: Directed Acyclic Graph or **DAG**

Topological Sort

Problem: Given a **Directed Acyclic Graph** (DAG), order the vertices from 1 to n such that, if (u, v) is an edge, then u appears before v in the ordering.

- Equivalently, label vertices from 1 to n so that in any (directed) path from a node labelled k , all vertices in the path have labels $> k$.
- Many Applications
- Prerequisite requirements in a program
- Scheduling of tasks for any project
- Parallel algorithms;
- ...

Topological Sorting: A first algorithm

Property exploited: An acyclic Digraph must have at least one vertex with indegree = 0.

 5 Prove this

Algorithm:

- First label these vertices as $1, 2, \dots, k$;
- Remove these vertices and all edges incident from them
- Resulting graph is again acyclic ... $\exists$ nodes with indegree = 0. label these nodes as $k + 1, k + 2, \dots$,
- Repeat..

 6 Explore implementation aspects.

Alternative method: Topological sort from DFS

- Depth first search traversal of graph.
- Do a 'post-order traversal' of the DFS tree.

Algorithm $Lst = Tsort(G)$

(post-order DFS from v)

Mark = zeros($n,1$); Lst = $\emptyset$

for $v=1:n$ do:

 if (Mark(v))== 0)

 [Lst, Mark] = dfs(v , G, Lst, Mark);

 end

end

- dfs(v , G, Lst, Mark) is the DFS(G,v) which adds v to the top of Lst after finishing the traversal from v

Lst = DFS(G, v)

- Visit and Mark v ;
- for all edges (v, w) do
 - if w is not marked then $Lst = DFS(G, w)$
- $Lst = [v, Lst]$

➤ Topological order given by the final Lst array of **Tsort**

 7 Explore implementation issue

 8 Implement in matlab

 9 Show correctness [i.e.: is this indeed a topol. order? hint: no back-edges in a DAG]

GRAPH MODELS FOR SPARSE MATRICES

- *See Chap. 3 of text*
- *Sparse matrices and graphs.*
- *Bipartite model, hypergraphs*
- *Application: back propagation*

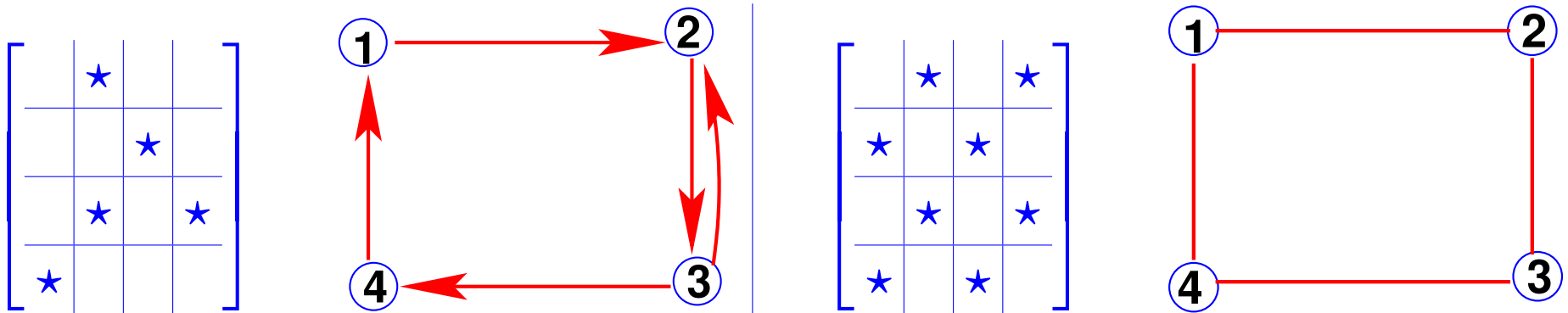
Graph Representations of Sparse Matrices. Recall:

Adjacency Graph $G = (V, E)$ of an $n \times n$ matrix A :

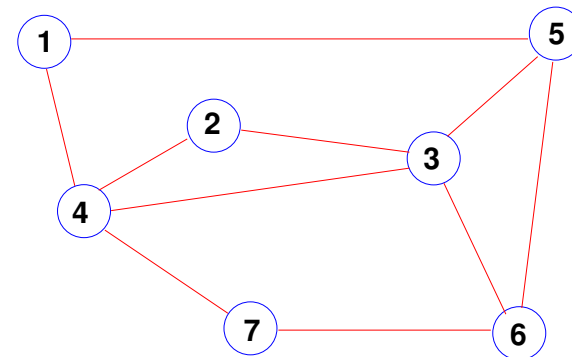
$$V = \{1, 2, \dots, N\} \quad E = \{(i, j) | a_{ij} \neq 0\}$$

➤ G == undirected if A has a symmetric pattern

Example:



 10 Show the matrix pattern for the graph on the right and give an interpretation of the path v_4, v_2, v_3, v_5, v_1 on the matrix



➤ A separator is a set Y of vertices such that the graph G_{X-Y} is disconnected.

Example: $Y = \{v_3, v_4, v_5\}$ is a separator in the above figure

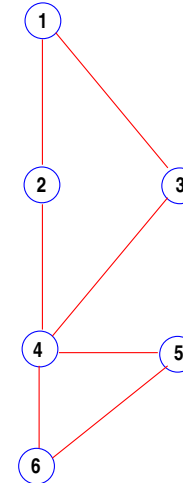
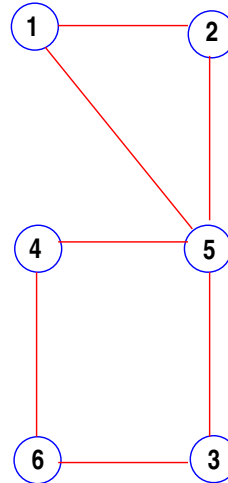
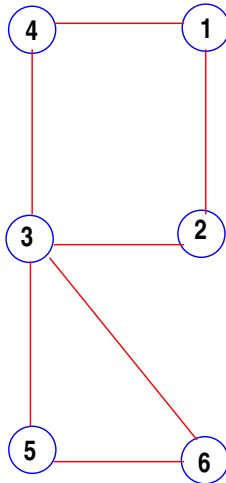
Example: Adjacency graph of:

$$A = \begin{bmatrix} & \star & & \star & & \\ \star & & \star & & & \\ & \star & & \star & \star & \star \\ \star & & \star & & & \\ & & \star & & & \star \\ & & \star & & \star & \end{bmatrix}.$$

Example: For any **adjacency** matrix A , what is the graph of A^2 ? [interpret in terms of paths in the graph of A]

➤ Two graphs are **isomorphic** if there is a mapping between the vertices of the two graphs that preserves adjacency.

 11 Are the following 3 graphs isomorphic? If yes find the mappings between them.



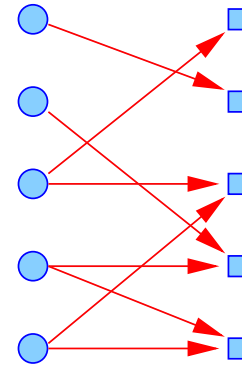
- Graphs are identical – labels are different
- Determining graph isomorphism is a **hard** problem

Bipartite graph representation

- Rows and columns are (both) represented by vertices;
- Relations only between rows and columns: Row i is connected to column j if $a_{ij} \neq 0$

Example:

$$\begin{bmatrix} & \star & & & \\ & & & \star & \\ \star & & \star & & \\ & & & \star & \star \\ & & \star & & \star \end{bmatrix}$$



- Bipartite models used only for specific cases [e.g. rectangular matrices, ...] - By default we use the standard definition of graphs.

Interpretation of graphs of matrices

 12 What is the graph of $A + B$ (for two $n \times n$ matrices)?

 13 What is the graph of A^T ?

 14 What is the graph of $A.B$?

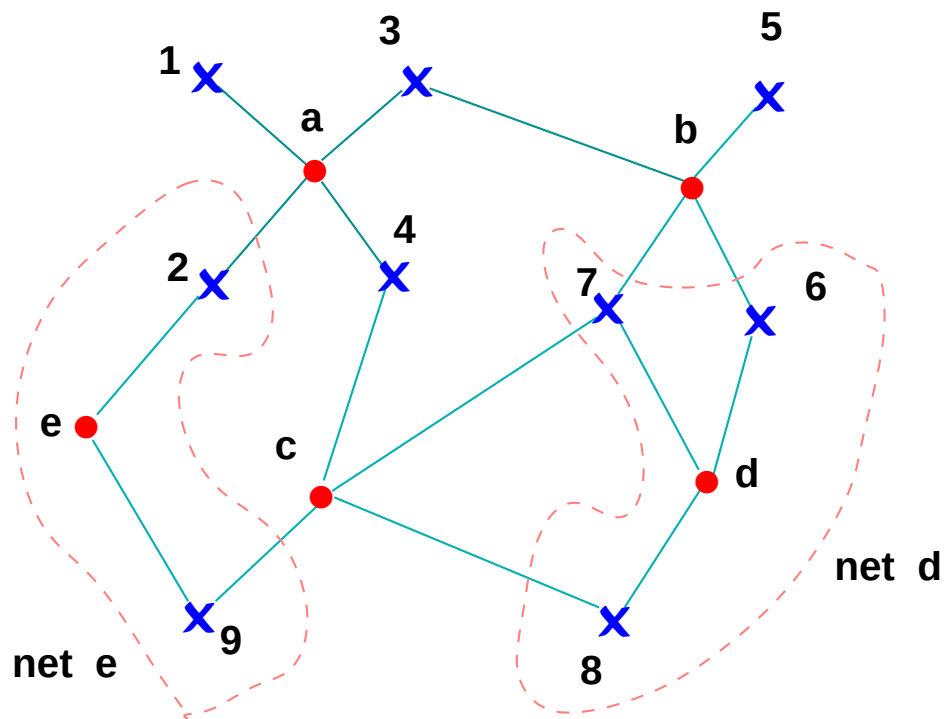
A few words on hypergraphs

- Hypergraphs are very general.. Ideas borrowed from VLSI work
- Main motivation: to better represent communication volumes when partitioning a graph. Standard models face many limitations
- Hypergraphs can better express complex graph partitioning problems and provide better solutions.
- Example: completely nonsymmetric patterns ...
- .. Even rectangular matrices. Best illustration: Hypergraphs are ideal for text data

Example: $V = \{1, \dots, 9\}$ and $E = \{a, \dots, e\}$ with

$a = \{1, 2, 3, 4\}$, $b = \{3, 5, 6, 7\}$, $c = \{4, 7, 8, 9\}$,

$d = \{6, 7, 8\}$, and $e = \{2, 9\}$



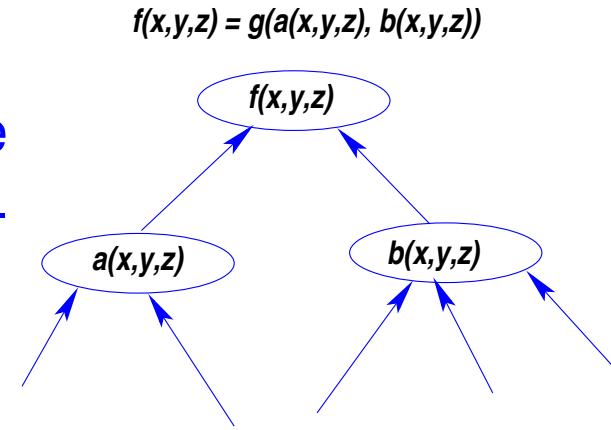
Boolean matrix:

	1	2	3	4	5	6	7	8	9	
1	1	1	1	1						a
			1		1	1	1			b
				1			1	1	1	c
						1	1	1		d
	1								1	e

$A =$

A few words on computational graphs

- Computational graphs: graphs where nodes represent computations whose evaluation depend on other (incoming) nodes.

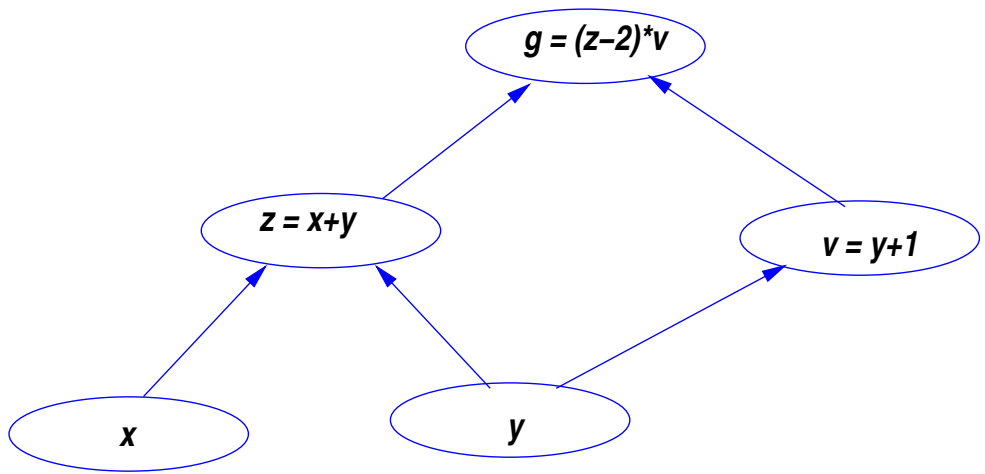


- Consider the following expression:

$$g(x, y) = (x + y - 2) * (y + 1)$$

We can decompose this as
$$\begin{cases} z = x + y \\ v = y + 1 \\ g = (z - 2) * v \end{cases}$$

- Computational graph →
- Given x, y we want:
 - (a) Evaluate the nodes and
 - (b) derivatives w.r.t x, y



(a) is trivial - just follow the graph up - starting from the leaves (that contain x and y)

(b): Use the chain rule – here shown for x only using previous setting

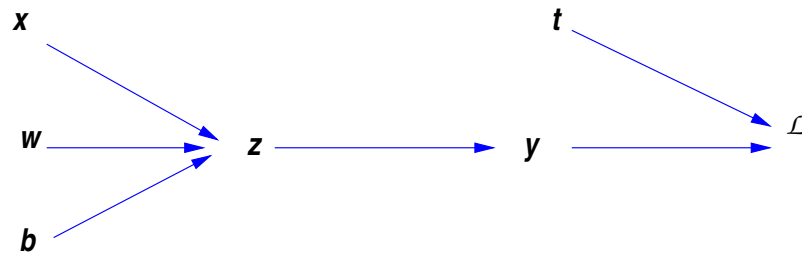
$$\frac{\partial g}{\partial x} = \frac{\partial g}{\partial a} \frac{da}{dx} + \frac{\partial g}{\partial b} \frac{db}{dx}$$

 15 For the above example compute values and derivatives at all nodes when $x = -1, y = 2$.

Back-Propagation

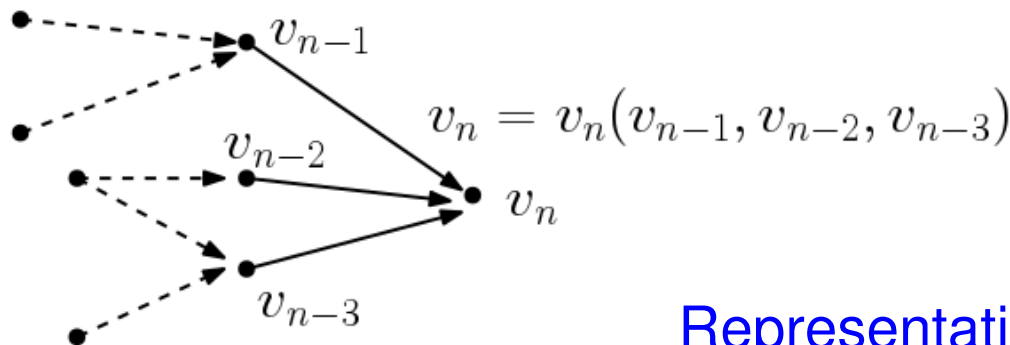
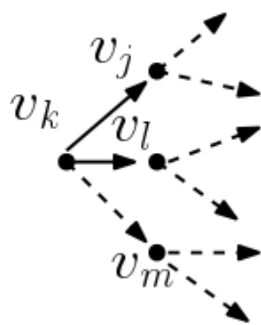
- Often we want to compute the gradient of the function at the root, once the nodes have been evaluated
- The derivatives can be calculated by going backward (or down the tree)
- Here is a very simple example from Neural Networks

$$\begin{cases} L = \frac{1}{2}(y - t)^2 \\ y = \sigma(z) \\ z = wx + b \end{cases}$$



- Note that t (desired output) and x (input) are constant.

Back-Propagation: General computational graphs



Representation: *a DAG*

- Last node (v_n) is the target function. Let us rename it f .
- Nodes $v_i, i = 1, \dots, e$ with indegree 0 are the variables
- Want to compute $\partial f / \partial v_1, \partial f / \partial v_2, \dots, \partial f / \partial v_e$

- Use the chain rule.



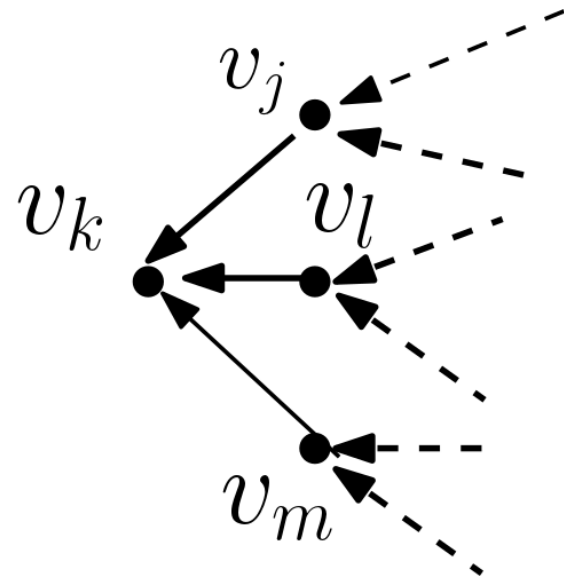
$$\frac{\partial f}{\partial v_k} = \frac{\partial f}{\partial v_j} \frac{\partial v_j}{\partial v_k} + \frac{\partial f}{\partial v_l} \frac{\partial v_l}{\partial v_k} + \frac{\partial f}{\partial v_m} \frac{\partial v_m}{\partial v_k}$$

- Let $\delta_k = \frac{\partial f}{\partial v_k}$ (called ‘errors’). Then

$$\delta_k = \delta_j \frac{\partial v_j}{\partial v_k} + \delta_l \frac{\partial v_l}{\partial v_k} + \delta_m \frac{\partial v_m}{\partial v_k}$$

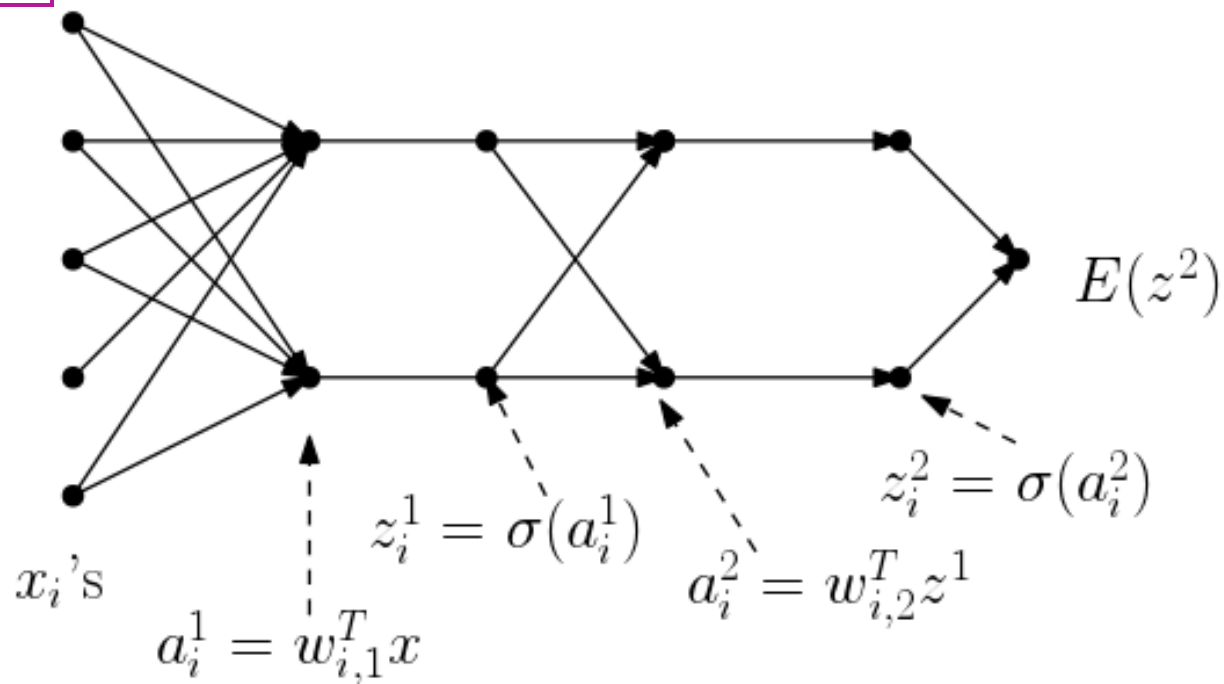
- To compute the δ_k ’s once the v_j ’s have been computed (in a ‘forward’ propagation) – proceed backward.

- $\delta_j, \delta_l, \delta_m$ available and $\partial v_i / \partial v_k$ computable. Note $\delta_n \equiv 1$.



- However: cannot just do this in any order. Must follow a **topological order** in order to obey dependencies.

Example:



Sparse Triangular Systems

- *Triangular systems*
- *Sparse triangular systems with dense right-hand sides*
- *Sparse triangular systems with sparse right-hand sides*
- *A sparse factorization based on sparse triangular solves*

Sparse Triangular linear systems: the problem

The Problem: A is an $n \times n$ matrix, and b a vector of $\mathbb{R}^n$. Find x such that:

$$Ax = b$$

- x is the unknown vector, b the right-hand side, and A is the coefficient matrix
- We consider the case when A is upper (or lower)triangular.

Two cases:

1. A sparse, b dense vector [solve once or many times]
2. A sparse, b sparse vector [solve once or many times]

Triangular linear systems

Example:

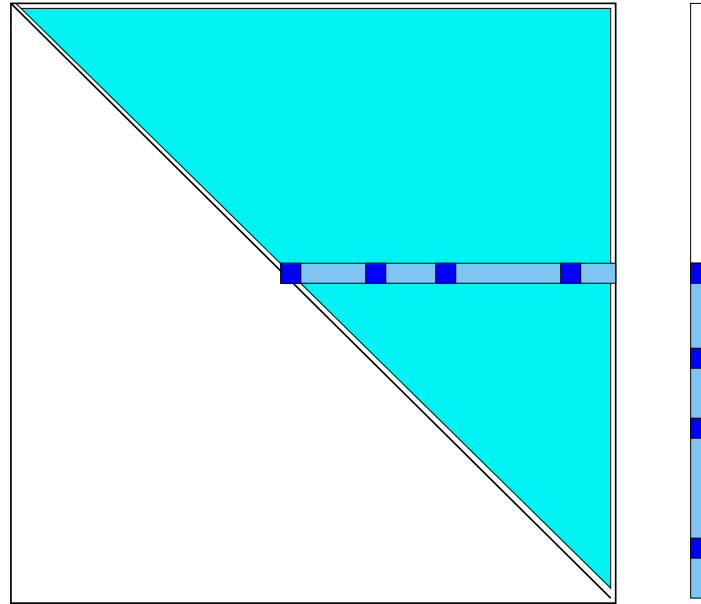
$$\begin{bmatrix} 2 & 4 & 4 \\ 0 & 5 & -2 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix}$$

Back-Substitution
Row version

 1 Operation count?

```
For  $i = n : -1 : 1$  do:  
     $t := b_i$   
    For  $j = i + 1 : n$  do  
         $t := t - a_{ij}x_j$   
    End  
     $x_i = t/a_{ii}$   
End
```

Illustration for sparse case (Sparse A , dense b)



- This will use the CSR data structure
- Inner product of a sparse row with a dense column
- Sparse BLAS: Sparse 'sdot'

➤ Recall:

```
typedef struct SpaFmt {  
/*-----  
| C-style CSR format - used internally  
| for all matrices in CSR format  
|-----*/  
    int n;  
    int *nzcount; /* length of each row */  
    int **ja;     /* to store column indices */  
    double **ma;  /* to store nonzero entries */  
} CsMat, *csptr;
```

- Can store rows of a matrix (CSR) or its columns (CSC)
- Assume that diagonal entry is stored in first location in inverted form.
- Result:

```

void Usol(csptr mata, double *b, double *x)
{
    int i, k, *ki;
    double *ma;
    for (i=mata->n-1; i>=0; i--) {
        ma = mata->ma[i];
        ki = mata->ja[i];
        x[i] = b[i] ;
        // Note: diag. entry avoided
        for (k=1; k<mata->nzcount[i]; k++)
            x[i] -= ma[k] * x[ki[k]];
        x[i] *= ma[0];
    }
}

```

➤ Operation count?

Column version

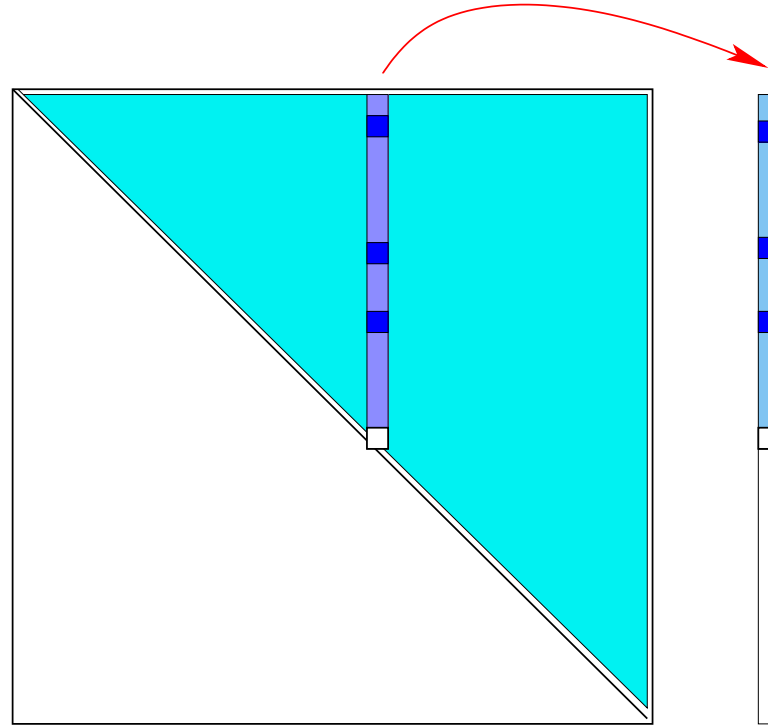
➤ Column version of back-substitution:

Back-Substitution
Column version

```
For  $j = n : -1 : 1$  do:  
     $x_j = b_j / a_{jj}$   
    For  $i = 1 : j - 1$  do  
         $b_i := b_i - x_j * a_{ij}$   
    End  
End
```

 2 Justify the above algorithm [Show that it does indeed give the solution]

Illustration for sparse case (Sparse A , dense b)



- Uses the CSC format – (CsMat struct for columns of A)
- Sparse BLAS : sparse ‘saxpy’

➤ Assumes diagonal entry stored first in inverted form

```
void UsolC(csptr mata, double *b, double *x)
{
    int i, k, *ki;
    double *ma;
    for (i=mata->n-1; i>=0; i--) {
        ja = U->ja[i];
        ma = U->ma[i];
        x[i] *= ma[0];
        // Note: diag. entry avoided
        for( j = 1; j < U->nzcount[i]; j++ )
            x[ja[j]] -= ma[j] * x[i];
    }
}
```

 3 Operation count ?

Sparse A and sparse b

Illustration: Consider solving $Lx = b$ in the situation:

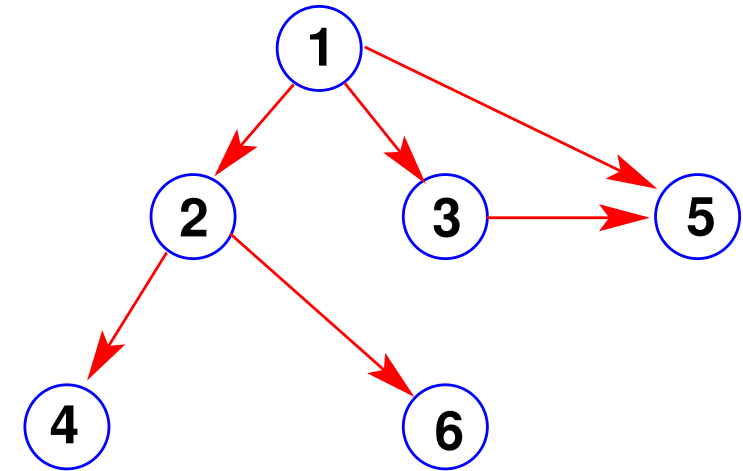
$$L = \begin{bmatrix} * & & & & & \\ * & * & & & & \\ * & & * & & & \\ & * & & * & & \\ * & & * & & * & \\ & * & & & & * \end{bmatrix} \quad b = \begin{bmatrix} * \\ \\ \\ \\ \\ \end{bmatrix}$$

 4 Show progress of the pattern of $x = L^{-1}b$ by performing symbolically a column solve for system $Lx = b$.

 5 Show how this pattern can be determined with Topological sorting. Generalize to any sparse b .

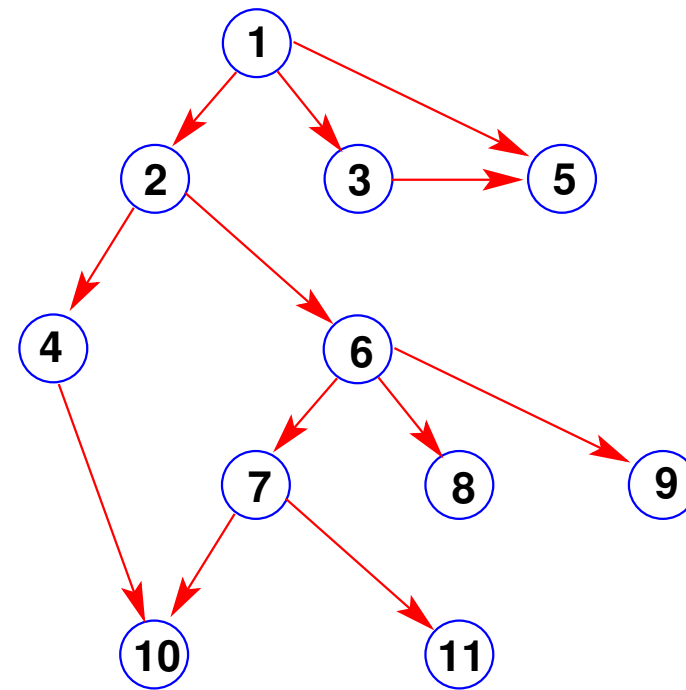
Sparse A and sparse b: Example

- Triangular system of previous example
- DAG shown in next figure
- Sets dependencies between tasks:
- Edge $i \rightarrow j$ means $a(j, i) = 1$ (j requires i)
- Root: node 1 (see right-hand side b)
- Topological sort: 1, 3, 5, 2, 6, 4 [as produced by a DFS from 1]
- In many cases, this leads to a short traversal



 6 Example: remove link $1 \rightarrow 2$ and redo

 7 Consider a triangular system with the following graph where b has nonzero entries in positions 3 and 7. (1) Progress of solution based on Topolog. sort; (2) Pattern of solution. (3) Verify pattern with matlab.



 8 Same questions if b has (only) a nonzero entry in position 1.

LU factorization from sparse triangular solves

- LU factorization built one column at a time. At step k :

We want:
$$\underbrace{L_k}_{n \times n} \underbrace{U_k}_{n \times k} = \underbrace{A_k}_{n \times k} \quad (\equiv A(1:n, 1:k))$$

$$\left[\begin{array}{ccc|c|ccc} 1 & & & & & & \\ * & 1 & & & & & \\ * & * & 1 & & & & \\ * & * & * & 1 & & & \\ * & * & * & ? & 1 & & \\ * & * & * & ? & & 1 & \\ * & * & * & ? & & & 1 \end{array} \right] \left[\begin{array}{ccc|c} x & x & x & ? \\ & x & x & ? \\ & & x & ? \\ & & & ? \\ & & & 0 \\ & & & 0 \\ & & & 0 \end{array} \right] = A_k$$

- In blue: has been determined. In red: to be determined

- Step 0: Set the terms $?$ in L_k to zero. Result $\equiv \tilde{L}_k$
- Step 1 : Solve $\tilde{L}_k w = a_k$ [Sparse $\tilde{L}_k$, sparse RHS]
- Step 2: set

$$u = \begin{array}{|c|} \hline w_1 \\ w_2 \\ \vdots \\ w_k \\ \hline 0 \\ \vdots \\ 0 \\ 0 \\ \hline \end{array} \quad z = \frac{1}{w_k} \begin{array}{|c|} \hline 0 \\ \vdots \\ 0 \\ 0 \\ \hline w_{k+1} \\ w_{k+2} \\ \vdots \\ w_n \\ \hline \end{array}$$

- Then $L_k U_k = A_k$ with

$$\underbrace{\left[\begin{array}{ccc|c|c} 1 & & & & \\ * & 1 & & & \\ * & * & 1 & & \\ * & * & * & 1 & \\ * & * & * & z_{k+1} & 1 \\ * & * & * & \vdots & \\ * & * & * & z_n & \end{array} \right]}_{L_k}; \quad \underbrace{\left[\begin{array}{ccc|c} x & x & x & u_1 \\ & x & x & u_2 \\ & & x & \vdots \\ & & & u_k \\ & & & 0 \\ & & & 0 \\ & & & 0 \end{array} \right]}_{U_k}$$

- Verification: Note $L_k = \tilde{L}_k + ze_k^T$; Also $\tilde{L}_k z = z$
- Must verify only $L_k U_k(:, k) = a_k$, i.e., $L_k u = a_k$

$$L_k u = (\tilde{L}_k + ze_k^T)u = \tilde{L}_k(I + ze_k^T)u = \tilde{L}_k(u + w_k z) = \tilde{L}_k w = a_k$$

- Key step: solve triangular system
- In sparse case: sparse triangular system with sparse right-hand side
- Use topological sorting at each step
- Scheme derived from this known as ‘left-looking’ sparse LU –
- Also known as ‘Gilbert and Peierls’ approach
- Reference: J. R. Gilbert and T. Peierls, Sparse partial pivoting in time proportional to arithmetic operations, SIAM J. Sci. Statist. Comput., 9 (1988), pp. 862-874

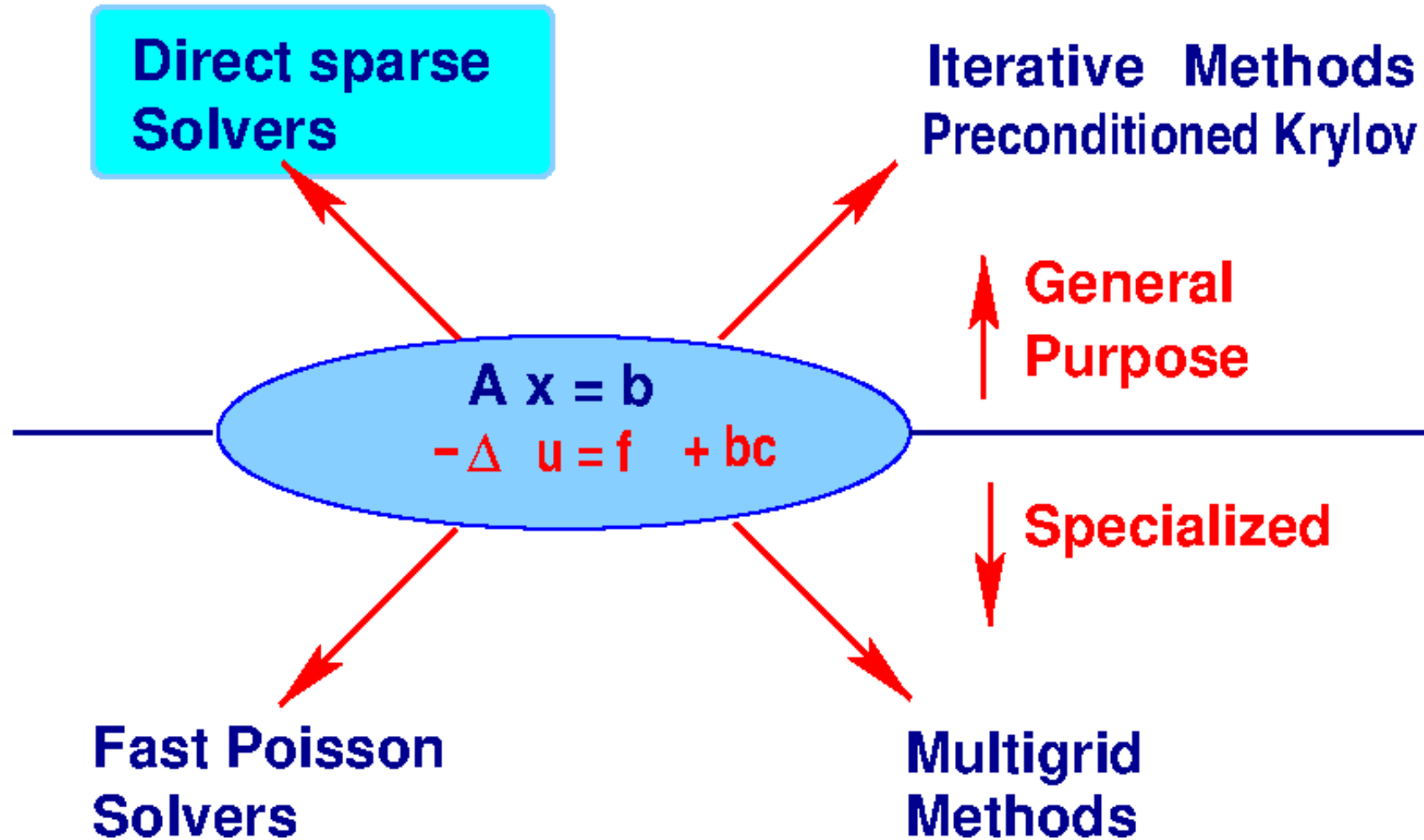
 Benefit of this approach: Partial pivoting is easy. Show how you would do it.

SPARSE GAUSSIAN ELIMINATION

(introduction)

- *Gaussian elimination - dense case*
- *Variants*
- *Main computational kernels*
- *Gaussian elimination in sparse case: a preview*
- *Graph model of elimination*

Solving sparse systems today



Background. Three types of methods:

- Direct methods : based on sparse Gaussian elimination, sparse Cholesky,..
- Iterative methods: compute a sequence of iterates which converge to the solution - preconditioned Krylov methods..
- Special purpose methods: Multigrid, Fast-Poisson solvers, ...

Remark: The first 2 classes of methods have always been in competition.

- 40 years ago solving a system with $n = 10,000$ was a challenge [Now you can solve this in a fraction of a second on a laptop]

Quotation from R. Varga's book on iterative methods [1962]

“As an example of the magnitude of problems that have been successfully solved by cyclic iterative methods, the Bettis Atomic Power Laboratory of the Westinghouse Electric Corporation had in daily use in 1960 a 2-dimensional program which would treat as a special case Laplacean-type matrix equations of order 20,000.”

He adds in footnote: (paraphrase) the program was written for the Philco-2000 computer which had 32,000 words of core storage (32Kwords!). “Even more staggering”: Bettis had a 3-D code which could treat coupled matrix equations of order 108,000.

- Sparse direct methods made huge gains in efficiency. They are very competitive for 2-D problems.
- 3-D problems lead to more challenging systems [inherent to the underlying graph]

Difficulty:

- No robust ‘black-box’ iterative solvers.

At issue Robustness in conflict with efficiency.

- Iterative methods are starting to use some of the tools of direct solvers to gain ‘robustness’

Gaussian Elimination. Variants

Recall: Gaussian Elimination has 3 nested loops.

- The three loop indices are denoted by k, i, j
- We can order each of the loops differently.
- A total of 6 different algorithms [more if we add blocking]: $kij, kji, ijk, ikj, jki, jik$.
- **IMPORTANT:** these algorithms are equivalent. Same operations are done in a different order
- Therefore same operation counts



Find the analogous algorithms for Matrix-matrix multiplication

For $k = 1 : n - 1$ Do:

For $i = k + 1 : n$ Do:

$a_{ik} := a_{ik} / a_{kk}$

For $j = k + 1 : n$ Do

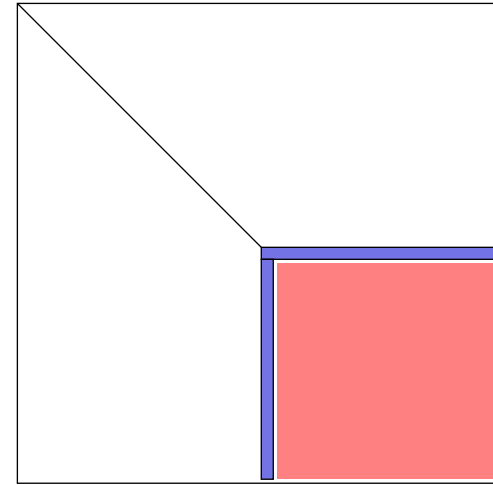
$a_{ij} := a_{ij} - a_{ik} * a_{kj}$

End

End

End

KIJ



- This is the **KIJ** variant of GE [hint: read the loop indices]
- Also known as the “Outer product” form
- Main drawback: rank-one update matrix at each step.
- Can also have a KJI version [flip loops i and j]

For $k = 1 : n - 1$ Do:

For $i := k + 1 : n$ Do

$a_{ik} := a_{ik} / a_{kk}$

EndDo

For $j = k + 1 : n$ Do:

For $i := k + 1 : n$ Do

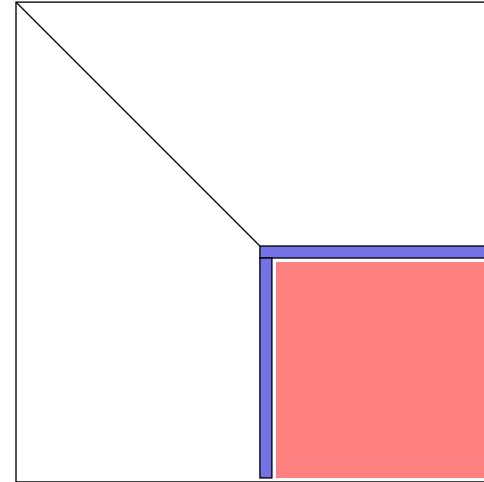
$a_{ij} := a_{ij} - a_{ik} * a_{kj}$

End

End

End

KJI



- Can we swap loops k and j [‘delay the k-loop’]
- Consider first the KIJ (row) version [flip loops k and i]

Gaussian Elimination. IKJ variant

For $i = 2, \dots, n$ Do:

For $k = 1, \dots, i - 1$ Do:

$$a_{ik} := a_{ik} / a_{kk}$$

For $j = k + 1, \dots, n$ Do:

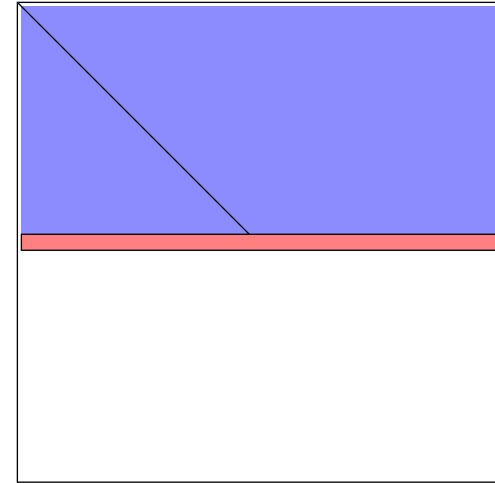
$$a_{ij} := a_{ij} - a_{ik} * a_{kj}$$

EndDo

EndDo

EndDo

IKJ

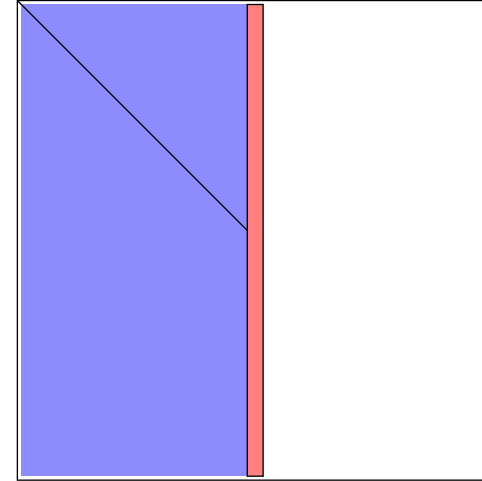


- Also known as the ‘up-looking LU’
- Row-oriented ‘delayed update’ algorithm
- Can also have a column version [very common]

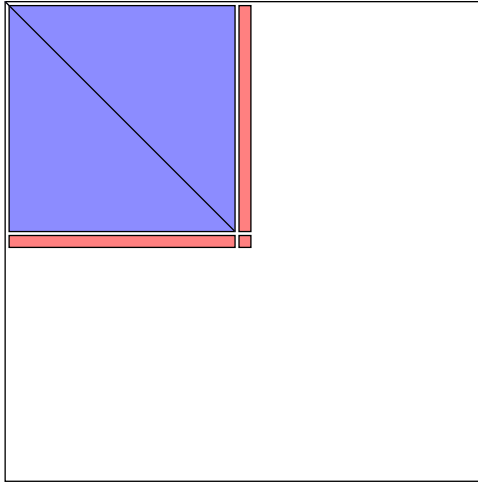
Gaussian Elimination. JKI or 'Left-looking LU'

```
for j=1:n
    for k=1:j-1
        A(k+1:n,j) = A(k+1:n,j) - ...
            A(k,j)*A(k+1:n,k);
    end
    // i loop:
    A(j+1:n,j) = A(j+1:n,j) / A(j,j);
end
```

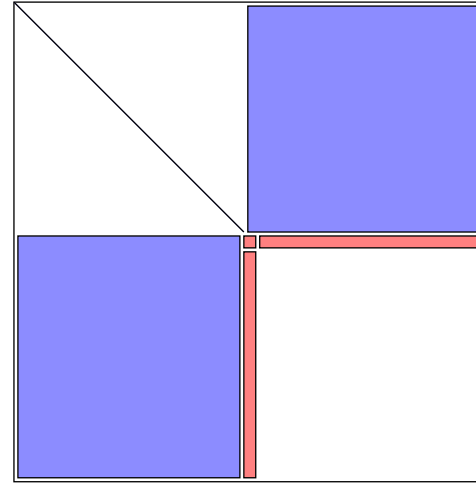
JKI



Other variants



IJK variant (dot product)

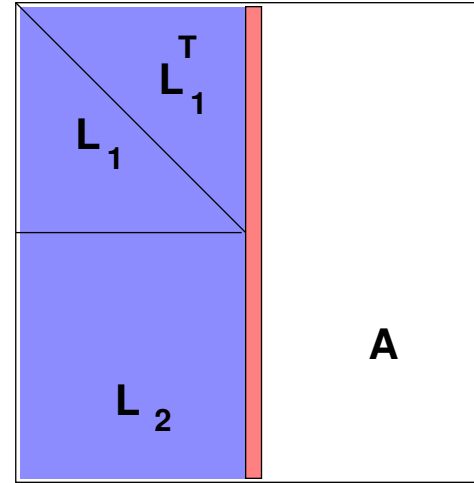


Crout

- Also: corresponding variants of Cholesky – We will only look at the adaptation of the JKI variant of Cholesky..

Column Cholesky (compare with JKI – Gauss)

```
for j=1:n
  for k=1:j-1
    A(j:n,j)=A(j:n,j)-A(j:n,k)*A(j,k);
  end
  A(j,j) = sqrt(A(j,j));
  A(j+1:n,j) = A(j+1:n,j)/A(j,j);
end
```



➤ Column j of A [L-part] becomes column j of L .

- We will often consider Sparse Cholesky because: 1) the SPD case is important; 2) certain aspects are simpler than Gauss; 3) Generalizations are easy..
- Sparse Column **Cholesky**: same as above algorithm but implemented in sparse mode

Number of operations. The total number of multiplications required to compute the Cholesky factor L of a matrix A is given by

$$\sum_{k=1}^{n-1} (\eta_k^2 - 1)$$

where η_k is the number of nonzero entries in the k -th column of L

Proof:

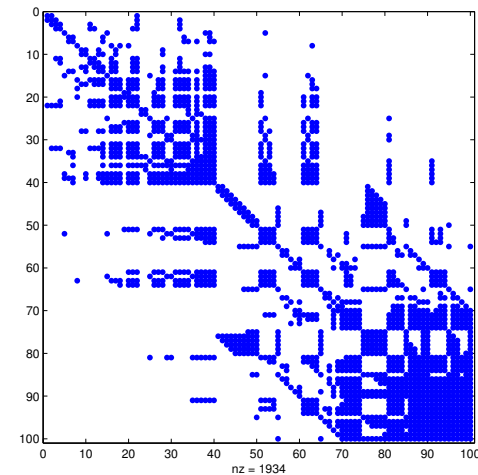
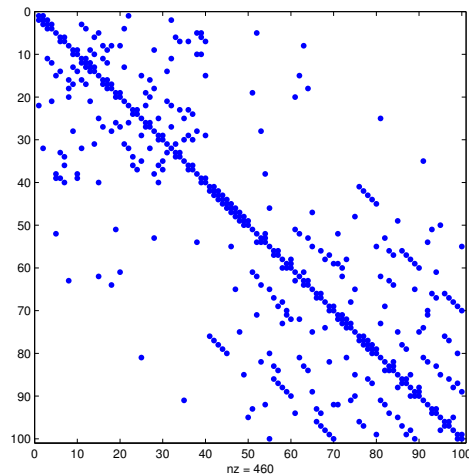
- Consider only the KIJ version of Cholesky which is equivalent.
- Rank-1 update at each step is $A^{(k)} = A^{(k-1)} - v_k v_k^T$, where [using matlab notation]

$$v_k = [\text{zeros}(1, k), A^{(k-1)}(k, k+1 : n)]^T$$

- Only lower part is done - so cost is $(\eta_k - 1)\eta_k$.
- Must add $\eta_k - 1$ scaling operations (mult. by inverse). Total: $\eta_k^2 - 1$ ■
- OK – but η_k 's not known in advance. Dense case $\eta_k = n - k + 1$
- Storage: $\sum_{k=1}^n \eta_k$

Sparse solvers

- Need to develop sparse versions: sparse operations
- Major new consideration: Fill-in.



- These variants will lead to different computation kernels

Gaussian Elimination. Graph model

- What happens to the graph when a node is eliminated?

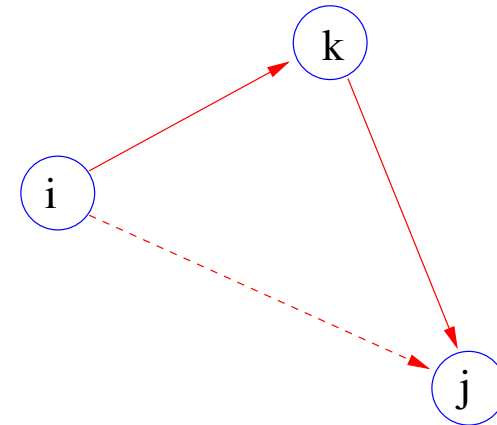
Eliminating a node x in Gaussian elimination amounts to

1. Deleting node x and its adjacent edges from the graph
2. Adding edges between any two nodes that were adjacent to x .

- Notation: $A^{(k)}$ = matrix at k -th step of elimination

Another viewpoint: At step k of Gaussian Elimination, a fill-in is introduced in position (i, j) iff

$$a_{ik}^{(k-1)} \neq 0 \ \& \ a_{kj}^{(k-1)} \neq 0$$



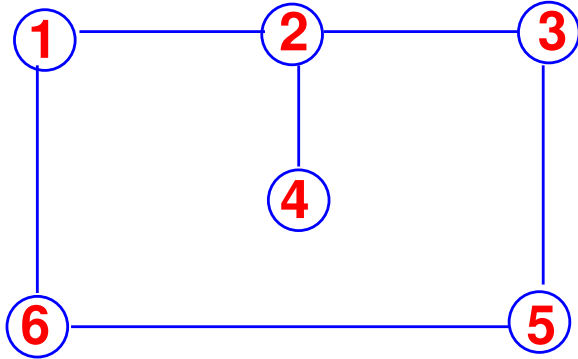
A seminal paper:

S. Parter, “*The use of linear graphs in Gauss elimination*”, SIAM review, vol. 3, (1961), pp. 119-130.

- Gave a major impetus to the use of the graph theory approach to sparse matrix techniques.
- Foundation for some of the major ideas (e.g. elimination trees) to come even much later.

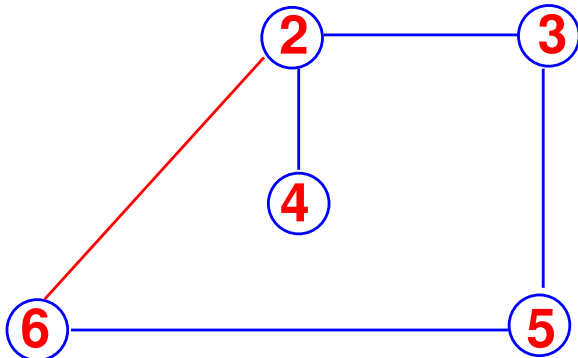
Example:

Gaussian elimination steps for following graph



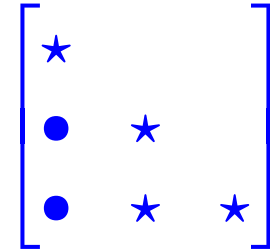
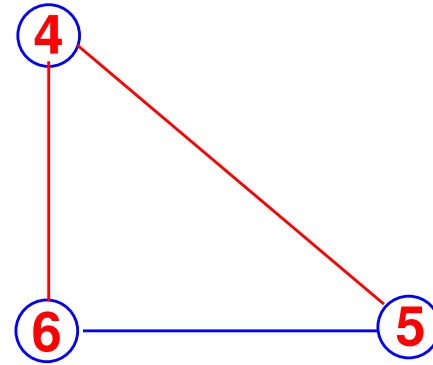
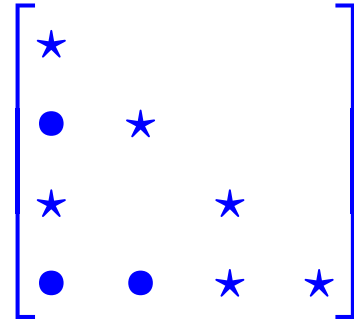
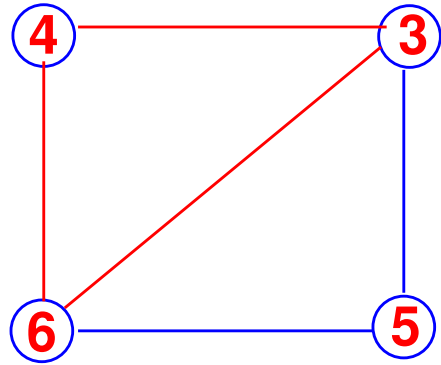
$$\begin{bmatrix} \star & \star & & & & \star \\ \star & \star & \star & \star & & \\ & \star & \star & & \star & \\ & \star & & \star & & \\ & & \star & & \star & \star \\ \star & & & & \star & \star \end{bmatrix}$$

Eliminate 1:



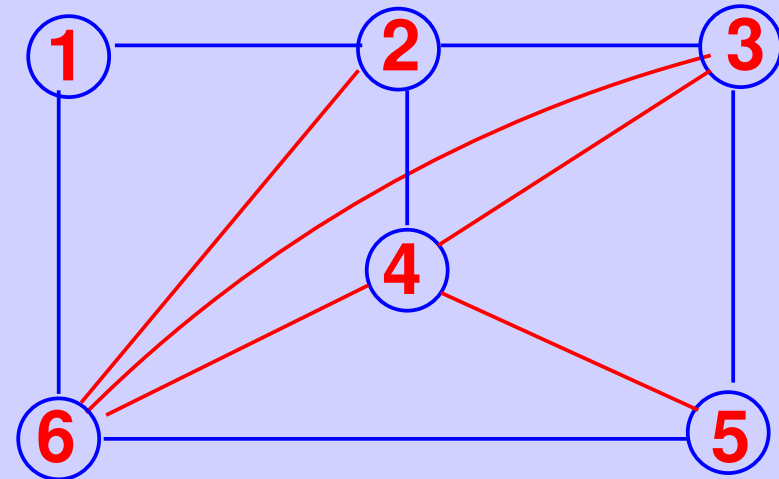
$$\begin{bmatrix} \star & & & & \\ \star & \star & & & \\ \star & & \star & & \\ & \star & & \star & \\ \bullet & & & \star & \star \end{bmatrix}$$

Eliminate 2 and then 3:



➤ Two more steps [omitted - they involve no fill-in]

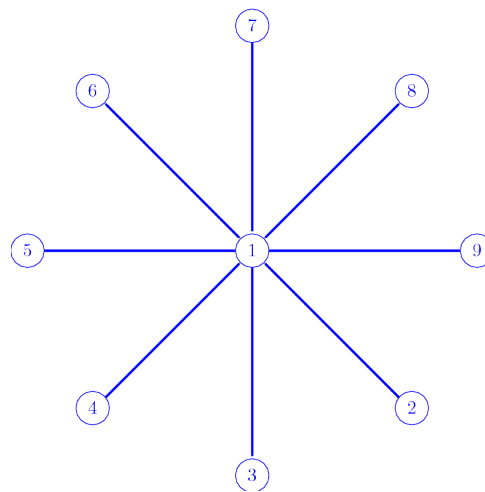
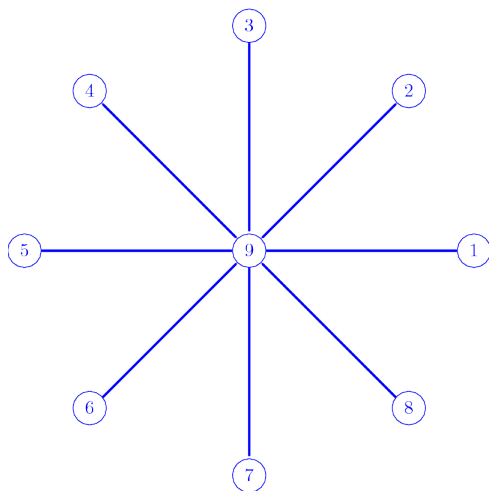
Filled graph = final graph including initial graph and all fill edges. Notation $G^F = (V, E^F)$. Note: $|E^F|$ measures the memory required for GE to solve the problem.



Gaussian Elimination. The fill path theorem

➤ A **Fill-path** is a path between two vertices i and j in the graph of A such that all vertices in the path, except the end points i and j , are numbered less than i and j .

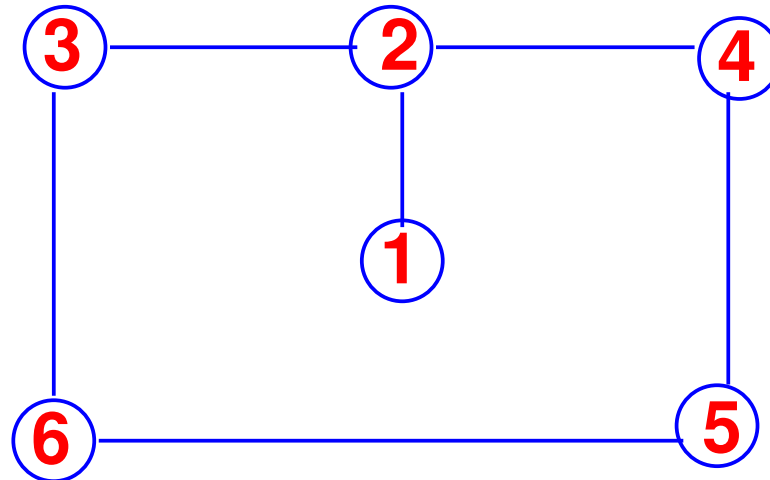
 2 What are all the fill-paths for the two examples below



THEOREM [Rose-Tarjan] There is a fill-in in entry (i, j) at the completion of the Gaussian elimination process **if and only if**, there exists a fill-path between i and j .

➤ Example of application: Separating a graph $\equiv$ finding 3 sets of vertices: V_1, V_2, S such that $V = V_1 \cup V_2 \cup S$ and V_1 and V_2 have no couplings. Labeling nodes of S last prevents fill-ins between nodes of V_1 and V_2 .

 3 What are all the fill-edges for the previous examples (star graphs)



 4 Fill-edges for:

REORDERINGS FOR FILL-REDUCTION

- *Permutations and reorderings - graph interpretations*
- *Band-reduction orderings: Cuthill-Mc Kee, Reverse Cuthill Mc Kee*
- *Profile/envelope methods. Profile reduction.*
- *Multicoloring and independent sets [for iterative methods]*
- *Minimal degree ordering*
- *Nested Dissection*

Reorderings and graphs

- Let $\pi = \{i_1, \dots, i_n\}$ a permutation
- $A_{\pi,*} = \{a_{\pi(i),j}\}_{i,j=1,\dots,n}$ = matrix A with its i -th row replaced by row number $\pi(i)$.
- $A_{*,\pi}$ = matrix A with its j -th column replaced by column $\pi(j)$.
- Define $P_\pi = I_{\pi,*}$ = “Permutation matrix” – Then:

- (1) Each row (column) of P_π consists of zeros and exactly one “1”
- (2) $A_{\pi,*} = P_\pi A$
- (3) $P_\pi P_\pi^T = I$
- (4) $A_{*,\pi} = A P_\pi^T$

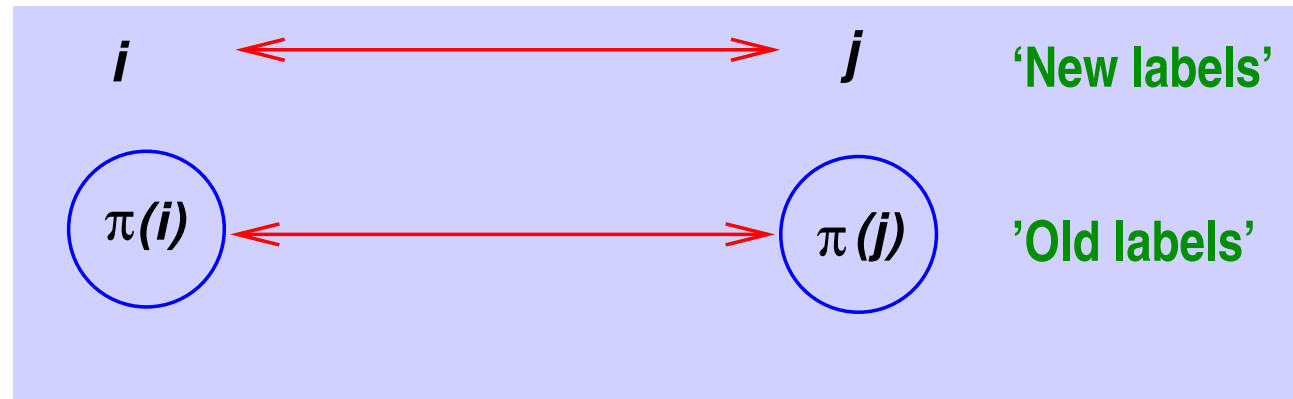
Consider now:

$$A' = A_{\pi,\pi} = P_{\pi} A P_{\pi}^T$$

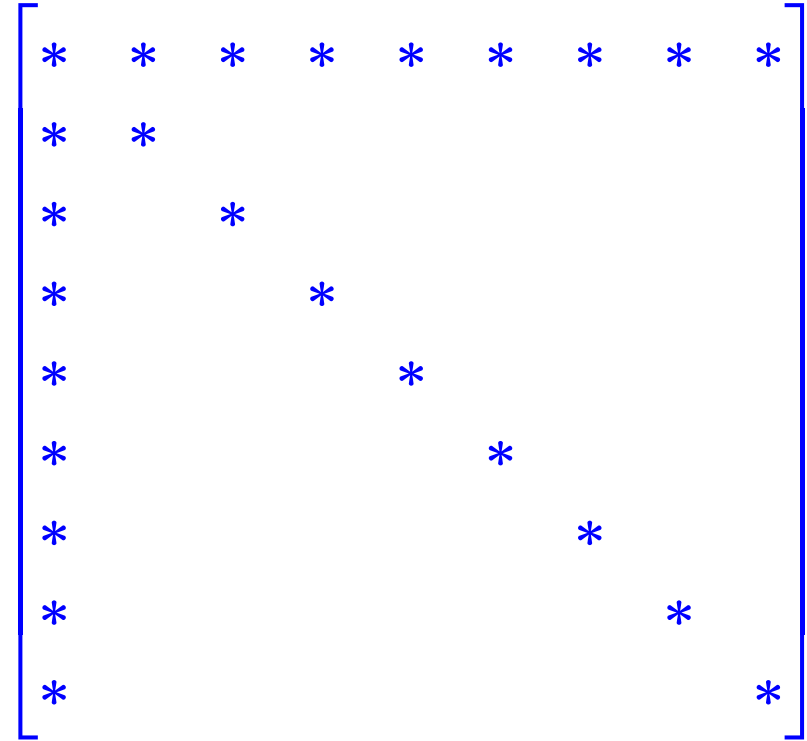
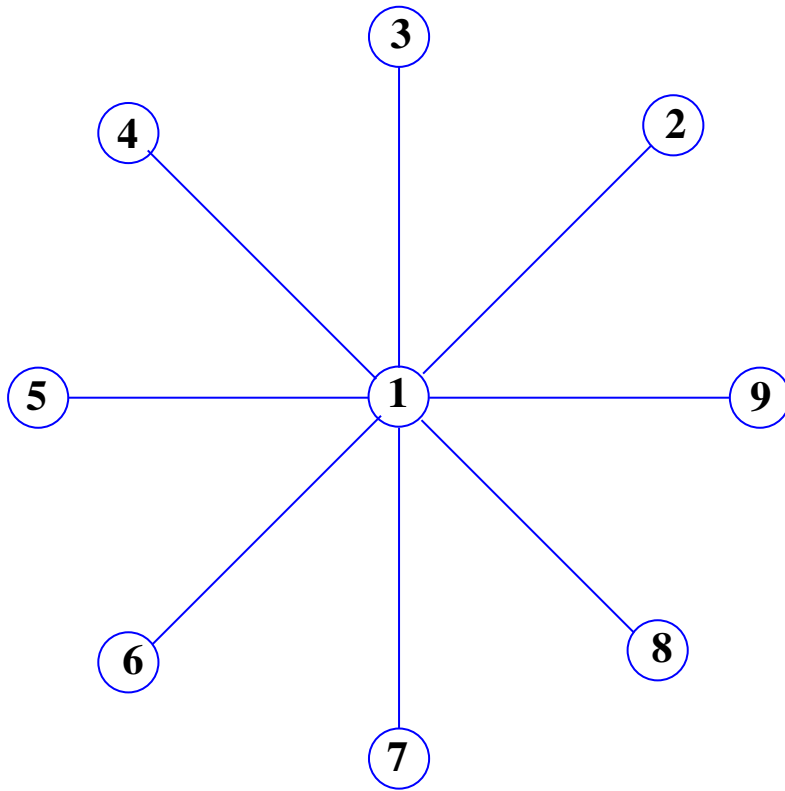
- Element in position (i, j) in matrix A' is exactly element in position $(\pi(i), \pi(j))$ in A . ($a'_{ij} = a_{\pi(i), \pi(j)}$)

$$(i, j) \in E_{A'} \iff (\pi(i), \pi(j)) \in E_A$$

- General Picture:

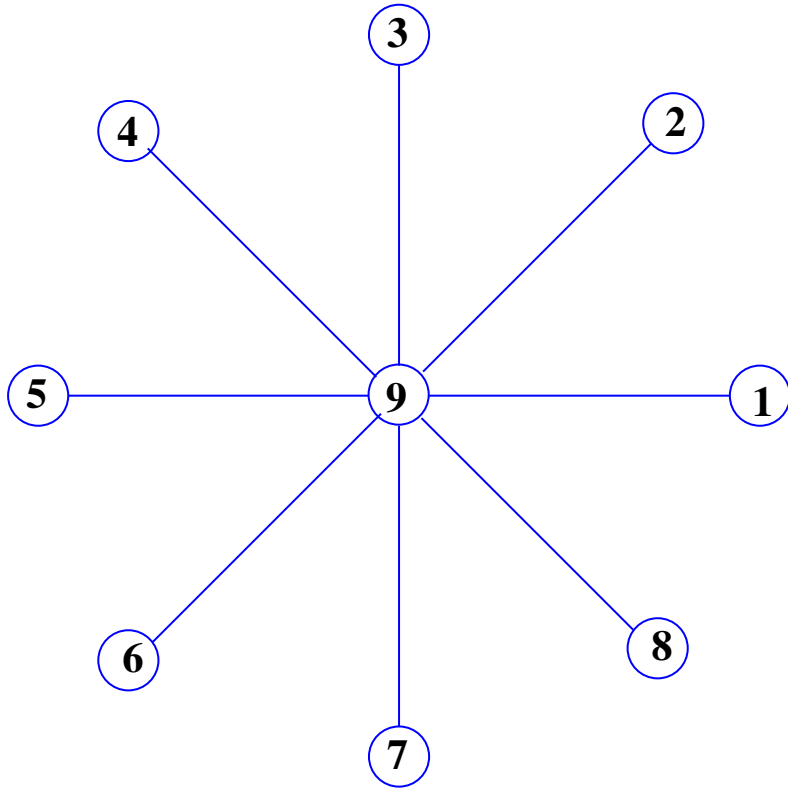


Example: A 9×9 'arrow' matrix and its adjacency graph.



 1 Fill-in?

➤ Graph and matrix after swapping nodes 1 and 9:



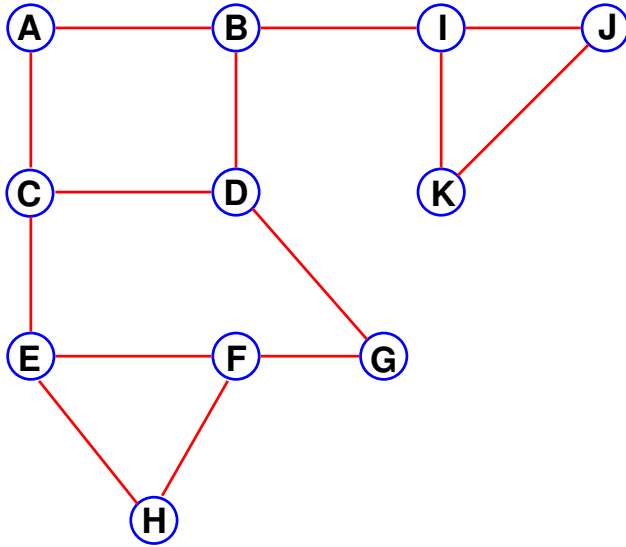
$$\begin{bmatrix} * & & & & & & & & * \\ & * & & & & & & & * \\ & & * & & & & & & * \\ & & & * & & & & & * \\ & & & & * & & & & * \\ & & & & & * & & & * \\ & & & & & & * & & * \\ & & & & & & & * & * \\ * & * & * & * & * & * & * & * & * \end{bmatrix}$$

 2 Fill-in?

The Cuthill-McKee and its reverse orderings

- A class of reordering techniques which proceed by levels in the graph.
- Related to **Breadth First Search** (BFS) traversal in graph theory.
- Idea of BFS is to visit the nodes by 'levels'. Level 0 = level of starting node.
- Start with a node, visit its neighbors, then the (unmarked) neighbors of its neighbors, etc...

Example:



h	Queue
A	A* B, C
B	A, B* C, D, I
C	A, B, C* D, I, E
D	A, B, C, D* I, E, G
I	A, B, C, D, I* E, G, J, K
E	A, B, C, D, I, E* G, J, K, F, H
G	...
E	A, B, C, D, I, E, G, J, K, F, H*

A,	B, C,	D, I, E,	G, J, K, F, H
Level 0	Level 1	Level 2	Level 3

➤ Final traversal order:

- Levels represent distances from the root
- Algorithm can be implemented by crossing levels 1,2, ...
- More common: Queue implementation

Algorithm $BFS(G, v)$ – Queue implementation

- Initialize: $Queue := \{v\}$; Mark v ; $ptr = 1$;
- While $ptr < length(Queue)$ do
 - $head = Queue(ptr)$;
 - ForEach Unmarked $w \in Adj(head)$:
 - * Mark w ;
 - * Add w to Queue: $Queue = \{Queue, w\}$;
 - $ptr ++$;

```

function [p] = bfs(A,init )
%% BFS traversal. queue implementation
%%----- enqueue first node
p=[init];
n = size(A,1);
mask = zeros(n,1);
mask(init) = 1;
%%----- main loop
for h=1:n
%%----- scan nodes in adj(p(h))
    [ii, jj, rr] = find(A(:,p(h)));
    for v=ii'
        if (mask(v)==0)
            p = [p, v] ;
            mask(v) = 1;
        end
    end
end
end

```

A few properties of Breadth-First-Search

- If G is a connected undirected graph then each vertex will be visited once; each edge will be inspected at least once
- Therefore, for a connected undirected graph,

The cost of BFS is $O(|V| + |E|)$

- Distance = level number; ➤ For each node v we have:

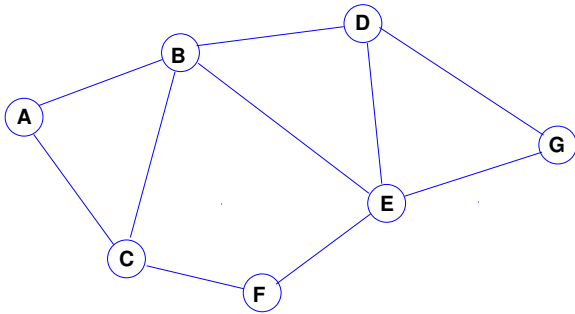
$$\textit{min_dist}(s, v) = \textit{level_number}(v) = \textit{depth}_T(v)$$

- Several reordering algorithms are based on variants of Breadth-First-Search

Cuthill McKee ordering

Same as BFS except: $\text{Adj}(\text{head})$ always sorted by increasing degree

Example:



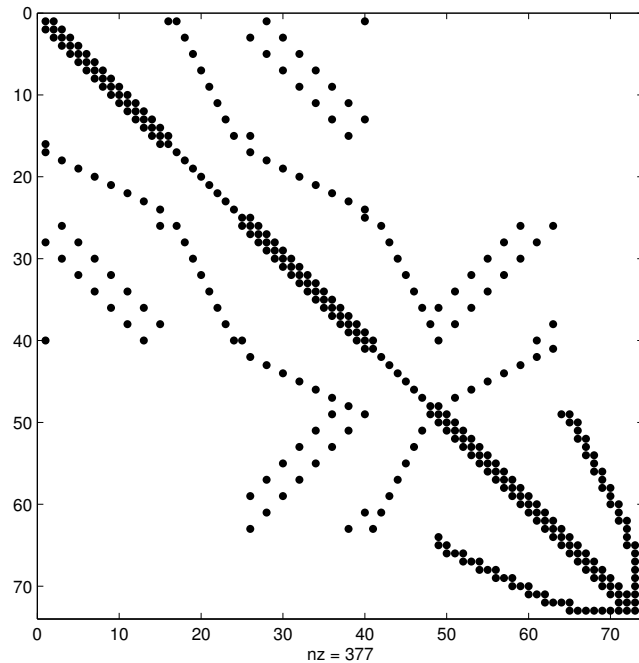
A	C(3) B(4)
A, C	B, F(2)
A, C, B	F, D(3), E(4)
A, C, B, F	D, E
A, C, B, F, D	E, G(2)
A, C, B, F, D, E	G
A, C, B, F, D, E, G	

Rule: when adding nodes to the queue list them in $\uparrow$ deg.

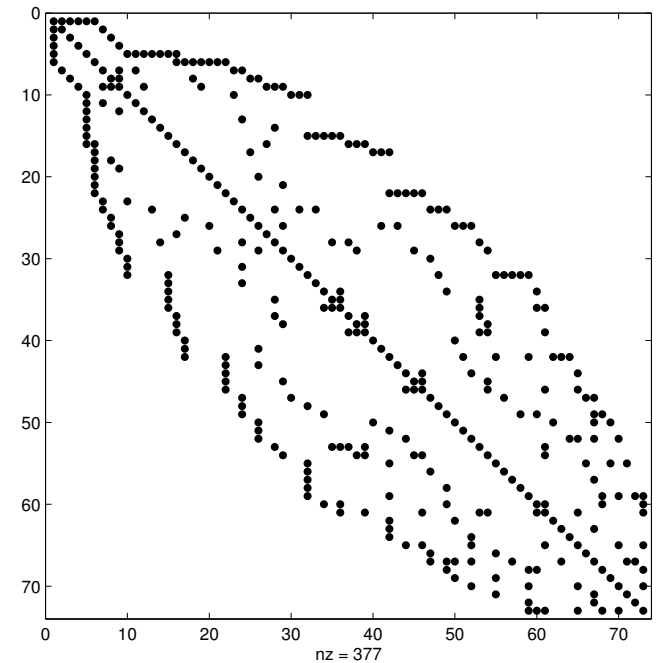
Reverse Cuthill McKee ordering

- The Cuthill - Mc Kee ordering has a tendency to create small arrow matrices (going the wrong way):

Original matrix

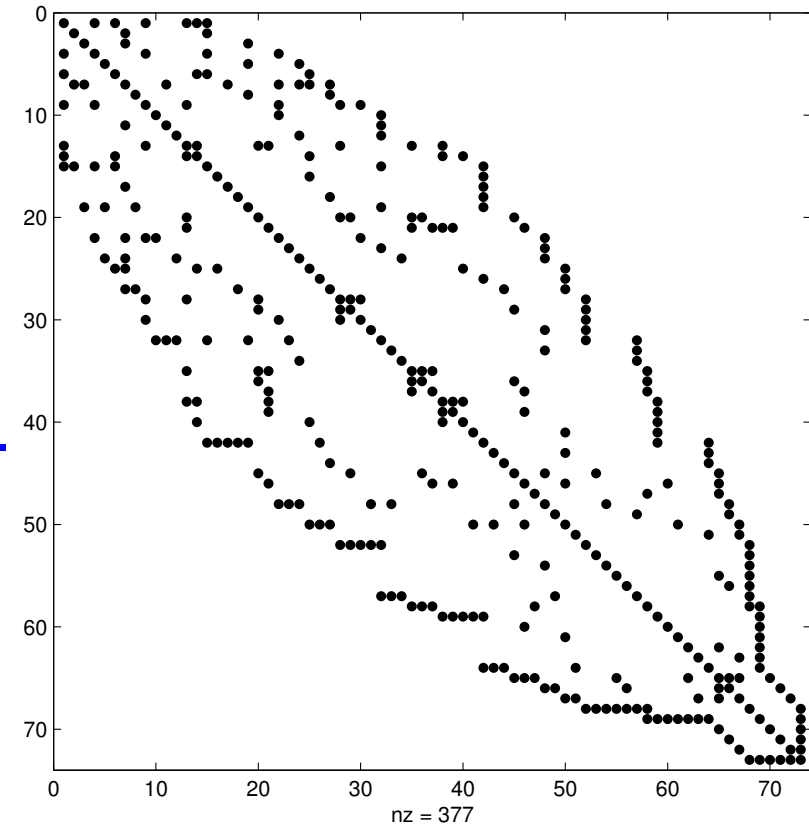


CM ordering



RCM ordering

- Idea: Take the reverse ordering
- Reverse Cuthill M Kee ordering (RCM).



Envelope/Profile methods

Many terms used for the same methods: Profile, Envelope, Skyline, ...

- Generalizes band methods
- Consider only the symmetric (in fact SPD) case
- Define bandwith of row i . (“ i -th bandwidth of A):

$$\beta_i(A) = \max_{j \leq i; a_{ij} \neq 0} |i - j|$$

Definition: Envelope of A is the set of all pairs (i, j) such that $0 < i - j \leq \beta_i(A)$. The quantity $|Env(A)|$ is called profile of A .

Main result The envelope is preserved by GE (no-pivoting)

Theorem: Let $A = LL^T$ the Cholesky factorization of A . Then

$$Env(A) = Env(L + L^T)$$

➤ An envelope / profile/ Skyline method is a method which treats any entry a_{ij} , with $(i, j) \in Env(A)$ as nonzero.

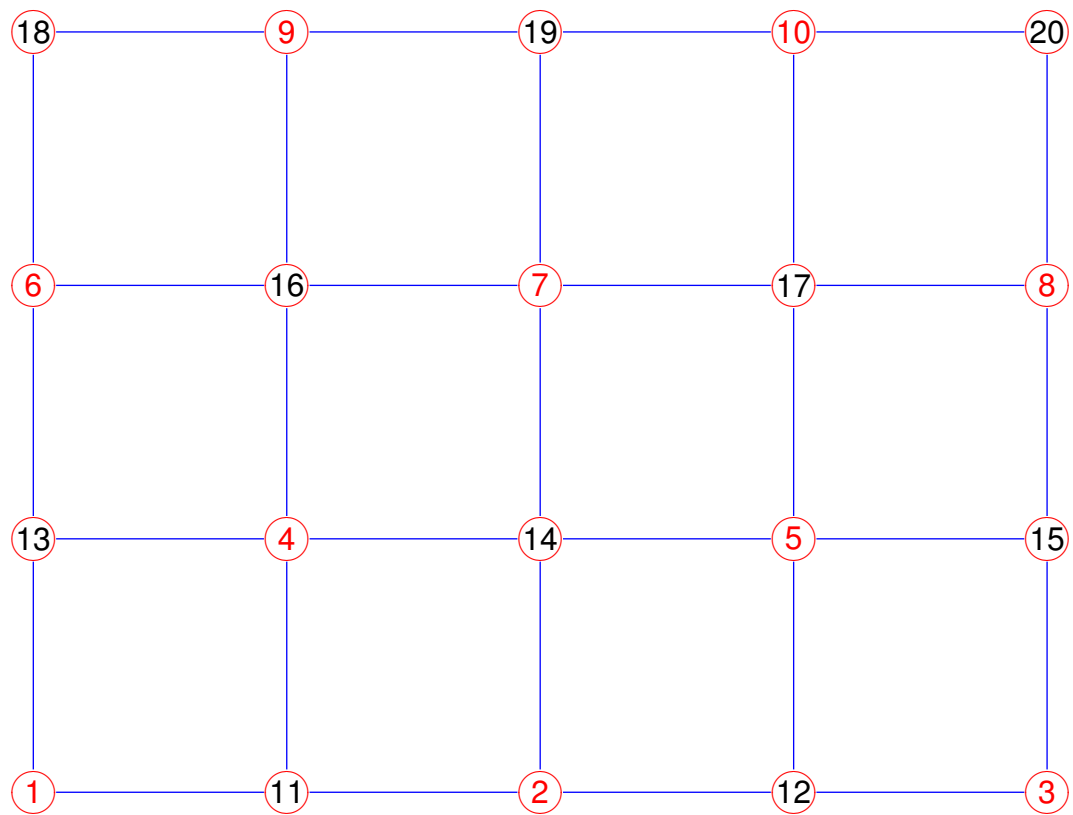
Matlab test: do the following

1. Generate `A = Lap2D(64, 64)`
2. Compute `R = chol(A)`
3. show `nnz(R)`
4. Compute RCM permutation (sym-rcm)
5. Compute `B = A(p, p)`
6. `spy(B)`
7. compute `R1 = chol(B)`
8. Show `nnz(R)`
9. `spy(R1)`

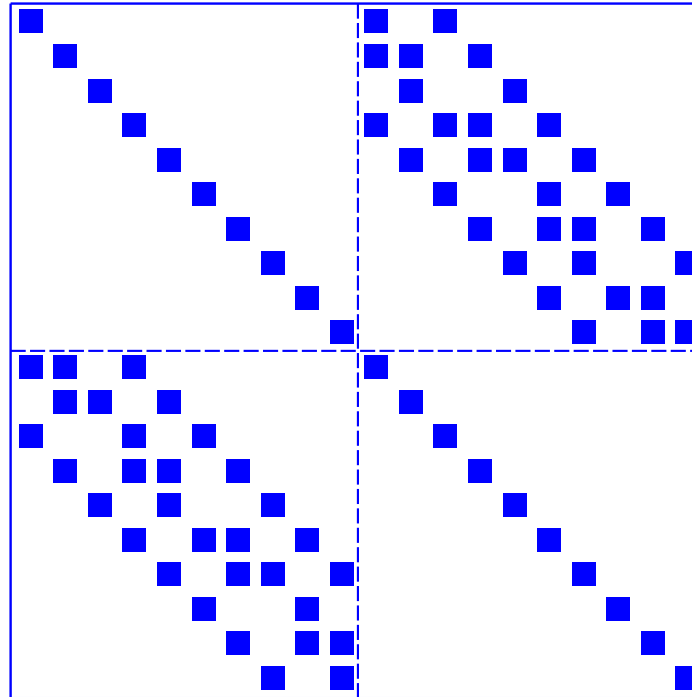
Orderings for parallelism: Multicoloring

- General technique that can be exploited in many different ways to introduce parallelism – generally of order N .
- Constitutes one of the most successful techniques for introducing vector computations for iterative methods..
- Want: assign colors so that no two adjacent nodes have the same color.

Simple example: Red-Black ordering.



Corresponding matrix



- Observe: L-U solves with lower and upper parts of A will require only diagonal scalings + matrix-vector products with matrices of size $N/2$.

How to generalize Red-Black ordering?

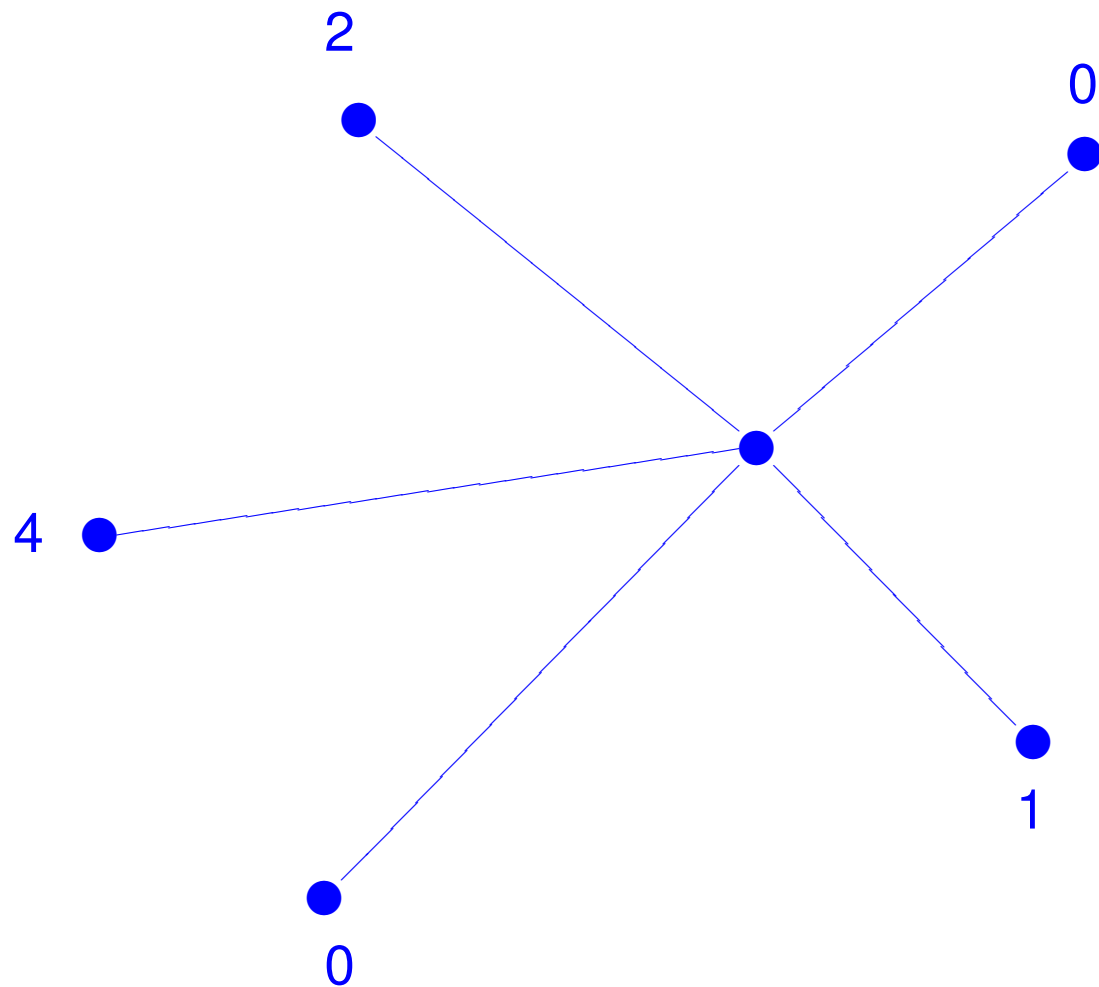
Answer: **Multicoloring** & **independent sets**

A greedy multicoloring technique:

- Initially assign color number zero (uncolored) to every node.
- Choose an order in which to traverse the nodes.
- Scan all nodes in the chosen order and at every node i do

$$Color(i) = \min\{k \neq 0 \mid k \neq Color(j), \forall j \in Adj(i)\}$$

$Adj(i)$ = set of nearest neighbors of $i = \{k \mid a_{ik} \neq 0\}$.



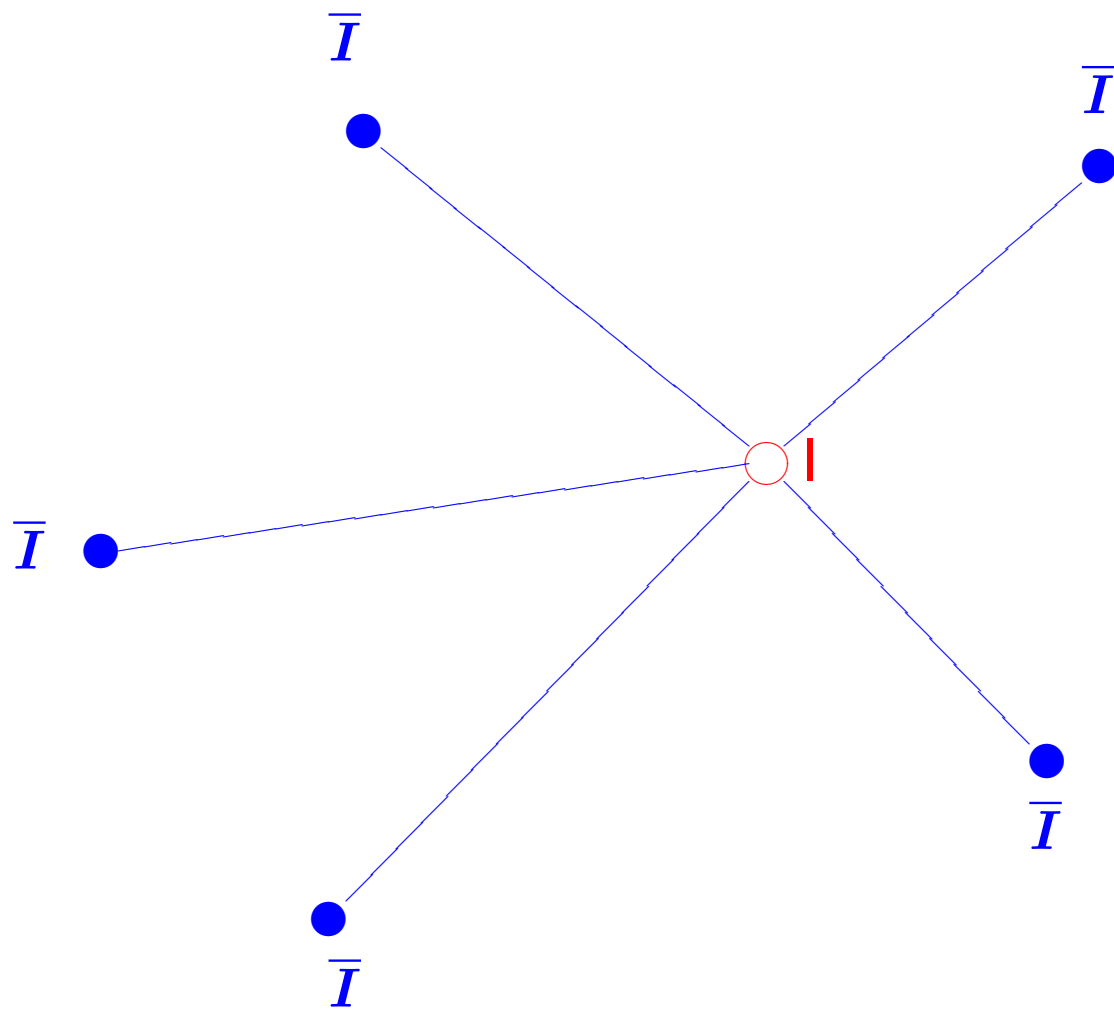
Independent Sets

An independent set (IS) is a set of nodes that are not coupled by an equation. The set is maximal if all other nodes in the graph are coupled to a node of IS. If the unknowns of the IS are labeled first, then the matrix will have the form:

$$\begin{bmatrix} B & F \\ E & C \end{bmatrix}$$

in which B is a diagonal matrix, and E , F , and C are sparse.

Greedy algorithm: Scan all nodes in a certain order and at every node i do: if i is not colored color it **Red** and color all its neighbors **Black**. Independent set: set of red nodes. Complexity: $O(|E| + |V|)$.



 3 Show that the size of the independent set I is such that

$$|I| \geq \frac{n}{1 + d_I}$$

where d_I is the maximum degree of each vertex in I (not counting self cycle).

 4 According to the above inequality what is a good (heuristic) order in which to traverse the vertices in the greedy algorithm?

 5 Are there situations when the greedy algorithm for independent sets yield the same sets as the multicoloring algorithm?

Orderings used in direct solution methods

- Two broad types of orderings used:
 - Minimal degree ordering + many variations
 - Nested dissection ordering + many variations
- Minimal degree ordering is easiest to describe:

At each step of GE, select next node to eliminate, as the node v of smallest degree. After eliminating node v , update degrees and repeat.

Minimal Degree Ordering

At any step i of Gaussian elimination define for any candidate pivot row j

$$Cost(j) = (nz_c(j) - 1)(nz_r(j) - 1)$$

where $nz_c(j)$ = number of nonzero elements in column j of 'active' matrix,
 $nz_r(j)$ = number of nonzero elements in row j of 'active' matrix.

- Heuristic: fill-in at step j is $\leq cost(j)$
- Strategy: select pivot with minimal cost.
- Local, greedy algorithm
- Good results in practice.

Many improvements made over the years

- Alan George and Joseph W-H Liu, THE EVOLUTION OF THE MINIMUM DEGREE ORDERING ALGORITHM, SIAM Review, vol 31 (1989), pp. 1-19.

Min. Deg. Algorithm	Storage (words)	Order. time
Final min. degree	1,181 K	43.90
Above w/o multiple elimn.	1,375 K	57.38
Above w/o elimn. absorption	1,375 K	56.00
Above w/o incompl. deg. update	1,375 K	83.26
Above w/o indistinguishable nodes	1,308 K	183.26
Above w/o mass-elimination	1,308 K	2289.44

➤ Results for a 180×180 9-point mesh problem

- Since this article, many important developments took place.
- In particular the idea of “Approximate Min. Degree” and “Approximate Min. Fill”, see
 - E. Rothberg and S. C. Eisenstat, NODE SELECTION STRATEGIES FOR BOTTOM-UP SPARSE MATRIX ORDERING, SIMAX, vol. 19 (1998), pp. 682-695.
 - Patrick R. Amestoy, Timothy A. Davis, and Iain S. Duff. AN APPROXIMATE MINIMUM DEGREE ORDERING ALGORITHM. SIAM Journal on Matrix Analysis and Applications, 17 (1996), pp. 886-905.

Practical Minimal degree algorithms

First Idea: Use quotient graphs

- * Avoids elimination graphs which are not economical
- * Elimination creates **cliques**
- * Represent each clique by a node termed an *element* (recall FEM methods)
- * No need to create fill-edges and elimination graph
- * Still expensive: updating the degrees

Second idea: Multiple Minimum degree

- * Many nodes will have the same degree. Idea: eliminate many of them **simultaneously** –
- * Specifically eliminate **independent sets** of nodes with same degree.

Third idea: Approximate Minimum degree

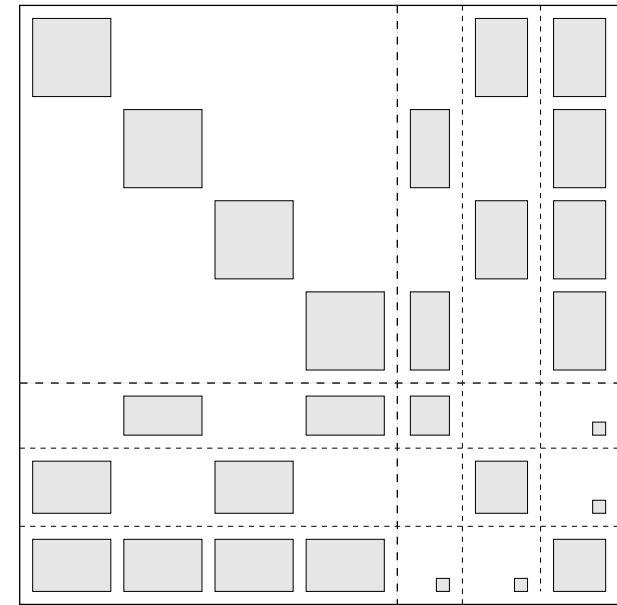
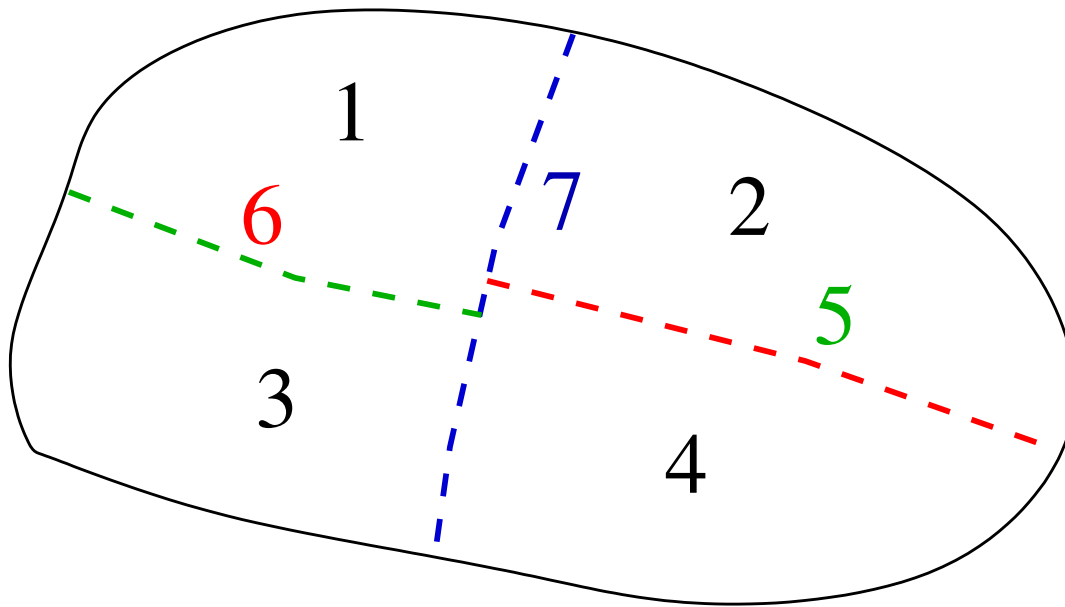
- * Degree updates are expensive –
- * Goal: To save time.
- * Approach: only compute an approximation (upper bound) to degrees.
- * Details are complex and can be found in Tim Davis' book

 6 Explore *symamd* and *amd* in matlab

Nested Dissection Reordering (Alan George)

- Computer science 'Divide-and-Conquer' strategy.
- Best illustration: PDE finite difference grid.
- Easily described by using recursivity and by exploiting 'separators': 'separate' the graph in three parts, two of which have no coupling between them. The 3rd set ('the separator') has couplings with vertices from both of the first 2 sets.
- Key idea: dissect the graph; take the subgraphs and dissect them recursively.
- Nodes of separators always labeled last after those of the parents

Nested dissection ordering: illustration

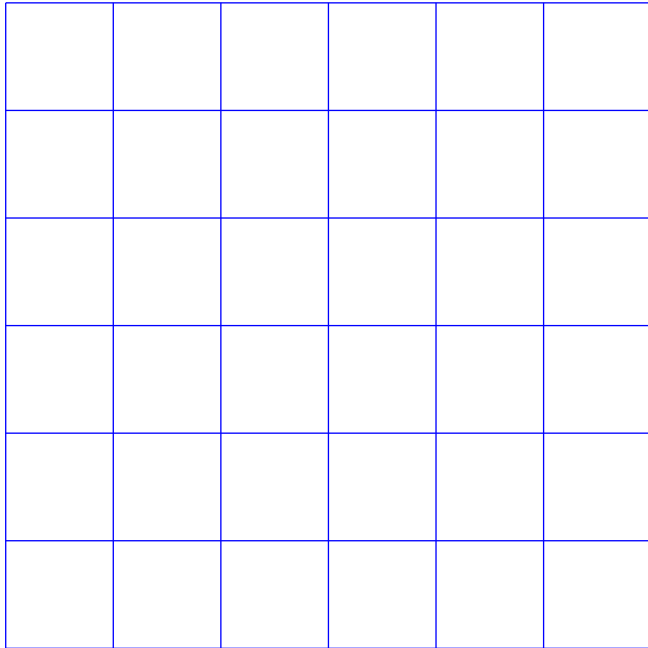


➤ For regular $n \times n$ meshes, can show: fill-in is of order $n^2 \log n$ and computational cost of factorization is $O(n^3)$

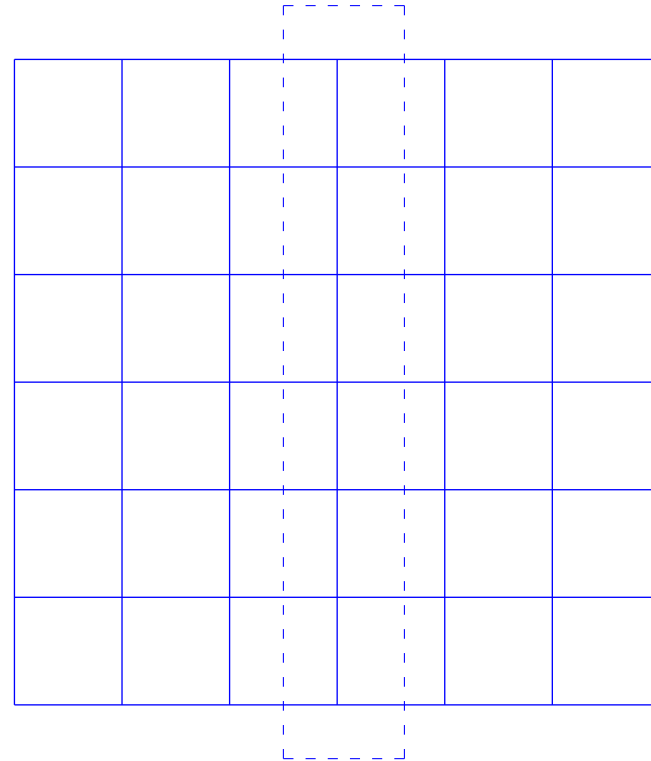
 7 How does this compare with a standard band solver?

Nested dissection for a small mesh

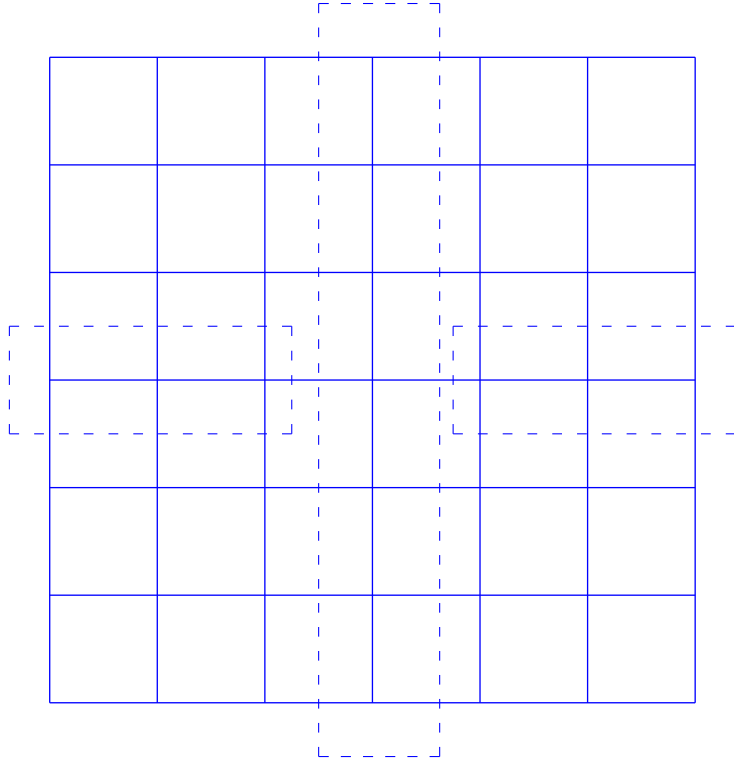
Original Grid



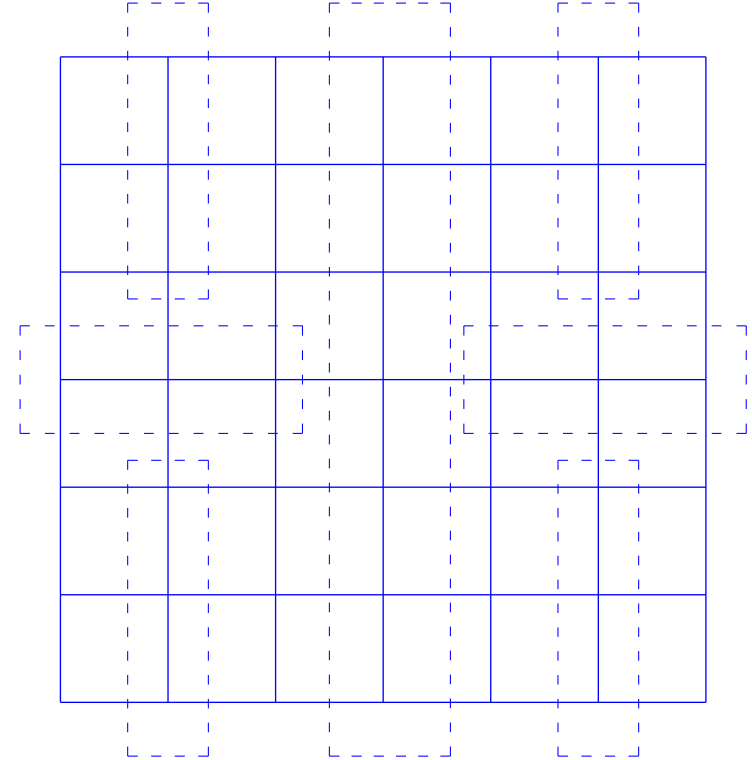
First dissection



Second Dissection



Third Dissection



Nested dissection: cost for a regular mesh

- In 2-D consider an $n \times n$ problem, $N = n^2$
- In 3-D consider an $n \times n \times n$ problem, $N = n^3$

	2-D	3-D
space (fill)	$O(N \log N)$	$O(N^{4/3})$
time (flops)	$O(N^{3/2})$	$O(N^2)$

- Significant difference in complexity between 2-D and 3-D

Nested dissection and separators

- Nested dissection methods depend on finding a good graph separator: $V = T_1 \cup UT_2 \cup S$ such that the removal of S leaves T_1 and T_2 disconnected.
 - Want: S small and T_1 and T_2 of about the same size.
 - Simplest version of the graph partitioning problem.
- A theoretical result:* If G is a planar graph with N vertices, then there is a separator S of size $\leq \sqrt{N}$ such that $|T_1| \leq 2N/3$ and $|T_2| \leq 2N/3$.
- In other words “Planar graphs have $O(\sqrt{N})$ separators”
- Many techniques for finding separators: Spectral, iterative swapping (K-L), multilevel (Metis), BFS, ...

SPARSE DIRECT METHODS

- *Building blocks for sparse direct solvers*
- *SPD case. Sparse Column Cholesky/*
- *Elimination Trees - Symbolic factorization*

Direct Sparse Matrix Methods

Problem addressed: Linear systems

$$Ax = b$$

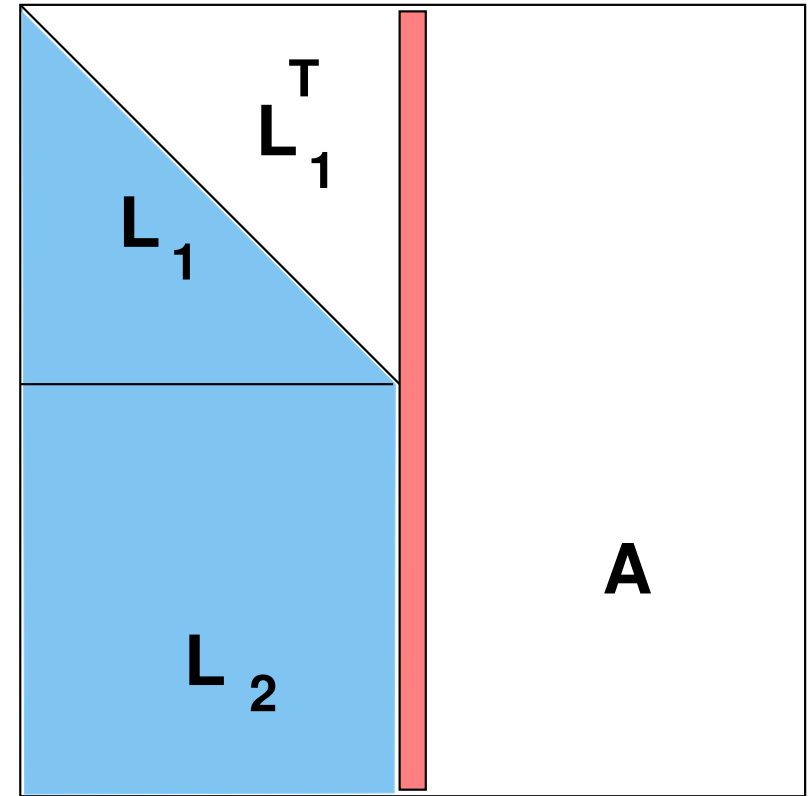
- We will consider mostly Cholesky –
- We will consider some implementation details and tricks used to develop efficient solvers

Basic principles:

- Separate computation of structure from rest [symbolic factorization]
- Do as much work as possible statically
- Take advantage of clique formation (supernodes, mass-elimination).

Sparse Column Cholesky

```
For  $j = 1, \dots, n$  Do:  
   $l(j : n, j) = a(j : n, j)$   
  For  $k = 1, \dots, j - 1$  Do:  
    // cmod(k,j):  
     $l_{j:n,j} := l_{j:n,j} - l_{j,k} * l_{j:n,k}$   
  EndDo  
  // cdiv (j) [Scale]  
   $l_{j,j} = \sqrt{l_{j,j}}$   
   $l_{j+1:n,j} := l_{j+1:n,j} / l_{j,j}$   
EndDo
```



The four essential stages of a solve

1. Reordering: $A \longrightarrow A := PAP^T$

- Preprocessing: uses graph [Min. deg, AMD, Nested Dissection]

2. Symbolic Factorization: Build static data structure.

- Exploits 'elimination tree', uses graph only.
- Also: 'supernodes'

3. Numerical Factorization: Actual factorization $A = LL^T$

- Pattern of L known. Use static data structure. Exploit supernodes

4. Triangular solves: Solve $Ly = b$ then $L^T x = y$

ELIMINATION TREES

The notion of elimination tree

➤ Elimination trees are useful in many different ways [theory, symbolic factorization, etc..]

➤ For a matrix whose graph is a tree, parent of column $j < n$ is defined by

$$\text{Parent}(j) = i, \text{ where } a_{ij} \neq 0 \text{ and } i > j$$

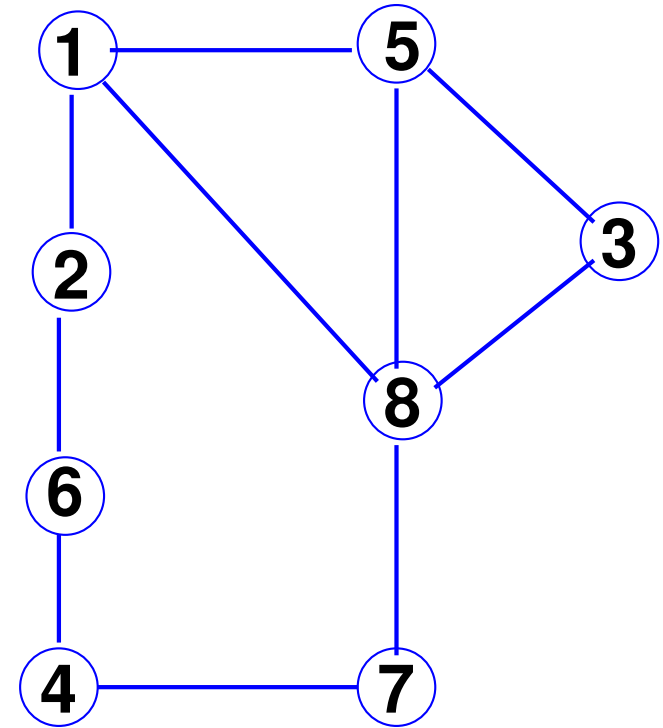
➤ For a general matrix matrix, consider $A = LL^T$, and G^F = 'filled' graph = graph of $L + L^T$. Then

$$\text{Parent}(j) = \min(i) \text{ s.t. } a_{ij} \neq 0 \text{ and } i > j$$

➤ Defines a tree rooted at column n (Elimintion tree).

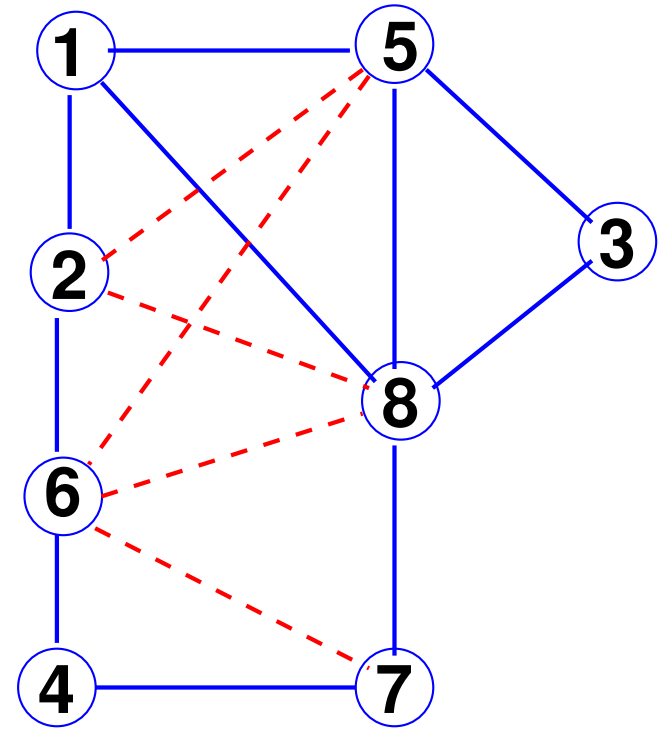
Example: Original matrix and Graph

1	★			★			★
★	2				★		
		3		★			★
			4		★	★	
★		★		5			★
	★		★		6		
			★			7	★
★		★		★		★	8

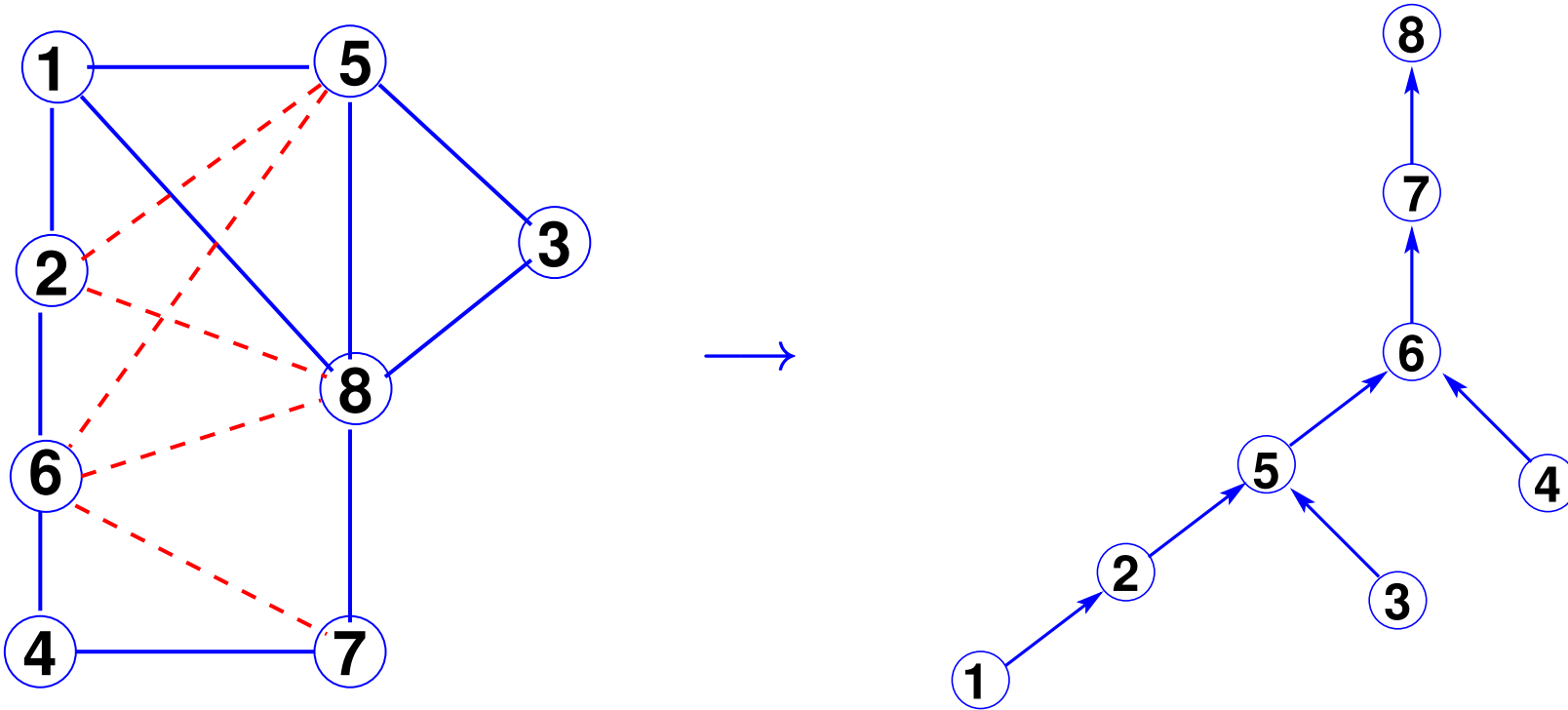


Filled matrix+graph

1	★			★			★
★	2			■	★		■
		3		★			★
			4		★	★	
★	■	★		5	■		★
	★		★	■	6	■	■
			★		■	7	★
★	■	★		★	■	★	8



Corresponding Elimination Tree

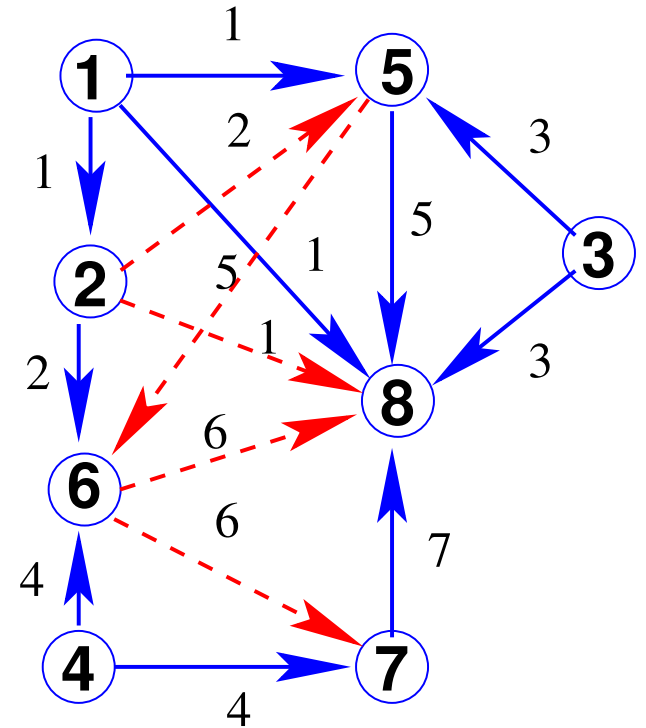


- $\text{Parent}(i) = \text{'first nonzero entry in } L(i+1:n,i)\text{'}$
- $\text{Parent}(i) = \min \{j > i \mid j \in \text{Adj}_{G^F}(i)\}$

Where does the elimination tree come from?

➤ Answer in the form of an exercise.

Consider the elimination steps for the previous example. A directed edge means a row (column) modification. It shows the task dependencies. There are unnecessary dependencies. For example: $1 \rightarrow 5$ can be removed because it is subsumed by the path $1 \rightarrow 2 \rightarrow 5$.



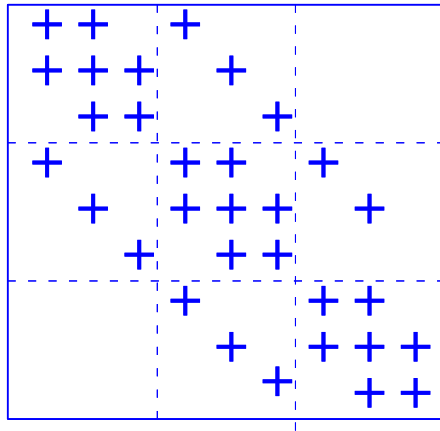
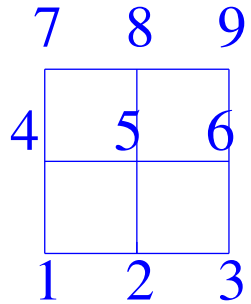
To do: Remove all the redundant dependencies.. What is the result?

Facts about elimination trees

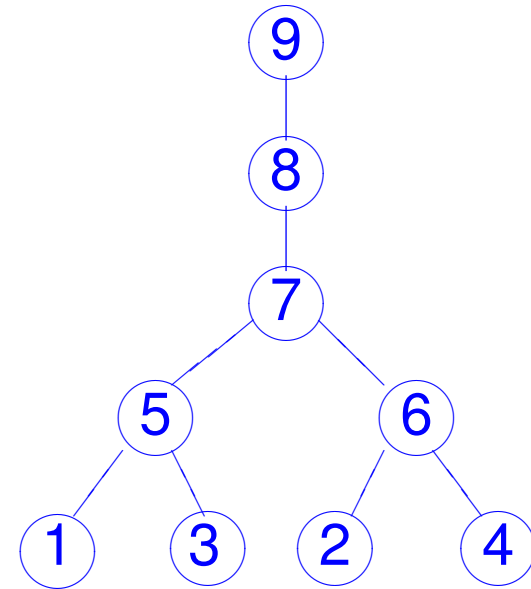
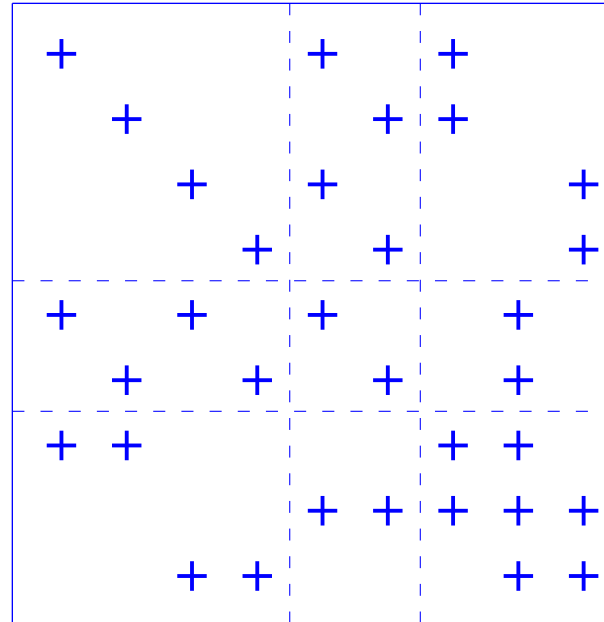
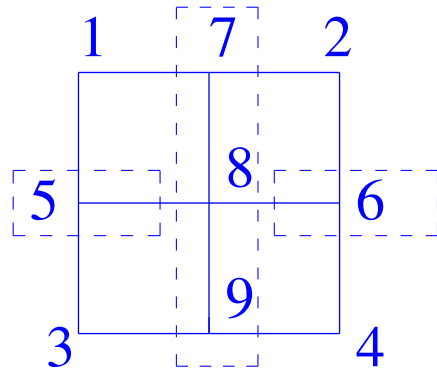
- Elimination Tree defines dependencies between columns.
- The root of a subtree cannot be used as pivot before any of its descendants is processed.
- Elimination tree depends on ordering;
- Can be used to define 'parallel' tasks.
- For parallelism: flat and wide trees → good; thin and tall (e.g. of tridiagonal systems) → Bad.
- For parallel executions, Nested Dissection gives better trees than Minimum Degree ordering.

Elim. tree depends on ordering (Not just the graph)

Example: 3×3 grid for 5-point stencil [natural ordering]



➤ Same example with nested dissection ordering

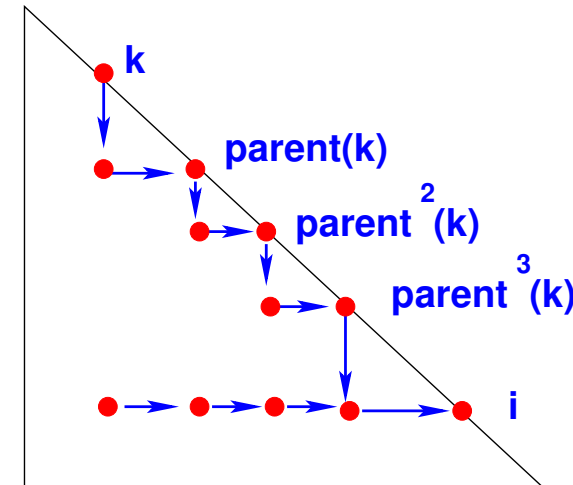


Properties

➤ The elimination tree is a spanning tree of the filled graph [a tree containing all vertices] - obtained by removing edges.

➤ If $l_{ik} \neq 0$ then i is an ancestor of k in the tree

 In the previous example: follow the creation of the fill-in (6,8).



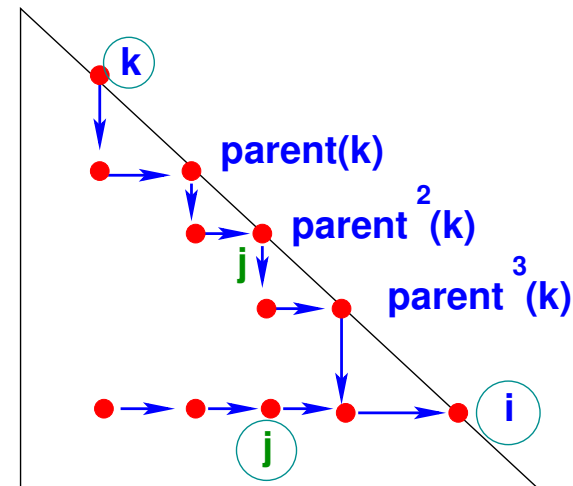
In particular: if $a_{ik} \neq 0, k < i$ then $i \rightsquigarrow k$

➤ **Consequence:** no fill-in between branches of the same subtree

Elimination trees and the pattern of L

- It is easy to determine the sparsity pattern of L because the pattern of a given column is “inherited” by the ancestors in the tree.

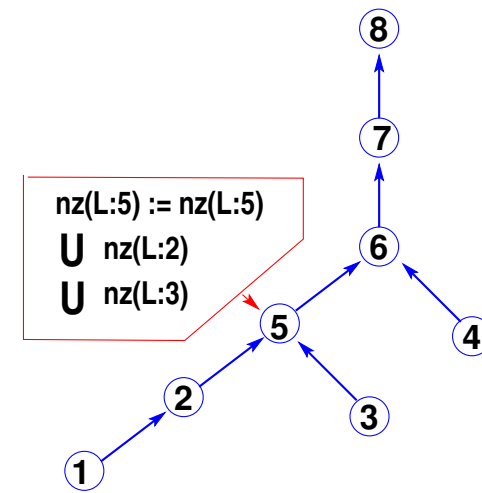
Theorem: For $i > j$, $l_{ij} \neq 0$ iff j is an ancestor of some $k \in Adj_A(i)$ in the elimination tree.



In other words:

$l_{ij} \neq 0, i > j$ iff	$\exists k \in Adj_A(i) \text{ s.t. } j \rightsquigarrow k$
----------------------------	---

In theory: To construct the pattern of L , go up the tree and accumulate the patterns of the columns. Initially L has the same pattern as $TRIL(A)$.



- **However:** Let us assume tree is not available ahead of time
- Solution: Parents can be obtained dynamically as the pattern is being built.
- This is the basis of symbolic factorization.

Notation :

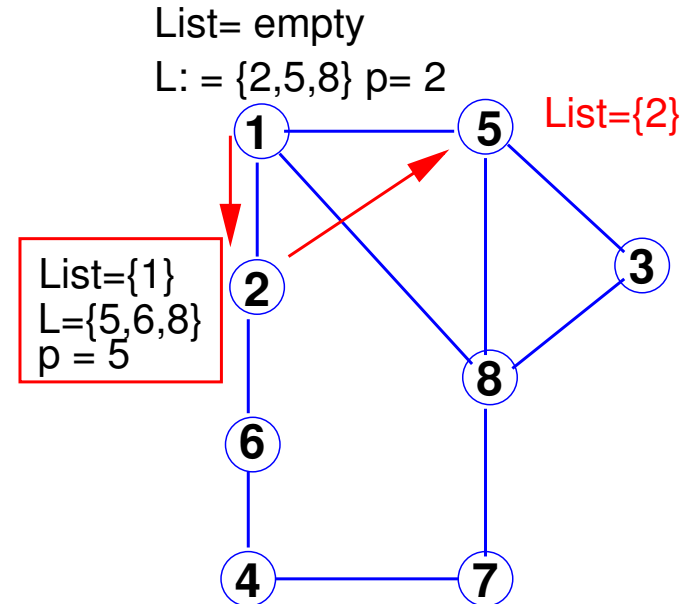
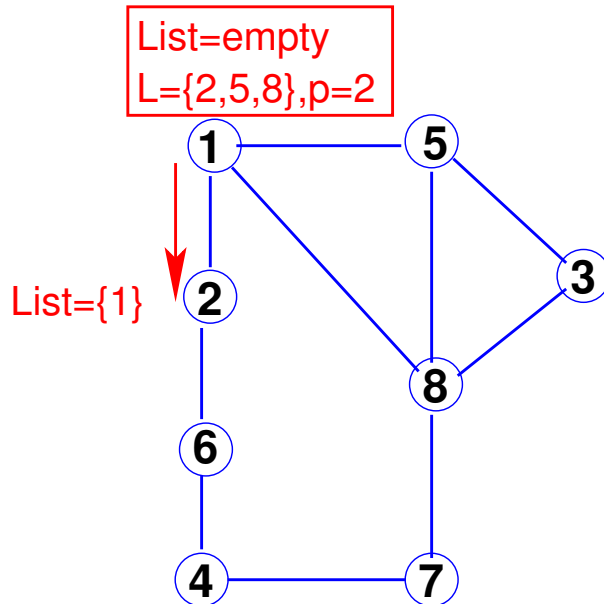
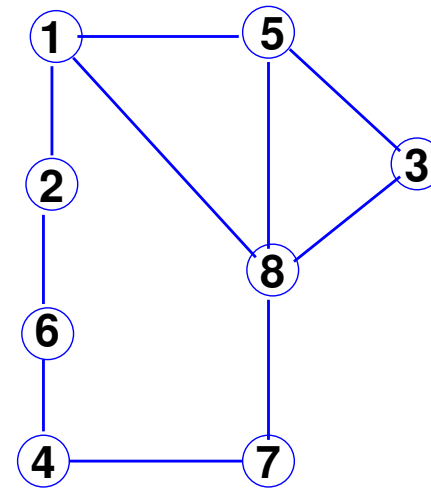
- $nz(X)$ is the pattern of X (matrix or column, or row). A set of pairs (i, j)
- $tril(X)$ = Lower triangular part of pattern [matlab notation] $\{(i, j) \in X \mid i > j\}$
- $list(j)$ = list of nodes (columns) that update column j
- Idea: dynamically create the list of nodes needed to update $L_{:,j}$.

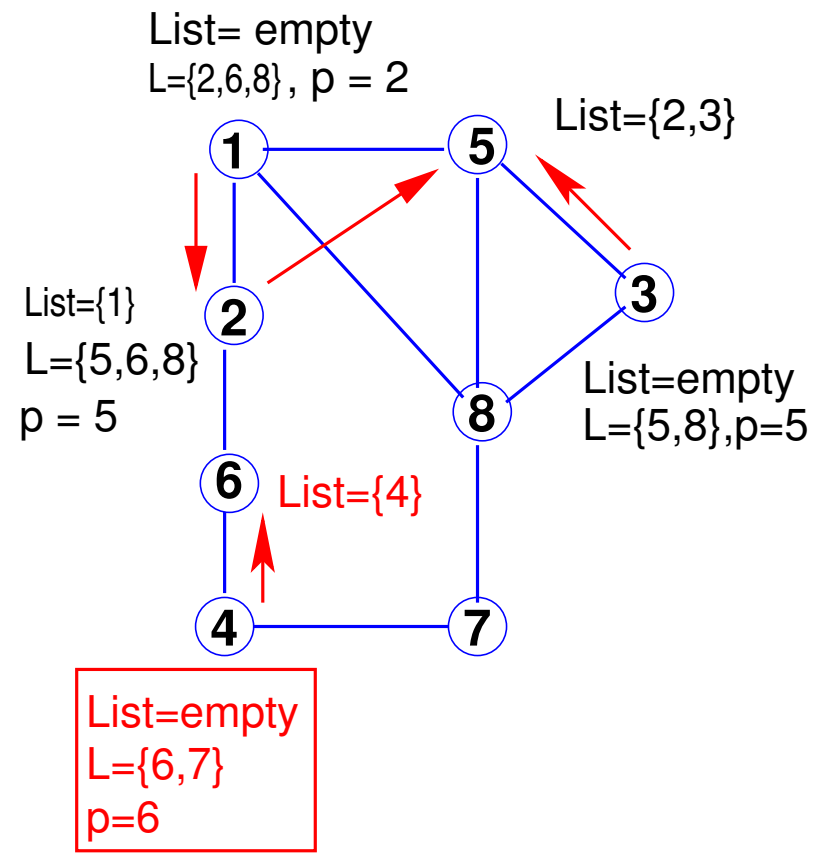
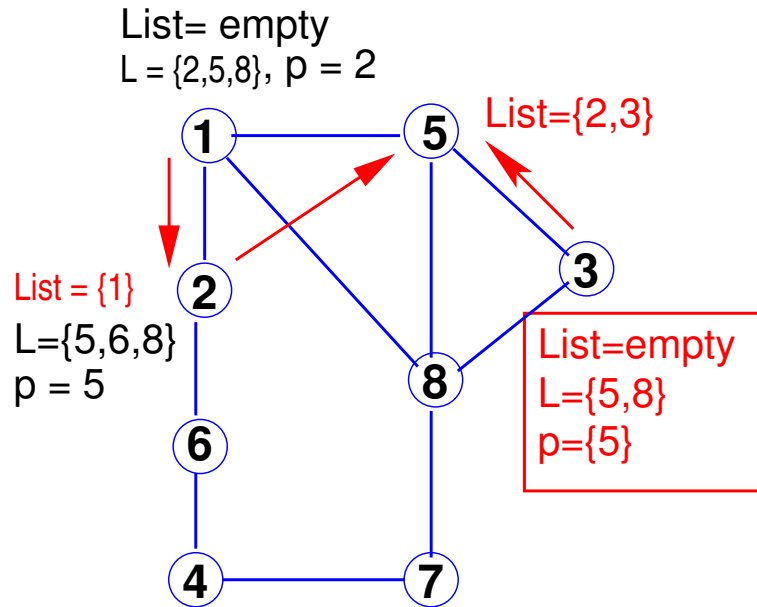
ALGORITHM : 1. *Symbolic factorization*

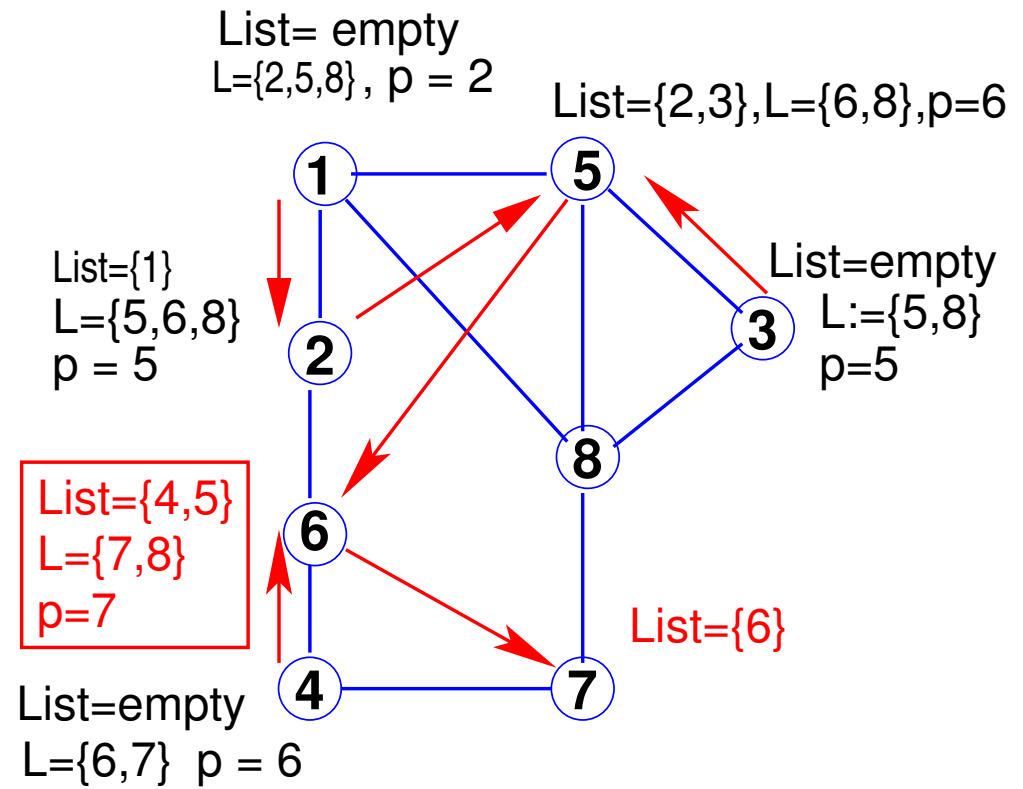
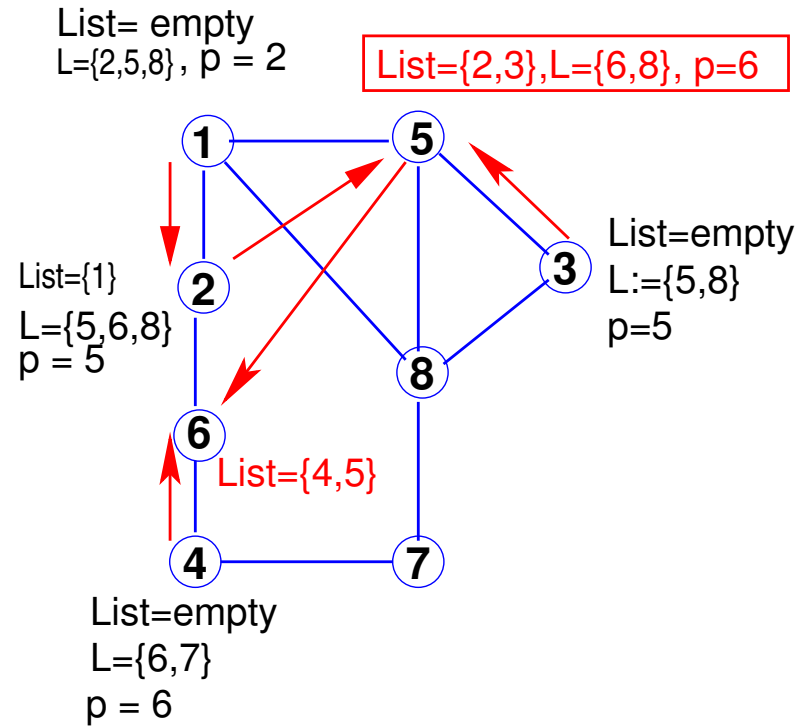
1. Set: $nz(L) = tril(nz(A))$,
2. Set: $list(j) = \emptyset, j = 1, \dots, n$
3. For $j = 1 : n$
4. for $k \in list(j)$ do
5. $nz(L_{:,j}) := nz(L_{:,j}) \cup nz(L_{:,k})$
6. end
7. $p = \min\{i > j \mid L_{i,j} \neq 0\}$
8. $list(p) := list(p) \cup \{j\}$
9. End

Example:

Consider the earlier example:







Multifrontal methods

- Start with the frontal method.

- Recall: Finite element matrix:

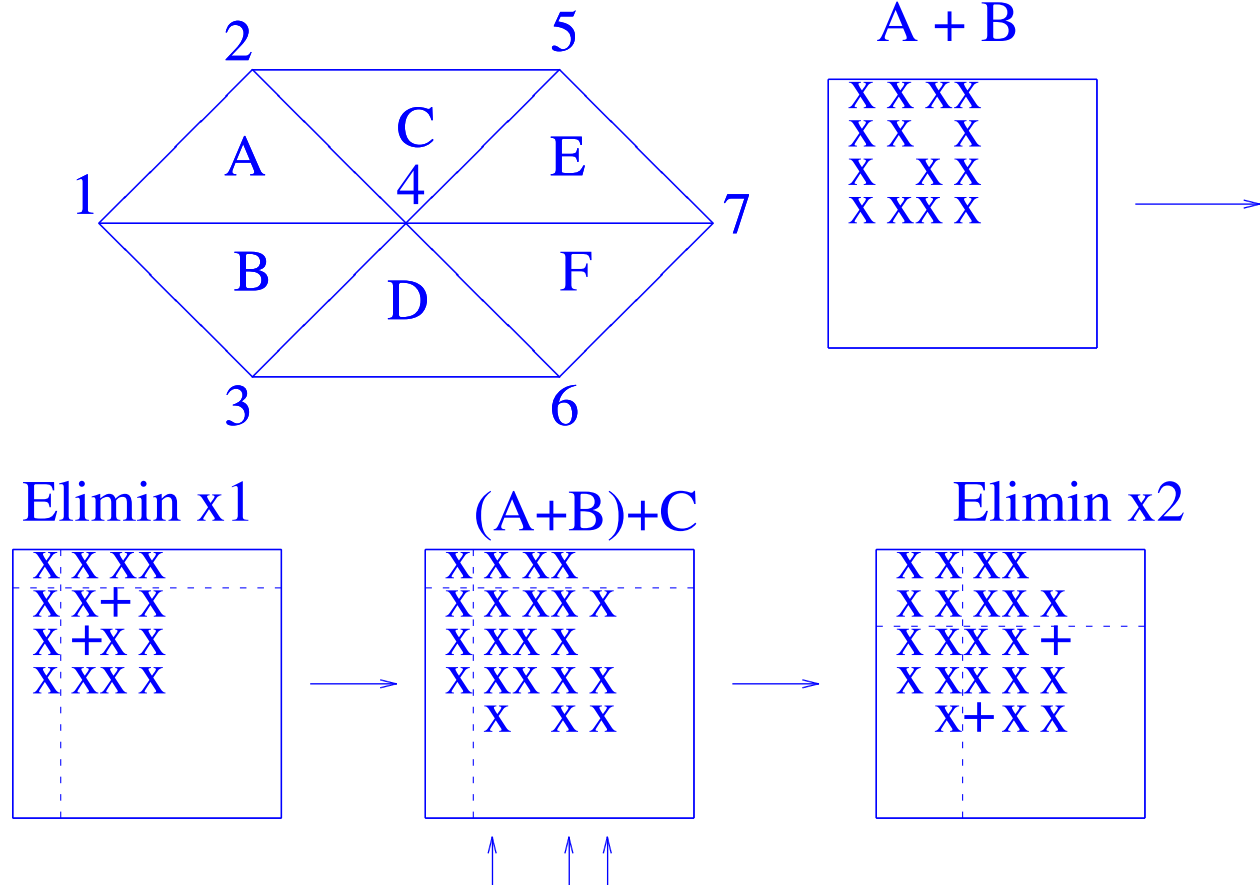
$$\mathbf{A} = \sum \mathbf{A}^{[e]}$$

$\mathbf{A}^{[e]}$ = element matrix associated with element e .

- An old idea: Execute Gaussian elimination as the elements are being assembled
- Dependency: variables $\leftrightarrow$ elements, creates an **assembly tree**.
- Method is called *the frontal method*
- Very popular among finite element users: **saves storage**

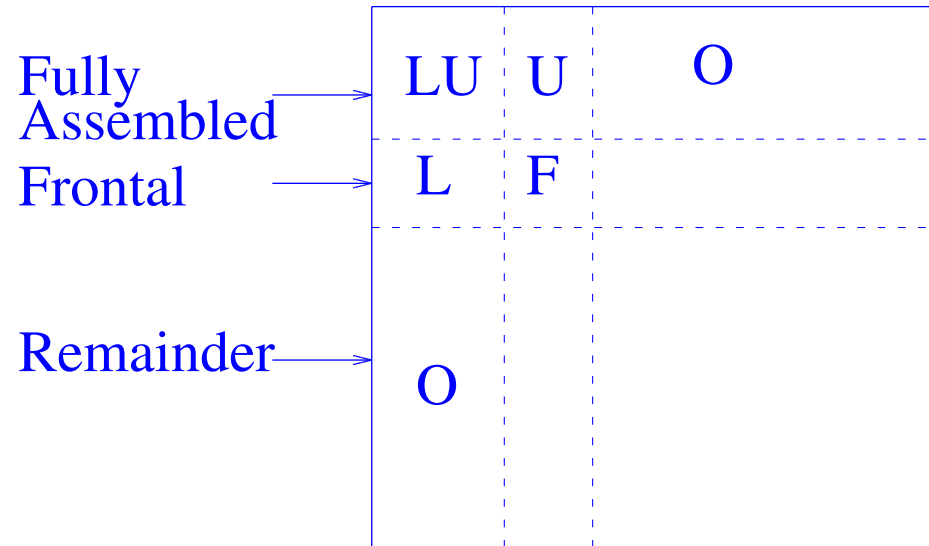
The origin: Frontal method (circa 1970s)

- Assemble $A + B$ then eliminate x_1
- Elimination of x_1 creates an **update matrix**



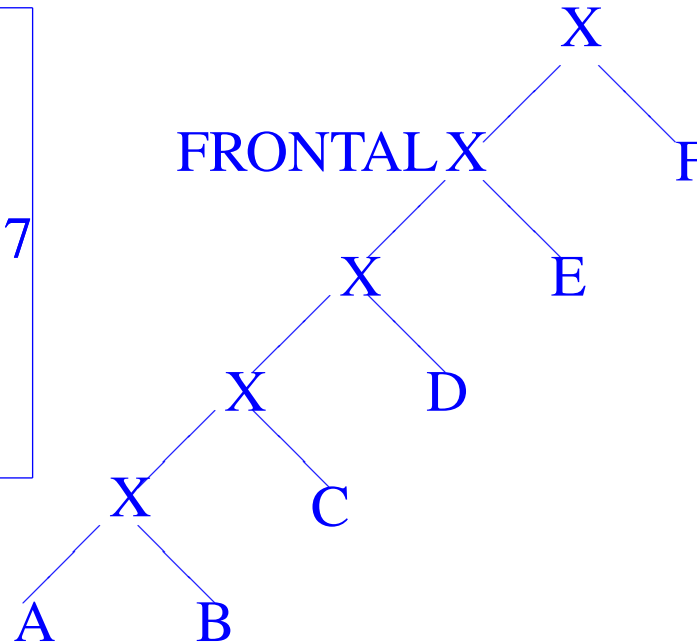
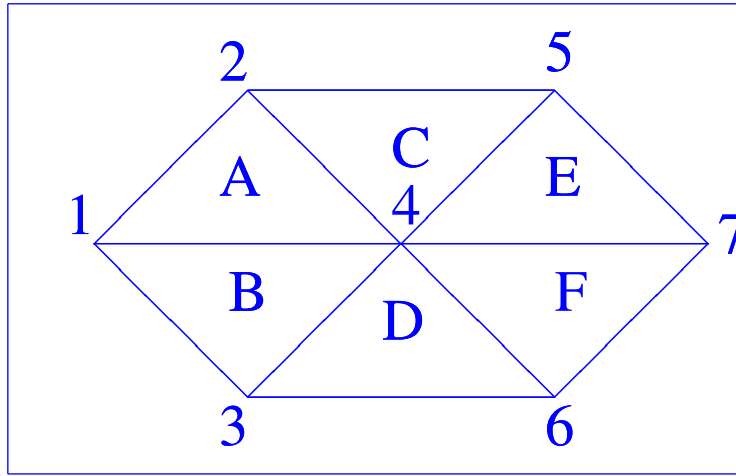
➤ Matrix has 3 parts:

- 1) Fully assembled (no longer modified)
- 2) Frontal matrix: undergoes assembly + updates
- 3) Remainder: not accessed yet.



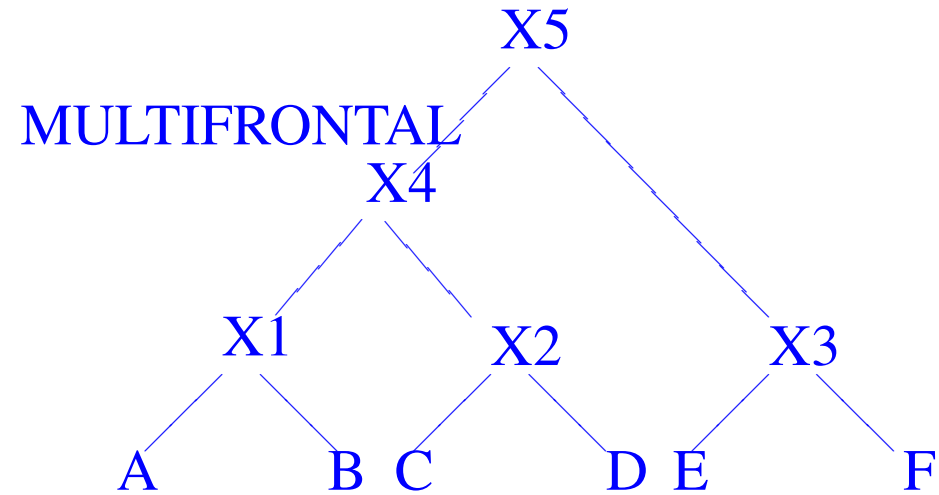
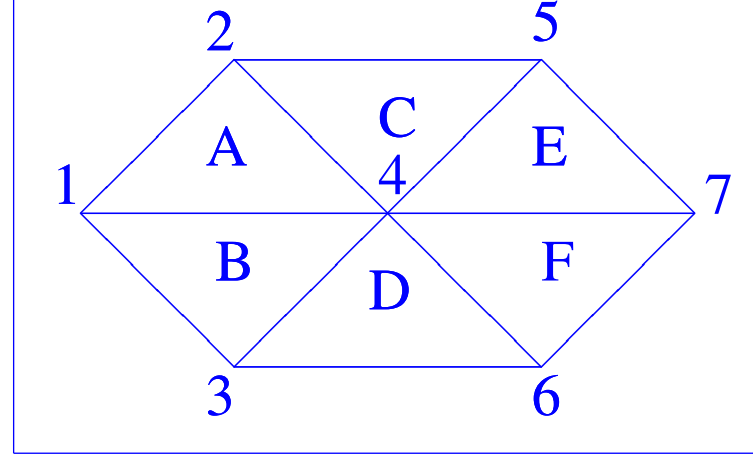
Assembly tree:

- analogue to elimination tree



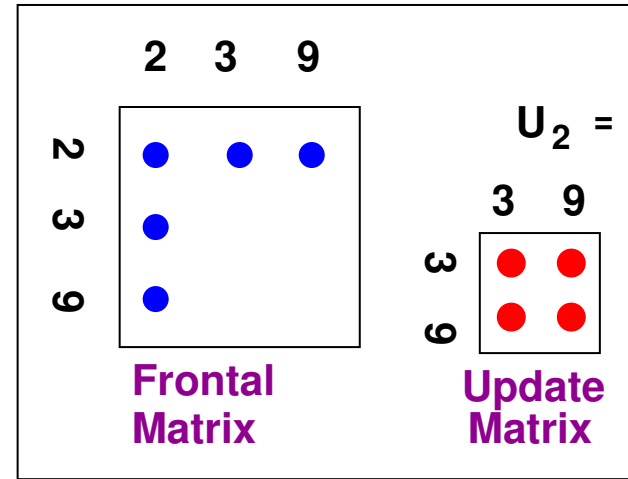
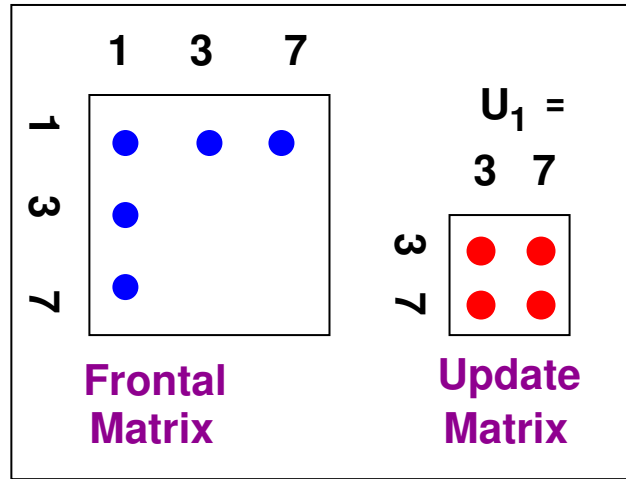
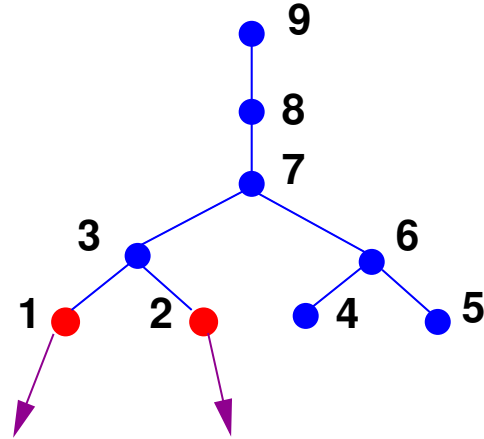
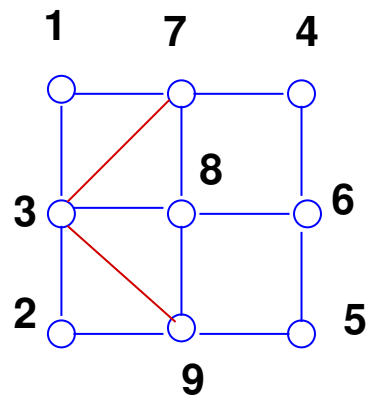
- Can proceed from several incoupled elements at the same time → multi-frontal technique [Duff & Reid, 1983]

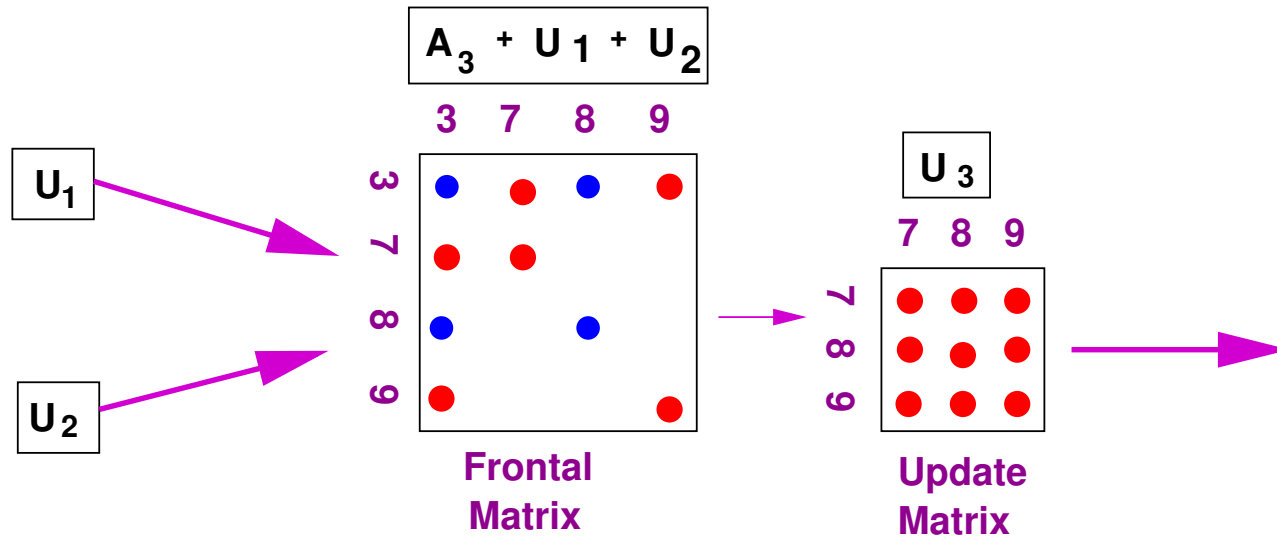
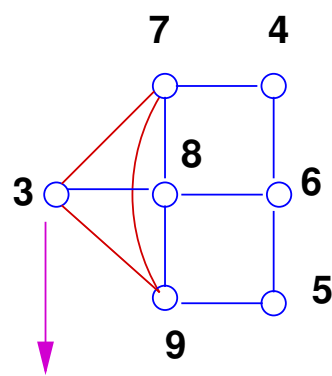
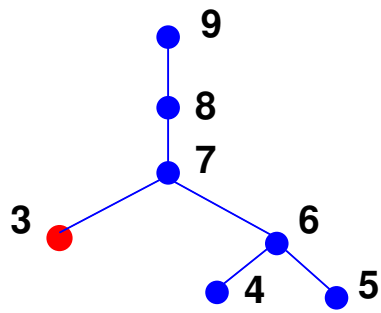
Assembly tree
for Multifrontal
Method



Multifrontal methods: extension to general matrices

- Elimination tree replaces assembly tree
- Proceed in post-order traversal of elimination tree in order not to violate task dependencies.
- When a node is eliminated an **update matrix** is created.
- This matrix is passed to the parent which adds it to its **frontal matrix**.
- Requires a stack of pending update matrices
- Update matrices popped out as they are needed
- Often implemented with nested dissection-type ordering
- More complex than a left-looking algorithm





Eliminating nodes 1 and 2:

What happens on matrix

1		★			★		
	2	★					★
★	★	3			■	★	■
			4	★	★		
				5	★		★
			★	★	6		★
★		■	★		7	★	
		★		★	★	8	★
	★	■		★		★	9

$\leftarrow U_1(3, :) \leftarrow U_2(3, :)$

$\leftarrow U_1(7, :)$

$\leftarrow U_2(9, :)$

Supernodes

Columns inherit patterns of the columns from which they are updated → Many columns with same sparsity pattern. Supernode = a set of contiguous columns in the Cholesky factor L that have the same sparsity pattern.

- The set $\{j, j + 1, \dots, j + s\}$ is a supernode if

$$NZ(L_{*,k}) = NZ(L_{*,k+1}) \cup \{k + 1\} \quad j \leq k < j + s$$

where $NZ(L_{*,k})$ is nonzero set of column k of L .

- Other terms used: Mass elimination, indistinguishable nodes, active variables in front, subscript compression,...
- Gain in performance due to savings in Gather-Scatter operations.

A few existing solvers (among many)

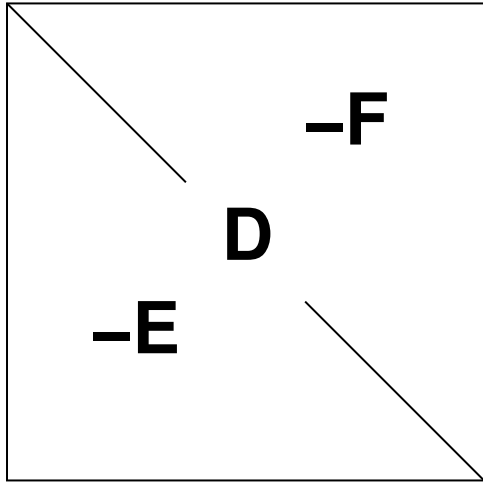
Code	Method	Scope	Developer
CHOLMOD	Left-Looking	SPD	T. Davis
MA67	Multifrontal	Symm	HSL
MA48	Right-Looking	UnSymm	HSL
SuperLU	Left-Looking	UnSymm	S. Li (LBL)
Pardiso	Left-Looking	Symm. Patt.	O. Schenk (Lugano)
MA41	Multifrontal	Symm Patt.	HSL
MUMPS	Multifrontal	Symm Patt.	Amestoy (Toulouse)
Pastix	Left+Right-Looking	Symm, symm. patt.	Labri (Bordeaux)
SuperLU_Dist	Right-Looking	UnSymm	S. Li (LBL)

Iterative methods: relaxation techniques and projection methods

- *Basic relaxation methods: Jacobi, Gauss-Seidel, SOR*
- *Convergence results*
- *Introduction to projection-type techniques*
- *Sample one-dimensional Projection methods*
- *Some theory and interpretation –*
- *See Chapter 4 and Chapter 5 of text for details.*

Linear Systems: Basic Relaxation Schemes

Relaxation schemes: based on the decomposition $A = D - E - F$



$D = \text{diag}(A)$, $-E$ = strict lower part of A and $-F$ its strict upper part.

➤ For example, Gauss-Seidel iteration :

$$(D - E)x^{(k+1)} = Fx^{(k)} + b$$

➤ Most common techniques 60 years ago.

➤ Now: used as smoothers in Multigrid or as preconditioners

Note: If $\rho_i^{(k)}$ = i th component of current residual $b - Ax$ then **relaxation** version of GS is:

$$\xi_i^{(k+1)} = \xi_i^{(k)} + \frac{\rho_i^{(k)}}{a_{ii}}$$

for $i = 1, \dots, n$

Iteration matrices

➤ Jacobi, Gauss-Seidel, SOR, & SSOR iterations are of the form

$$x^{(k+1)} = Mx^{(k)} + f$$

- $M_{Jac} = D^{-1}(E + F) = I - D^{-1}A$
- $M_{GS}(A) = (D - E)^{-1}F = I - (D - E)^{-1}A$

SOR relaxation: $\xi_i^{(k+1)} = \omega \xi_i^{(GS,k+1)} + (1 - \omega) \xi_i^{(k)}$

- $M_{SOR}(A) = (D - \omega E)^{-1}(\omega F + (1 - \omega)D) = I - (\omega^{-1}D - E)^{-1}A$
- Related Splitting: $(D - \omega E)x^{(k+1)} = [\omega F + (1 - \omega)D]x^{(k)} + \omega b$

 1 Matlab: take a look at: *gs.m*, *sor.m*, and *sorRelax.m* in *iters/*

Iteration matrices

Previous methods based on splitting A as:

$$A = M - N$$

$$Mx = Nx + b \quad \rightarrow \quad Mx^{(k+1)} = Nx^{(k)} + b \rightarrow$$

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b \equiv Gx^{(k)} + f$$

Jacobi, Gauss-Seidel, SOR, & SSOR iterations are of the form

$$G_{Jac} = D^{-1}(E + F) = I - D^{-1}A$$

$$G_{GS} = (D - E)^{-1}F = I - (D - E)^{-1}A$$

$$\begin{aligned} G_{SOR} &= (D - \omega E)^{-1}(\omega F + (1 - \omega)D) \\ &= I - (\omega^{-1}D - E)^{-1}A \end{aligned}$$

$$G_{SSOR} = I - \omega(2 - \omega)(D - \omega F)^{-1}D(D - \omega E)^{-1}A$$

General convergence result

Consider the iteration:

$$x^{(k+1)} = Gx^{(k)} + f$$

(1) Assume that $\rho(G) < 1$. Then $I - G$ is non-singular and G has a fixed point. Iteration converges to a fixed point for any f and $x^{(0)}$.

(2) If iteration converges for any f and $x^{(0)}$ then $\rho(G) < 1$.

Example: Richardson's iteration

$$x^{(k+1)} = x^{(k)} + \alpha(b - Ax^{(k)})$$

 2 Assume $\Lambda(A) \subset \mathbb{R}$. When does the iteration converge?

A few well-known results

- Jacobi and Gauss-Seidel converge for diagonal dominant matrices, i.e., matrices such that

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, i = 1, \dots, n$$

- SOR converges for $0 < \omega < 2$ for SPD matrices
- The optimal ω is known in theory for an important class of matrices called **2-cyclic matrices** or **matrices with property A**.
- A matrix has property A if it can be (symmetrically) permuted into a 2×2 block matrix whose diagonal blocks are diagonal.

$$PAP^T = \begin{bmatrix} D_1 & E \\ E^T & D_2 \end{bmatrix}$$

➤ Let A be a matrix which has property A . Then the eigenvalues λ of the SOR iteration matrix and the eigenvalues μ of the Jacobi iteration matrix are related by

$$(\lambda + \omega - 1)^2 = \lambda \omega^2 \mu^2$$

➤ The optimal ω for matrices with property A is given by

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho(B)^2}}$$

where B is the Jacobi iteration matrix.

An observation & Introduction to Preconditioning

- The iteration $x^{(k+1)} = Mx^{(k)} + f$ is attempting to solve $(I - M)x = f$. Since M is of the form $M = I - P^{-1}A$ this system can be rewritten as

$$P^{-1}Ax = P^{-1}b$$

where for SSOR, we have

$$P_{SSOR} = (D - \omega E)D^{-1}(D - \omega F)$$

referred to as the SSOR ‘preconditioning’ matrix.

In other words:

Relaxation Scheme $\iff$ *Preconditioned Fixed Point Iteration*

Projection Methods

- The main idea of projection methods is to extract an approximate solution from a subspace.
- We define a subspace of approximants of dimension m and a set of m conditions to extract the solution
- These conditions are typically expressed by orthogonality constraints.
- This defines one basic step which is repeated until convergence (alternatively the dimension of the subspace is increased until convergence).

Example:

Each relaxation step in Gauss-Seidel can be viewed as a projection step

Background on projectors

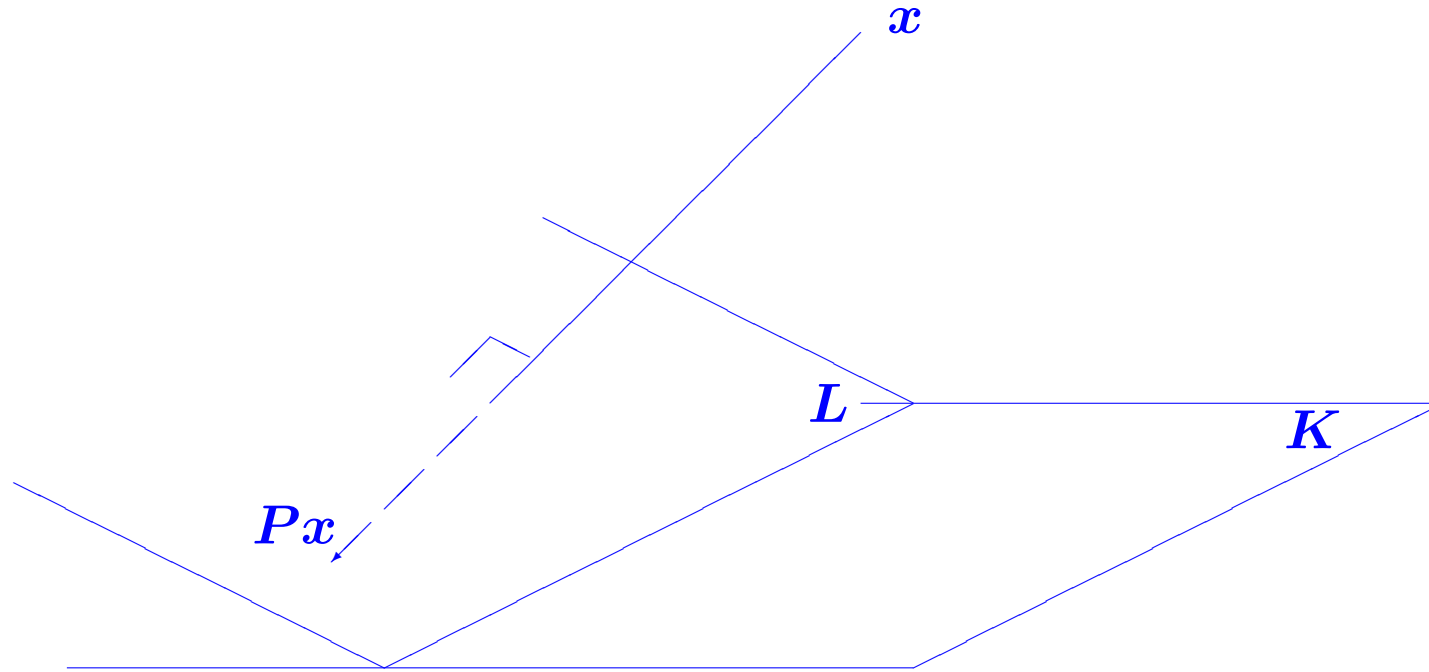
- P is a projector if it is idempotent: $P^2 = P$

Decomposition $\mathbb{R}^n = K \oplus S$ defines a (unique) projector P :

- From $x = x_1 + x_2$, set $Px = x_1$. In this case:
 - $\text{Ran}(P) = K$ and $\text{Null}(P) = S$; $\dim(K) = m \rightarrow \dim(S) = n - m$.
- Pb: express mapping $x \rightarrow u = Px$ in terms of K, S
- Note $u \in K, x - u \in S$
- Express 2nd part with m constraints: let $L = S^\perp$, then

$$u = Px \text{ iff } \begin{cases} u \in K \\ x - u \perp L \end{cases}$$

➤ Projection onto K and orthogonally to L



- Illustration: P projects **onto** K and **orthogonally** to L
- When $L = K$ projector is orthogonal.
- Note: $Px = 0$ iff $x \perp L$.

Projection methods for linear systems

- Initial Problem:

$$b - Ax = 0$$

- Given two subspaces K and L of $\mathbb{R}^N$ of dimension m , define ...

Approximate problem:

Find $\tilde{x} \in K$ such that $\underbrace{b - A\tilde{x} \perp L}_{\text{Petrov-Galerkin cond.}}$

- m degrees of freedom (K) + m constraints (L) $\rightarrow$
- To solve: A small linear system ('projected problem')
- Basic projection step. Typically a sequence of such steps are applied

➤ With a nonzero initial guess x_0 , approximate problem is

Find $\tilde{x} \in x_0 + K$ such that $b - A\tilde{x} \perp L$

Write $\tilde{x} = x_0 + \delta$ and $r_0 = b - Ax_0$. $\rightarrow$ system for δ :

Find $\delta \in K$ such that $r_0 - A\delta \perp L$

 3 Formulate Gauss-Seidel as a projection method -

 4 Generalize Gauss-Seidel by defining subspaces consisting of ‘blocks’ of coordinates $\text{span}\{e_i, e_{i+1}, \dots, e_{i+p}\}$

Matrix representation:

Let

- $V = [v_1, \dots, v_m]$ a basis of K &
- $W = [w_1, \dots, w_m]$ a basis of L

➤ Write approximate solution as $\tilde{x} = x_0 + \delta \equiv x_0 + Vy$ where $y \in \mathbb{R}^m$.
Then Petrov-Galerkin condition yields:

$$W^T(r_0 - AVy) = 0$$

➤ Therefore,

$$\tilde{x} = x_0 + V[W^TAV]^{-1}W^Tr_0$$

Remark: In practice W^TAV is known from algorithm and has a simple structure [tridiagonal, Hessenberg,..]

Prototype Projection Method

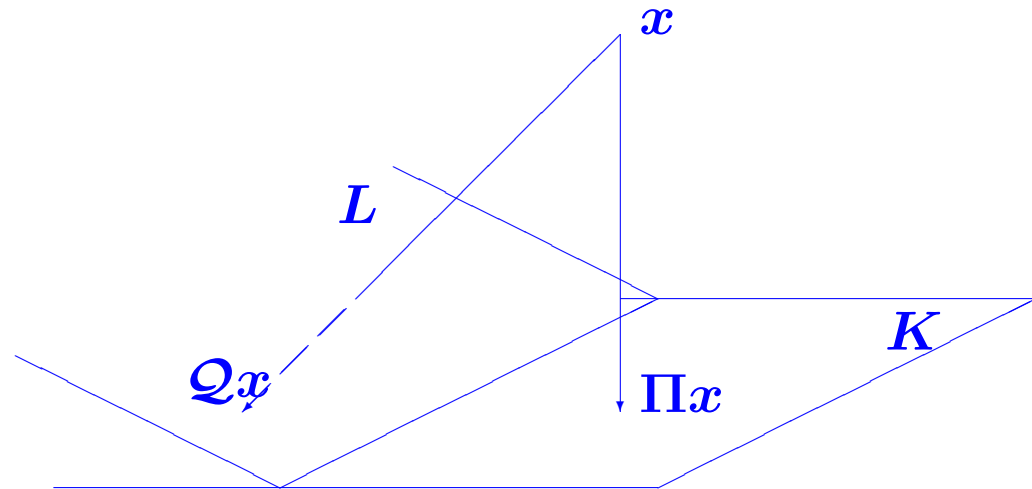
Until Convergence Do:

1. Select a pair of subspaces K , and L ;
2. Choose bases: $V = [v_1, \dots, v_m]$ for K and $W = [w_1, \dots, w_m]$ for L .
3. Compute :
$$r \leftarrow b - Ax,$$
$$y \leftarrow (W^T AV)^{-1} W^T r,$$
$$x \leftarrow x + Vy.$$

Projection methods: Operator form representation

- Let Π = the orthogonal projector onto K and $\mathcal{Q}$ the (oblique) projector onto K and orthogonally to L .

$$\begin{array}{l} \Pi x \in K, \quad x - \Pi x \perp K \\ \mathcal{Q}x \in K, \quad x - \mathcal{Q}x \perp L \end{array}$$



Assumption: no vector of K is $\perp$ to L

In the case $x_0 = 0$, approximate problem amounts to solving

$$\mathcal{Q}(b - Ax) = 0, \quad x \in K$$

or in operator form (solution is Πx)

$$\mathcal{Q}(b - A\Pi x) = 0$$

Question: what accuracy can one expect? Let x^* be the exact solution

- 1) We can't do better than $\|(I - \Pi)x^*\|_2$: $\|\tilde{x} - x^*\|_2 \geq \|(I - \Pi)x^*\|_2$
- 2) The residual of the exact solution for the approximate problem satisfies:

$$\|b - \mathcal{Q}A\Pi x^*\|_2 \leq \|\mathcal{Q}A(I - \Pi)\|_2 \|(I - \Pi)x^*\|_2$$

Two Important Particular Cases.

1. $L = K$

- When A is SPD then $\|x^* - \tilde{x}\|_A = \min_{z \in K} \|x^* - z\|_A$.
- Class of Galerkin or Orthogonal projection methods
- Important member of this class: Conjugate Gradient (CG) method

2. $L = AK$

In this case $\|b - A\tilde{x}\|_2 = \min_{z \in K} \|b - Az\|_2$

- Class of Minimal Residual Methods: CR, GCR, ORTHOMIN, GMRES, CGNR, ...

One-dimensional projection processes

$$\begin{aligned} K &= \text{span}\{d\} \\ &\text{and} \\ L &= \text{span}\{e\} \end{aligned}$$

Then $\tilde{x} = x + \alpha d$. Condition $r - A\delta \perp e$ yields

$$\alpha = \frac{(r, e)}{(Ad, e)}$$

➤ Three popular choices:

- (1) Steepest descent
- (2) Minimal residual iteration
- (3) Residual norm steepest descent

1. Steepest descent.

A is SPD. Take at each step $d = r$ and $e = r$.

Iteration:

$$\begin{aligned} r &\leftarrow b - Ax, \\ \alpha &\leftarrow (r, r) / (Ar, r) \\ x &\leftarrow x + \alpha r \end{aligned}$$

➤ Each step minimizes $f(x) = \|x - x^*\|_A^2 = (A(x - x^*), (x - x^*))$ in direction $-\nabla f$.

➤ Convergence guaranteed if A is SPD.

 5 As is formulated, the above algorithm requires 2 ‘matvecs’ per step. Reformulate it so only one is needed.

Convergence based on the Kantorovitch inequality: Let B be an SPD matrix, λ_{max} , λ_{min} its largest and smallest eigenvalues. Then,

$$\frac{(Bx, x)(B^{-1}x, x)}{(x, x)^2} \leq \frac{(\lambda_{max} + \lambda_{min})^2}{4 \lambda_{max} \lambda_{min}}, \quad \forall x \neq 0.$$

➤ This helps establish the convergence result

Let A an SPD matrix. Then, the A -norms of the error vectors $d_k = x_* - x_k$ generated by steepest descent satisfy:

$$\|d_{k+1}\|_A \leq \frac{\lambda_{max} - \lambda_{min}}{\lambda_{max} + \lambda_{min}} \|d_k\|_A$$

➤ Algorithm converges for any initial guess x_0 .

Proof: Observe $\|d_{k+1}\|_A^2 = (Ad_{k+1}, d_{k+1}) = (r_{k+1}, d_{k+1})$

➤ by substitution,

$$\|d_{k+1}\|_A^2 = (r_{k+1}, d_k - \alpha_k r_k)$$

➤ By construction $r_{k+1} \perp r_k$ so we get $\|d_{k+1}\|_A^2 = (r_{k+1}, d_k)$. Now:

$$\begin{aligned}\|d_{k+1}\|_A^2 &= (r_k - \alpha_k Ar_k, d_k) \\ &= (r_k, A^{-1}r_k) - \alpha_k (r_k, r_k) \\ &= \|d_k\|_A^2 \left(1 - \frac{(r_k, r_k)}{(r_k, Ar_k)} \times \frac{(r_k, r_k)}{(r_k, A^{-1}r_k)} \right).\end{aligned}$$

Result follows by applying the Kantorovich inequality. ■

2. Minimal residual iteration.

A positive definite ($A + A^T$ is SPD). Take at each step $d = r$ and $e = Ar$.

Iteration:

$$\begin{aligned} r &\leftarrow b - Ax, \\ \alpha &\leftarrow (Ar, r) / (Ar, Ar) \\ x &\leftarrow x + \alpha r \end{aligned}$$

- Each step minimizes $f(x) = \|b - Ax\|_2^2$ in direction r .
- Converges under the condition that $A + A^T$ is SPD.

 6 As is formulated, the above algorithm would require 2 'matvecs' at each step. Reformulate it so that only one matvec is required

Convergence

Let A be a real positive definite matrix, and let

$$\mu = \lambda_{\min}(A + A^T)/2, \quad \sigma = \|A\|_2.$$

Then the residual vectors generated by the Min. Res. Algorithm satisfy:

$$\|r_{k+1}\|_2 \leq \left(1 - \frac{\mu^2}{\sigma^2}\right)^{1/2} \|r_k\|_2$$

➤ In this case Min. Res. converges for any initial guess x_0 .

Proof: Similar to steepest descent. Start with

$$\begin{aligned}\|r_{k+1}\|_2^2 &= (r_{k+1}, r_k - \alpha_k A r_k) \\ &= (r_{k+1}, r_k) - \alpha_k (r_{k+1}, A r_k).\end{aligned}$$

By construction, $r_{k+1} = r_k - \alpha_k A r_k$ is $\perp A r_k$, so:

$\|r_{k+1}\|_2^2 = (r_{k+1}, r_k) = (r_k - \alpha_k A r_k, r_k)$. Then:

$$\begin{aligned}\|r_{k+1}\|_2^2 &= (r_k, r_k) - \alpha_k (A r_k, r_k) \\ &= \|r_k\|_2^2 \left(1 - \frac{(A r_k, r_k)}{(r_k, r_k)} \frac{(A r_k, r_k)}{(A r_k, A r_k)} \right) \\ &= \|r_k\|_2^2 \left(1 - \frac{(A r_k, r_k)^2}{(r_k, r_k)^2} \frac{\|r_k\|_2^2}{\|A r_k\|_2^2} \right).\end{aligned}$$

Result follows from the inequalities $(Ax, x)/(x, x) \geq \mu > 0$ and $\|A r_k\|_2 \leq \|A\|_2 \|r_k\|_2$. ■

3. Residual norm steepest descent.

A is arbitrary (nonsingular). Take at each step $d = A^T r$ and $e = Ad$.

Iteration:

$$\begin{aligned} r &\leftarrow b - Ax, d = A^T r \\ \alpha &\leftarrow \|d\|_2^2 / \|Ad\|_2^2 \\ x &\leftarrow x + \alpha d \end{aligned}$$

- Each step minimizes $f(x) = \|b - Ax\|_2^2$ in direction $-\nabla f$.
- Important Note: equivalent to usual steepest descent applied to normal equations $A^T Ax = A^T b$.
- Converges under the condition that A is nonsingular.

 7 Take a look at *demo1DProj.m* in /iters.

Krylov subspace methods

- *Introduction to Krylov subspace techniques*
- *FOM, GMRES, practical details.*
- *Symmetric case: Conjugate gradient*
- *See Chapter 6 of text for details.*

Motivation

- One-dimensional projection techniques:

$$x_{new} = x + \alpha d$$

where d = a certain direction.

- α is defined to optimize a certain function.
- Equivalently: determine α by an orthogonality constraint

In MR:

Example

$$x(\alpha) = x + \alpha d, \text{ with } d = b - Ax.$$

$$\min_{\alpha} \|b - Ax(\alpha)\|_2 \text{ reached iff } b - Ax(\alpha) \perp r$$

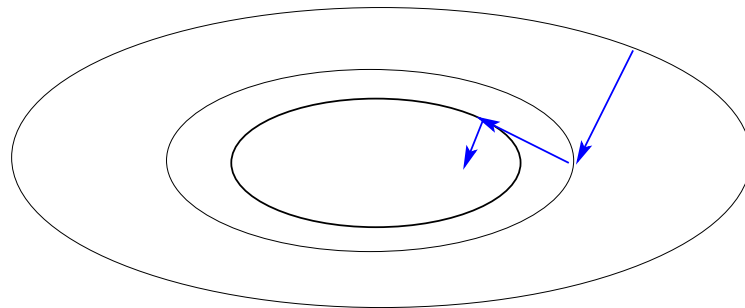
- One-dimensional projection methods are greedy methods. They are ‘short-sighted’.

Example:

Recall in Steepest Descent: New direction of search $\tilde{r}$ is $\perp$ to old direction of search r .

$$\begin{aligned} r &\leftarrow b - Ax, \\ \alpha &\leftarrow (r, r) / (Ar, r) \\ x &\leftarrow x + \alpha r \end{aligned}$$

Isosurfaces for $\|x - x_*\|_A^2$ are ellipsoids. Center == sol. Gradient of $f(x)$ is $\perp$ to isosurface



Question: can we do better by combining successive iterates?

➤ Yes: Krylov subspace methods..

Krylov subspace methods: Introduction

➤ Consider MR (or steepest descent). At each iteration:

$$\begin{aligned}r_{k+1} &= b - A(x^{(k)} + \alpha_k r_k) \\&= r_k - \alpha_k A r_k \\&= (I - \alpha_k A) r_k\end{aligned}$$

➤ In the end: $r_{k+1} = (I - \alpha_k A)(I - \alpha_{k-1} A) \cdots (I - \alpha_0 A) r_0 = p_{k+1}(A) r_0$ where $p_{k+1}(t)$ is a polynomial of degree $k + 1$ of the form

$$p_{k+1}(t) = 1 - tq_k(t)$$

 Show that: $x^{(k+1)} = x^{(0)} + q_k(A) r_0$, with $\deg(q_k) = k$

➤ Krylov subspace methods: iterations of this form that are ‘optimal’ [from m -dimensional projection methods]

Krylov subspace methods

Principle: Projection methods on Krylov subspaces:

$$K_m(A, v_1) = \text{span}\{v_1, Av_1, \dots, A^{m-1}v_1\}$$

- The most important class of iterative methods.
- Many variants exist depending on the subspace L .

Simple properties of K_m [$\mu \equiv \text{deg. of minimal polynomial of } v_1$.]

- $K_m = \{p(A)v_1 | p = \text{polynomial of degree } \leq m - 1\}$
- $K_m = K_\mu$ for all $m \geq \mu$. Moreover, K_μ is invariant under A .
- $\dim(K_m) = m$ iff $\mu \geq m$.

A little review: Gram-Schmidt process

Goal: given $X = [x_1, \dots, x_m]$ compute an orthonormal set $Q = [q_1, \dots, q_m]$ which spans the same subspace.

ALGORITHM : 2. *Classical Gram-Schmidt*

1. *For $j = 1, \dots, m$ Do:*
2. *Compute $r_{ij} = (x_j, q_i)$ for $i = 1, \dots, j - 1$*
3. *Compute $\hat{q}_j = x_j - \sum_{i=1}^{j-1} r_{ij} q_i$*
4. *$r_{jj} = \|\hat{q}_j\|_2$ If $r_{jj} == 0$ exit*
5. *$q_j = \hat{q}_j / r_{jj}$*
6. *EndDo*

ALGORITHM : 3. *Modified Gram-Schmidt*

1. *For* $j = 1, \dots, m$ *Do*:
2. $\hat{q}_j := x_j$
3. *For* $i = 1, \dots, j - 1$ *Do*
4. $r_{ij} = (\hat{q}_j, q_i)$
5. $\hat{q}_j := \hat{q}_j - r_{ij}q_i$
6. *EndDo*
7. $r_{jj} = \|\hat{q}_j\|_2$. *If* $r_{jj} == 0$ *exit*
8. $q_j := \hat{q}_j / r_{jj}$
9. *EndDo*

Let:

$X = [x_1, \dots, x_m]$ ($n \times m$ matrix)

$Q = [q_1, \dots, q_m]$ ($n \times m$ matrix)

$R = \{r_{ij}\}$ ($m \times m$ upper triangular matrix)

➤ At each step,

$$x_j = \sum_{i=1}^j r_{ij} q_i$$

Result:

$$X = QR$$

Arnoldi's algorithm

- Goal: to compute an orthogonal basis of K_m .
- Input: Initial vector v_1 , with $\|v_1\|_2 = 1$ and m .

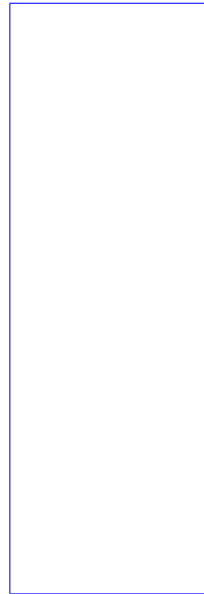
ALGORITHM : 4. *Arnoldi*

```
1: for  $j = 1, \dots, m$  do  
2:   Compute  $w := Av_j$   
3:   for  $i = 1, \dots, j$  do  
4:      $h_{i,j} := (w, v_i)$   
5:      $w := w - h_{i,j}v_i$   
6:   end for  
7:   Compute:  $h_{j+1,j} = \|w\|_2$  and  $v_{j+1} = w/h_{j+1,j}$   
8: end for
```

Result of orthogonalization process (Arnoldi):

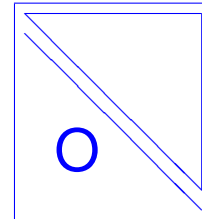
1. $V_m = [v_1, v_2, \dots, v_m]$ orthonormal basis of K_m .
2. $AV_m = V_{m+1}\overline{H}_m$
3. $V_m^T AV_m = H_m \equiv \overline{H}_m$ – last row.

$$V_m =$$



$$AV_m = V_{m+1}\overline{H}_m$$

$$\overline{H}_m =$$



$$V_{m+1} = [V_m, v_{m+1}]$$

Arnoldi's Method for linear systems ($L_m = K_m$)

From Petrov-Galerkin condition when $L_m = K_m$, we get

$$x_m = x_0 + V_m H_m^{-1} V_m^T r_0$$

➤ Select $v_1 = r_0 / \|r_0\|_2 \equiv r_0 / \beta$ in Arnoldi's. Then

$$x_m = x_0 + \beta V_m H_m^{-1} e_1$$

 2 What is the residual vector $r_m = b - Ax_m$?

Several algorithms mathematically equivalent to this approach:

- * FOM [Y. Saad, 1981] (above formulation), Young and Jea's ORTHORES [1982], Axelsson's projection method [1981],...
- * Also Conjugate Gradient method [see later]

Minimal residual methods ($L_m = AK_m$)

When $L_m = AK_m$, we let $W_m \equiv AV_m$ and obtain relation

$$\begin{aligned}x_m &= x_0 + V_m[W_m^T AV_m]^{-1}W_m^T r_0 \\ &= x_0 + V_m[(AV_m)^T AV_m]^{-1}(AV_m)^T r_0.\end{aligned}$$

► Use again $v_1 := r_0/(\beta := \|r_0\|_2)$ and the relation

$$AV_m = V_{m+1}\bar{H}_m$$

► $x_m = x_0 + V_m[\bar{H}_m^T \bar{H}_m]^{-1}\bar{H}_m^T \beta e_1 = x_0 + V_m y_m$
where y_m minimizes $\|\beta e_1 - \bar{H}_m y\|_2$ over $y \in \mathbb{R}^m$.

- Gives the Generalized Minimal Residual method (GMRES) ([YS-Schultz,'86]):

$$\begin{aligned} x_m &= x_0 + V_m y_m \quad \text{where} \\ y_m &= \min_y \|\beta e_1 - \bar{H}_m y\|_2 \end{aligned}$$

- Several Mathematically equivalent methods:

- Axelsson's CGLS
- Orthomin (1980)
- Orthodir
- GCR

A few implementation details: GMRES

Issue 1 : How to solve the least-squares problem ?

Issue 2: How to compute residual norm (without computing solution at each step)?

- Several solutions to both issues. Simplest: use Givens rotations.
- Recall: We want to solve least-squares problem

$$\min_y \|\beta e_1 - \overline{H}_m y\|_2$$

- Transform the problem into upper triangular one.

- Rotation matrices of dimension $m + 1$. Define (with $s_i^2 + c_i^2 = 1$):

$$\Omega_i = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & c_i & s_i & \\ & & & -s_i & c_i & \\ & & & & & 1 & \\ & & & & & & \ddots & \\ & & & & & & & 1 \end{bmatrix} \quad \begin{array}{l} \leftarrow \text{row } i \\ \leftarrow \text{row } i + 1 \end{array}$$

- Multiply $\bar{H}_m$ and right-hand side $\bar{g}_0 \equiv \beta e_1$ by a sequence of such matrices from the left. ➤ s_i, c_i selected to eliminate $h_{i+1,i}$

$$\bar{H}_5 = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} \\ & h_{32} & h_{33} & h_{34} & h_{35} \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \\ & & & & h_{65} \end{bmatrix}, \quad \bar{g}_0 = \begin{bmatrix} \beta \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

► 1-st Rotation:

$$\Omega_1 = \begin{bmatrix} c_1 & s_1 & & & \\ -s_1 & c_1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix} \quad \text{with:} \quad \begin{aligned} s_1 &= \frac{h_{21}}{\sqrt{h_{11}^2 + h_{21}^2}}, \\ c_1 &= \frac{h_{11}}{\sqrt{h_{11}^2 + h_{21}^2}} \end{aligned}$$

$$\bar{H}_m^{(1)} = \begin{bmatrix} h_{11}^{(1)} & h_{12}^{(1)} & h_{13}^{(1)} & h_{14}^{(1)} & h_{15}^{(1)} \\ & h_{22}^{(1)} & h_{23}^{(1)} & h_{24}^{(1)} & h_{25}^{(1)} \\ & h_{32} & h_{33} & h_{34} & h_{35} \\ & & h_{43} & h_{44} & h_{45} \\ & & & h_{54} & h_{55} \\ & & & & h_{65} \end{bmatrix}, \quad \bar{g}_1 = \begin{bmatrix} c_1\beta \\ -s_1\beta \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Repeat
with Ω_2 ,
 $\dots$,
 Ω_5 . Result:

$$\bar{H}_5^{(5)} = \begin{bmatrix} h_{11}^{(5)} & h_{12}^{(5)} & h_{13}^{(5)} & h_{14}^{(5)} & h_{15}^{(5)} \\ & h_{22}^{(5)} & h_{23}^{(5)} & h_{24}^{(5)} & h_{25}^{(5)} \\ & & h_{33}^{(5)} & h_{34}^{(5)} & h_{35}^{(5)} \\ & & & h_{44}^{(5)} & h_{45}^{(5)} \\ & & & & h_{55}^{(5)} \\ & & & & 0 \end{bmatrix}, \quad \bar{g}_5 = \begin{bmatrix} \gamma_1 \\ \gamma_2 \\ \gamma_3 \\ \cdot \\ \cdot \\ \gamma_6 \end{bmatrix}$$

Define

$$Q_m = \Omega_m \Omega_{m-1} \dots \Omega_1$$

$$\bar{R}_m = \bar{H}_m^{(m)} = Q_m \bar{H}_m,$$

$$\bar{g}_m = Q_m(\beta e_1) = (\gamma_1, \dots, \gamma_{m+1})^T.$$

➤ Since Q_m is unitary,

$$\min \|\beta e_1 - \bar{H}_m y\|_2 = \min \|\bar{g}_m - \bar{R}_m y\|_2.$$

➤ Delete last row and solve resulting triangular system.

$$R_m y_m = g_m$$

Proposition:

1. The rank of AV_m is equal to the rank of R_m . In particular, if $r_{mm} = 0$ then A must be singular.

2. The vector y_m that minimizes $\|\beta e_1 - \bar{H}_m y\|_2$ is given by

$$y_m = R_m^{-1} g_m.$$

3. The residual vector at step m satisfies

$$\begin{aligned} b - Ax_m &= V_{m+1} [\beta e_1 - \bar{H}_m y_m] \\ &= V_{m+1} Q_m^T (\gamma_{m+1} e_{m+1}) \end{aligned}$$

4. As a result, $\|b - Ax_m\|_2 = |\gamma_{m+1}|$.

Krylov subspace methods (Continued)

- *Practical variants: restarting and truncating*
- *Symmetric case: The link with the Lanczos algorithm*
- *The Conjugate Gradient algorithm*
- *See Chapter 6 of text for details.*

Restarting and Truncating

Difficulty: As m increases, storage and work per step increase fast.

First remedy: Restart. Fix m (dim. of subspace)

ALGORITHM : 5 . *Restarted GMRES (resp. Arnoldi)*

1. **(Re)-Start:** Compute $r_0 = b - Ax_0$, Set: $v_1 = r_0 / (\beta := \|r_0\|_2)$.
2. **Arnoldi Process:** generate $\bar{H}_m$ and V_m .
3. Compute $y_m = H_m^{-1} \beta e_1$ (FOM), or
 $y_m = \operatorname{argmin} \|\beta e_1 - \bar{H}_m y\|_2$ (GMRES)
4. $x_m = x_0 + V_m y_m$
5. If $\|r_m\|_2 \leq \epsilon \|r_0\|_2$ stop
else set $x_0 := x_m$ and go to 1.

Second remedy: Truncate the orthogonalization

Formula for v_{j+1} replaced by:

$$h_{j+1,j}v_{j+1} = Av_j - \sum_{i=j-k+1}^j h_{ij}v_i$$

- Each v_j is made orthogonal to the previous k v_i 's.
- x_m still computed as $x_m = x_0 + V_m H_m^{-1} \beta e_1$.
- It can be shown that this is an oblique projection process.
- IOM (Incomplete Orthogonalization Method) = replace orthogonalization in FOM, by the above truncated (or 'incomplete') orthogonalization.

The direct version of IOM [DIOM]:

► Write the LU decomposition of H_m as $H_m = L_m U_m$

$$x_m = x_0 + V_m U_m^{-1} L_m^{-1} \beta e_1 \equiv x_0 + P_m z_m$$

Structure of L_m, U_m when $k = 3$

$$L_m = \begin{bmatrix} 1 & & & & \\ x & 1 & & & \\ & x & 1 & & \\ & & x & 1 & \\ & & & x & 1 \\ & & & & x & 1 \end{bmatrix} \quad U_m = \begin{bmatrix} x & x & x & & & \\ & x & x & x & & \\ & & x & x & x & \\ & & & x & x & x \\ & & & & x & x \\ & & & & & x \end{bmatrix}$$

► $p_m = u_{mm}^{-1} [v_m - \sum_{i=m-k+1}^{m-1} u_{im} p_i]$ $z_m = \begin{bmatrix} z_{m-1} \\ \zeta_m \end{bmatrix}$

➤ Can update x_m at each step:

$$x_m = x_{m-1} + \zeta_m p_m$$

Algorithm:

Until convergence do:

Update LU factorization of $H_m \rightarrow H_m = L_m U_m$

$$p_m = u_{mm}^{-1} [v_m - \sum_{i=m-k+1}^{m-1} u_{im} p_i]$$

$$x_m = x_{m-1} + \zeta_m p_m$$

$$h_{m+1,m} v_{m+1} = A v_m - \sum_{i=m-k+1}^m h_{im} v_i \text{ (Arnoldi step)}$$

Enddo

➤ Requires $2k + 1$ vectors [in addition to solution]



What can you say about the matrix $P_m^T A P_m$?

Note: Several existing pairs of methods have a similar link: they are based on the LU, or other, factorizations of the H_m matrix

- CG-like formulation of IOM called DIOM [YS, 1982]
- ORTHORES(k) [Young & Jea '82] equivalent to DIOM(k)
- SYMMLQ [Paige and Saunders, '77] uses LQ factorization of H_m .
- Can incorporate partial pivoting in LU factorization of H_m

The symmetric case: Observation

Observe: When A is real symmetric then in Arnoldi's method:

$$H_m = V_m^T A V_m$$

must be symmetric. Therefore

Theorem. When Arnoldi's algorithm is applied to a (real) symmetric matrix then the matrix H_m is symmetric tridiagonal:

$$\begin{aligned} h_{ij} &= 0 \quad 1 \leq i < j - 1; \quad \text{and} \\ h_{j,j+1} &= h_{j+1,j}, \quad j = 1, \dots, m \end{aligned}$$

➤ We can write

$$H_m = \begin{bmatrix} \alpha_1 & \beta_2 & & & & \\ \beta_2 & \alpha_2 & \beta_3 & & & \\ & \beta_3 & \alpha_3 & \beta_4 & & \\ & & \cdot & \cdot & \cdot & \\ & & & \cdot & \cdot & \cdot \\ & & & & \beta_m & \alpha_m \end{bmatrix} \quad (3)$$

The v_i 's satisfy a 3-term recurrence [Lanczos Algorithm]:

$$\beta_{j+1}v_{j+1} = Av_j - \alpha_jv_j - \beta_jv_{j-1}$$

➤ Simplified version of Arnoldi's algorithm for sym. systems.

Symmetric matrix + Arnoldi $\rightarrow$ Symmetric Lanczos

The Lanczos algorithm

ALGORITHM : 6. *Lanczos*

1. Choose an initial vector v_1 , s.t. $\|v_1\|_2 = 1$
Set $\beta_1 \equiv 0, v_0 \equiv 0$
2. For $j = 1, 2, \dots, m$ Do:
3. $w_j := Av_j - \beta_j v_{j-1}$
4. $\alpha_j := (w_j, v_j)$
5. $w_j := w_j - \alpha_j v_j$
6. $\beta_{j+1} := \|w_j\|_2$. If $\beta_{j+1} = 0$ then Stop
7. $v_{j+1} := w_j / \beta_{j+1}$
8. EndDo

Lanczos algorithm for linear systems

➤ Usual orthogonal projection method setting:

- $L_m = K_m = \text{span}\{r_0, Ar_0, \dots, A^{m-1}r_0\}$
- Basis $V_m = [v_1, \dots, v_m]$ of K_m generated by the Lanczos algorithm

➤ Three different possible implementations.

(1) Arnoldi-like;

(2) Exploit tridiagonal nature of H_m (DIOM);

(3) Conjugate gradient - derived from (2)

ALGORITHM : 7. *Lanczos Method for Linear Systems*

1. Compute $r_0 = b - Ax_0$, $\beta := \|r_0\|_2$, and $v_1 := r_0/\beta$
2. For $j = 1, 2, \dots, m$ Do:
3. $w_j = Av_j - \beta_j v_{j-1}$ (If $j = 1$ set $\beta_1 v_0 \equiv 0$)
4. $\alpha_j = (w_j, v_j)$
5. $w_j := w_j - \alpha_j v_j$
6. $\beta_{j+1} = \|w_j\|_2$. If $\beta_{j+1} = 0$ set $m := j$ and go to 9
7. $v_{j+1} = w_j/\beta_{j+1}$
8. EndDo
9. Set $T_m = \text{tridiag}(\beta_i, \alpha_i, \beta_{i+1})$, and $V_m = [v_1, \dots, v_m]$.
10. Compute $y_m = T_m^{-1}(\beta e_1)$ and $x_m = x_0 + V_m y_m$

➤ Equivalent to FOM.

ALGORITHM : 8. *D-Lanczos*

1. Compute $r_0 = b - Ax_0$, $\zeta_1 := \beta := \|r_0\|_2$, and $v_1 := \frac{r_0}{\beta}$
2. Set $\lambda_1 = \beta_1 = 0$, $p_0 = 0$
3. For $m = 1, 2, \dots$, until convergence Do:
 4. Compute $w := Av_m - \beta_m v_{m-1}$ and $\alpha_m = (w, v_m)$
 5. If $m > 1$ compute $\lambda_m = \frac{\beta_m}{\eta_{m-1}}$ and $\zeta_m = -\lambda_m \zeta_{m-1}$
 6. $\eta_m = \alpha_m - \lambda_m \beta_m$
 7. $p_m = \eta_m^{-1} (v_m - \beta_m p_{m-1})$
 8. $x_m = x_{m-1} + \zeta_m p_m$
 9. If x_m has converged then Stop
 10. $w := w - \alpha_m v_m$
 11. $\beta_{m+1} = \|w\|_2$, $v_{m+1} = w / \beta_{m+1}$
12. EndDo

► Equivalent to DIOM with $k = 2$

The Conjugate Gradient Algorithm (A S.P.D.)

➤ In D-Lanczos, $r_m = \text{scal.} \times v_{m+1}$ and $p_m = \text{scal.} \times [v_m - \beta_{m-1}p_{m-1}]$

➤ And we have $x_m = x_{m-1} + \xi_m p_m$

➤ So there must exist an update of the form:

1. $p_{m+1} = r_m - \beta_m p_m$
2. $x_{m+1} = x_m + \xi_{m+1} p_{m+1}$
3. $r_{m+1} = r_m - \xi_{m+1} A p_{m+1}$

➤ Note: p_m is scaled differently and β_m is not the same

➤ .. In CG, index of p_m aligned with that of r_m – so p_j replaced by p_{j-1} .

➤ Note: the p_i 's are A -orthogonal

➤ The r_i 's are orthogonal.

The Conjugate Gradient Algorithm (A S.P.D.)

1. **Start:** $r_0 := b - Ax_0$, $p_0 := r_0$.

2. **Iterate:** Until convergence do,

(a) $\alpha_j := (r_j, r_j) / (Ap_j, p_j)$

(b) $x_{j+1} := x_j + \alpha_j p_j$

(c) $r_{j+1} := r_j - \alpha_j Ap_j$

(d) $\beta_j := (r_{j+1}, r_{j+1}) / (r_j, r_j)$

(e) $p_{j+1} := r_{j+1} + \beta_j p_j$

- $r_j = \text{scaling} \times v_{j+1}$. The r_j 's are orthogonal.

- The p_j 's are A -conjugate, i.e., $(Ap_i, p_j) = 0$ for $i \neq j$.

➤ Question: How to apply preconditioning?

A bit of history. From the 1952 CG article:

“The method of conjugate gradients was developed independently by E. Stiefel of the Institute of Applied Mathematics at Zurich and by M. R. Hestenes with the cooperation of J. B. Rosser, G. Forsythe, and L. Paige of the Institute for Numerical Analysis, National Bureau of Standards. The present account was prepared jointly by M. R. Hestenes and E. Stiefel during the latter’s stay at the National Bureau of Standards. The first papers on this method were given by E. Stiefel [1952] and by M. R. Hestenes [1951]. Reports on this method were given by E. Stiefel and J. B. Rosser at a Symposium on August 23-25, 1951. Recently, C. Lanczos [1952] developed a closely related routine based on his earlier paper on eigenvalue problem [1950]. Examples and numerical tests of the method have been by R. Hayes, U. Hoschstrasser, and M. Stein.”

CONVERGENCE THEORY

- *Background: Best uniform approximation;*
- *Chebyshev polynomials;*
- *Analysis of the CG algorithm;*
- *Analysis in the non-Hermitian case (short)*

Background: Best uniform approximation

We seek a function ϕ (e.g. polynomial) which deviates as little as possible from f in the sense of the $\|\cdot\|_\infty$ -norm, i.e., we seek the

$$\min_{\phi} \max_{t \in [a,b]} |f(t) - \phi(t)| = \min_{\phi} \|f - \phi\|_\infty$$

where ϕ is in a finite dimensional space (e.g., space of polynomials of degree $\leq n$)

- Solution is the “best uniform approximation to f ”
- Important case: ϕ is a polynomial of degree $\leq n$
- In this case ϕ belongs to $\mathbb{P}_n$

The Min-Max Problem:

$$\rho_n(f) = \min_{p \in \mathbb{P}_n} \max_{x \in [a,b]} |f(x) - p(x)|$$

- If f is continuous, best approximation to f on $[a, b]$ by polynomials of degree $\leq n$ exists and is unique
- ... and $\lim_{n \rightarrow \infty} \rho_n(f) = 0$ (Weierstrass theorem).

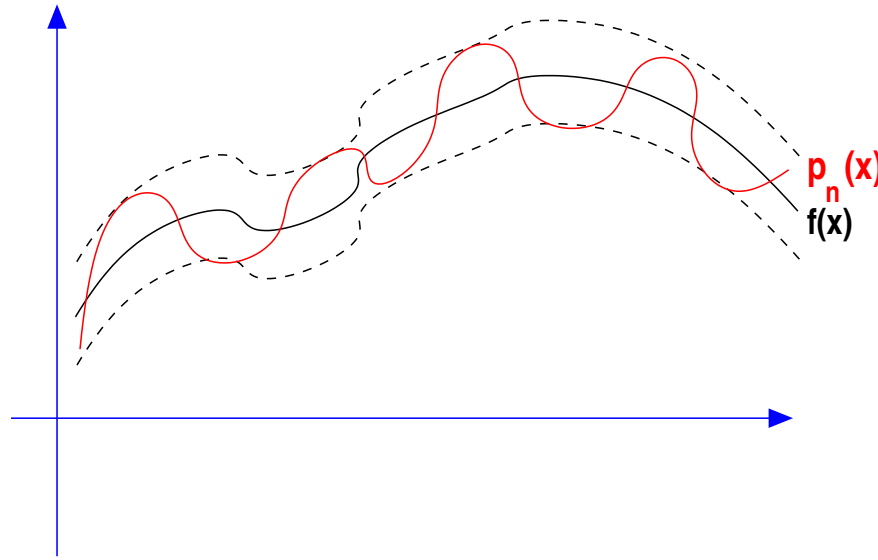
Question: How to find the best polynomial?

Answer: Chebyshev's equi-oscillation theorem.

Chebyshev equi-oscillation theorem: p_n is the best uniform approximation to f in $[a, b]$ **if and only if** there are $n + 2$ points $t_0 < t_1 < \dots < t_{n+1}$ in $[a, b]$ such that

$$f(t_j) - p_n(t_j) = c(-1)^j \|f - p_n\|_\infty \quad \text{with} \quad c = \pm 1$$

$[p_n$ 'equi-oscillates' $n + 2$ times around f]



Application: Chebyshev polynomials

Question: Among all **monic** polynomials of degree $n + 1$ which one minimizes the infinity norm? Problem:

$$\text{Minimize } \|t^{n+1} - a_n t^n - a_{n-1} t^{n-1} - \dots - a_0\|_\infty$$

Reformulation: Find the best uniform approximation to t^{n+1} by polynomials p of degree $\leq n$.

➤ $t^{n+1} - p(t)$ should be a polynomial of degree $n + 1$ which equi-oscillates $n + 2$ times.

➤ Define Chebyshev polynomials:

$$C_k(t) = \cos(k \cos^{-1} t) \text{ for } k = 0, 1, \dots, \text{ and } t \in [-1, 1]$$

➤ Observation: C_k is a polynomial of degree k , because:

➤ The C_k 's satisfy the three-term recurrence :

$$C_{k+1}(t) = 2tC_k(t) - C_{k-1}(t)$$

with $C_0(t) = 1$, $C_1(t) = t$.

 1 Show the above recurrence relation

 2 Compute $C_2, C_3, \dots, C_8$

 3 Show that for $|t| > 1$ we have

$$C_k(t) = \operatorname{ch}(k \operatorname{ch}^{-1}(t))$$

➤ C_k Equi-Oscillates $k + 1$ times around zero.

➤ Normalize C_{n+1} so that leading coefficient is 1

The minimum of $\|t^{n+1} - p(t)\|_\infty$ over $p \in \mathbb{P}_n$ is achieved when $t^{n+1} - p(t) = \frac{1}{2^n} C_{n+1}(t)$.

➤ Another important result:

Let $[\alpha, \beta]$ be a non-empty interval in $\mathbb{R}$ and let γ be any real scalar outside the interval $[\alpha, \beta]$. Then the minimum

$$\min_{p \in \mathbb{P}_k, p(\gamma)=1} \max_{t \in [\alpha, \beta]} |p(t)|$$

is reached by the polynomial: $\hat{C}_k(t) \equiv \frac{C_k \left(1 + 2 \frac{\alpha-t}{\beta-\alpha}\right)}{C_k \left(1 + 2 \frac{\alpha-\gamma}{\beta-\alpha}\right)}.$

Convergence Theory for CG

- Approximation of the form $x = x_0 + p_{m-1}(A)r_0$. with $x_0 =$ initial guess, $r_0 = b - Ax_0$;
- Recall property: x_m minimizes $\|x - x_*\|_A$ over $x \in x_0 + K_m$

Consequence: Standard result

Let $x_m = m$ -th CG iterate, $x_* =$ exact solution and

$$\eta = \frac{\lambda_{\min}}{\lambda_{\max} - \lambda_{\min}}$$

$$\text{Then: } \|x_* - x_m\|_A \leq \frac{\|x_* - x_0\|_A}{C_m(1 + 2\eta)}$$

where $C_m =$ Chebyshev polynomial of degree m .

➤ Alternative expression. From $C_k = ch(kch^{-1}(t))$:

$$\begin{aligned} C_m(t) &= \frac{1}{2} \left[\left(t + \sqrt{t^2 - 1} \right)^m + \left(t + \sqrt{t^2 - 1} \right)^{-m} \right] \\ &\geq \frac{1}{2} \left(t + \sqrt{t^2 - 1} \right)^m. \quad \text{Then:} \end{aligned}$$

$$\begin{aligned} C_m(1 + 2\eta) &\geq \frac{1}{2} \left(1 + 2\eta + \sqrt{(1 + 2\eta)^2 - 1} \right)^m \\ &\geq \frac{1}{2} \left(1 + 2\eta + 2\sqrt{\eta(\eta + 1)} \right)^m. \end{aligned}$$

➤ Next notice that:

$$\begin{aligned} 1 + 2\eta + 2\sqrt{\eta(\eta + 1)} &= \left(\sqrt{\eta} + \sqrt{\eta + 1} \right)^2 \\ &= \frac{\left(\sqrt{\lambda_{\min}} + \sqrt{\lambda_{\max}} \right)^2}{\lambda_{\max} - \lambda_{\min}} \end{aligned}$$

$$\begin{aligned}
&= \frac{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}}{\sqrt{\lambda_{\max}} - \sqrt{\lambda_{\min}}} \\
&= \frac{\sqrt{\kappa} + 1}{\sqrt{\kappa} - 1}
\end{aligned}$$

where $\kappa = \kappa_2(A) = \lambda_{\max}/\lambda_{\min}$.

➤ Substituting this in previous result yields

$$\|x_* - x_m\|_A \leq 2 \left[\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right]^m \|x_* - x_0\|_A.$$

➤ Compare with steepest descent!

Theory for Nonhermitian case

- Much more difficult!
- No convincing results on '*global convergence*' for most algorithms: FOM, GMRES(k), BiCG (to be seen) etc..
- Can get a general a-priori – a-posteriori error bound

Convergence results for nonsymmetric case

- Methods based on minimum residual better understood.
- If $(A + A^T)$ is positive definite ($(Ax, x) > 0 \forall x \neq 0$), all minimum residual-type methods (ORTHOMIN, ORTHODIR, GCR, GMRES,...), + their restarted and truncated versions, converge.
- Convergence results based on comparison with one-dim. MR [Eisenstat, Elman, Schultz 1982] $\rightarrow$ not sharp.

MR-type methods: if $A = X\Lambda X^{-1}$, Λ diagonal, then

$$\|b - Ax_m\|_2 \leq \text{Cond}_2(X) \min_{p \in \mathcal{P}_{m-1}, p(0)=1} \max_{\lambda \in \Lambda(A)} |p(\lambda)|$$

($\mathcal{P}_{m-1} \equiv$ set of polynomials of degree $\leq m - 1$, $\Lambda(A) \equiv$ spectrum of A)

Preconditioning

- *Introduction to preconditioning*
- *Preconditioned iterations*
- *Preconditioned CG and GMRES.*
- *Basic preconditioners.*
- *ILU(0), ILU(p), ILUT preconditioners*
- *See Chapters 9, 10 of text for details.*

Preconditioning – Basic principles

Basic idea: Use Krylov subspace method on a modified system such as:

$$M^{-1}Ax = M^{-1}b.$$

- The matrix $M^{-1}A$ need not be formed explicitly; only need to solve $Mw = v$ whenever needed.
- Consequence: fundamental requirement is that it should be easy to compute $M^{-1}v$ for an arbitrary vector v .
- We want: M close to A (system easier to solve) but operation $v \rightarrow M^{-1}v$ inexpensive (added cost not too high).

Left, Right, and Split preconditioning

Left preconditioning

$$M^{-1}Ax = M^{-1}b$$

Right preconditioning

$$AM^{-1}u = b, \text{ with } x = M^{-1}u$$

Split preconditioning: M is factored as $M = M_L M_R$.

$$M_L^{-1}AM_R^{-1}u = M_L^{-1}b, \text{ with } x = M_R^{-1}u$$

Preconditioned CG (PCG)

- Assume: A and M are both SPD.
- Can apply CG directly to systems $M^{-1}Ax = M^{-1}b$ or $AM^{-1}u = b$
- Problem: loss of symmetry
- Alternative: when $M = LL^T$ use split preconditioner option
- Second alternative: Observe that $M^{-1}A$ is self-adjoint with respect to M inner product:

$$(M^{-1}Ax, y)_M = (Ax, y) = (x, Ay) = (x, M^{-1}Ay)_M$$

Preconditioned CG (PCG)

ALGORITHM : 9. *Preconditioned CG*

1. Compute $r_0 := b - Ax_0$, $z_0 = M^{-1}r_0$, and $p_0 := z_0$
2. For $j = 0, 1, \dots$, until convergence Do:
3. $\alpha_j := (r_j, z_j) / (Ap_j, p_j)$
4. $x_{j+1} := x_j + \alpha_j p_j$
5. $r_{j+1} := r_j - \alpha_j Ap_j$
6. $z_{j+1} := M^{-1}r_{j+1}$
7. $\beta_j := (r_{j+1}, z_{j+1}) / (r_j, z_j)$
8. $p_{j+1} := z_{j+1} + \beta_j p_j$
9. EndDo

Note $M^{-1}A$ is also self-adjoint with respect to $(\cdot, \cdot)_A$:

$$(M^{-1}Ax, y)_A = (AM^{-1}Ax, y) = (x, AM^{-1}Ay) = (x, M^{-1}Ay)_A$$

- Can obtain a similar algorithm
- Assume that $M =$ Cholesky product $M = LL^T$.

Then, another possibility: Split preconditioning option, which applies CG to the system

$$L^{-1}AL^{-T}u = L^{-1}b, \text{ with } x = L^T u$$

- Notation: $\hat{A} = L^{-1}AL^{-T}$. All quantities related to the preconditioned system are indicated by $\hat{\cdot}$.

ALGORITHM : 10. *CG with Split Preconditioner*

1. *Compute $r_0 := b - Ax_0$; $\hat{r}_0 = L^{-1}r_0$; $p_0 := L^{-T}\hat{r}_0$.*
2. *For $j = 0, 1, \dots$, until convergence Do:*
3. $\alpha_j := (\hat{r}_j, \hat{r}_j) / (Ap_j, p_j)$
4. $x_{j+1} := x_j + \alpha_j p_j$
5. $\hat{r}_{j+1} := \hat{r}_j - \alpha_j L^{-1}Ap_j$
6. $\beta_j := (\hat{r}_{j+1}, \hat{r}_{j+1}) / (\hat{r}_j, \hat{r}_j)$
7. $p_{j+1} := L^{-T}\hat{r}_{j+1} + \beta_j p_j$
8. *EndDo*

➤ The x_j 's produced by the above algorithm and PCG are identical (if same initial guess is used).



Show this

1. *Start:* Choose x_0 and a dimension m

2. *Arnoldi process:*

- Compute $r_0 = b - Ax_0$, $\beta = \|r_0\|_2$ and $v_1 = r_0/\beta$.
- For $j = 1, \dots, m$ do
 - Compute $z_j := M^{-1}v_j$
 - Compute $w := Az_j$
 - for $i = 1, \dots, j$, do : $\left\{ \begin{array}{l} h_{i,j} := (w, v_i) \\ w := w - h_{i,j}v_i \end{array} \right\}$
 - $h_{j+1,1} = \|w\|_2$; $v_{j+1} = w/h_{j+1,1}$
- Define $V_m := [v_1, \dots, v_m]$ and $\bar{H}_m = \{h_{i,j}\}$.

3. *Form the approximate solution:* $\boxed{x_m = x_0 + M^{-1}V_my_m}$ where $y_m = \operatorname{argmin}_y \|\beta e_1 - \bar{H}_m y\|_2$ and $e_1 = [1, 0, \dots, 0]^T$.
4. *Restart:* If satisfied stop, else set $x_0 \leftarrow x_m$ and goto 2.

Remark: M is assumed to be the same at each step j . Situations may arise where M varies: $M \rightarrow M_j$. We need a 'Flexible' accelerator that allows this. Changes needed:

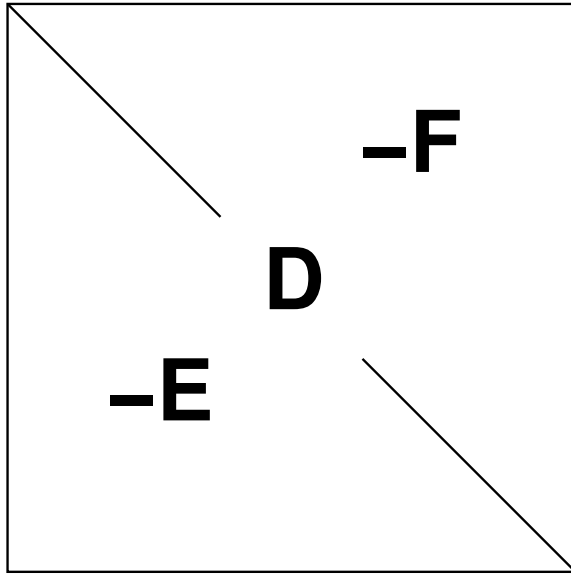
- 1) Save each z_j into matrix $Z_m = [z_1, \dots, z_m]$.
- 2) Replace $M^{-1}V_m$ by Z_m to form solution in step 3.

 2 What optimality property is satisfied with (1) Left Preconditioned GMRES, (2) Right Preconditioned GMRES; (3) Flexible GMRES?

Standard preconditioners

- Simplest preconditioner: $M = \text{Diag}(A)$ ➤ poor convergence.
- Next to simplest: SSOR. $M = (D - \omega E)D^{-1}(D - \omega F)$
- Still simple but often more efficient: ILU(0).
- ILU(p) – ILU with level of fill p – more complex.
- Class of ILU preconditioners with threshold
- Class of approximate inverse preconditioners
- Class of Multilevel ILU preconditioners
- Algebraic Multigrid Preconditioners

The SOR/SSOR preconditioner



- SOR preconditioning

$$M_{SOR} = (D - \omega E)$$

- SSOR preconditioning

$$M_{SSOR} = (D - \omega E)D^{-1}(D - \omega F)$$

- $M_{SSOR} = LU$, L = lower unit matrix, U = upper triangular. One solve with $M_{SSOR} \approx$ same cost as a MAT-VEC.

- k -step SOR (resp. SSOR) preconditioning:

k steps of SOR (resp. SSOR)

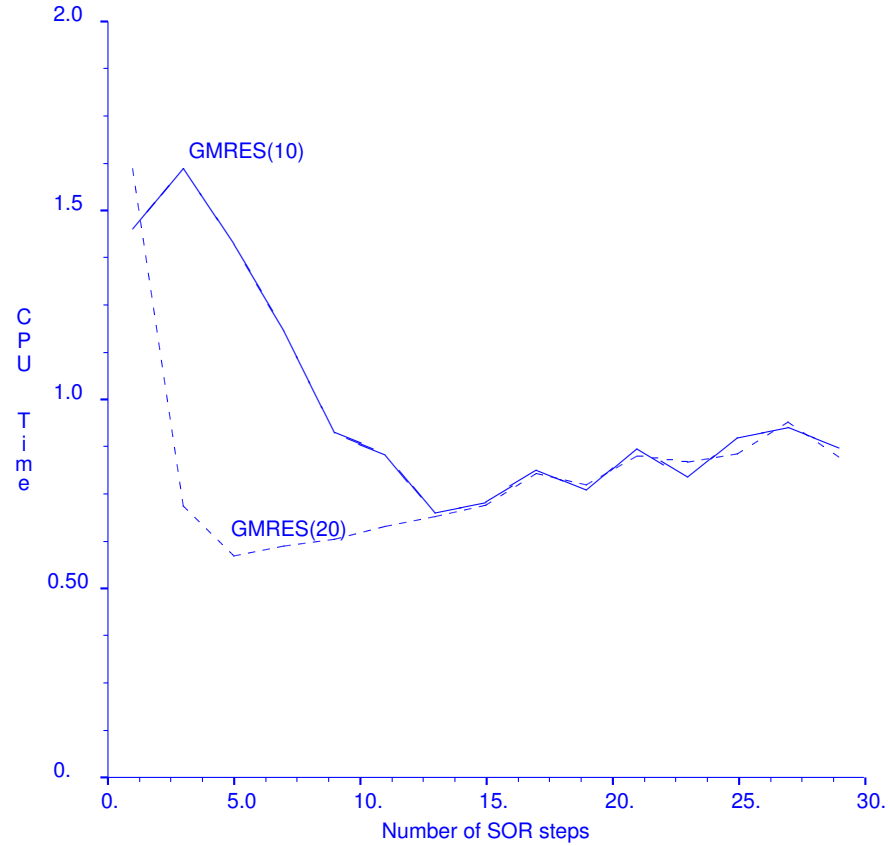
- Questions: Best ω ? For preconditioning can take $\omega = 1$ [not optimal]

$$M = (D - E)D^{-1}(D - F)$$

Observe: $M = LU + R$ with $R = ED^{-1}F$.

- Best k ? $k = 1$ is rarely the best. Substantial difference in performance.

Iteration times versus k for
SOR(k) preconditioned GM-
RES



ILU(0) and IC(0) preconditioners

- **Notation:** $NZ(X) = \{(i, j) \mid X_{i,j} \neq 0\}$
- Formal definition of ILU(0):

$$\begin{aligned} A &= LU + R \\ NZ(L) \cup NZ(U) &= NZ(A) \\ r_{ij} &= 0 \text{ for } (i, j) \in NZ(A) \end{aligned}$$

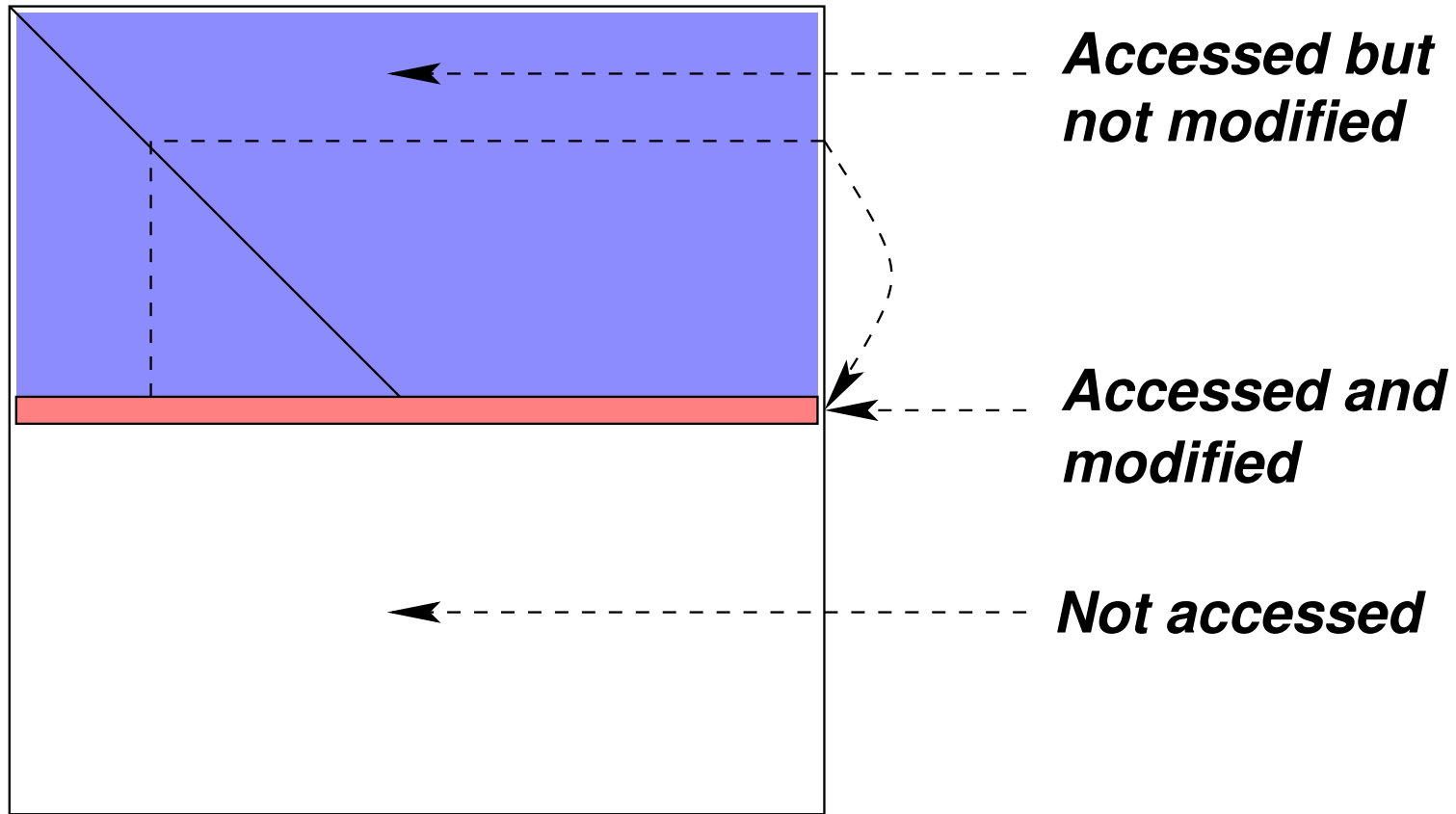
Constructive definition: Compute the LU factorization of A but drop any fill-in in L and U outside of $\text{Struct}(A)$.

- ILU factorizations are often based on i, k, j version of GE.

What is the IKJ version of GE?

ALGORITHM : 12. *Gaussian Elimination – IKJ Variant*

1. *For* $i = 2, \dots, n$ *Do*:
2. *For* $k = 1, \dots, i - 1$ *Do*:
3. $a_{ik} := a_{ik} / a_{kk}$
4. *For* $j = k + 1, \dots, n$ *Do*:
5. $a_{ij} := a_{ij} - a_{ik} * a_{kj}$
6. *EndDo*
7. *EndDo*
8. *EndDo*



ALGORITHM : 13. *ILU(0)*

For $i = 1, \dots, N$ *Do*:

For $k = 1, \dots, i - 1$ *and if* $(i, k) \in NZ(A)$ *Do*:

Compute $a_{ik} := a_{ik} / a_{kk}$

For $j = k + 1, \dots$ *and if* $(i, j) \in NZ(A)$, *Do*:

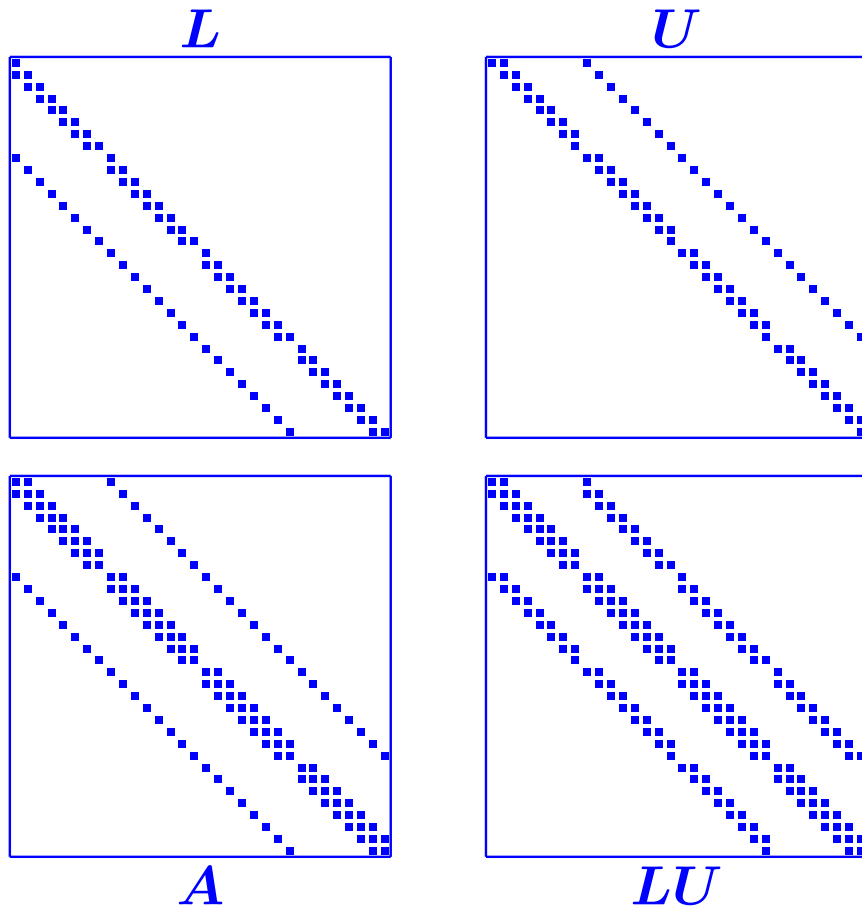
compute $a_{ij} := a_{ij} - a_{ik}a_{k,j}$.

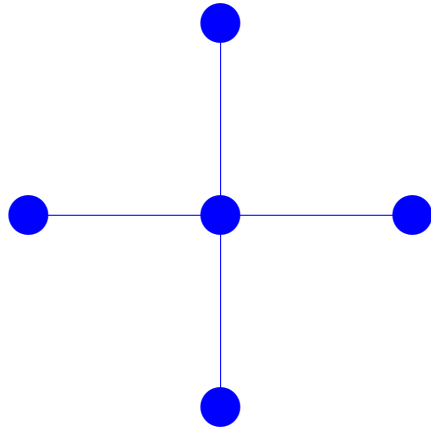
EndFor

EndFor

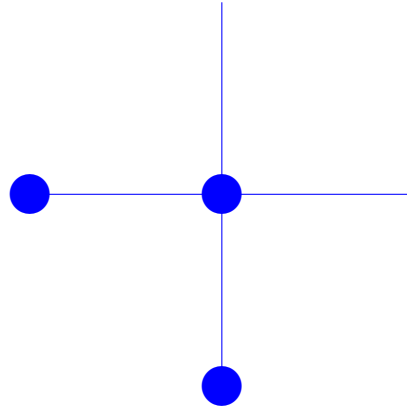
➤ When A is SPD then the ILU factorization = Incomplete Choleski factorization – IC(0). Meijerink and Van der Vorst [1977].

Pattern of ILU(0) for 5-point matrix. 'Stencil' viewpoint

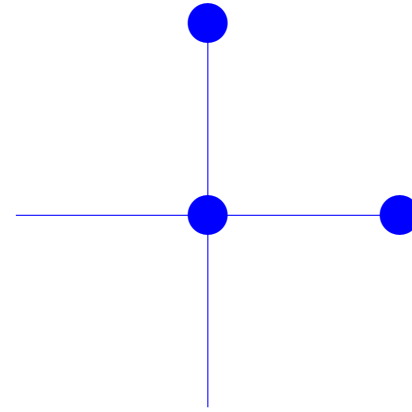




Stencil of A



Stencil of L



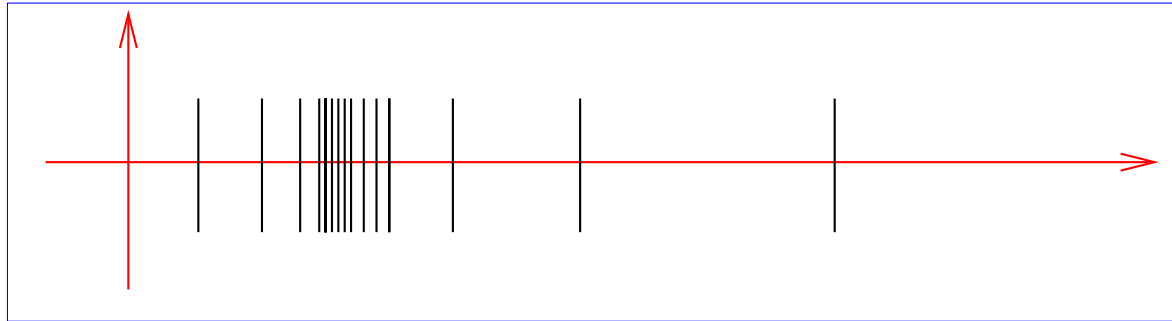
Stencil of U

- Stencil: local connectivity for a graph with a regular pattern.
- Example: For 5-point matrix A each node is coupled with its East, West, North, South neighbors (when then exist)

 Interpret fill-ins in the ILU(0) and ILU(1) preconditioners using only stencils/

Typical eigenvalue distribution

- More than anything else, what determines the convergence of an iterative method is the **distribution** of the eigenvalues of the matrix.
- Need to consider eigenvalues of preconditioned matrix $M^{-1}A$



- Clustering around 1 results in fast convergence

 4 If A is SPD with only k distinct eigenvalues, what is the minimal polynomial p of A ? Show that $p(0) \neq 0$. How many steps will it take CG to converge for any linear system $Ax = b$?

Higher order ILU factorization

- Higher accuracy incomplete Choleski: for regularly structured problems, IC(p) allows p additional diagonals in L .
- Can be generalized to irregular sparse matrices using the notion of level of fill-in [Watts III, 1979]

- Initially $Lev_{ij} = \begin{cases} 0 & \text{for } a_{ij} \neq 0 \\ \infty & \text{for } a_{ij} == 0 \end{cases}$
- At a given step i of Gaussian elimination:

$$Lev_{ij} = \min\{Lev_{ij}; Lev_{ik} + Lev_{kj} + 1\}$$

ALGORITHM : 14. $ILU(p)$

For $i = 2, N$ Do

For each $k = 1, \dots, i - 1$ and if $a_{ij} \neq 0$ do

Compute $a_{ik} := a_{ik} / a_{jj}$

Compute $a_{i,} := a_{i,*} - a_{ik}a_{k,*}$.*

Update the levels of $a_{i,}$*

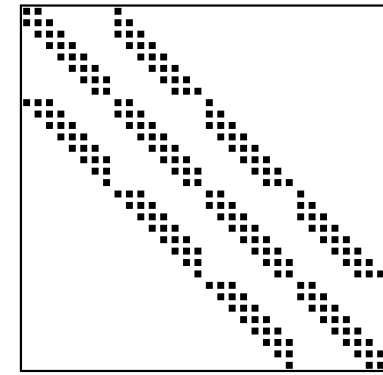
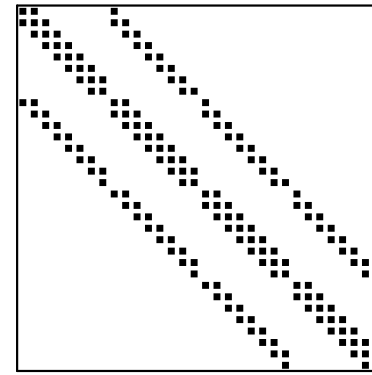
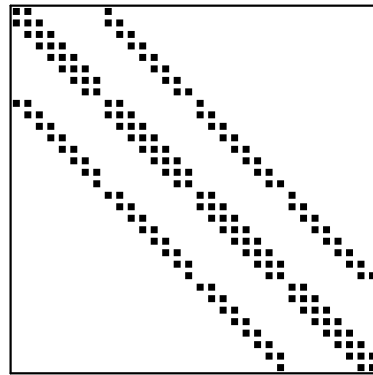
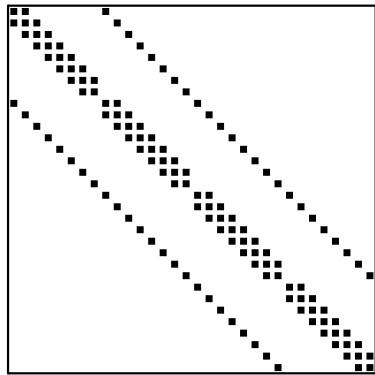
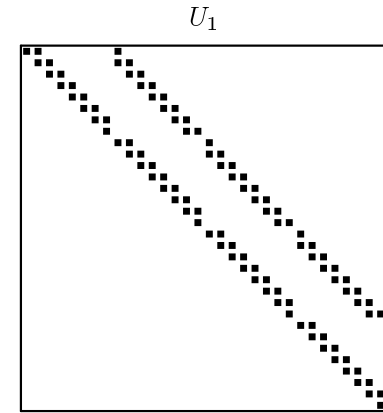
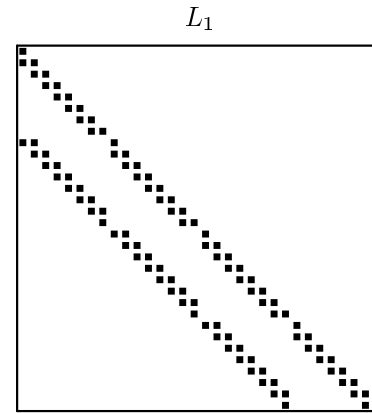
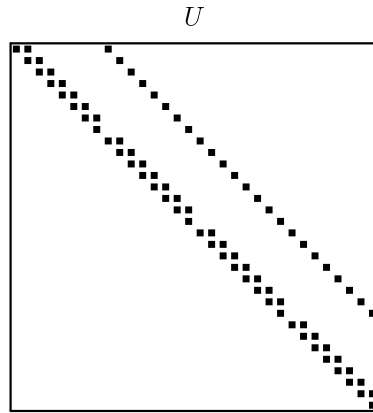
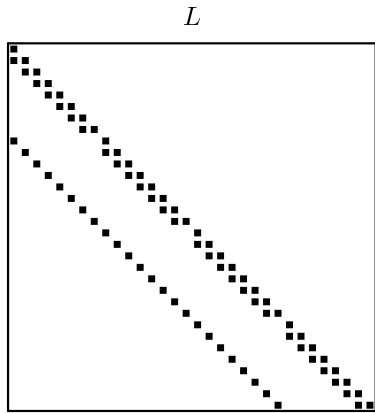
In row i : if $lev(a_{ij}) > p$ set $a_{ij} = 0$

EndFor

EndFor

- Algorithm can be split into symbolic and a numerical phase.
- Higher level of fill-in $\rightarrow$ typically fewer iterations - but more expensive set-up cost

➤ Augmented pattern used for $ILU(1) = \text{pattern of } L \text{ } U \text{ from } ILU(0)$



A

LU

Augmented A

L_1U_1

$ILU(0)$

$ILU(1)$

ILU with threshold: $ILUT(k, \epsilon)$

ILU(p) factorizations are based on structure only and not numerical values
➤ potential problems for non M-matrices.

Alternative: ILU with Threshold, ILUT

- During each i -th step in GE (i, k, j version), discard pivots or fill-ins whose value is below $\epsilon \|row_i(A)\|$.
 - Once the i -th row of $L + U$, (L-part + U-part) is computed retain only the k largest elements in both parts.
- Advantages: controlled fill-in. Smaller memory overhead.
- Easy to implement and can be made quite inexpensive.

Other preconditioners

Many other techniques have been developed:

- Approximate inverse methods
- Polynomial preconditioners
- Multigrid - type methods
- Incomplete LU based on Crout factorization
- Multi-elimination and multilevel ILU (ARMS)

Solution of eigenvalue problems

- *Introduction – motivation*
- *Projection methods for eigenvalue problems*
- *Subspace iteration, The symmetric Lanczos algorithm*
- *Nonsymmetric Lanczos procedure;*
- *Implicit restarts*
- *Harmonic Ritz values, Jacobi-Davidson's method*
- *Text: Chaps. 4 to 8 of:*

https://www-users.cse.umn.edu/~saad/eig_book_2ndEd.pdf

Background. Origins of Eigenvalue Problems

- Structural Engineering [$Ku = \lambda Mu$] (Goal: frequency response)
 - Electronic structure calculations [Schrödinger equation..]
 - Stability analysis [e.g., electrical networks, mechanical system,..]
 - Bifurcation analysis [e.g., in fluid flow]
- Large eigenvalue problems in quantum chemistry use up biggest portion of the time in supercomputer centers

Background. New applications in data analytics

- Machine learning problems often require a (partial) *Singular Value Decomposition* -
- Somewhat different issues in this case:
 - Very large matrices, update the SVD
 - Compute dominant singular values/vectors
 - Many problems of approximating a matrix (or a **tensor**) by one of lower rank (Dimension reduction, ...)
- But: Methods for computing SVD often based on those for standard eigenvalue problems

Background. The Problem (s)

- Standard eigenvalue problem:

$$Ax = \lambda x$$

Often: A is symmetric real (or Hermitian complex)

- Generalized problem $Ax = \lambda Bx$ Often: B is symmetric positive definite, A is symmetric or nonsymmetric

- Quadratic problems:

$$(A + \lambda B + \lambda^2 C)u = 0$$

- Nonlinear eigenvalue problems (NEVP)

$$\left[A_0 + \lambda B_0 + \sum_{i=1}^n f_i(\lambda) A_i \right] u = 0$$

➤ General form of NEVP $A(\lambda)x = 0$

➤ Nonlinear **eigenvector** problems:

$$[A + \lambda B + F(u_1, u_2, \dots, u_k)]u = 0$$

What to compute:

- A few λ_i 's with smallest or largest real parts;
- All λ_i 's in a certain region of $\mathbb{C}$;
- A few of the dominant eigenvalues;
- All λ_i 's (rare).

Large eigenvalue problems in applications

- Some applications require the computation of a large number of eigenvalues and vectors of very large matrices.
- Density Functional Theory in electronic structure calculations: '*ground states*'
- *Excited states* involve transitions and invariably lead to much more complex computations. → Large matrices, *many* eigen-pairs to compute

Background: The main tools

Projection process:

- (a) Build a 'good' subspace $K = \text{span}(V)$;
- (b) get approximate eigenpairs by a Rayleigh-Ritz process:
 $\tilde{\lambda}, \tilde{u} \in K$ satisfy: $(A - \tilde{\lambda}I)\tilde{u} \perp K \longrightarrow$

$$V^H(A - \tilde{\lambda}I)V y = 0$$

- $\tilde{\lambda} = \text{Ritz value}$, $\tilde{u} = Vy = \text{Ritz vector}$
- Two common choices for K :
 - 1) **Power** subspace $K = \text{span}\{A^k X_0\}$; or $\text{span}\{P_k(A)X_0\}$;
 - 2) **Krylov** subspace $K = \text{span}\{v, Av, \dots, A^{k-1}v\}$

Background. The main tools (cont)

Shift-and-invert: If we want eigenvalues near σ , replace A by $(A - \sigma I)^{-1}$.

Example: power method: $v_j = Av_{j-1}/\text{scaling}$ replaced by

$$v_j = \frac{(A - \sigma I)^{-1} v_{j-1}}{\text{scaling}}$$

- Works well for computing *a few* eigenvalues near σ /
- Used in commercial package NASTRAN (for decades!)
- Requires factoring $(A - \sigma I)$ (or $(A - \sigma B)$ in generalized case.) But convergence will be much faster.
- A solve each time - Factorization done once (ideally).

Background. The main tools (cont)

Deflation:

- Once eigenvectors converge remove them from the picture (e.g., with power method, second largest becomes largest eigenvalue after deflation).

Restarting Strategies :

- Restart projection process by using information gathered in previous steps
-

- ALL available methods use some combination of these ingredients.

[e.g. ARPACK: Arnoldi/Lanczos + ‘implicit restarts’ + shift-and-invert (option).]

Current state-of-the art in eigensolvers

- Eigenvalues at one end of the spectrum:
 - Subspace iteration + filtering [e.g. **FEAST**, **Cheb**,...]
 - Lanczos+variants (no restart, thick restart, implicit restart, Davidson,...), e.g., **ARPACK** code, **PRIMME**.
 - Block Algorithms [Block Lanczos, **TraceMin**, **LOBPCG**, **SlepSc**,...]
 - + Many others - more or less related to above
- ‘Interior’ eigenvalue problems (middle of spectrum):
 - Combine shift-and-invert + Lanczos/block Lanczos. Used in, e.g., **NASTRAN**
 - Rational filtering [**FEAST**, Sakurai et al.,...]

Projection Methods for Eigenvalue Problems

General formulation:

- Projection method onto K orthogonal to L
- Given: Two subspaces K and L of same dimension.
- Find: $\tilde{\lambda}, \tilde{u}$ such that:

$$\tilde{\lambda} \in \mathbb{C}, \tilde{u} \in K; \quad (\tilde{\lambda}I - A)\tilde{u} \perp L$$

Two types of methods:

- Orthogonal projection methods: situation when $L = K$.
- Oblique projection methods: When $L \neq K$.

Rayleigh-Ritz projection

Given: a subspace X known to contain good approximations to eigenvectors of A .

Question: How to extract good approximations to eigenvalues/ eigenvectors from this subspace?

Answer: Rayleigh Ritz process.

Let $Q = [q_1, \dots, q_m]$ an orthonormal basis of X . Then write an approximation in the form $\tilde{u} = Qy$ and obtain y by writing

$$Q^H(A - \tilde{\lambda}I)\tilde{u} = 0$$

$$\triangleright Q^H A Q y = \tilde{\lambda} y$$

Procedure:

1. Obtain an orthonormal basis of X
2. Compute $C = Q^H A Q$ (an $m \times m$ matrix)
3. Obtain Schur factorization of C , $C = Y R Y^H$
4. Compute $\tilde{U} = Q Y$

Property: if X is (exactly) invariant, then procedure will yield exact eigenvalues and eigenvectors.

Proof: Since X is invariant, $(A - \tilde{\lambda}I)u = Qz$ for a certain z . $Q^H Qz = 0$ implies $z = 0$ and therefore $(A - \tilde{\lambda}I)u = 0$.

➤ Can use this procedure in conjunction with the subspace obtained from subspace iteration algorithm

Subspace Iteration

- *Original idea:* projection technique onto a subspace of the form $Y = A^k X$
- In practice: Replace A^k by suitable polynomial [Chebyshev]

Advantages:

- Easy to implement (in symmetric case);
- Easy to analyze;

Disadvantage: Slow.

- Often used with polynomial acceleration: $A^k X$ replaced by $C_k(A)X$. Typically C_k = Chebyshev polynomial.

Algorithm: *Subspace Iteration with Projection*

1. **Start:** Choose an initial system of vectors $X = [x_0, \dots, x_m]$ and an initial polynomial C_k .
2. **Iterate:** Until convergence do:
 - (a) Compute $\hat{Z} = C_k(A)X_{old}$.
 - (b) Orthonormalize $\hat{Z}$ into Z .
 - (c) Compute $B = Z^H A Z$ and use the QR algorithm to compute the Schur vectors $Y = [y_1, \dots, y_m]$ of B .
 - (d) Compute $X_{new} = ZY$.
 - (e) Test for convergence. If satisfied stop. Else select a new polynomial $C'_{k'}$ and continue.

THEOREM: Let $S_0 = \text{span}\{x_1, x_2, \dots, x_m\}$ and assume that S_0 is such that the vectors $\{Px_i\}_{i=1,\dots,m}$ are linearly independent where P is the spectral projector associated with $\lambda_1, \dots, \lambda_m$. Let $\mathcal{P}_k$ the orthogonal projector onto the subspace $S_k = \text{span}\{X_k\}$. Then for each eigenvector u_i of A , $i = 1, \dots, m$, there exists a unique vector s_i in the subspace S_0 such that $Ps_i = u_i$. Moreover, the following inequality is satisfied

$$\|(I - \mathcal{P}_k)u_i\|_2 \leq \|u_i - s_i\|_2 \left(\left| \frac{\lambda_{m+1}}{\lambda_i} \right| + \epsilon_k \right)^k, \quad (4)$$

where ϵ_k tends to zero as k tends to infinity.

Krylov subspace methods

Principle: Projection methods on Krylov subspaces:

$$K_m(A, v_1) = \text{span}\{v_1, Av_1, \dots, A^{m-1}v_1\}$$

- The most important class of iterative methods.
- Many variants exist depending on the subspace L .

Simple properties of K_m [$\mu \equiv \text{deg. of minimal polynomial of } v_1$.]

- $K_m = \{p(A)v_1 | p = \text{polynomial of degree } \leq m - 1\}$
- $K_m = K_\mu$ for all $m \geq \mu$. Moreover, K_μ is invariant under A .
- $\dim(K_m) = m$ iff $\mu \geq m$.

Arnoldi's Algorithm

- Goal: to compute an orthogonal basis of K_m .
- Input: Initial vector v_1 , with $\|v_1\|_2 = 1$ and m .

ALGORITHM : 15. *Arnoldi's procedure*

For $j = 1, \dots, m$ do

Compute $w := Av_j$

For $i = 1, \dots, j$, do
$$\begin{cases} h_{i,j} := (w, v_i) \\ w := w - h_{i,j}v_i \end{cases}$$

$h_{j+1,j} = \|w\|_2$; $v_{j+1} = w/h_{j+1,j}$

End

Result of Arnoldi's algorithm

Let

$$\overline{H}_m = \begin{bmatrix} x & x & x & x & x \\ x & x & x & x & x \\ & x & x & x & x \\ & & x & x & x \\ & & & x & x \\ & & & & x \end{bmatrix}; \quad H_m = \overline{H}_m(1:m, 1:m)$$

1. $V_m = [v_1, v_2, \dots, v_m]$ orthonormal basis of K_m .
2. $AV_m = V_{m+1}\overline{H}_m = V_m H_m + h_{m+1,m}v_{m+1}e_m^T$
3. $V_m^T AV_m = H_m \equiv \overline{H}_m$ – last row.

Appliaction to eigenvalue problems

- Write approximate eigenvector as $\tilde{u} = V_m y$ + Galerkin condition

$$(A - \tilde{\lambda}I)V_m y \perp \mathcal{K}_m \rightarrow V_m^H (A - \tilde{\lambda}I)V_m y = 0$$

- Approximate eigenvalues are eigenvalues of H_m

$$H_m y_j = \tilde{\lambda}_j y_j$$

Associated approximate eigenvectors are

$$\tilde{u}_j = V_m y_j$$

Typically a few of the outermost eigenvalues will converge first.

Restarted Arnoldi

In practice: Memory requirement of algorithm implies restarting is necessary

➤ Restarted Arnoldi for computing rightmost eigenpair:

ALGORITHM : 16. Restarted Arnoldi

1. **Start:** Choose an initial vector v_1 and a dimension m .
2. **Iterate:** Perform m steps of Arnoldi's algorithm.
3. **Restart:** Compute the approximate eigenvector $u_1^{(m)}$
4. associated with the rightmost eigenvalue $\lambda_1^{(m)}$.
5. If satisfied stop, else set $v_1 \equiv u_1^{(m)}$ and goto 2.

Example:

Small Markov Chain matrix [Mark(10) , dimension = 55]. Restarted Arnoldi procedure for computing the eigenvector associated with the eigenvalue with algebraically largest real part. We use $m = 10$.

m	$\Re(\lambda)$	$\Im(\lambda)$	Res. Norm
10	0.9987435899D+00	0.0	0.246D-01
20	0.9999523324D+00	0.0	0.144D-02
30	0.1000000368D+01	0.0	0.221D-04
40	0.1000000025D+01	0.0	0.508D-06
50	0.99999999996D+00	0.0	0.138D-07

Deflation

- Very useful in practice.
- Different forms: locking (subspace iteration), selective orthogonalization (Lanczos), Schur deflation, ...

A little background

Consider Schur canonical form $A = URU^H$

where U is a (complex) upper triangular matrix.

- Vector columns $u_1, \dots, u_n$ called **Schur vectors**.
- Note: Schur vectors are not unique. In particular, they depend on the order of the eigenvalues

Wiedlandt Deflation: Assume we have computed a right eigenpair λ_1, u_1 .
Wielandt deflation considers eigenvalues of

$$A_1 = A - \sigma u_1 v^H$$

Note:

$$\Lambda(A_1) = \{\lambda_1 - \sigma, \lambda_2, \dots, \lambda_n\}$$

Wielandt deflation preserves u_1 as an eigenvector as well all the left eigenvectors not associated with λ_1 .

- An interesting choice for v is to take simply $v = u_1$. In this case Wielandt deflation preserves Schur vectors as well.
- Can apply above procedure successively.

ALGORITHM : 17. *Explicit Deflation*

1. $A_0 = A$
2. *For* $j = 0 \dots \mu - 1$ *Do*:
3. *Compute a dominant eigenvector of* A_j
4. *Define* $A_{j+1} = A_j - \sigma_j u_j u_j^H$
5. *End*

- Computed $u_1, u_2, \dots$ form a set of Schur vectors for A .
- In Arnoldi: Accumulate each new converged eigenvector in columns 1, 2, 3, ... ['locked' set of eigenvectors.] + maintain orthogonality
- Alternative: implicit deflation (within a procedure such as Arnoldi).

Deflated Arnoldi

For $k = 1, \dots, NEV$ do: /* Eigenvalue loop */

1. For $j = k, k + 1, \dots, m$ do: /* Arnoldi loop*/

- Compute $w := Av_j$.
- Orthonormalize w against $v_1, v_2, \dots, v_j \rightarrow v_{j+1}$

2. Compute next approximate eigenpair $\tilde{\lambda}, \tilde{u}$.

3. Orthonormalize $\tilde{u}$ against $v_1, \dots, v_j$ ➤ Result = $\tilde{s}$ = approximate Schur vector.

4. Define $v_k := \tilde{s}$.

5. If approximation not satisfactory go to 1.

6. Else define $h_{i,k} = (Av_k, v_i)$, $i = 1, \dots, k$,

Thus, for $k = 2$:

$$V_m = \left[\underbrace{v_1, v_2}_{\text{Locked}}, \overbrace{v_3, \dots, v_m}^{\text{active}} \right]$$

$$H_m = \left(\begin{array}{cc|cccc} * & * & * & * & * & * \\ & * & * & * & * & * \\ \hline & & * & * & * & * \\ & & * & * & * & * \\ & & & * & * & * \\ & & & & * & * \end{array} \right)$$

➤ Similar techniques in Subspace iteration [G. Stewart's SRRIT]

 1 Run example with restarted Arnoldi with Deflation in *testArnRD*

Example:Matrix Mark(10) – small Markov chain matrix ($N = 55$).

➤ Continued from earlier example. [First eigenpair by iterative Arnoldi with $m = 10$] We now compute next 2 eigenvalues

Eig.	Mat-Vec's	$\Re(\lambda)$	$\Im(\lambda)$	Res. Norm
2	60	0.9370509474	0.0	0.870D-03
	69	0.9371549617	0.0	0.175D-04
	78	0.9371501442	0.0	0.313D-06
	87	0.9371501564	0.0	0.490D-08
3	96	0.8112247133	0.0	0.210D-02
	104	0.8097553450	0.0	0.538D-03
	112	0.8096419483	0.0	0.874D-04
	⋮	⋮	⋮	⋮
	152	0.8095717167	0.0	0.444D-07

Hermitian case: The Lanczos Algorithm

- The Hessenberg matrix becomes tridiagonal :

$$A = A^H \quad \text{and} \quad V_m^H A V_m = H_m \quad \rightarrow \quad H_m = H_m^H \longrightarrow$$

$$H_m = \begin{bmatrix} \alpha_1 & \beta_2 & & & \\ \beta_2 & \alpha_2 & \beta_3 & & \\ & \beta_3 & \alpha_3 & \beta_4 & \\ & & \cdot & \cdot & \cdot \\ & & & \cdot & \cdot & \cdot \\ & & & & \beta_m & \alpha_m \end{bmatrix}$$

Consequence:

3-term recurrence:

$$\beta_{j+1} v_{j+1} = A v_j - \alpha_j v_j - \beta_j v_{j-1}$$

Hermitian matrix + Arnoldi $\rightarrow$ Hermitian Lanczos

ALGORITHM : 18. *Lanczos*

1. Choose v_1 of norm unity. Set $\beta_1 \equiv 0, v_0 \equiv 0$
2. For $j = 1, 2, \dots, m$ Do:
3. $w_j := Av_j - \beta_j v_{j-1}$
4. $\alpha_j := (w_j, v_j)$
5. $w_j := w_j - \alpha_j v_j$
6. $\beta_{j+1} := \|w_j\|_2$. If $\beta_{j+1} = 0$ then Stop
7. $v_{j+1} := w_j / \beta_{j+1}$
8. EndDo

- In theory v_i 's defined by 3-term recurrence are orthogonal.
- However: in practice severe loss of orthogonality;

Lanczos with reorthogonalization

Observation [Paige, 1981]: Loss of orthogonality starts suddenly, when the first eigenpair converges. It indicates loss of linear independence of the v_i s. When orthogonality is lost, then several copies of the same eigenvalue start appearing.

Forms of Re-orthogonalization

Full – reorthogonalize v_{j+1} against all previous v_i 's every time.

Partial – reorthogonalize v_{j+1} against all previous v_i 's only when needed

Selective – reorthogonalize v_{j+1} against computed eigenvectors

None – Do not reorthogonalize - but take measures to deal with 'spurious' eigenvalues.

Partial reorthogonalization

- Partial reorthogonalization: reorthogonalize only when deemed necessary.
- Main question is **when**?
- Uses an inexpensive recurrence relation
- Work done in the 80's [Parlett, Simon, and co-workers] + more recent work [Larsen, '98]
- Package: PROPACK [Larsen] V 1: 2001, most recent: V 2.1 (Apr. 05)
- Often, need for reorthogonalization not too strong

The Lanczos Algorithm in the Hermitian Case

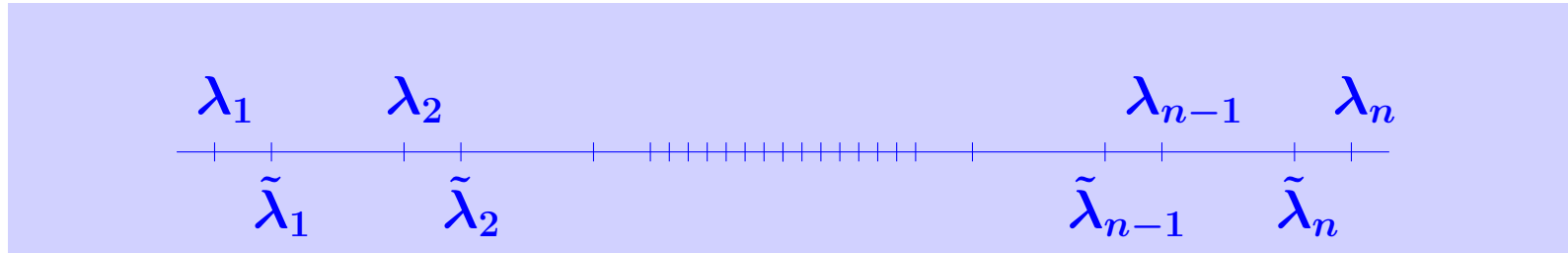
Assume eigenvalues sorted increasingly

$$\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$$

- Orthogonal projection method onto K_m ;
- To derive error bounds, use the Courant characterization

$$\begin{aligned}\tilde{\lambda}_1 &= \min_{u \in K, u \neq 0} \frac{(Au, u)}{(u, u)} = \frac{(A\tilde{u}_1, \tilde{u}_1)}{(\tilde{u}_1, \tilde{u}_1)} \\ \tilde{\lambda}_j &= \min_{\substack{u \in K, u \neq 0 \\ u \perp \tilde{u}_1, \dots, \tilde{u}_{j-1}}} \frac{(Au, u)}{(u, u)} = \frac{(A\tilde{u}_j, \tilde{u}_j)}{(\tilde{u}_j, \tilde{u}_j)}\end{aligned}$$

- Bounds for λ_1 easy to find – similar to linear systems.
- Ritz values approximate eigenvalues of A inside out:



Run *testLan* to see an illustration

A-priori error bounds

Theorem [Kaniel, 1966]: Let $\gamma_1 = \frac{\lambda_2 - \lambda_1}{\lambda_N - \lambda_2}$; Then:

$$0 \leq \lambda_1^{(m)} - \lambda_1 \leq (\lambda_N - \lambda_1) \left[\frac{\tan \angle(v_1, u_1)}{T_{m-1}(1 + 2\gamma_1)} \right]^2$$

Theorem [Kaniel, Paige, YS]. Let $\gamma_i = \frac{\lambda_{i+1} - \lambda_i}{\lambda_N - \lambda_{i+1}}$, $\kappa_i^{(m)} = \prod_{j < i} \frac{\lambda_j^{(m)} - \lambda_N}{\lambda_j^{(m)} - \lambda_i}$ Then:

$$0 \leq \lambda_i^{(m)} - \lambda_i \leq (\lambda_N - \lambda_1) \left[\kappa_i^{(m)} \frac{\tan \angle(v_i, u_i)}{T_{m-i}(1 + 2\gamma_i)} \right]^2$$

The Lanczos biorthogonalization ($A^H \neq A$)

ALGORITHM : 19. *Lanczos bi-orthogonalization*

1. Choose two vectors v_1, w_1 such that $(v_1, w_1) = 1$.
2. Set $\beta_1 = \delta_1 \equiv 0, w_0 = v_0 \equiv 0$
3. For $j = 1, 2, \dots, m$ Do:
 4. $\alpha_j = (Av_j, w_j)$
 5. $\hat{v}_{j+1} = Av_j - \alpha_j v_j - \beta_j v_{j-1}$
 6. $\hat{w}_{j+1} = A^T w_j - \alpha_j w_j - \delta_j w_{j-1}$
 7. $\delta_{j+1} = |(\hat{v}_{j+1}, \hat{w}_{j+1})|^{1/2}$. If $\delta_{j+1} = 0$ Stop
 8. $\beta_{j+1} = (\hat{v}_{j+1}, \hat{w}_{j+1}) / \delta_{j+1}$
 9. $w_{j+1} = \hat{w}_{j+1} / \beta_{j+1}$
 10. $v_{j+1} = \hat{v}_{j+1} / \delta_{j+1}$
11. EndDo

- Builds a pair of biorthogonal bases for the two subspaces

$$\mathcal{K}_m(A, v_1) \quad \text{and} \quad \mathcal{K}_m(A^H, w_1)$$

- Many choices for $\delta_{j+1}, \beta_{j+1}$ in lines 7 and 8. Only constraint:

$$\delta_{j+1}\beta_{j+1} = (\hat{v}_{j+1}, \hat{w}_{j+1})$$

Let

$$T_m = \begin{bmatrix} \alpha_1 & \beta_2 & & & \\ \delta_2 & \alpha_2 & \beta_3 & & \\ & \cdot & \cdot & \cdot & \\ & & \delta_{m-1} & \alpha_{m-1} & \beta_m \\ & & & \delta_m & \alpha_m \end{bmatrix} \cdot$$

- $v_i \in \mathcal{K}_m(A, v_1)$ and $w_j \in \mathcal{K}_m(A^T, w_1)$.

If the algorithm does not break down before step m , then the vectors $v_i, i = 1, \dots, m$, and $w_j, j = 1, \dots, m$, are biorthogonal, i.e.,

$$(v_j, w_i) = \delta_{ij} \quad 1 \leq i, j \leq m .$$

Moreover, $\{v_i\}_{i=1,2,\dots,m}$ is a basis of $\mathcal{K}_m(A, v_1)$ and $\{w_i\}_{i=1,2,\dots,m}$ is a basis of $\mathcal{K}_m(A^H, w_1)$ and

$$AV_m = V_m T_m + \delta_{m+1} v_{m+1} e_m^H,$$

$$A^H W_m = W_m T_m^H + \bar{\beta}_{m+1} w_{m+1} e_m^H,$$

$$W_m^H AV_m = T_m .$$

➤ If θ_j, y_j, z_j are, respectively an eigenvalue of T_m , with associated right and left eigenvectors y_j and z_j respectively, then corresponding approximations for A are

Ritz value	Right Ritz vector	Left Ritz vector
θ_j	$V_m y_j$	$W_m z_j$

[Note: terminology is abused slightly - Ritz values and vectors normally refer to Hermitian cases.]

Advantages and disadvantages

Advantages:

- Nice three-term recurrence – requires little storage in theory.
- Computes left and a right eigenvectors at the same time

Disadvantages:

- Algorithm can break down or nearly break down.
- Convergence not too well understood. Erratic behavior
- Not easy to take advantage of the tridiagonal form of T_m .

 3 Explore the literature on “Look-ahead Lanczos” which aims at resolving some of these issues.

Preconditioning eigenvalue problems and other approaches

- *Preconditioning eigenvalue problems: Shift-invert, polynomial*
- *Polynomial filters, Implicit restarts*
- *The Davidson approach*
- *Jacobi-Davidson*
- *Harmonic Ritz values*

Preconditioning eigenvalue problems

➤ Goal: To extract good approximations to add to a subspace in a projection process. Result: faster convergence.

➤ Best known technique: Shift-and-invert; Work with

$$B = (A - \sigma I)^{-1}$$

➤ Some success with polynomial preconditioning [Chebyshev iteration / least-squares polynomials]. Work with

$$B = p(A)$$

➤ Above preconditioners preserve eigenvectors. Other methods (Davidson) use a more general preconditioner M .

Shift-and-invert preconditioning

Main idea: to use Arnoldi, or Lanczos, or subspace iteration for the matrix $B = (A - \sigma I)^{-1}$. The matrix B need not be computed explicitly. Each time we need to apply B to a vector we solve a system with B .

- Factor $B = A - \sigma I = LU$. Then each solution $Bx = y$ requires solving $Lz = y$ and $Ux = z$.

How to deal with complex shifts?

- If A is complex need to work in complex arithmetic.
- If A is real, then instead of $(A - \sigma I)^{-1}$ use

$$\Re(A - \sigma I)^{-1} = \frac{1}{2} \left[(A - \sigma I)^{-1} + (A - \bar{\sigma} I)^{-1} \right]$$

Preconditioning by polynomials

Main idea:

Iterate with $p(A)$ instead of A in Arnoldi or Lanczos,...

- Used very early on in subspace iteration [Rutishauser, 1959.]
- Usually not as reliable as Shift-and-invert techniques but less demanding in terms of storage.

Question: How to find a good polynomial (dynamically)?

- 1 Use of Chebyshev polynomials over ellipses
- 2 Use polynomials based on Leja points
- 3 Least-squares polynomials over polygons
- 4 Polynomials from previous Arnoldi decompositions

Approaches:

Polynomial filters and implicit restart

Goal: exploit the Arnoldi procedure to apply polynomial filter of the form:
 $p(t) = (t - \theta_1)(t - \theta_2) \dots (t - \theta_q)$

Assume

$$AV_m = V_m H_m + \hat{v}_{m+1} e_m^T$$

and consider first factor: $(t - \theta_1)$

$$(A - \theta_1 I)V_m = V_m(H_m - \theta_1 I) + \hat{v}_{m+1} e_m^T$$

Let $H_m - \theta_1 I = Q_1 R_1$. Then,

$$\begin{aligned}(A - \theta_1 I)V_m &= V_m Q_1 R_1 + \hat{v}_{m+1} e_m^T \rightarrow \\(A - \theta_1 I)(V_m Q_1) &= (V_m Q_1) R_1 Q_1 + \hat{v}_{m+1} e_m^T Q_1 \rightarrow \\A(V_m Q_1) &= (V_m Q_1)(R_1 Q_1 + \theta_1 I) + \hat{v}_{m+1} e_m^T Q_1\end{aligned}$$

Notation:

$$R_1 Q_1 + \theta_1 I \equiv H_m^{(1)}; \quad (b_{m+1}^{(1)})^T \equiv e_m^T Q_1; \quad V_m Q_1 \equiv V_m^{(1)}$$

➤
$$A V_m^{(1)} = V_m^{(1)} H_m^{(1)} + v_{m+1} (b_{m+1}^{(1)})^T$$

➤ Note that $H_m^{(1)}$ is upper Hessenberg.

➤ Similar to an Arnoldi decomposition.

Observe:

- $R_1 Q_1 + \theta_1 I \equiv$ matrix resulting from one step of the QR algorithm with shift θ_1 applied to H_m .
- First column of $V_m^{(1)}$ is a multiple of $(A - \theta_1 I)v_1$.
- The columns of $V_m^{(1)}$ are orthonormal.

Can now apply second shift in same way:

$$(A - \theta_2 I)V_m^{(1)} = V_m^{(1)}(H_m^{(1)} - \theta_2 I) + v_{m+1}(b_{m+1}^{(1)})^T \rightarrow$$

Similar process: $(H_m^{(1)} - \theta_2 I) = Q_2 R_2$ then $\times Q_2$ to the right:

$$(A - \theta_2 I)V_m^{(1)}Q_2 = (V_m^{(1)}Q_2)(R_2Q_2) + v_{m+1}(b_{m+1}^{(1)})^T Q_2$$

$$AV_m^{(2)} = V_m^{(2)}H_m^{(2)} + v_{m+1}(b_{m+1}^{(2)})^T$$

Now:

$$\begin{aligned} \text{1st column of } V_m^{(2)} &= \text{scalar} \times (A - \theta_2 I)v_1^{(1)} \\ &= \text{scalar} \times (A - \theta_2 I)(A - \theta_1 I)v_1 \end{aligned}$$

➤ Note that

$$(b_{m+1}^{(2)})^T = e_m^T Q_1 Q_2 = [0, 0, \dots, 0, \eta_1, \eta_2, \eta_3]$$

➤ Let: $\hat{V}_{m-2} = [\hat{v}_1, \dots, \hat{v}_{m-2}]$ consist of first $m - 2$ columns of $V_m^{(2)}$ and $\hat{H}_{m-2} = H_m(1 : m - 2, 1 : m - 2)$. Then

$$A\hat{V}_{m-2} = \hat{V}_{m-2}\hat{H}_{m-2} + \hat{\beta}_{m-1}\hat{v}_{m-1}e_m^T \quad \text{with} \\ \hat{\beta}_{m-1}\hat{v}_{m-1} \equiv \eta_1 v_{m+1} + h_{m-1,m-2}^{(2)} v_{m-1}^{(2)} \quad \|\hat{v}_{m-1}\|_2 = 1$$

- Result: An Arnoldi process of $m - 2$ steps with the initial vector $p(A)v_1$.
- In other words: We know how to apply polynomial ‘filtering’ via a form of the Arnoldi process, combined with the QR algorithm.

The Davidson approach

Goal: to use a more general preconditioner to introduce good new components to the subspace.

- Ideal new vector would be eigenvector itself!
- Next best thing: an approximation to $(A - \mu I)^{-1}r$ where $r = (A - \mu I)z$, current residual.
- Approximation written in the form $M^{-1}r$. Note that M can vary at every step if needed.

ALGORITHM : 20. Davidson's method ($A = A^T$)

1. Choose an initial unit vector v_1 . Set $V_1 = [v_1]$.
2. For $j = 1, \dots, m$ Do:
3. $w := Av_j$.
4. Update $H_j \equiv V_j^T AV_j$
5. Compute the smallest eigenpair μ, y of H_j .
6. $z := V_j y$ $r := Az - \mu z$
7. Test for convergence. If satisfied Return
8. Compute $t := M_j^{-1} r$
9. Compute $V_{j+1} := ORTHN([V_j, t])$
10. EndDo

- Note: Traditional Davidson uses diagonal preconditioning: $M_j = D - \sigma_j I$.
- Will work only for some matrices

Other options:

- Shift-and-invert using ILU [negatives: expensive + hard to parallelize.]
- Filtering (by averaging)
- Filtering by using smoothers (multigrid style)
- Iterative solves [e.g., Jacobi-Davidson]

Jacobi-Davidson: Introduction via Newton's method

Assumptions: $M = A + E$ and $Az \approx \mu z$

Goal: to find an improved eigenpair $(\mu + \eta, z + v)$.

- Write $A(z + v) = (\mu + \eta)(z + v)$ and neglect second order terms + rearrange ➤

$$(M - \mu I)v - \eta z = -r \quad \text{with} \quad r \equiv (A - \mu I)z$$

- Unknowns: η and v .
- Underdetermined system. Need one constraint.
- Add the condition: $w^H v = 0$ for some vector w .

In matrix form:

$$\begin{bmatrix} M - \mu I & -z \\ w^H & 0 \end{bmatrix} \begin{bmatrix} v \\ \eta \end{bmatrix} = \begin{bmatrix} -r \\ 0 \end{bmatrix}$$

➤ Eliminate v from
second equation:

$$\begin{aligned} (M - \mu I)v - \eta z &= -r \\ w^H(M - \mu I)^{-1}z \cdot \eta &= w^H(M - \mu I)^{-1}r \end{aligned}$$

➤ Solution: [Olsen's method]

$$\eta = \frac{w^H(M - \mu I)^{-1}r}{w^H(M - \mu I)^{-1}z} \quad v = -(M - \mu I)^{-1}(r - \eta z)$$

When $M = A$, corresponds to Newton's method for solving

$$\begin{cases} (A - \lambda I)u = 0 \\ w^T u = \textit{Constant} \end{cases}$$

Another characterization of the solution:

$$v = -(M - \mu I)^{-1}r + \eta(M - \mu I)^{-1}z, \\ \eta \text{ such that } w^H v = 0$$

Alternative expression using projectors.

➤ Let P_z = projector in direction of z ,
s.t. $P_z r = r$: $P_z = I - \frac{zs^H}{s^H z}$ with $s \perp r$

➤ Similarly let P_w any projector that leaves v unchanged. Then Olsen's solution can be rewritten in mathematically equivalent form:

$$[P_z(M - \mu I)P_w]v = -r \quad w^H v = 0$$

The Jacobi-Davidson approach

- In orthogonal projection methods (e.g. Arnoldi) we have $r \perp z$
- Also it is natural to take $w \equiv z$. Assume $\|z\|_2 = 1$

With the above assumptions, Olsen's correction equation is mathematically equivalent to finding v such that :

$$(I - zz^H)(M - \mu I)(I - zz^H)v = -r \quad v \perp z$$

- Main attraction: can use iterative method for the solution of the correction equation. (M -solves not explicitly required).

Harmonic Ritz values

Main idea: take $L = AK$ in projection process

➤ In context of Arnoldi's method.
Write $\tilde{u} = V_m y$ then:

$$(A - \tilde{\lambda}I)V_m y \perp \{AV_m\}$$

Using $AV_m = V_{m+1}\underline{H}_m$ ➤

$$\underline{H}_m^H V_{m+1}^H [V_{m+1}\underline{H}_m y - \tilde{\lambda}V_m y] = 0$$

Notation: $H_m = \underline{H}_m$ — last row. Then

$$\underline{H}_m^H \underline{H}_m y - \tilde{\lambda} \underline{H}_m^H y = 0$$

or

$$\left(H_m^H H_m + h_{m+1,m}^2 e_m e_m^H \right) y = \tilde{\lambda} H_m^H y$$

Remark:

Assume H_m is nonsingular and multiply both sides by H_m^{-H} . Then, the problem is equivalent to

$$\left(H_m + z_m e_m^H \right) y = \tilde{\lambda} y$$

with $z_m = h_{m+1,m}^2 H_m^{-H} e_m$.

➤ Modified from H_m only in the last column.

Implementation within Davidson framework

- Slight variation to standard Davidson: Introduce $z_i = M_i^{-1}r_i$ to subspace. Proceed as in FGMRES: $v_{j+1} = \text{Orthn}(Az_j, V_j)$.
- From Gram-Schmidt process: $Az_j = \sum_{i=1}^{j+1} h_{ij}v_i$
- Hence the relation

$$AZ_m = V_{m+1}\bar{H}_m$$

Approximation: $\lambda, \tilde{u} = Z_m y$

Galerkin Condition: $r \perp AZ_m$ gives the generalized problem

$$\bar{H}_m^H \bar{H}_m y = \lambda \bar{H}_m^H V_{m+1}^H Z_m y$$

BACK TO GRAPHS: PATHS, CENTRALITY, PAGERANK

- *Back to graph models of sparse matrices*
- *Paths and powers of matrices*
- *Perron Frobenius theorem*
- *Application: Markov chains*
- *PageRank*
- *Notions of centrality*

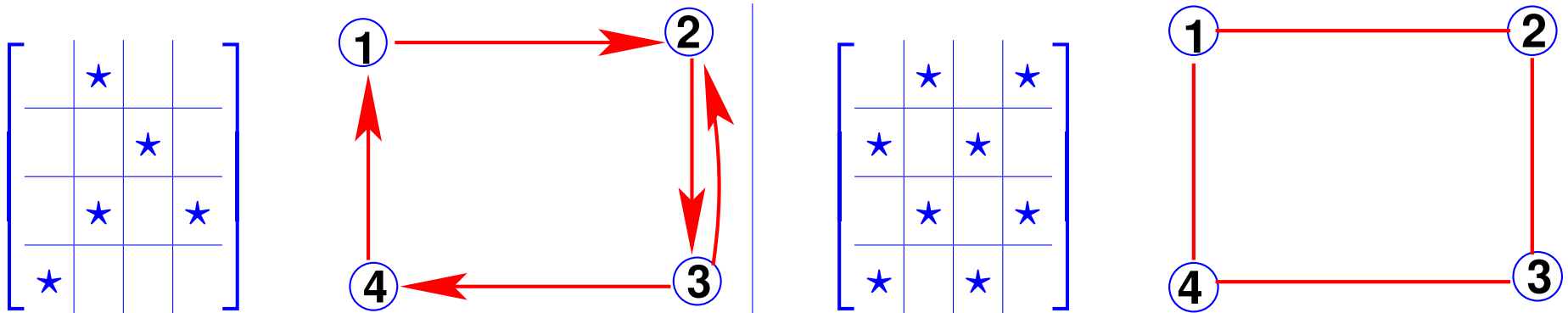
Graph Representations of Sparse Matrices. Recall:

Adjacency Graph $G = (V, E)$ of an $n \times n$ matrix A :

$$V = \{1, 2, \dots, N\} \quad E = \{(i, j) | a_{ij} \neq 0\}$$

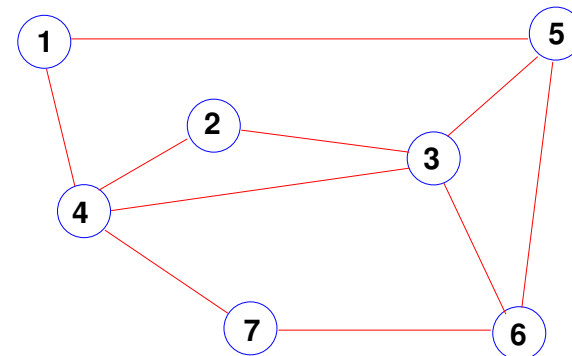
➤ G == undirected if A has a symmetric pattern

Example:





Show the matrix pattern for the graph on the right and give an interpretation of the path v_4, v_2, v_3, v_5, v_1 on the matrix



Example: Adjacency graph of:

$$A = \begin{bmatrix} & \star & & \star & & & \\ \star & & \star & & & & \\ & \star & & \star & \star & \star & \\ \star & & \star & & & & \\ & & \star & & & \star & \\ & & \star & & \star & & \\ & & \star & & & & \end{bmatrix}.$$

Example: For any **adjacency** matrix A , what is the graph of A^2 ? [interpret in terms of paths in the graph of A]

Interpretation of graphs of matrices

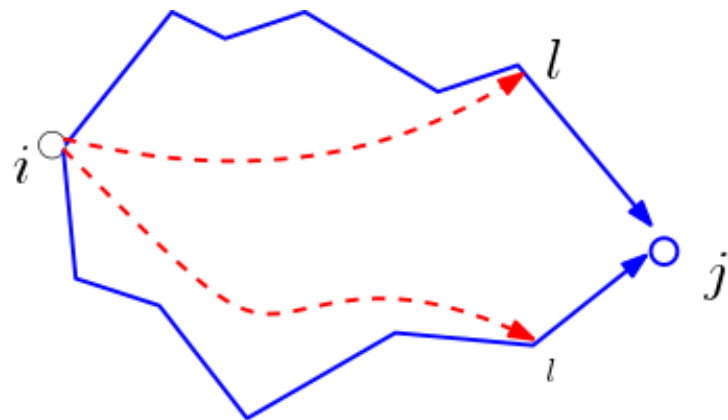
-  2 What is the graph of $A + B$ (for two $n \times n$ matrices)?
-  3 What is the graph of A^T ?
-  4 What is the graph of $A.B$?

Paths in graphs

 5 What is the graph of A^k ?

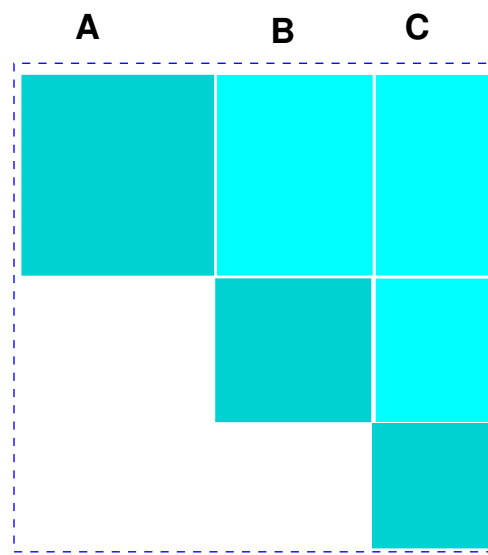
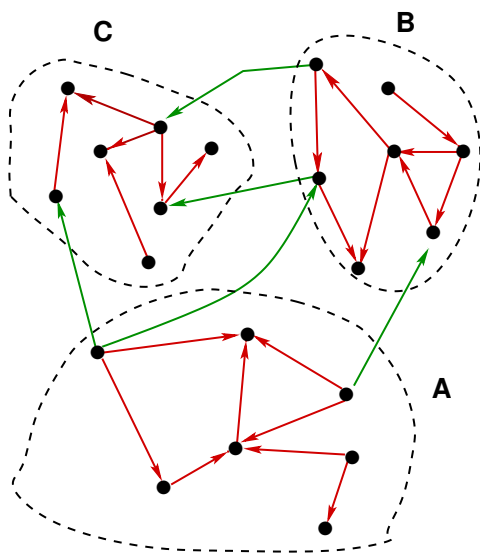
Theorem Let A be the adjacency matrix of a graph $G = (V, E)$. Then for $k \geq 0$ and vertices u and v of G , the number of paths of length k starting at u and ending at v is equal to $(A^k)_{u,v}$.

Proof: Proof is by induction. ■



If $C = BA$ then $c_{ij} = \sum_l b_{il}a_{lj}$. Take $B = A^{k-1}$ and use induction. Any path of length k is formed as a path of length $k-1$ to some node l completed by an edge from l to j . Because a_{lj} is one for that last edge, c_{ij} is just the sum of all possible paths of length k from i to j

- Recall (definition): A matrix is *reducible* if it can be permuted into a block upper triangular matrix.
- Note: A matrix is reducible iff its adjacency graph is not (strongly) connected, i.e., iff it has more than one connected component.



➤ No edges from C to A or B . No edges from B to A .

Theorem: Perron-Frobenius An irreducible, nonnegative $n \times n$ matrix A has a real, positive eigenvalue λ_1 such that:

- (i) λ_1 is a simple eigenvalue of A ;
- (ii) λ_1 admits a positive eigenvector u_1 ; and
- (iii) $|\lambda_i| \leq \lambda_1$ for all other eigenvalues λ_i where $i > 1$.

➤ The spectral radius is equal to the eigenvalue λ_1

➤ Definition : a graph is d regular if each vertex has the same degree d .

Proposition: The spectral radius of a d regular graph is equal to d .

Proof: The vector e of all ones is an eigenvector of A associated with the eigenvalue $\lambda = d$. In addition this eigenvalue is the largest possible (consider the infinity norm of A). Therefore e is the Perron-Frobenius vector u_1 . ■

Application: Markov Chains

- Read about Markov Chains in Sect. 10.9 of:
https://www-users.cs.umn.edu/~saad/eig_book_2ndEd.pdf
- Let $\pi \equiv$ row vector of stationary probabilities
- Then π satisfies the equation $\rightarrow \pi P = \pi$
- P is the probability transition matrix and it is ‘stochastic’:

A matrix P is said to be *stochastic* if :

- (i) $p_{ij} \geq 0$ for all i, j
- (ii) $\sum_{j=1}^n p_{ij} = 1$ for $i = 1, \dots, n$
- (iii) No column of P is a zero column.

➤ Spectral radius is ≤ 1

 6 Why?

➤ Assume P is irreducible. Then:

➤ Perron Frobenius $\rightarrow \rho(P) = 1$ is an eigenvalue and associated eigenvector has positive entries.

➤ Probabilities are obtained by scaling π by its sum.

➤ Example: One of the 2 models used for page rank.

Example: A college Fraternity has 50 students at various stages of college (Freshman, Sophomore, Junior, Senior). There are 6 potential stages for the following year: Freshman, Sophomore, Junior, Senior, graduated, or left-without degree. Following table gives probability of transitions from one stage to next

To From	Fr	So.	Ju.	Sr.	Grad	lwd
Fr.	.2	0	0	0	0	0
So.	.6	.1	0	0	0	0
Ju.	0	.7	.1	0	0	0
Sr.	0	0	.8	.1	0	0
Grad	0	0	0	.75	1	0
lwd	.2	.2	.1	.15	0	1

 7 What is P ? Assume initial population is $x_0 = [10, 16, 12, 12, 0, 0]$ and follow the stages of the students for a few years. What is the probability that a student will graduate? What is the probability that s/he leaves without a degree?

- Can be viewed as an application of Markov Chains

Main point: A page is important if it is pointed to by other important pages.

- Importance of your page (its PageRank) is determined by summing the page ranks of all pages which point to it.
- Weighting: If a page points to several other pages, then the weighting should be distributed proportionally.
- Imagine many tokens doing a random walk on this graph:
 - (δ/n) chance to follow one of the n links on a page,
 - $(1 - \delta)$ chance to jump to a random page.
 - What's the chance a token will land on each page?

Page-Rank - definitions

If $T_1, \dots, T_n$ point to page T_i then

$$\rho(T_i) = 1 - \delta + \delta \left[\frac{\rho(T_1)}{|T_1|} + \frac{\rho(T_2)}{|T_2|} + \dots + \frac{\rho(T_n)}{|T_n|} \right]$$

➤ $|T_j|$ = count of links going out of Page T_j . So the 'vote' $\rho(T_j)$ is spread evenly among $|T_j|$ links.

➤ Sum of all PageRanks == 1: $\sum_T \rho(T) = 1$

➤ δ is a 'damping' parameter close to 1 – e.g. 0.85

➤ Defines a (possibly huge) Hyperlink matrix H

$$h_{ij} = \begin{cases} \frac{1}{|T_i|} & \text{if } i \text{ points to } j \\ 0 & \text{otherwise} \end{cases}$$



4 Nodes

A points to B and D

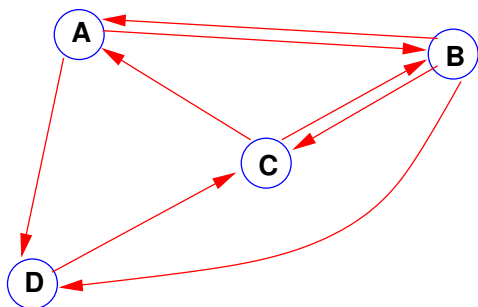
B points to A, C, and D

C points to A and B

D points to C

1) What is the H matrix?

2) the graph?



	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>		1/2		1/2
<i>B</i>	1/3		1/3	1/3
<i>C</i>	1/2	1/2		
<i>D</i>			1	

➤ Row- sums of H are = 1.

➤ Sum of all PageRanks will be one:

$$\sum_{\text{All-Pages}_A} \rho(A) = 1.$$

➤ H is a stochastic matrix [actually it is forced to be by changing zero rows]

Algorithm (PageRank)

1. Select initial **row** vector v ($v \geq 0$)
2. For $i=1:\text{maxitr}$
- 3 $v := (1 - \delta)e^T + \delta vH$
4. end

 Do a few steps of this algorithm for previous example with $\delta = 0.85$.

➤ This is a row iteration..

$$\boxed{v} = \boxed{(1 - \delta)e^T} + \boxed{v} \cdot \boxed{\delta H}$$

A few properties:

- v will remain ≥ 0 . [combines non-negative vectors]
- More general iteration is of the form

$$v := v \underbrace{[(1 - \delta)E + \delta H]}_G \quad \text{with} \quad E = ez^T$$

where z is a probability vector $e^T z = 1$ [Ex. $z = \frac{1}{n}e$]

- A variant of the power method.
- e is a right-eigenvector of G associated with $\lambda = 1$. We are interested in the left eigenvector.



Run *test_pr* + other drivers in matlab page

Kleinberg's Hubs and Authorities

- Idea is to put order into the web by ranking pages by their degree of Authority or "Hubness".
- An Authority is a page pointed to by many important pages.
 - Authority Weight = sum of Hub Weights from In-Links.
- A Hub is a page that points to many important pages:
 - Hub Weight = sum of Authority Weights from Out-Links.
- Source:

<http://www.cs.cornell.edu/home/kleinber/auth.pdf>

Computation of Hubs and Authorities

- Simplify computation by forcing sum of squares of weights to be 1.
- $\text{Auth}_j = x_j = \sum_{i:(i,j) \in \text{Edges}} \text{Hub}_i.$
- $\text{Hub}_i = y_i = \sum_{j:(i,j) \in \text{Edges}} \text{Auth}_j.$
- Let A = Adjacency matrix: $a_{ij} = 1$ if $(i, j) \in \text{Edges}.$
- $y = Ax, x = A^T y.$
- Iterate ... to leading eigenvectors of $A^T A$ & $AA^T.$
- Answer: Leading Singular Vectors!

GRAPH CENTRALITY

Centrality in graphs

- Goal: measure importance of a node, edge, subgraph, .. in a graph
- Many measures introduced over the years
- Early Work: Freeman '77 [introduced 3 measures] – based on 'paths in graph'
- **Many** different ways of defining centrality! We will just see a few

Degree centrality: (simplest) 'Nodes with high degree are important'
(note: scaling $n - 1$ is unimportant)

$$C_D(v) = \deg(v)$$

Closeness centrality: 'Nodes that are close to many other nodes are important'

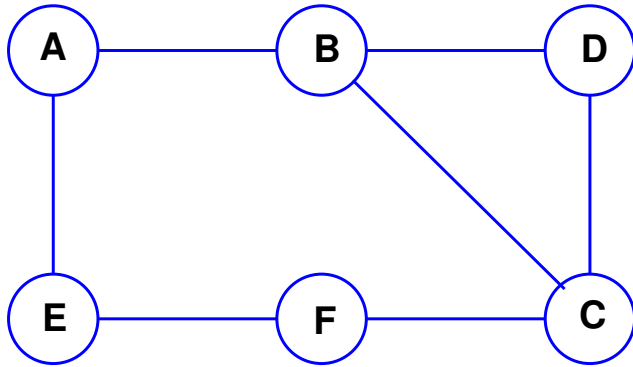
$$C_C(v) = \frac{1}{\sum_{w \neq v} d(v, w)}$$

Betweenness centrality:
(Freeman '77)

$$C_B(v) = \sum_{u \neq v, w \neq v} \frac{\sigma_{uw}(v)}{\sigma_{uw}}$$

- σ_{uw} = total # shortest paths from u to w
- $\sigma_{uw}(v)$ = total # shortest paths from u to w passing through v
- 'Nodes that are on many shortest paths are important'

Example: Find $C_D(v)$; $C_C(v)$; $C_B(v)$ when $v = C$



(u,w)	$\sigma_{uw}(v)$	σ_{uw}	/	(u,w)	$\sigma_{uw}(v)$	σ_{uw}	/
(A,B)	0	1	0	(B,E)	0	1	0
(A,D)	0	1	0	(B,F)	1	1	1
(A,E)	0	1	0	(D,E)	1	2	.5
(A,F)	0	1	0	(D,F)	1	1	1
(B,D)	0	1	0	(E,F)	0	1	0

➤ $C_D(v) = 3$;

➤ $C_C(v) = 1/[d_{CA} + d_{CB} + d_{CD} + d_{CE} + d_{CF}]$
 $= 1/[2 + 1 + 1 + 2 + 1] = 1/7$

➤ $C_B(v) = 2.5$ (add all ratios in table)

 11 Redo this for $v = B$

Eigenvector centrality:

- Suppose we have n nodes v_j , $j = 1, \dots, n$ — each with a measure of importance ('prestige') p_j
- Principle: prestige of i depends on that of its neighbors.
- Prestige x_i = multiple of sum of prestiges of neighbors pointing to it
- x_i = component of eigenvector associated with λ .
- Perron Frobenius theorem at play again: take largest eigenvalue $\rightarrow x_i$'s nonnegative

$$\lambda x_i = \sum_{j \in \mathcal{N}(i)} x_j = \sum_{j=1}^n a_{ji} x_j$$

GRAPH LAPLACEANS AND THEIR APPLICATIONS

- *Graph Laplaceans, definitions and basic properties*
- *Graph partitioning –*
- *Introduction to clustering*
- *Graph Embeddings, vertex embeddings . The problem*
- *Use of Graph Laplaceans, Laplacean Eigenmaps*
- *Use of similarity graphs: Locally Linear Embeddings*
- *Explicit dimension reduction method: PCA, LLP, ...*

Graph Laplaceans - Definition

➤ “Laplace-type” matrices associated with general undirected graphs – useful in many applications

➤ Given a graph $G = (V, E)$ define

- A matrix W of weights w_{ij} for each edge
- Assume $w_{ij} \geq 0$, $w_{ii} = 0$, and $w_{ij} = w_{ji} \forall (i, j)$
- The diagonal matrix $D = \text{diag}(d_i)$ with $d_i = \sum_{j \neq i} w_{ij}$

➤ Corresponding **graph Laplacean** of G is:

$$L = D - W$$

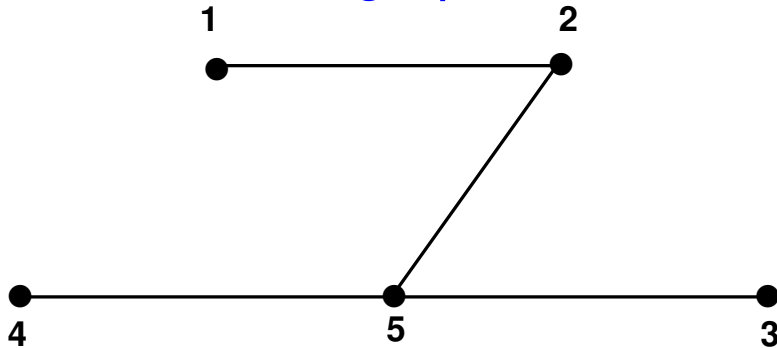
➤ Gershgorin's theorem $\rightarrow L$ is positive semidefinite.

➤ Simplest case:

$$w_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \text{ \& } i \neq j \\ 0 & \text{else} \end{cases} \quad D = \text{diag} \left[d_i = \sum_{j \neq i} w_{ij} \right]$$

Example:

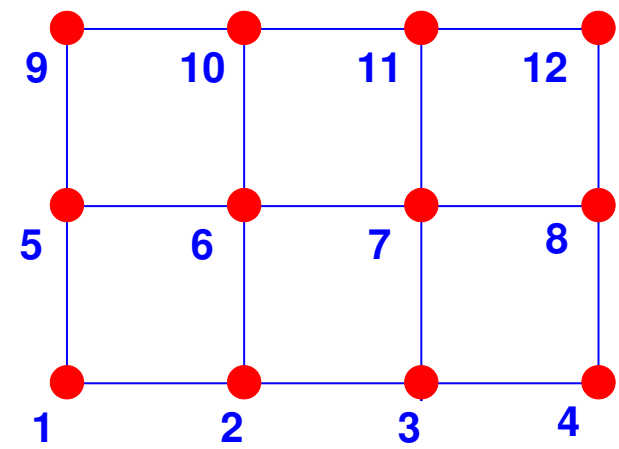
Consider the graph



$$L = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & -1 & -1 & -1 & 3 \end{bmatrix}$$



Define the graph Laplacean for the graph associated with the simple mesh shown next. [use the simple weights of 0 or 1]. What is the difference with the discretization of the Laplace operator for case when mesh is the same as this graph?



Proposition:

- (i) L is symmetric semi-positive definite.
- (ii) L is singular with $\mathbf{1}$ as a null vector.
- (iii) If G is connected, then $\text{Null}(L) = \text{span}\{\mathbf{1}\}$
- (iv) If G has $k > 1$ connected components $G_1, G_2, \dots, G_k$, then the nullity of L is k and $\text{Null}(L)$ is spanned by the vectors $z^{(j)}$, $j = 1, \dots, k$ defined by:

$$(z^{(j)})_i = \begin{cases} 1 & \text{if } i \in G_j \\ 0 & \text{if not.} \end{cases}$$

Proof: (i) and (ii) seen earlier and are trivial. (iii) Clearly $u = \mathbf{1}$ is a null vector for L . The vector $D^{-1/2}u$ is an eigenvector for the matrix $D^{-1/2}LD^{-1/2} = I - D^{-1/2}WD^{-1/2}$ associated with the smallest eigenvalue. It is also an eigenvector for $D^{-1/2}WD^{-1/2}$ associated with the largest eigenvalue. By the Perron Frobenius theorem this is a simple eigenvalue... (iv) Can be proved from the fact that L can be written as a direct sum of the Laplacian matrices for $G_1, \dots, G_k$. ■

A few properties of graph Laplaceans

Define: oriented incidence matrix H : (1) First orient the edges $i \sim j$ into $i \rightarrow j$ or $j \rightarrow i$. (2) Rows of H indexed by vertices of G . Columns indexed by edges. (3) For each (i, j) in E , define the corresponding column in H as $\sqrt{w(i, j)}(e_i - e_j)$.

Example: In previous example (4 p. back) orient $i \rightarrow j$ so that $j > i$ [lower triangular matrix representation]. Then matrix H is:

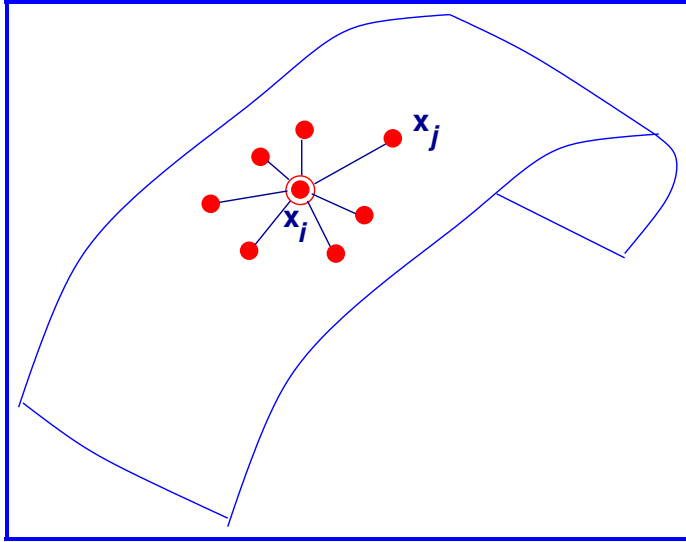
$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -1 & -1 & -1 \end{bmatrix}$$

Property 1

$$L = HH^T$$

 2 Re-prove part (iv) of previous proposition by using this property.

A few properties of graph Laplaceans



Strong relation between $x^T L x$ and local distances between entries of x

► Let $L =$ any matrix s.t. $L = D - W$, with $D = \text{diag}(d_i)$ and

$$w_{ij} \geq 0, \quad d_i = \sum_{j \neq i} w_{ij}$$

Property 2: for any $x \in \mathbb{R}^n$:

$$x^T L x = \frac{1}{2} \sum_{i,j} w_{ij} |x_i - x_j|^2$$

Property 3: (generalization) for any $Y \in \mathbb{R}^{d \times n}$:

$$\text{Tr}[YLY^\top] = \frac{1}{2} \sum_{i,j} w_{ij} \|y_i - y_j\|^2$$

► Note: y_j = j -th column of Y . Usually $d < n$. Each column can represent a data sample.

Property 4: For the particular $L = I - \frac{1}{n} \mathbf{1} \mathbf{1}^\top$

$$XLX^\top = \bar{X}\bar{X}^\top == n \times \text{Covariance matrix}$$

Property 5: L is singular and admits the null vector

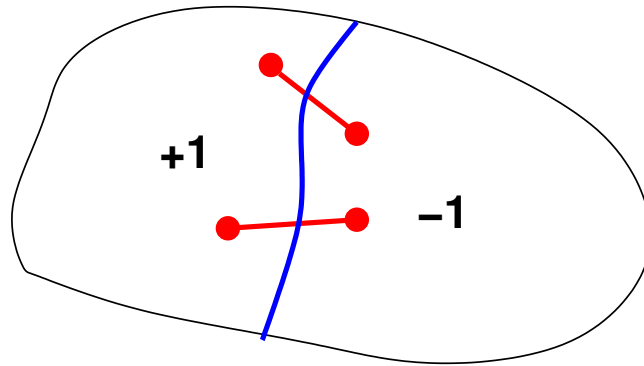
$$\mathbf{1} = \text{ones}(n, 1)$$

Property 6: (Graph partitioning) Consider situation when $w_{ij} \in \{0, 1\}$. If x is a vector of signs (± 1) then

$$x^\top Lx = 4 \times (\text{'number of edge cuts'})$$

edge-cut = pair (i, j) with $x_i \neq x_j$

➤ Consequence: Can be used to partition graphs



➤ Would like to minimize (Lx, x) subject to $x \in \{-1, 1\}^n$ and $e^T x = 0$
[balanced sets]

➤ Will solve a relaxed form of this problem

 3 What if we replace x by a vector of ones (representing one partition) and zeros (representing the other)?

 4 Let x be any vector and $y = x + \alpha \mathbf{1}$ and L a graph Laplacean. Compare (Lx, x) with (Ly, y) .

➤ Consider any symmetric (real) matrix A with eigenvalues $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ and eigenvectors $u_1, \dots, u_n$

➤ Recall that:

(Min reached for $x = u_1$)

$$\min_{x \in \mathbb{R}^n} \frac{(Ax, x)}{(x, x)} = \lambda_1$$

➤ In addition:

(Min reached for $x = u_2$)

$$\min_{x \perp u_1} \frac{(Ax, x)}{(x, x)} = \lambda_2$$

➤ For a graph Laplacean $u_1 = \mathbf{1}$ = vector of all ones and

➤ ...vector u_2 is called the Fiedler vector. It solves a **relaxed** form of the problem -

$$\min_{x \in \{-1, 1\}^n; \mathbb{1}^T x = 0} \frac{(Lx, x)}{(x, x)}$$

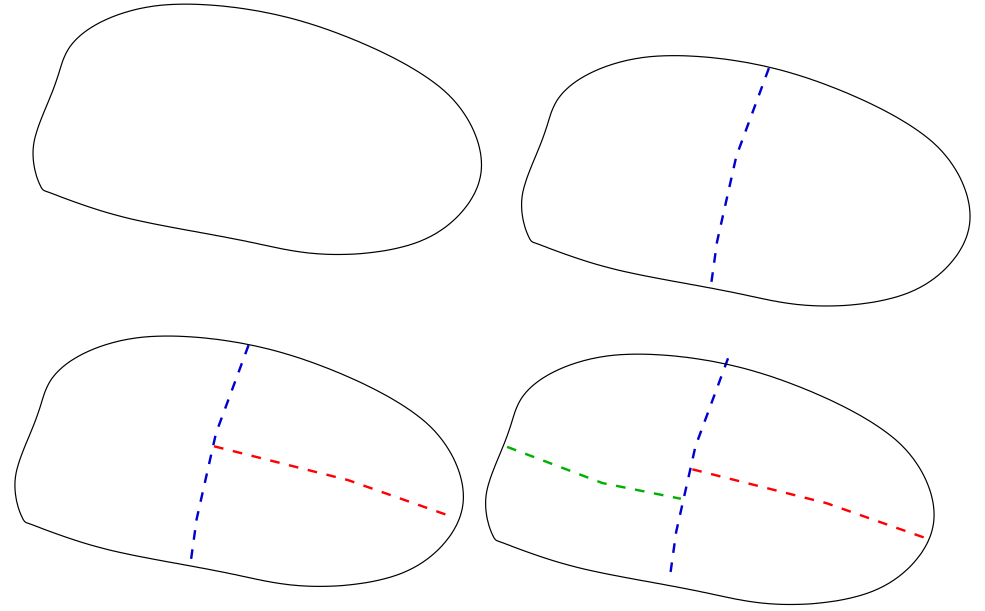
→

$$\min_{x \in \mathbb{R}^n; \mathbb{1}^T x = 0} \frac{(Lx, x)}{(x, x)}$$

➤ Define $v = u_2$ then $lab = \text{sign}(v - \text{med}(v))$

Recursive Spectral Bisection

- 1 Form graph Laplacean
- 2 Partition graph in 2 based on Fiedler vector
- 3 Partition largest subgraph in two recursively ...
- 4 ... Until the desired number of partitions is reached



Three approaches to graph partitioning:

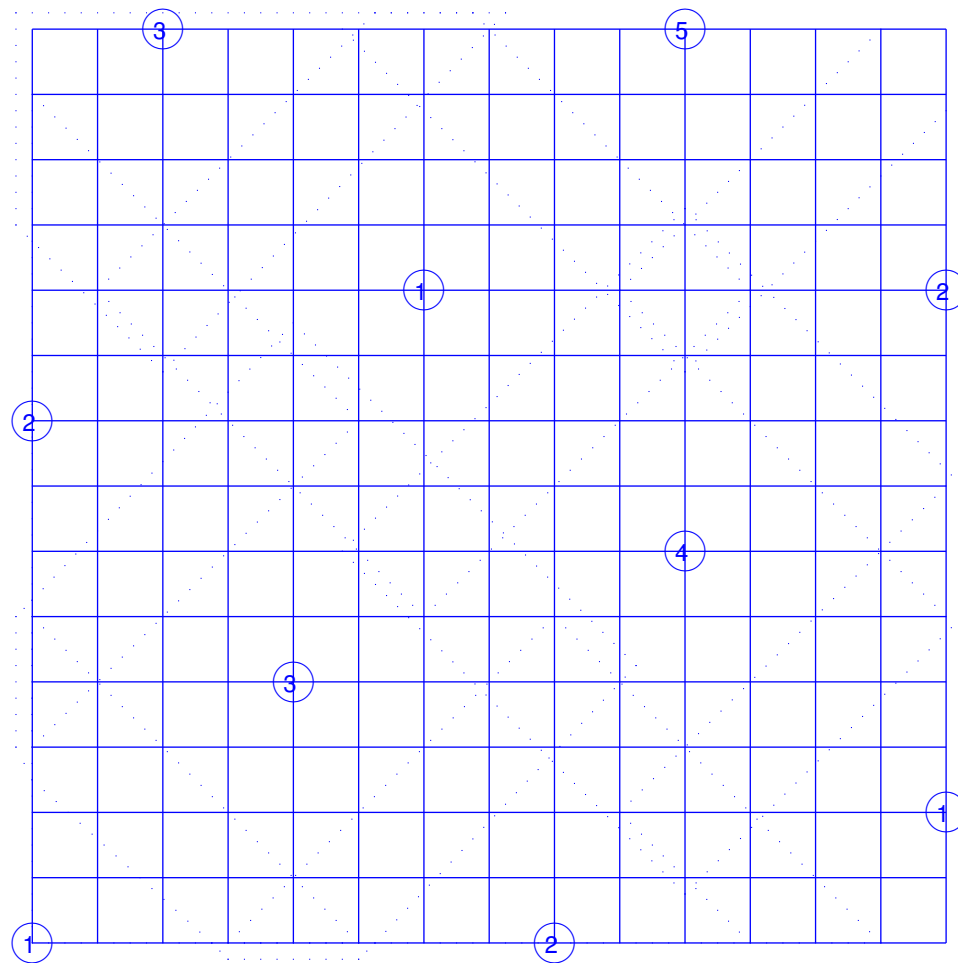
1. Spectral methods - Just seen + add Recursive Spectral Bisection.
2. Geometric techniques. Coordinates are required. [Houstis & Rice et al., Miller, Vavasis, Teng et al.]
3. Graph Theory techniques – multilevel,... [use graph, but no coordinates]
 - Currently best known technique is Metis (multi-level algorithm)
 - Simplest idea: Recursive Graph Bisection; Nested dissection (George & Liu, 1980; Liu 1992)
 - Advantages: simplicity – no coordinates required



Run *testBis_simple* and *testMeshPart* in matlab class site

Example of a graph theory approach

- Level Set Expansion Algorithm
- Given: p nodes ‘uniformly’ spread in the graph (roughly same distance from one another).
- Method: Perform a level-set traversal (BFS) from each node simultaneously.
- Best described for an example on a 15×15 five – point Finite Difference grid.
- See [Goehring-YS '94, See Cai-YS '95]
- Approach also known under the name ‘bubble’ algorithm and implemented in some packages [Party, DibaP]



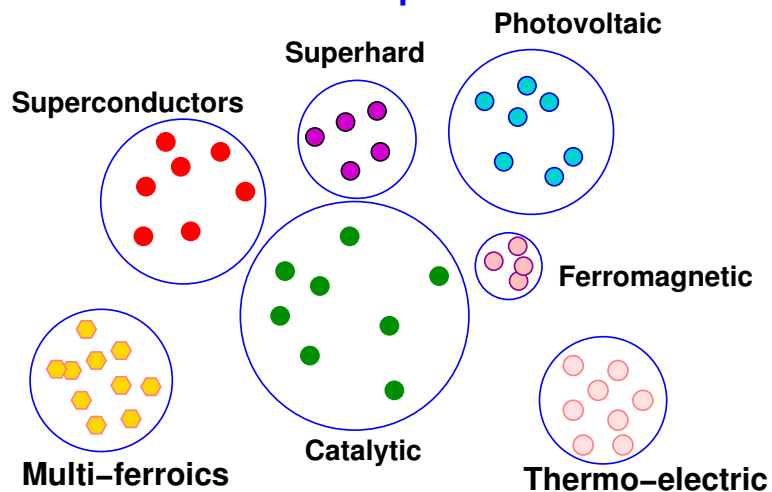
APPLICATIONS OF GRAPH LAPLACEANS: CLUSTERING, EMBEDDING

- *The clustering problem; Basic method: K-means*
- *Similarity graphs; kNN graphs*
- *Measures of separation: edge cuts, normalized cuts, etc*
- *Application: Segmentation*
- *Graph embeddings; Laplacean Eigenmaps*
- *Locally Linear Embeddings (LLE)*
- *Explicit mappings; PCA, LPP, ONPP,..*
- *Building a knn graph*

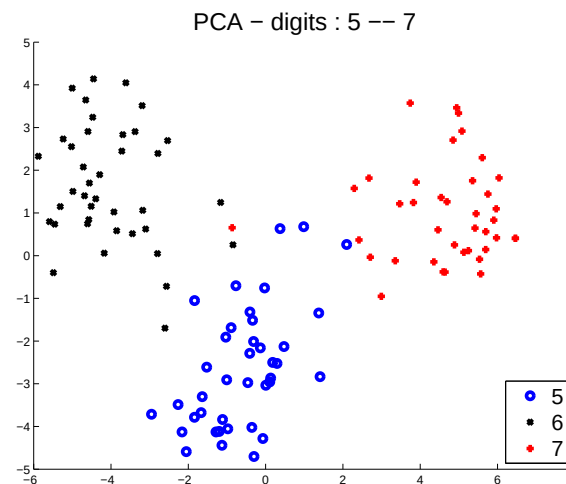
Clustering

➤ Problem: we are given n data items: $x_1, x_2, \dots, x_n$. Would like to ‘cluster’ them, i.e., group them so that each group or cluster contains items that are similar in some sense.

➤ Example: materials



➤ Example: Digits



➤ Each group is a ‘cluster’ or a ‘class’

➤ ‘Unsupervised learning’

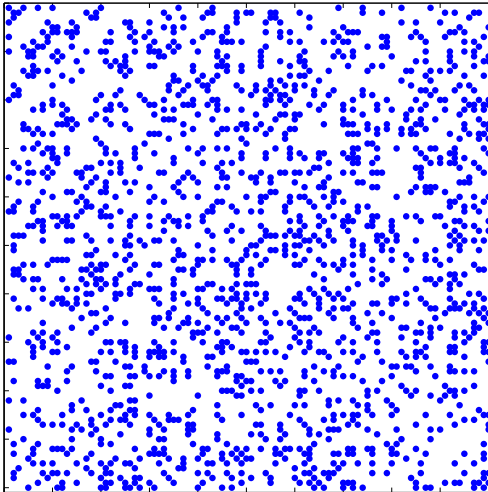
What is ‘Unsupervised Learning’?

Ans: Class of methods that do not exploit **labeled data**

- Example of digits: perform a 2-D projection
- Images of same digit tend to cluster (more or less)
- Such 2-D representations are popular for visualization
- Can also try to find natural clusters in data, e.g., in materials
- Basic clustering technique: K-means

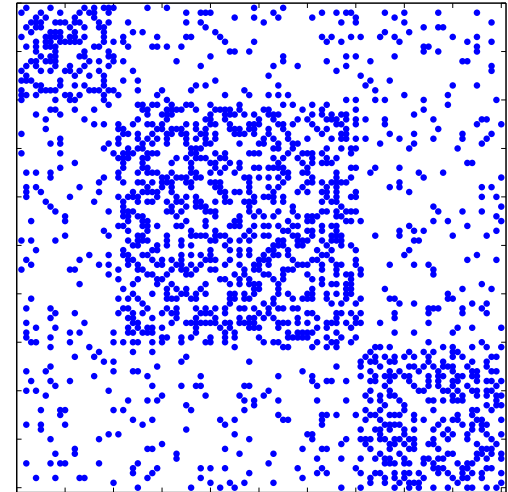
Example: Community Detection

- Communities modeled by an 'affinity' graph [e.g., 'user A sends frequent e-mails to user B ']. [data: www-personal.umich.edu/~mejn/netdata/]



← Original Adj. matrix

Goal: Find ordering so
blocks are as dense as
possible →



- Use 'blocking' techniques for sparse matrices
- Advantage of this viewpoint: need not know # of clusters.

Example of application

Data set from :

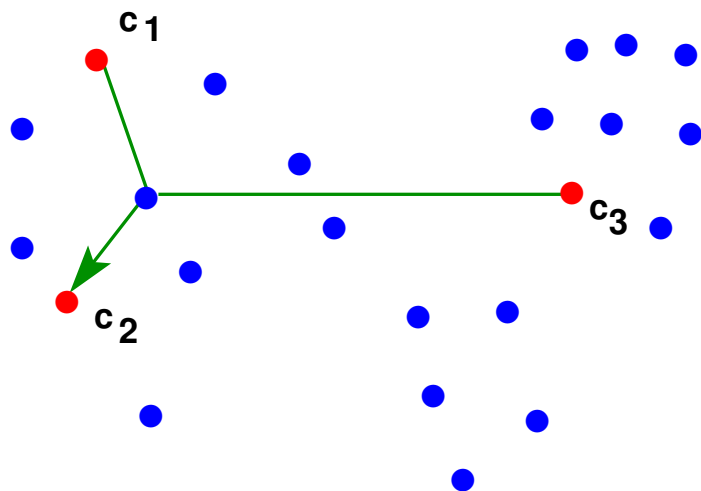
<http://www-personal.umich.edu/~mejn/netdata/>

- Network connecting bloggers of different political orientations [2004 US presidential election]
- ‘Communities’: liberal vs. conservative
- Graph: 1,490 vertices (blogs) : first 758: liberal, rest: conservative.
- Edge: $i \rightarrow j$: a citation between blogs i and j
- Blocking algorithm (Density threshold=0.4): subgraphs [note: density = $|E|/|V|^2$.]
- Smaller subgraph: conservative blogs, larger one: liberals

A basic method: K-means

- A basic algorithm that uses Euclidean distance

- 1 Select p initial centers: $c_1, c_2, \dots, c_p$ for classes $1, 2, \dots, p$
- 2 For each x_i do: determine *class* of x_i as $\operatorname{argmin}_k \|x_i - c_k\|$
- 3 Redefine each c_k to be the centroid of class k
- 4 Repeat until convergence



- Simple algorithm
- Works well (gives good results) but can be slow
- Performance depends on initialization

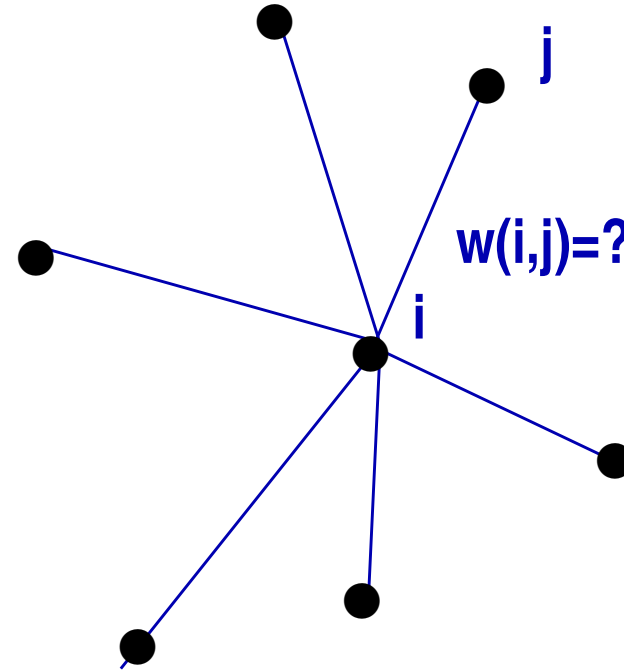
Methods based on similarity graphs

- Class of Methods that perform clustering by exploiting a graph that describes the similarities between any two items in the data.
- Need to:
 1. decide what nodes are in the neighborhood of a given node
 2. quantify their similarities - by assigning a weight to any pair of nodes.

Example: For text data: Can decide that any columns i and j with a cosine greater than 0.95 are 'similar' and assign that cosine value to w_{ij}

First task: build a ‘similarity’ graph

➤ Goal: to build a **similarity** graph, i.e., a graph that captures similarity between any two items



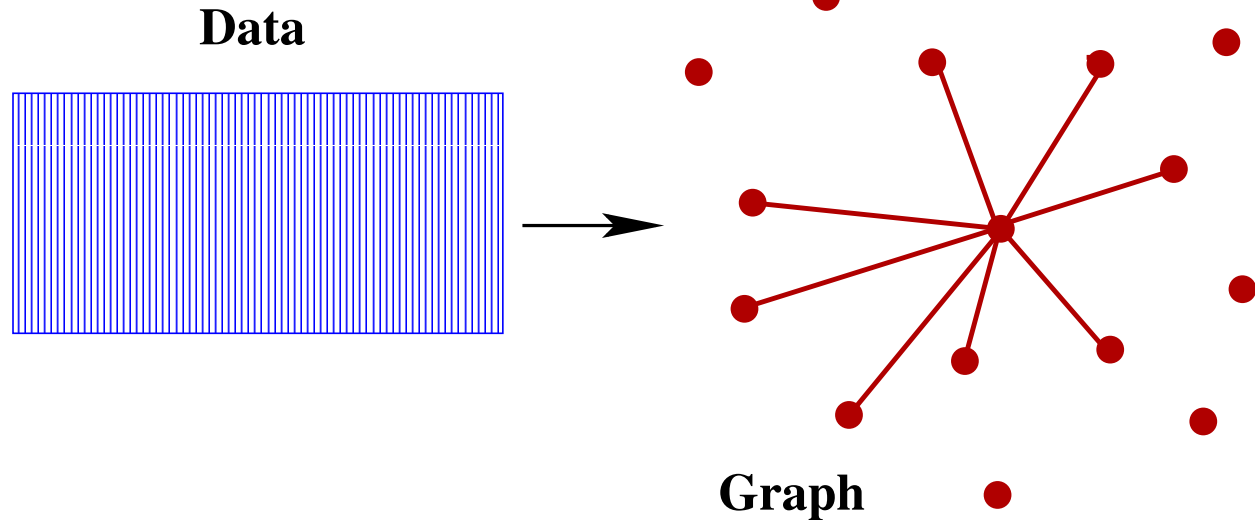
➤ Two methods: K-nearest Neighbor graphs or use Gaussian (‘heat’) kernel

K-nearest neighbor graphs

- Given: a set of n data points $X = \{x_1, \dots, x_n\} \rightarrow$ vertices
- Given: a **proximity** measure between two data points x_i and x_j – as measured by a quantity $dist(x_i, x_j)$
- Want: For each point x_i a list of the ‘nearest neighbors’ of x_i (edges between x_i and these nodes).
- Note: graph will usually be **directed** $\rightarrow$ need to symmetrize

Nearest neighbor graphs

- For each node, get a few of the nearest neighbors → Graph



- Problem: How to build a nearest-neighbor graph from given data
- We will revisit this later.

Two types of nearest neighbor graph often used:

ϵ -graph:

Edges consist of pairs (x_i, x_j) such that $\rho(x_i, x_j) \leq \epsilon$

k NN graph:

Nodes adjacent to x_i are those nodes x_ℓ with the k with smallest distances $\rho(x_i, x_\ell)$.

- ϵ -graph is undirected and is geometrically motivated. Issues: 1) may result in disconnected components 2) what ϵ ?
- k NN graphs are directed in general (can be trivially fixed).
- k NN graphs especially useful in practice.

Similarity graphs: Using 'heat-kernels'

Define weight between i and j as:

$$w_{ij} = f_{ij} \times \begin{cases} e^{\frac{-\|x_i - x_j\|^2}{\sigma_X^2}} & \text{if } \|x_i - x_j\| < r \\ 0 & \text{if not} \end{cases}$$

- Note $\|x_i - x_j\|$ could be any measure of distance...
- f_{ij} = optional = some measure of similarity - other than distance
- Only nearby points kept.
- Sparsity depends on parameters

Edge cuts, ratio cuts, normalized cuts, ...

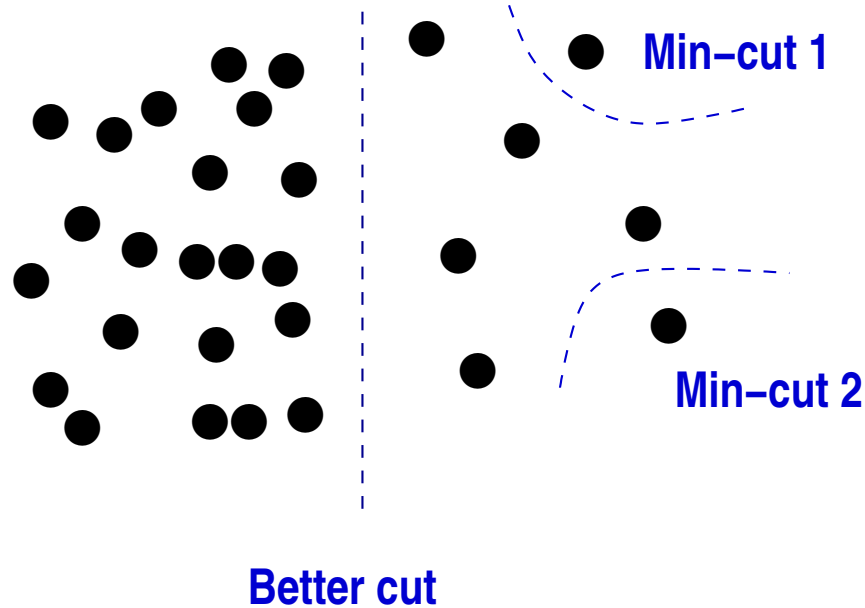
- Assume now that we have built a ‘similarity graph’
- Setting is identical with that of graph partitioning.
- Need a Graph Laplacean: $L = D - W$ with $w_{ii} = 0, w_{ij} \geq 0$ and $D = \text{diag}(W * \text{ones}(n, 1))$ [in matlab notation]
- Partition vertex set V in two sets A and B with

$$A \cup B = V, \quad A \cap B = \emptyset$$

- Define

$$\text{cut}(A, B) = \sum_{u \in A, v \in B} w(u, v)$$

- First (naive) approach: use this measure to partition graph, i.e.,
... Find A and B that minimize $cut(A, B)$.
- Issue: Small sets, isolated nodes, big imbalances,



- Standard Graph Partitioning approach: Find A, B by solving

$$\text{Minimize } \textit{cut}(A, B), \text{ subject to } |A| = |B|$$

- Condition $|A| = |B|$ not too meaningful in some applications - too restrictive in others.
- Minimum Ratio Cut approach. Find A, B by solving:

$$\text{Minimize } \frac{\textit{cut}(A, B)}{|A| \cdot |B|}$$

- Difficult to find solution (original paper [Wei-Cheng '91] proposes several heuristics)
- Approximate solution : spectral .

Theorem [Hagen-Kahng, 91] If λ_2 is the 2nd smallest eigenvalue of L , then a lower bound for the cost c of the optimal ratio cut partition, is:

$$c \geq \frac{\lambda_2}{n}.$$

Proof: Consider an optimal partition A, B and let $p = |A|/n, q = |B|/n$. Note that $p + q = 1$. Let x be the vector with coordinates

$$x_i = \begin{cases} q & \text{if } i \in A \\ -p & \text{if } i \in B \end{cases}$$

Note that $x \perp \mathbf{1}$. Also if (i, j) is an edge-cut then $|x_i - x_j| = |q - (-p)| = |q + p| = 1$, otherwise $x_i - x_j = 0$. Therefore:

$$x^T L x = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 = w(A, B).$$

$$\text{In addition: } \|x\|^2 = pq^2n + qp^2n = pq(p + q)n = pqn = \frac{|A| \cdot |B|}{n}.$$

Therefore, by the Courant-Fischer theorem:

$$\lambda_2 \leq \frac{(Lx, x)}{(x, x)} = n \times \frac{w(A, B)}{|A| \cdot |B|} = n \times c.$$

Hence result. ■

➤ Idea is to use eigenvector associated with λ_2 to determine partition, e.g., based on sign of entries. Use the ratio-cut measure to actually determine where to split.

Normalized cuts [Shi-Malik,2000]

- Recall notation $w(X, Y) = \sum_{x \in X, y \in Y} w(x, y)$ - then define:

$$\text{ncut}(A, B) = \frac{\text{cut}(A, B)}{w(A, V)} + \frac{\text{cut}(A, B)}{w(B, V)}$$

- Goal is to avoid small sets A, B

 1 What is $w(A, V)$ in the case when $w_{ij} == 1$?

- Let x be an indicator vector:

$$x_i = \begin{cases} 1 & \text{if } i \in A \\ 0 & \text{if } i \in B \end{cases}$$

- Recall that: $x^T L x = \sum_{(i,j) \in E} w_{ij} |x_i - x_j|^2$ (note: each edge counted once)

➤ Therefore:

$$\text{cut}(A, B) = \sum_{x_i=1, x_j=0} w_{ij} = x^T L x$$

$$w(A, V) = \sum_{x_i=1} d_i = x^T W \mathbb{1} = x^T D \mathbb{1}$$

$$w(B, V) = \sum_{x_j=0} d_j = (\mathbb{1} - x)^T W \mathbb{1} = (\mathbb{1} - x)^T D \mathbb{1}$$

➤ Goal now: to minimize ncut

$$\min_{A, B} \text{ncut}(A, B) = \min_{x_i \in \{0,1\}} \frac{x^T L x}{x^T D x} + \frac{x^T L x}{(\mathbb{1} - x)^T D x}$$

➤ Let

$$k = \frac{\sum_{x_i > 0} d_i}{\sum_i d_i}; \quad b = \frac{k}{1 - k}; \quad \text{and:} \quad y = (\mathbb{1} + x) - b(\mathbb{1} - x)$$

➤ Then it can be shown that we need to solve:

$$\begin{aligned} \min_{y_i \in \{1, -b\}} \quad & \frac{y^T L y}{y^T D y} \\ \text{Subject to} \quad & y^T D \mathbb{1} = 0 \end{aligned}$$

➤ + Relax $\rightarrow$ need to solve Generalized eigenvalue problem

$$Ly = \lambda Dy$$

➤ $y_1 = \mathbb{1}$ is eigenvector associated with eigenvalue $\lambda_1 = 0$

➤ y_2 associated with second eigenvalue solves problem.

A few properties

 2 Show that

$$ncut(A, B) = \sigma \times \frac{cut(A, B)}{w(A, V) \times w(B, V)}$$

where σ is a constant

 3 How do ratio-cuts and normalized cuts compare when the graph is d -regular (same degree for each node).

Extension to more than 2 clusters

➤ Just like graph partitioning we can:

1. Apply the method recursively [Repeat clustering on the resulted parts]
2. or compute a few eigenvectors and run K-means clustering on these eigenvectors to get the clustering.

Application: Image segmentation

- First task: obtain a graph from pixels.
- Common idea: use “Heat kernels”
- Let F_j = feature value (e.g., brightness), and Let X_j = spatial position.

Then define

$$w_{ij} = e^{\frac{-\|F_i - F_j\|^2}{\sigma_F^2}} \times \begin{cases} e^{\frac{-\|X_i - X_j\|^2}{\sigma_X^2}} & \text{if } \|X_i - X_j\| < r \\ 0 & \text{else} \end{cases}$$

- Sparsity depends on parameters

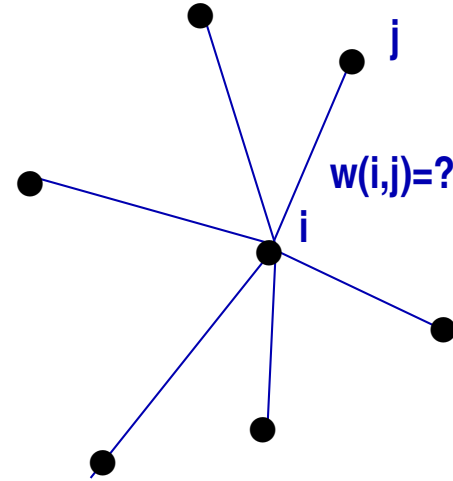


Run *test_mir* and *test_jump*

Spectral clustering: General approach

1 Given: Collection of data samples $\{x_1, x_2, \dots, x_n\}$

2 Build a **similarity** graph between items



3 Compute (smallest) eigenvector (s) of resulting graph Laplacean

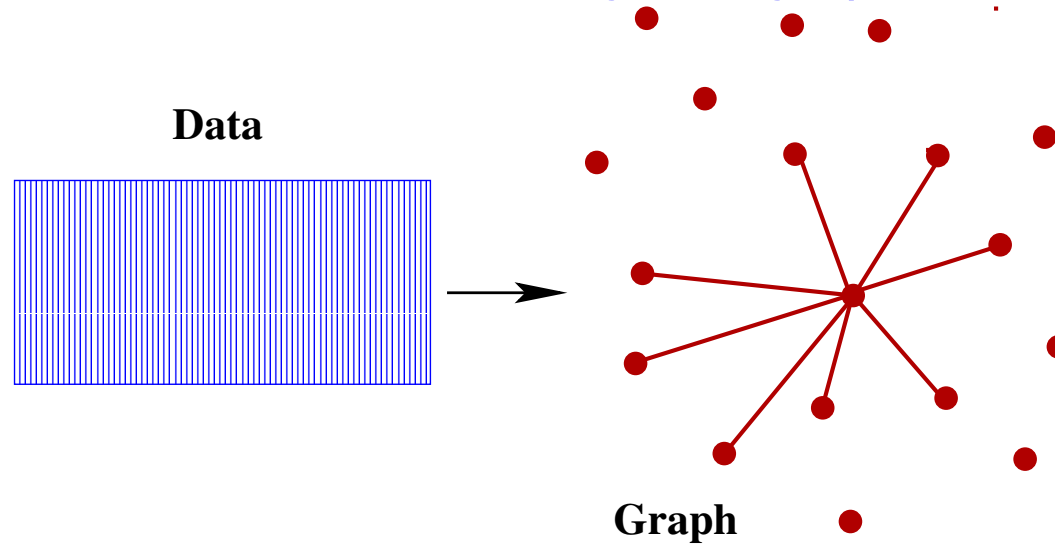
4 Use k-means on eigenvector (s) of Laplacean

➤ For Normalized cuts solve generalized eigen problem.

KNN GRAPHS

Building a nearest neighbor graph

- Question: How to build a nearest-neighbor graph from given data?



- Will demonstrate the power of a divide a conquer approach combined with the Lanczos algorithm.

Recall: Two common types of nearest neighbor graphs

ϵ -graph:

Edges consist of pairs (x_i, x_j) such that $\rho(x_i, x_j) \leq \epsilon$

k NN graph:

Nodes adjacent to x_i are those nodes x_ℓ with the k with smallest distances $\rho(x_i, x_\ell)$.

- ϵ -graph is undirected and is geometrically motivated. Issues: 1) may result in disconnected components 2) what ϵ ?
- k NN graphs are directed in general (can be trivially fixed).
- k NN graphs especially useful in practice.

Divide and conquer KNN: key ingredient

- Key ingredient is *Spectral bisection*
- Let the data matrix $X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$
- Each column == a data point.
- Center the data: $\hat{X} = [\hat{x}_1, \dots, \hat{x}_n] = X - ce^T$
where c == centroid; $e = \text{ones}(d, 1)$ (matlab)

Goal: Split $\hat{X}$ into halves using a hyperplane.

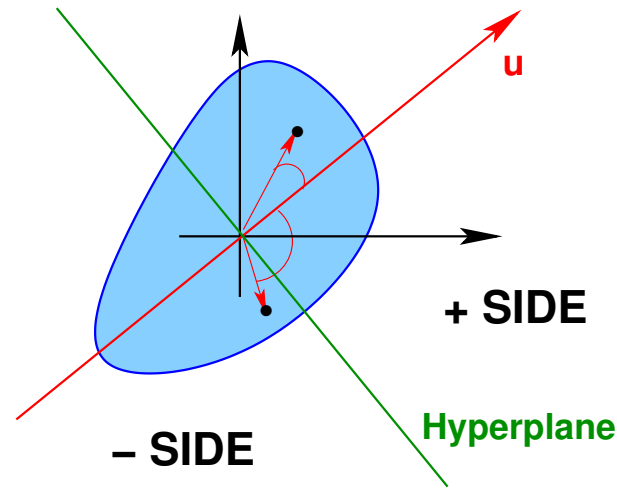
Method: Principal Direction Divisive Partitioning D. Boley, '98.

Idea: Use the (σ, u, v) = largest singular triplet of $\hat{X}$ with:

$$u^T \hat{X} = \sigma v^T.$$

- Hyperplane is defined as $\langle u, x \rangle = 0$, i.e., it splits the set of data points into two subsets:

$$X_+ = \{x_i \mid u^T \hat{x}_i \geq 0\} \quad \text{and} \quad X_- = \{x_i \mid u^T \hat{x}_i < 0\}.$$



- Note that $u^T \hat{x}_i = u^T \hat{X} e_i = \sigma v^T e_i \rightarrow$

$$X_+ = \{x_i \mid v_i \geq 0\} \quad \text{and} \quad X_- = \{x_i \mid v_i < 0\},$$

where v_i is the i -th entry of v .

➤ In practice: replace above criterion by

$$X_+ = \{x_i \mid v_i \geq \text{med}(v)\} \text{ \& } X_- = \{x_i \mid v_i < \text{med}(v)\}$$

where $\text{med}(v) == \text{median of the entries of } v$.

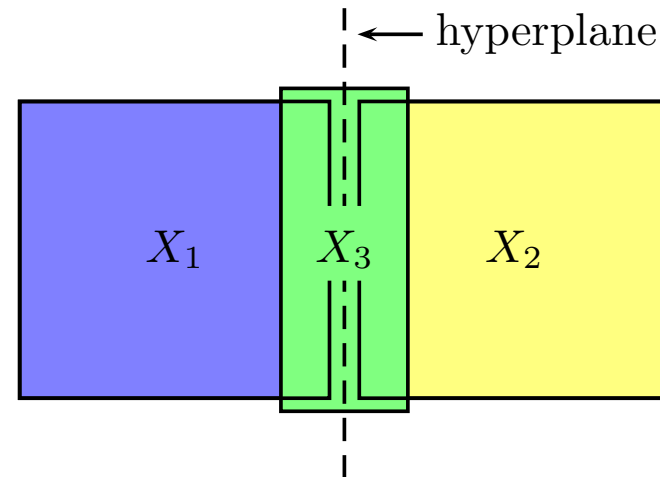
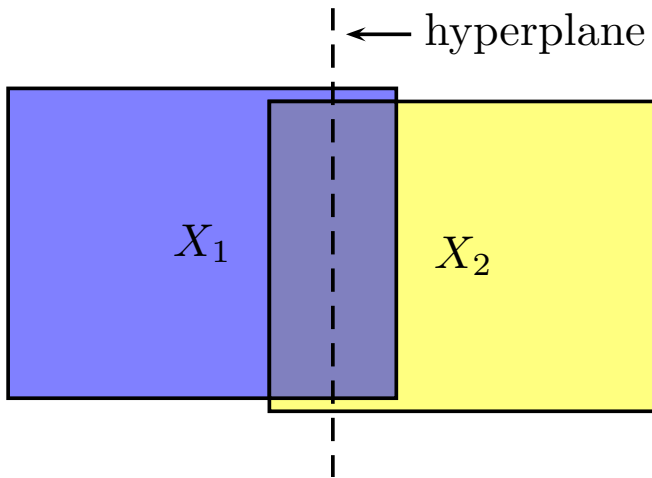
➤ For largest singular triplet (σ, u, v) of $\hat{X}$: use Golub-Kahan-Lanczos algorithm or Lanczos applied to $\hat{X}\hat{X}^T$ or $\hat{X}^T\hat{X}$

➤ Cost (assuming s Lanczos steps) : $O(n \times d \times s)$; Usually: d very small

Two divide and conquer algorithms

Overlap method: divide current set into two overlapping subsets X_1, X_2

Glue method: divide current set into two disjoint subsets X_1, X_2 plus a third set X_3 called gluing set.



➤ Exploit recursivity

The Overlap Method

- Divide current set X into two overlapping subsets:

$$X_1 = \{x_i \mid v_i \geq -h_\alpha(S_v)\} \quad \text{and} \quad X_2 = \{x_i \mid v_i < h_\alpha(S_v)\},$$

- where $S_v = \{|v_i| \mid i = 1, 2, \dots, n\}$.
 - and $h_\alpha(\cdot)$ is a function that returns an element larger than $(100\alpha)\%$ of those in S_v .
- Rationale: to ensure that the two subsets overlap $(100\alpha)\%$ of the data, i.e.,

$$|X_1 \cap X_2| = \lceil \alpha |X| \rceil .$$

The Glue Method

Divide the set X into two disjoint subsets X_1 and X_2 with a gluing subset X_3 :

$$X_1 \cup X_2 = X, \quad X_1 \cap X_2 = \emptyset, \quad X_1 \cap X_3 \neq \emptyset, \quad X_2 \cap X_3 \neq \emptyset.$$

Criterion used for splitting:

$$X_1 = \{x_i \mid v_i \geq 0\}, \quad X_2 = \{x_i \mid v_i < 0\}, \\ X_3 = \{x_i \mid -h_\alpha(S_v) \leq v_i < h_\alpha(S_v)\}.$$

Note: gluing subset X_3 here is just the intersection of the sets X_1, X_2 of the overlap method.

Theorem The time complexity for the overlap method is

$$T_o(n) = \Theta(dn^{t_o}), \quad \text{where:} \quad t_o = \log_{2/(1+\alpha)} 2 = \frac{1}{1 - \log_2(1 + \alpha)}.$$

Theorem The time complexity for the glue method is

$$T_g(n) = \Theta(dn^{t_g}/\alpha), \quad \text{where} \quad t_g \equiv \text{sol. to the equ.:} \quad \frac{2}{2^t} + \alpha^t = 1.$$

Example: When $\alpha = 0.1$, then $t_o = 1.16$ while $t_g = 1.12$.

Reference:

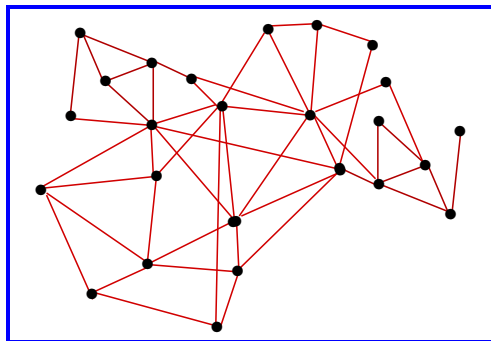
Jie Chen, Haw-Ren Fang and YS, “Fast Approximate k NN Graph Construction for High Dimensional Data via Recursive Lanczos Bisection” *JMLR*, vol. 10, pp. 1989-2012 (2009).

GRAPH EMBEDDING

Graph embedding

- We have seen how to build a graph to represent data
- *Graph embedding* does the opposite: maps a graph to data

Given: a graph that models some data (e.g., a kNN graph)



→ Data: $Y = [y_1, y_2, \dots, y_n]$ in $\mathbb{R}^d$

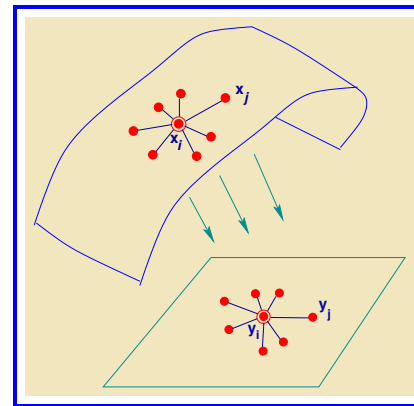
- Trivial use: visualize a graph ($d = 2$)
- Wish: mapping should preserve *similarities* in graph.

Vertex embedding: map every vertex x_i to a vector $y_i \in \mathbb{R}^d$

➤ Many applications [clustering, finding missing link, semi-supervised learning, community detection, ...]

➤ Graph captures similarities, closeness, ..., in data

Objective: Build a mapping of each vertex i to a data point $y_i \in \mathbb{R}^d$



➤ Many methods do this

➤ **Eigenmaps** and **LLE** are two of the best known

- Eigenmaps uses the *graph Laplacean*
- Recall: Graph Laplacean is a matrix defined by :

$$L = D - W$$

$$\begin{cases} w_{ij} \geq 0 & \text{if } j \in Adj(i) \\ w_{ij} = 0 & \text{else} \end{cases} \quad D = \text{diag} \left[d_{ii} = \sum_{j \neq i} w_{ij} \right]$$

with $Adj(i)$ = neighborhood of i (excludes i)

- Remember that vertex i represents data item x_i . We will use i or x_i to refer to the vertex.
- We will find the y_i 's by solving an optimization problem.

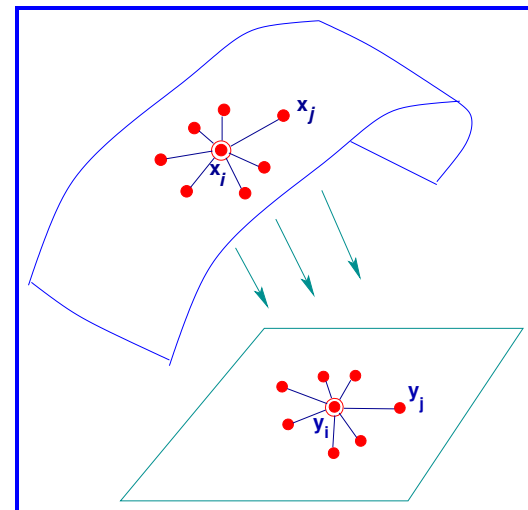
The Laplacean eigenmaps approach

Laplacean Eigenmaps [Belkin-Niyogi '01] *minimizes*

$$\mathcal{F}(Y) = \sum_{i,j=1}^n w_{ij} \|y_i - y_j\|^2 \quad \text{subject to} \quad YDY^\top = I$$

Motivation: if $\|x_i - x_j\|$ is small (orig. data), we want $\|y_i - y_j\|$ to be also small (low-Dim. data)

- Original data used indirectly through its graph
- Objective function can be translated to a trace (see Property 3 in Lecture notes 9) and will yield a sparse eigenvalue problem



➤ Problem translates to:

$$\begin{cases} \min & \text{Tr} [Y(D - W)Y^\top] \\ Y \in \mathbb{R}^{d \times n} \\ YD Y^\top = I \end{cases} .$$

➤ Solution (sort eigenvalues increasingly):

$$(D - W)u_i = \lambda_i D u_i ; \quad y_i = u_i^\top ; \quad i = 1, \dots, d$$

➤ An $n \times n$ sparse eigenvalue problem [In 'sample' space]

➤ Note: can assume $D = I$. Amounts to rescaling data. Problem becomes

$$(I - W)u_i = \lambda_i u_i ; \quad y_i = u_i^\top ; \quad i = 1, \dots, d$$

Locally Linear Embedding (Roweis-Saul-00)

➤ LLE is very similar to Eigenmaps. Main differences:

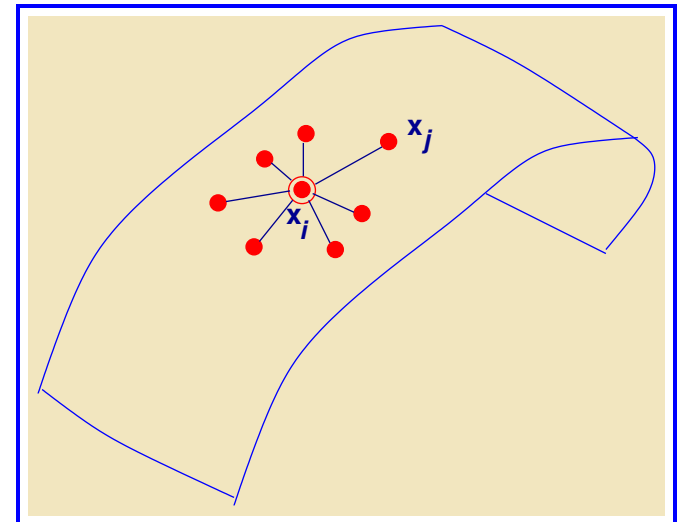
- 1) Graph Laplacean matrix is replaced by an 'affinity' graph
- 2) Objective function is changed: want to preserve graph

1. Graph: Each x_i is written as a convex combination of its k nearest neighbors:

$$x_i \approx \sum w_{ij} x_j, \quad \sum_{j \in N_i} w_{ij} = 1$$

➤ Optimal weights computed ('local calculation') by minimizing

$$\|x_i - \sum w_{ij} x_j\| \quad \text{for } i = 1, \dots, n$$



2. Mapping:

The y_i 's should obey the same 'affinity' as x_i 's $\rightsquigarrow$

Minimize:

$$\sum_i \left\| y_i - \sum_j w_{ij} y_j \right\|^2 \quad \text{subject to: } Y \mathbf{1} = 0, \quad Y Y^\top = I$$

Solution:

$$(I - W^\top)(I - W)u_i = \lambda_i u_i; \quad y_i = u_i^\top.$$

➤ $(I - W^\top)(I - W)$ replaces the graph Laplacean of eigenmaps

Implicit vs explicit mappings

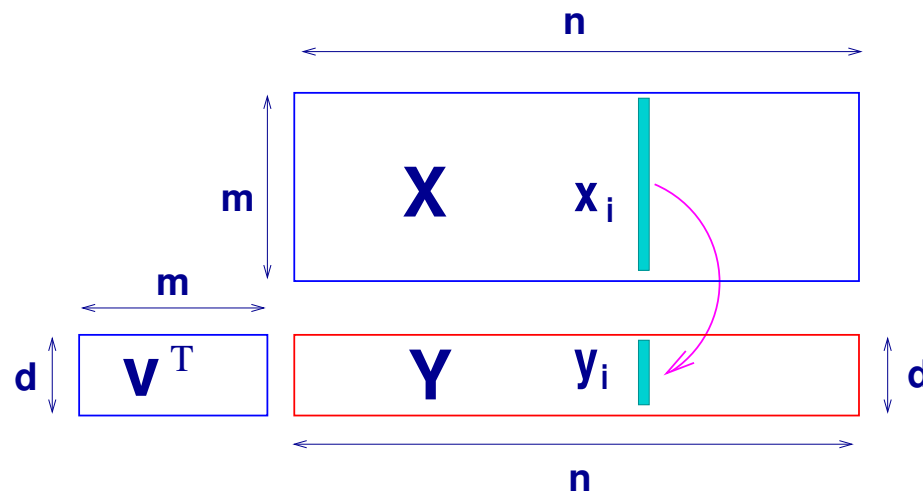
- In Eigenmaps and LLE we only determine a set of y'_i s in $\mathbb{R}^d$ from the data points $\{x_i\}$.
- The mapping $y_i = \phi(x_i), i = 1, \dots, n$ is **implicit**
- Difficult to compute a y for an x that is not one of the x_i 's
- Inconvenient for classification. Thus is known as the “**The out-of-sample extension**” problem
- In **Explicit** (also known as **linear**) methods: mapping ϕ is known explicitly (and it is linear.)

Locally Preserving Projections (He-Niyogi-03)

- LPP is a **linear** dimensionality reduction technique

- Recall the setting:

Want $V \in \mathbb{R}^{m \times d}$; $Y = V^T X$



- Starts with the same neighborhood graph as Eigenmaps: $L \equiv D - W =$ graph 'Laplacian'; with $D \equiv \text{diag}(\{\sum_i w_{ij}\})$.

- Optimization problem is to solve

$$\min_{Y \in \mathbb{R}^{d \times n}, YDY^\top = I} \sum_{i,j} w_{ij} \|y_i - y_j\|^2, \quad Y = V^\top X.$$

- Difference with eigenmaps: Y is an explicit projection of X
- Solution (sort eigenvalues increasingly)

$$XLX^\top v_i = \lambda_i XDX^\top v_i \quad y_{i,:} = v_i^\top X$$

- Note: essentially same method in [Koren-Carmel'04] called 'weighted PCA' [viewed from the angle of improving PCA]

ONPP (Kokopoulou and YS '05)

- Orthogonal Neighborhood Preserving Projections
- A linear (orthogonoal) version of LLE obtained by writing Y in the form $Y = V^T X$
- Same graph as LLE. Objective: preserve the affinity graph (as in LLE) *but* with the constraint $Y = V^T X$
- Problem solved to obtain mapping:

$$\begin{aligned} \min_V \text{Tr} [V^T X(I - W^T)(I - W)X^T V] \\ \text{s.t. } V^T V = I \end{aligned}$$

- In LLE replace $V^T X$ by Y

More recent methods

➤ Quite a bit of recent work - e.g., methods: node2vec, DeepWalk, GraRep, See the following papers ... among many others :

[1] *William L. Hamilton, Rex Ying, and Jure Leskovec Representation Learning on Graphs: Methods and Applications* arXiv:1709.05584v3

[2] *Shaosheng Cao, Wei Lu, and Qiongkai Xu GraRep: Learning Graph Representations with Global Structural Information*, CIKM, ACM Conference on Information and Knowledge Management, 24

[3] *Amr Ahmed, Nino Shervashidze, and Shravan Narayanamurthy, Distributed Large-scale Natural Graph Factorization* [Proc. WWW 2013, May 13–17, 2013, Rio de Janeiro, Brazil]

Example: Graph factorization

- Line of work in Papers [1] and [3] above + others
- Instead of minimizing $\sum w_{ij} \|y_i - y_j\|_2^2$ as before

... try to minimize

$$\sum_{ij} |w_{ij} - y_i^T y_j|^2$$

- In other words solve: $\min_Y \|W - Y^T Y\|_F^2$
- Referred to as *Graph factorization*
- Common in knowledge graphs

SUPERVISED LEARNING - (Brief)

- *Supervised learning; basics; labeled data*
- *Classification problems; KNN classification*
- *Linear Classifiers; Fisher Lin. Discriminants*
- *Support Vector Machines; Deep Neural Networks*

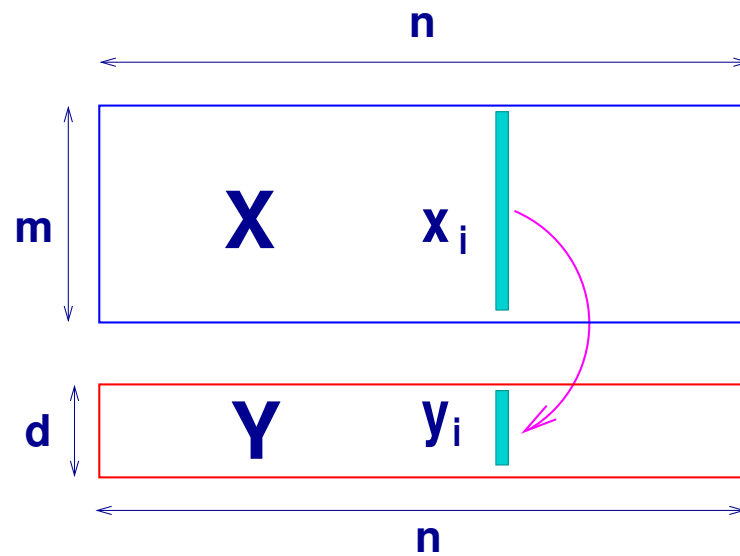
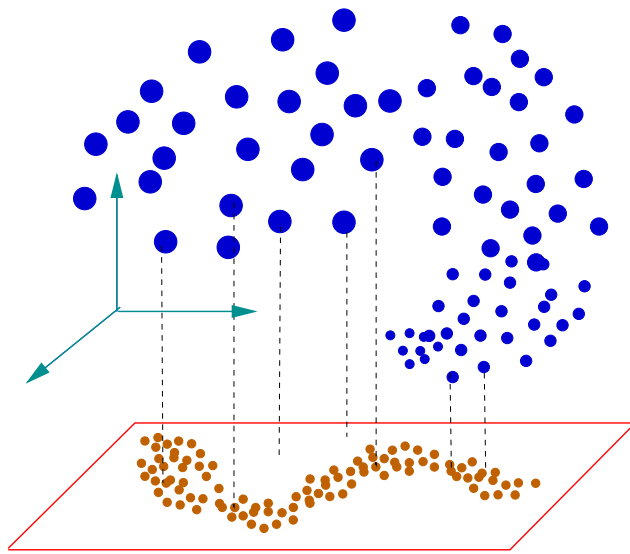
Major tool of Data Mining: Dimension reduction

- Goal is not as much to reduce size (& cost) but to:
 - Reduce noise and redundancy in data before performing a task [e.g., classification as in digit/face recognition]
 - Discover important ‘features’ or ‘parameters’

The problem: Given: $X = [x_1, \dots, x_n] \in \mathbb{R}^{m \times n}$, find a low-dimens. representation $Y = [y_1, \dots, y_n] \in \mathbb{R}^{d \times n}$ of X

- Achieved by a mapping $\Phi : x \in \mathbb{R}^m \longrightarrow y \in \mathbb{R}^d$ so:

$$\phi(x_i) = y_i, \quad i = 1, \dots, n$$



- Φ may be linear : $y_j = W^\top x_j, \forall j, \text{ or, } Y = W^\top X$
- ... or nonlinear (implicit).
- Mapping Φ required to: Preserve proximity? Maximize variance? Preserve a certain graph?

Basics: Principal Component Analysis (PCA)

PCA: Compute W to maximize variance of projected data:

$$\max_{W \in \mathbb{R}^{m \times d}; W^\top W = I} \sum_{i=1}^n \left\| y_i - \frac{1}{n} \sum_{j=1}^n y_j \right\|_2^2, \quad y_i = W^\top x_i.$$

➤ Leads to maximizing

$$\text{Tr} [W^\top (X - \mu e^\top)(X - \mu e^\top)^\top W], \quad \mu = \frac{1}{n} \sum_{i=1}^n x_i$$

➤ Solution $W = \{ \text{dominant eigenvectors} \}$ of the covariance matrix $\equiv$ Set of left singular vectors of $\bar{X} = X - \mu e^\top$

SVD:

$$\bar{X} = U\Sigma V^\top, \quad U^\top U = I, \quad V^\top V = I, \quad \Sigma = \text{Diag}$$

- Optimal $W = U_d \equiv$ matrix of first d columns of U
- Solution W also minimizes ‘reconstruction error’ ..

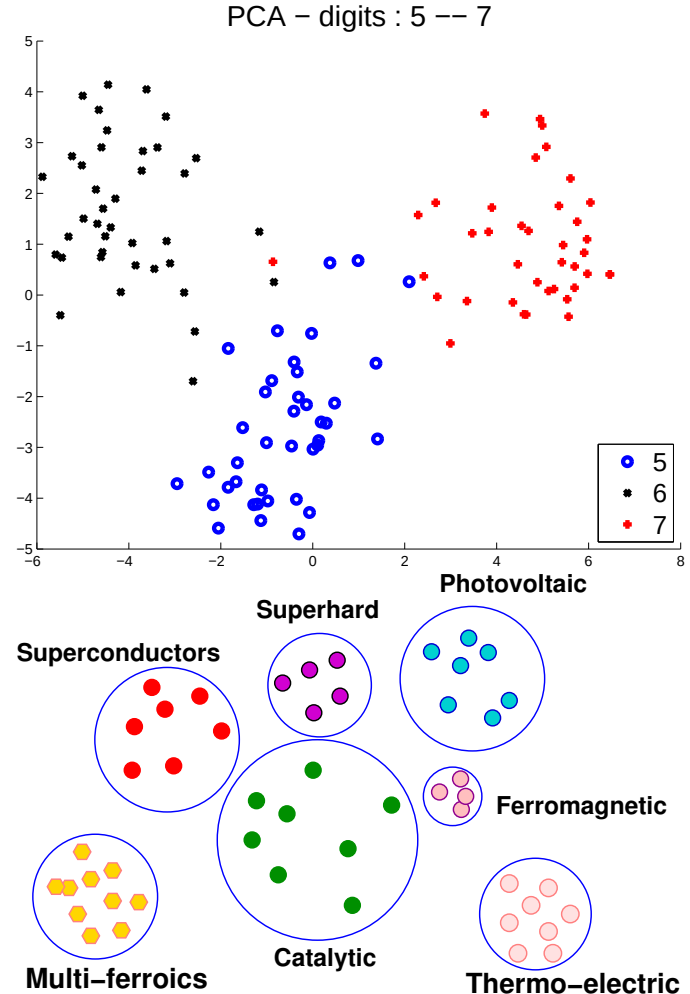
$$\sum_i \|x_i - WW^T x_i\|^2 = \sum_i \|x_i - Wy_i\|^2$$

- In some methods recentering to zero is not done, i.e., $\bar{X}$ replaced by X .

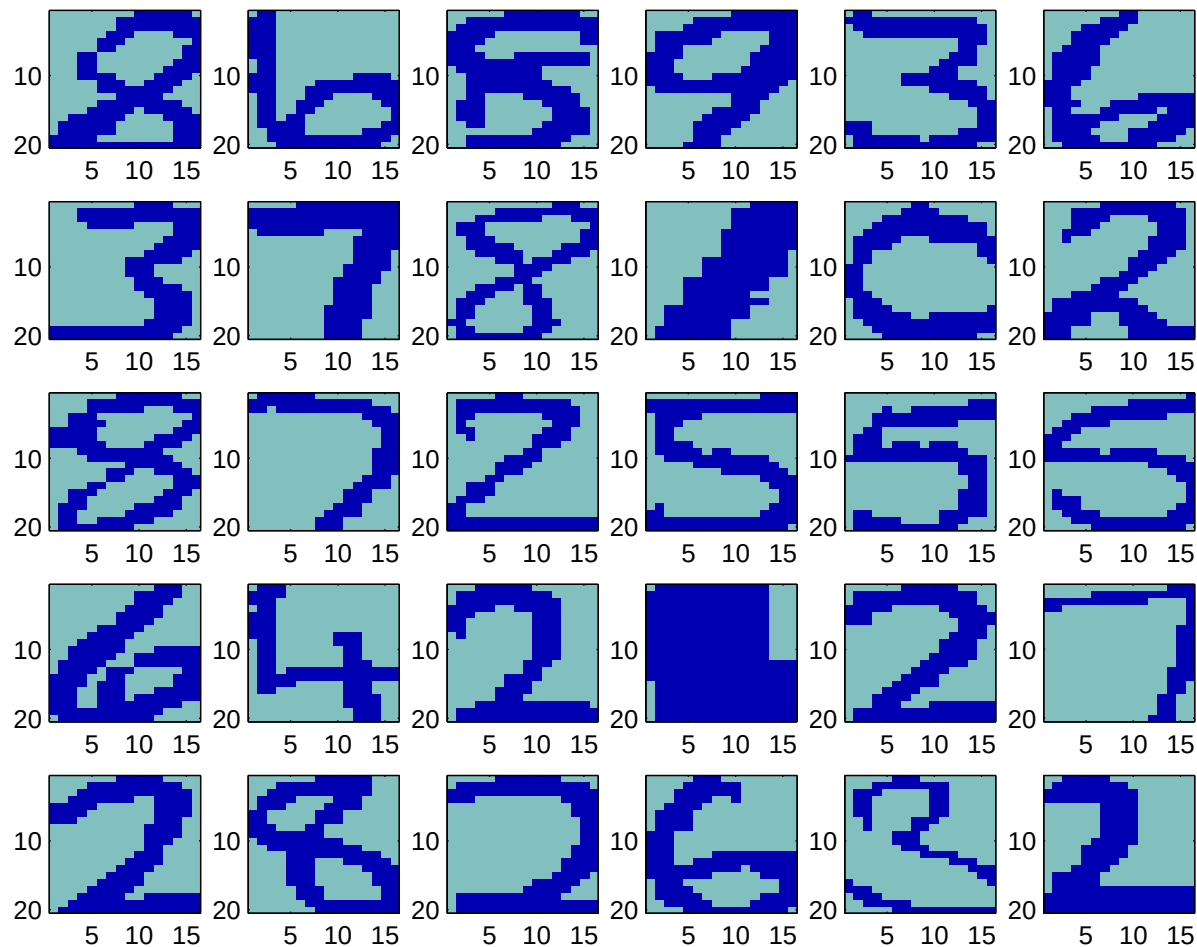
Unsupervised learning

“Unsupervised learning”: methods do not exploit labeled data

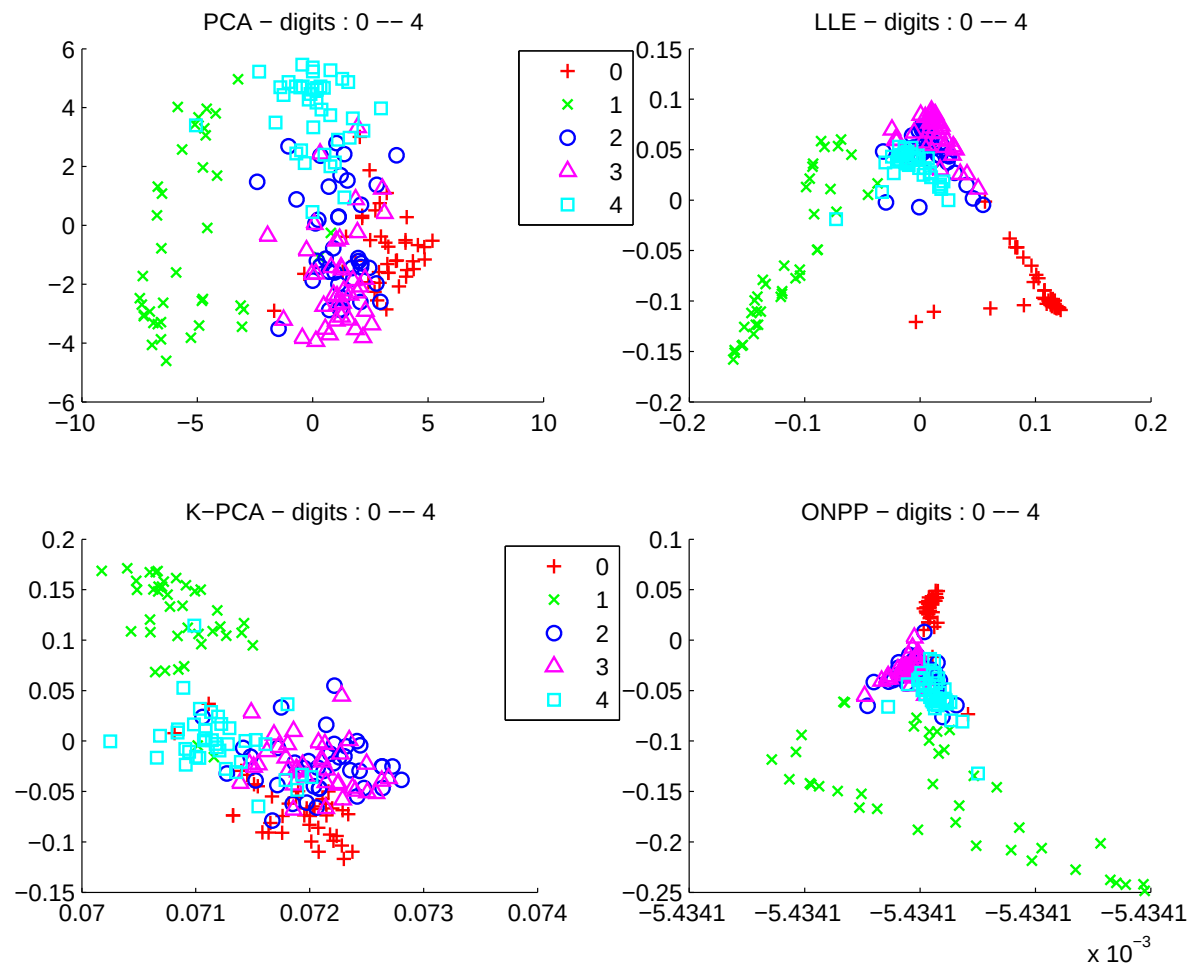
- Example of digits: perform a 2-D projection
- Images of same digit tend to cluster (more or less)
- Such 2-D representations are popular for visualization
- Can also try to find natural clusters in data, e.g., in materials
- Basic clustering technique: K-means



Example: Digit images (a random sample of 30)



2-D 'reductions':

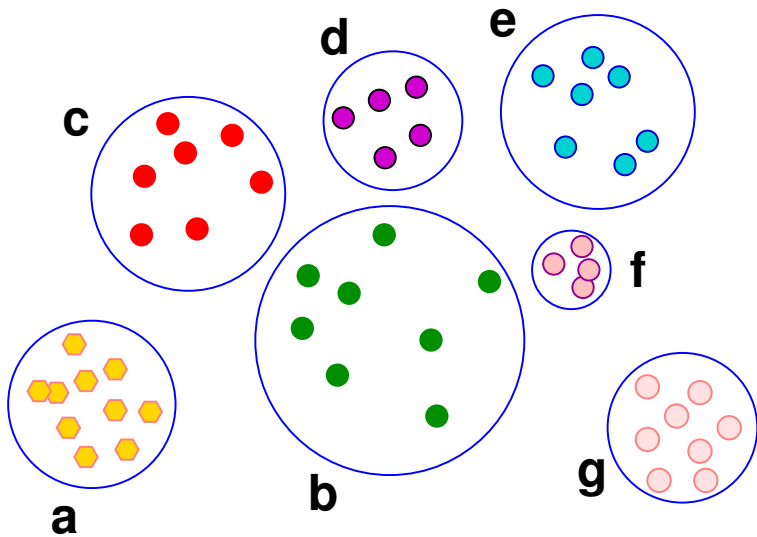


SUPERVISED LEARNING

Supervised learning

➤ We now have data that is 'labeled'

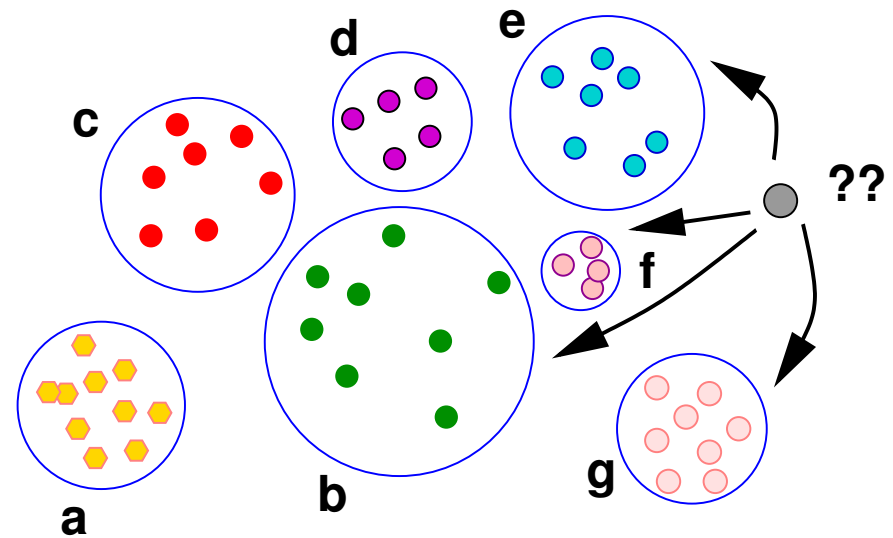
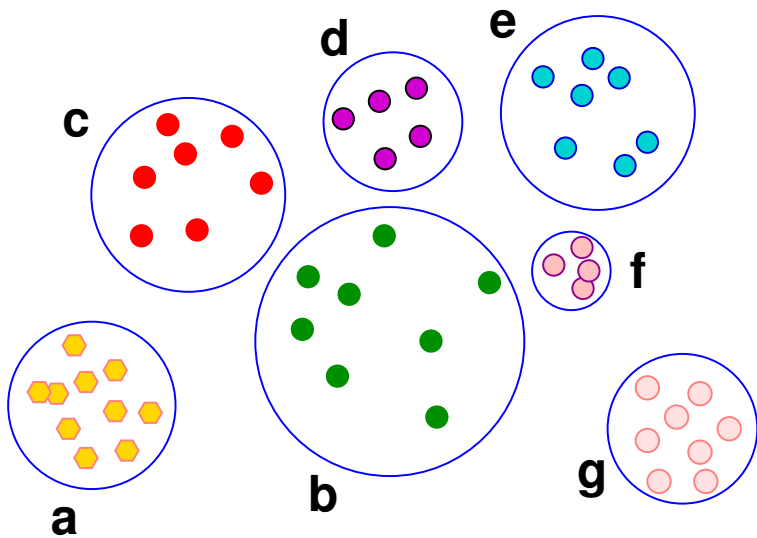
Examples: Health Sciences ('malignant'- 'non malignant') ; Materials ('photovoltaic', 'hard', 'conductor', ...) ; Digit Recognition ('0', '1', ..., '9')



Supervised learning

➤ We now have data that is 'labeled'

Examples: Health Sciences ('malignant'- 'non malignant') ; Materials ('photovoltaic', 'hard', 'conductor', ...) ; Digit Recognition ('0', '1', ..., '9')

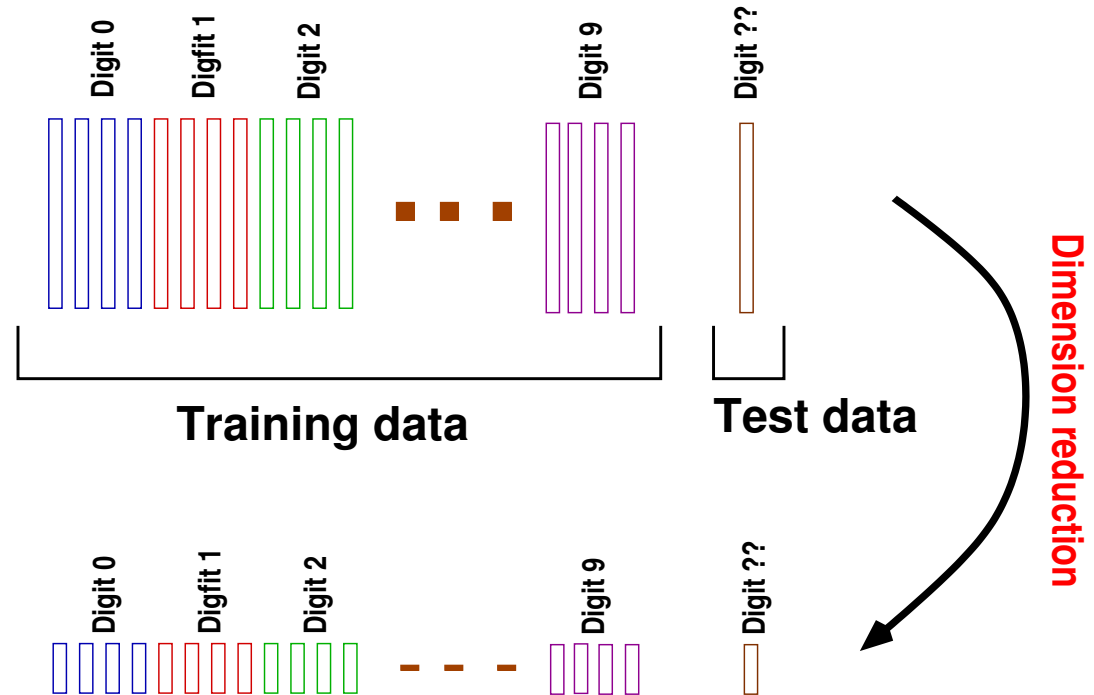


Supervised learning: classification

- Best illustration: written digits recognition example

Given: set of labeled samples (training set), and an (unlabeled) test image x .

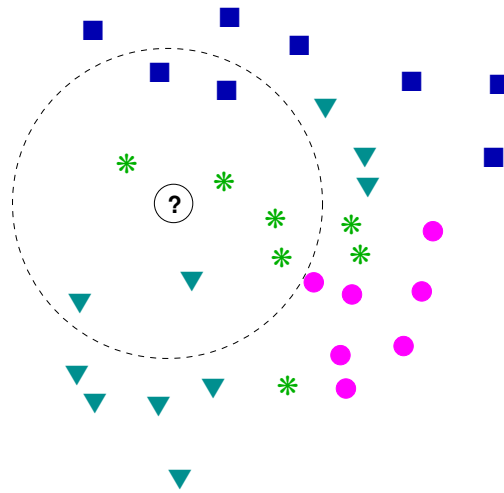
Problem: label of $x = ?$



- Roughly speaking: we seek dimension reduction so that recognition is 'more effective' in low-dim. space

Basic method: *K*-nearest neighbors (KNN) classification

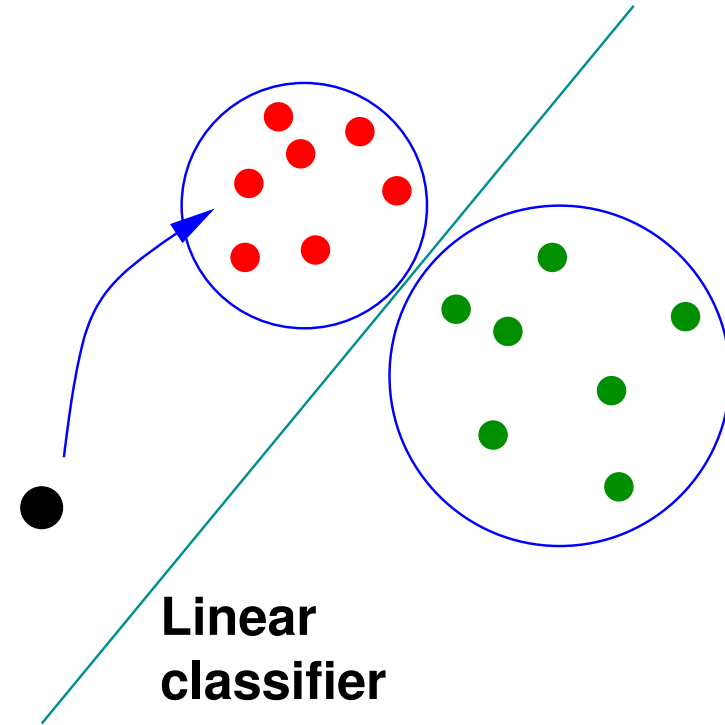
- Idea of a voting system: get distances between test sample and training samples
- Get the k nearest neighbors (here $k = 8$)
- Predominant class among these k items is assigned to the test sample (“*” here)



Supervised learning: Linear classification

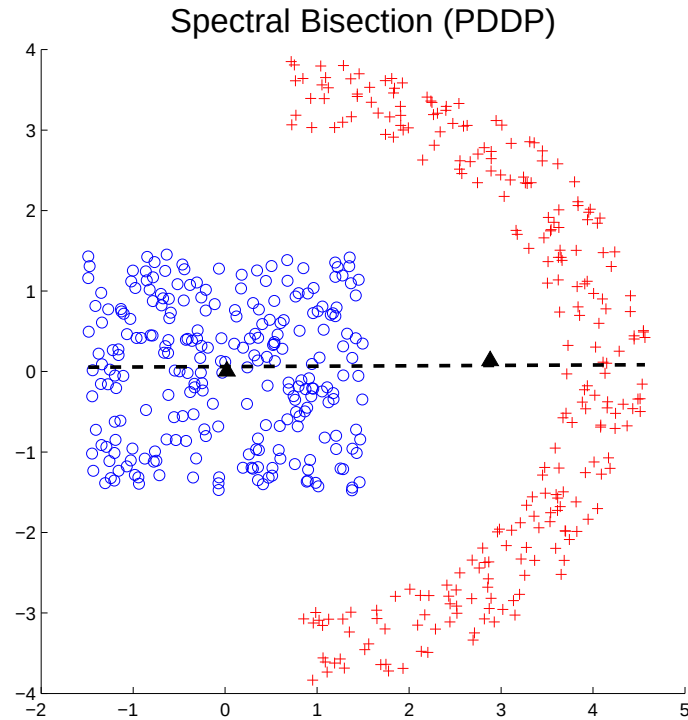
Linear classifiers: Find a hyperplane which best separates the data in classes A and B.

➤ Example of application: Distinguish between SPAM and non-SPAM e-mails

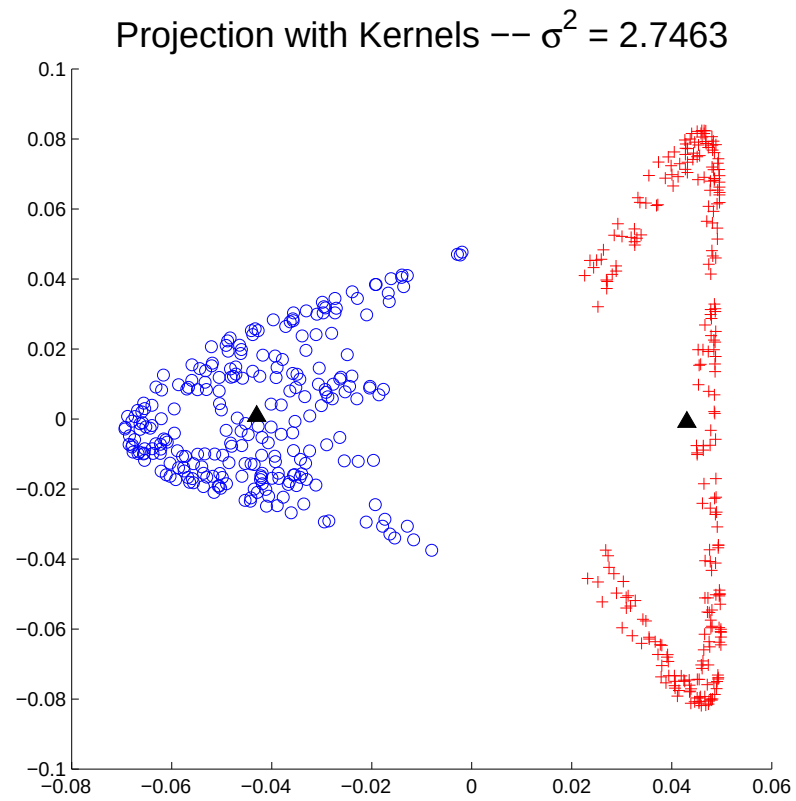


➤ Note: The world is non-linear. Often this is combined with **Kernels** – amounts to changing the inner product

A harder case:



➤ Use kernels to transform

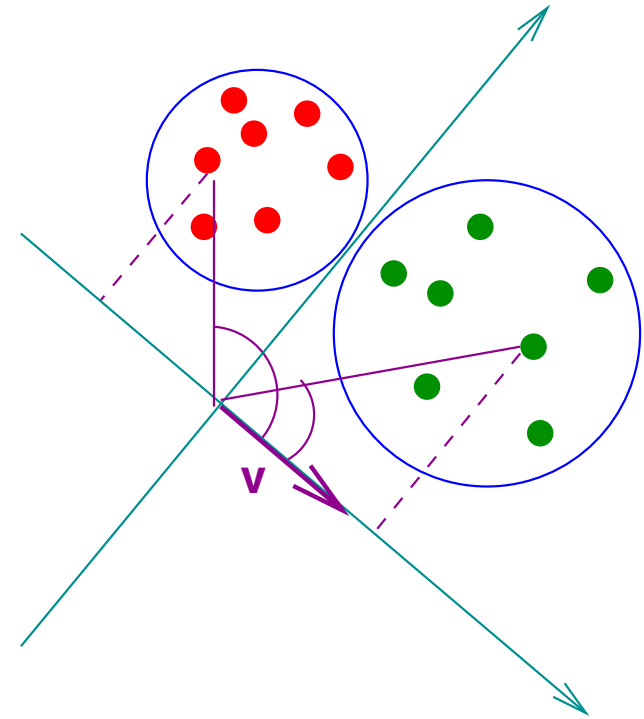


Transformed data with a Gaussian Kernel

Simple linear classifiers

- Let $X = [x_1, \dots, x_n]$ be the data matrix.
- and $L = [l_1, \dots, l_n]$ == labels. $l_i = \pm 1$
- 1st Solution: Find a vector u such that $u^T x_i$ close to $l_i, \forall i$
- Common solution: SVD to reduce dimension of data [e.g. 2-D] then do comparison in this space. e.g.

$$A: u^T x_i \geq 0, B: u^T x_i < 0$$



[For clarity: principal axis u drawn below where it should be]

Fisher's Linear Discriminant Analysis (LDA)

Principle: Use label information to build a good projector, i.e., one that can ‘discriminate’ well between classes

- Define “**between scatter**”: a measure of how well separated two distinct classes are.
- Define “**within scatter**”: a measure of how well clustered items of the same class are.
- Objective: make “between scatter” measure large **and** “within scatter” small.

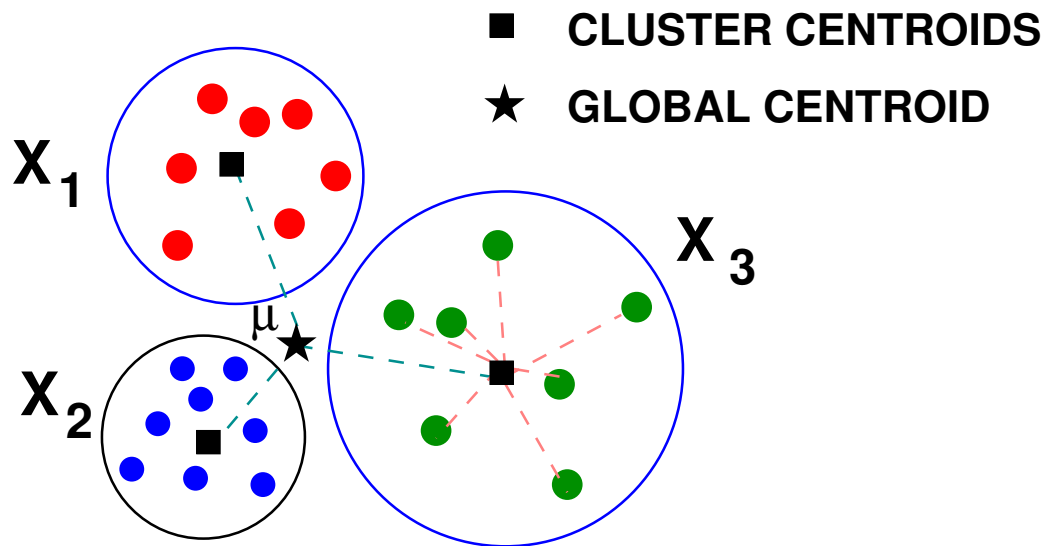
Idea: Find projector that maximizes the ratio of the “between scatter” measure over “within scatter” measure

$$S_B = \sum_{k=1}^c n_k (\mu^{(k)} - \mu)(\mu^{(k)} - \mu)^T,$$

$$S_W = \sum_{k=1}^c \sum_{x_i \in X_k} (x_i - \mu^{(k)})(x_i - \mu^{(k)})^T$$

where:

- $\mu = \text{mean}(X)$
- $\mu^{(k)} = \text{mean}(X_k)$
- $X_k = k\text{-th class}$
- $n_k = |X_k|$



- Consider 2nd moments for a vector a :

$$a^T S_B a = \sum_{i=1}^c n_k |a^T (\mu^{(k)} - \mu)|^2,$$
$$a^T S_W a = \sum_{k=1}^c \sum_{x_i \in X_k} |a^T (x_i - \mu^{(k)})|^2$$

- $a^T S_B a \equiv$ weighted variance of projected μ_j 's
- $a^T S_W a \equiv$ w. sum of variances of projected classes X_j 's

- LDA projects the data so as to maximize the ratio of these two numbers:

$$\max_a \frac{a^T S_B a}{a^T S_W a}$$

- Optimal a = eigenvector associated with top eigenvalue of:

$$S_B u_i = \lambda_i S_W u_i .$$

LDA – Extension to arbitrary dimensions

- Criterion: maximize the ratio of two traces:

$$\frac{\text{Tr} [U^T S_B U]}{\text{Tr} [U^T S_W U]}$$

- Constraint: $U^T U = I$ (orthogonal projector).
- Reduced dimension data: $Y = U^T X$.

Common viewpoint: hard to maximize, therefore ...

- ... alternative: Solve instead the ('easier') problem:

$$\max_{U^T S_W U = I} \text{Tr} [U^T S_B U]$$

- Solution: largest eigenvectors of $S_B u_i = \lambda_i S_W u_i$.

In Brief: Support Vector Machines (SVM)

➤ Similar in spirit to LDA. Formally, SVM finds a hyperplane that best separates two training sets belonging to two classes.

➤ If the hyperplane is:

$$w^T x + b = 0$$

➤ Then the classifier is $f(x) = \text{sign}(w^T x + b)$: assigns $y = +1$ to one class and $y = -1$ to other

➤ Normalize parameters w, b by looking for hyperplanes of the form $w^T x + b \geq 1$ to include one set and $w^T x + b \leq -1$ to include the other.

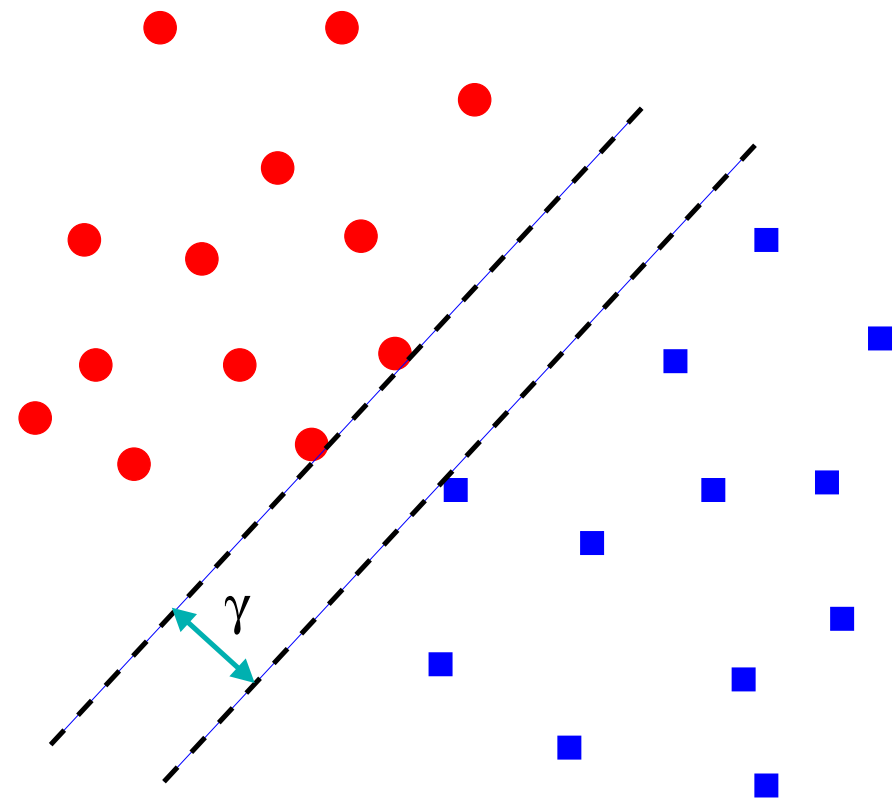
➤ With $y_i = +1$ for one class and $y_i = -1$ for the other, we can write the constraints as $y_i(w^T x_i + b) \geq 1$.

➤ The margin is the maximum distance between two such planes: goal find w, b to maximize margin.

➤ Maximize margin subject to the constraint $y_i(w^T x_i + b) \geq 1$.

➤ As it turns out the margin is equal to:

$$\gamma = \frac{2}{\|w\|_2}$$



 5 Prove it.

➤ Need to solve the constrained quadratic programming problem:

$$\begin{array}{ll} \min_{w,b} & \frac{1}{2} \|w\|_2^2 \\ \text{s.t.} & y_i(w^T x_i + b) \geq 1, \quad \forall x_i. \end{array}$$

Modification 1: Soft margin. Consider hinge loss: $\max\{0, 1 - y_i[w^T x_i + b]\}$

➤ Zero if constraint satisfied for pair x_i, y_i . Otherwise proportional to distance from corresponding hyperplane. Hence we can minimize

$$\lambda \|w\|^2 + \frac{1}{n} \sum_{i=1}^n \max\{0, 1 - y_i[w^T x_i + b]\}$$

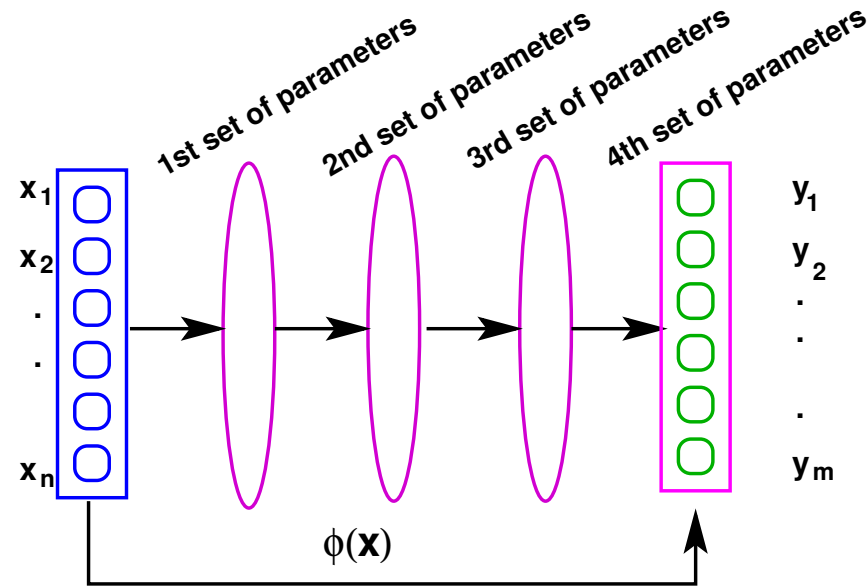
 6 Suppose $y_i = +1$ and let $d_i = 1 - y_i[w^T x_i + b]$. Show that the distance between x_i and hyperplane $w^T x_i + b = +1$ is $d_i / \|w\|$.

Modification 2: Use in combination with a Kernel to improve separability

A few words on Deep Neural Networks (DNNs)

- Ideas of neural networks goes back to the 1960s - were popularized in early 1990s – then laid dormant until recently.
- Two reasons for the come-back:
 - DNN are remarkably effective in some applications
 - big progress made in hardware [→ affordable ‘training cost’]

► Training a neural network can be viewed as a problem of approximating a function ϕ which is defined via sets of parameters:



Problem: find sets of parameters such that $\phi(x) \approx y$

Input: x , **Output:** y

Set: $z_0 = x$

For $l = 1 : L+1$ **Do:**

$$z_l = \sigma(W_l^T z_{l-1} + b_l)$$

End

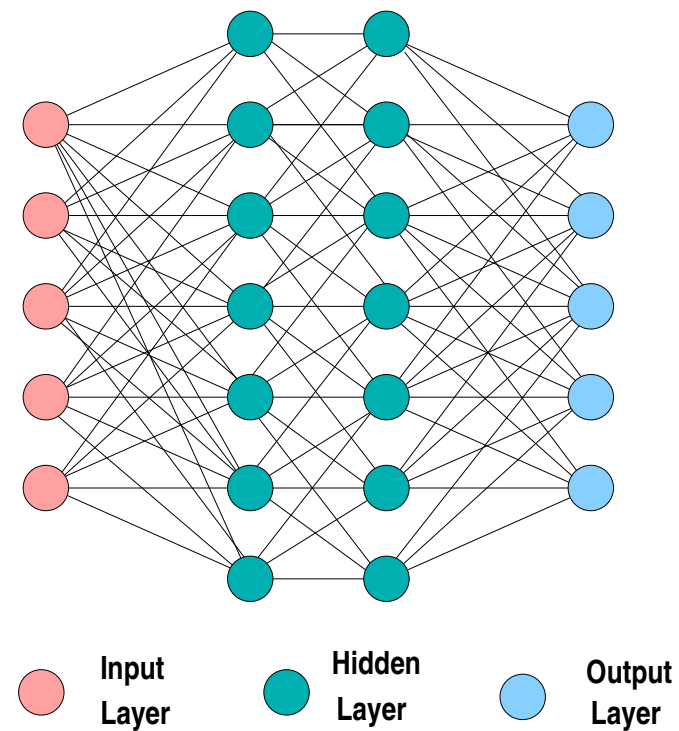
Set: $y = \phi(x) := z_{L+1}$

- layer # 0 = input layer
- layer # ($L + 1$) = output layer

➤ A matrix W_l is associated with layers 1,2, $L + 1$.

➤ Problem:

Find ϕ (i.e., matrices W_l) s.t. $\phi(x) \approx y$



DNN (continued)

- Problem is not convex, highly parameterized, ...,
- .. Main method used: Stochastic gradient descent [basic]
- It all looks like alchemy... but it works well for certain applications
- Training is still quite expensive – GPUs can help
- **Very** active area of research

DOMAIN DECOMPOSITION-TYPE METHODS

- *Back to scientific computing. Introduction – motivation*
- *Domain partitioning and distributed sparse matrices*
- *Basic algorithms: distributed Matvec*
- *Distributed preconditioners: additive Schwarz, multiplicative Schwarz.*
- *Schur complement techniques*

Introduction

- Back to scientific computing. So solve: PDE or $Ax = b$
- Thrust of parallel computing techniques in most applications areas.
- Programming model: Message-passing seems (MPI) dominates
- Open MP for small number of processors
- Also: GPUs (CUDA, ...) in most High-performance computers
- Parallel programming has penetrated most 'applications' areas [Sciences and Engineering, Data science, industry, ...]

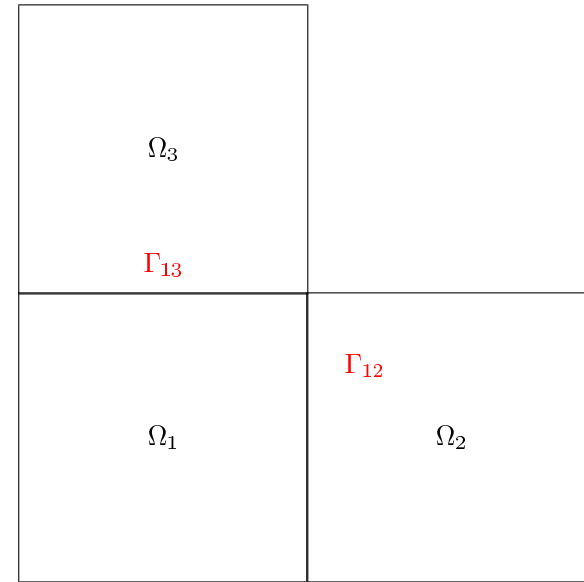
Domain Decomposition: A Model problem

Problem:

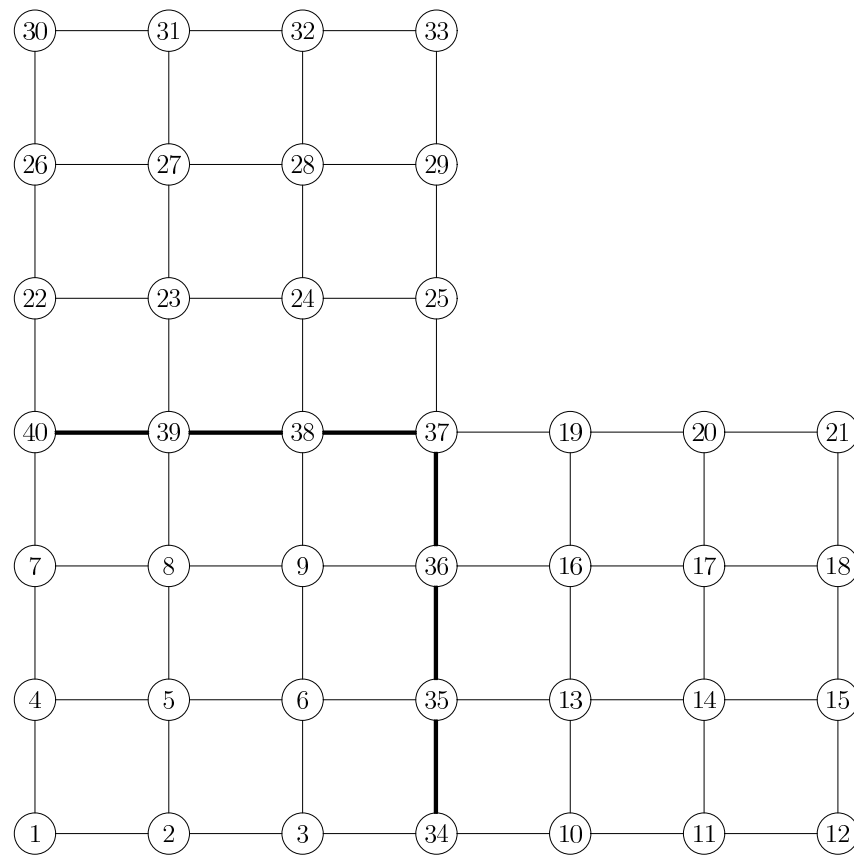
$$\begin{cases} \Delta u = f & \text{in } \Omega \\ u = u_\Gamma & \text{on } \Gamma = \partial\Omega. \end{cases}$$

Domain:

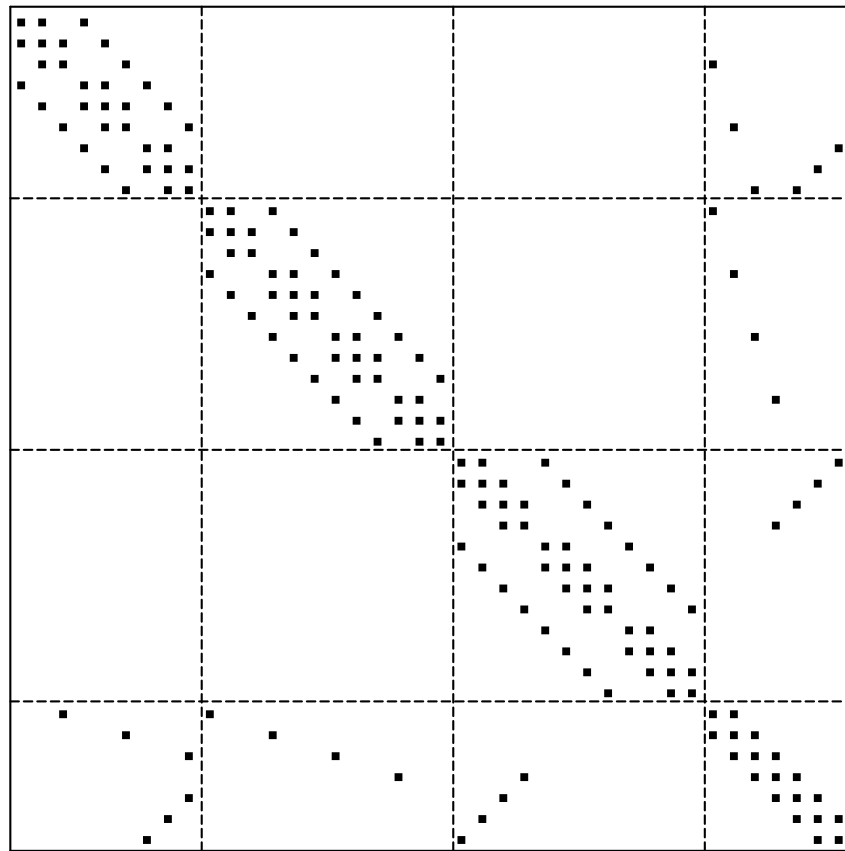
$$\Omega = \bigcup_{i=1}^s \Omega_i,$$



- Domain decomposition or substructuring methods attempt to solve a PDE problem (e.g.) on the entire domain from problem solutions on the subdomains Ω_i .

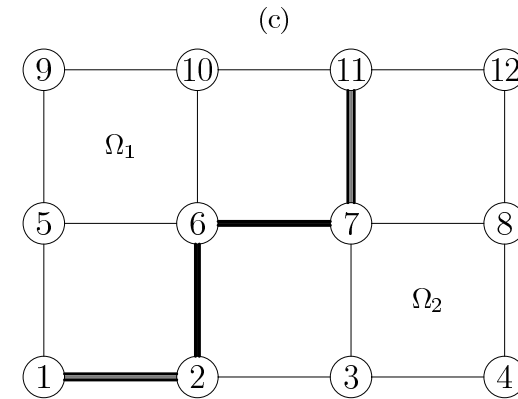
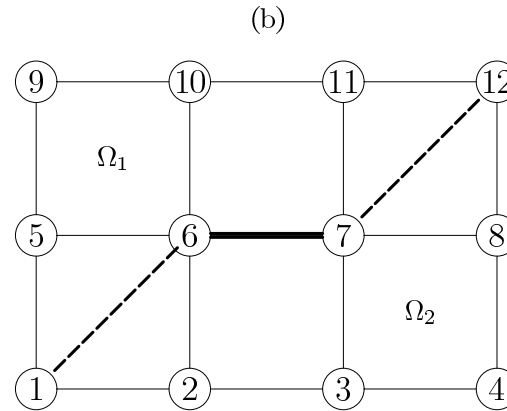
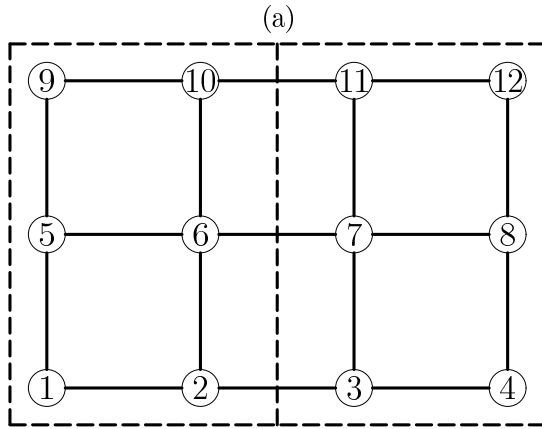


Discretization of domain



Coefficient Matrix

Types of mappings

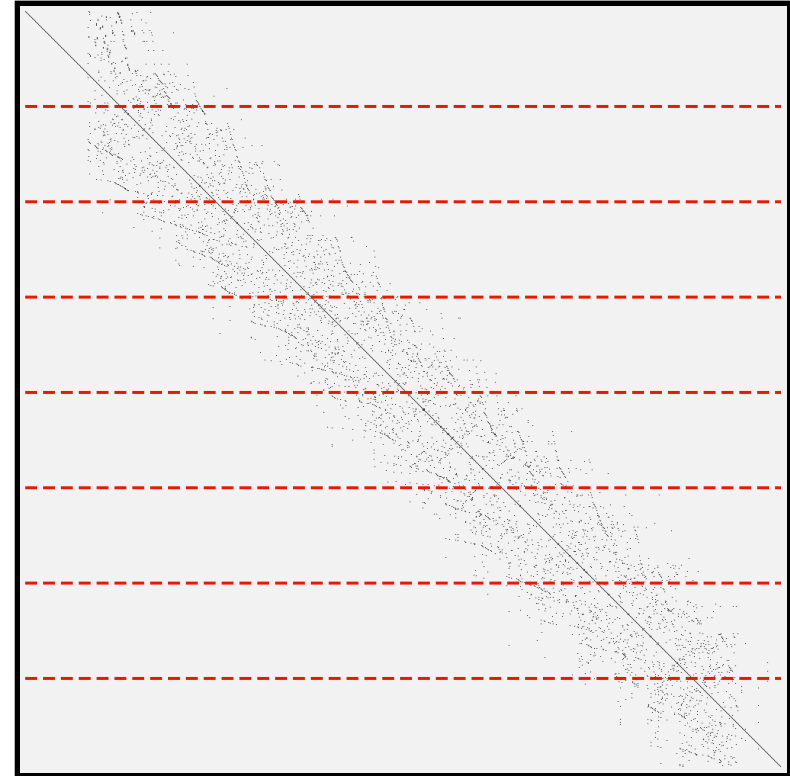


(a) Vertex-based; (b) edge-based; and (c) element-based partitioning

- Can adapt PDE viewpoint to general sparse matrices
- Will use the graph representation and 'vertex-based' viewpoint –

Generalization: Distributed Sparse Systems

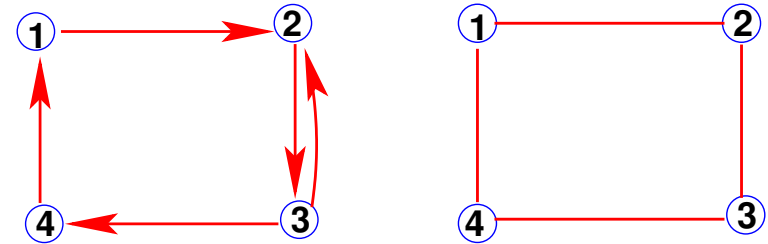
- Simple illustration: Block assignment. Assign equation i and unknown i to a given 'process'
- Naive partitioning - won't work well in practice



➤ Best idea is to use the adjacency graph of A :

Vertices = $\{1, 2, \dots, n\}$;

Edges: $i \rightarrow j$ iff $a_{ij} \neq 0$



Graph partitioning problem:

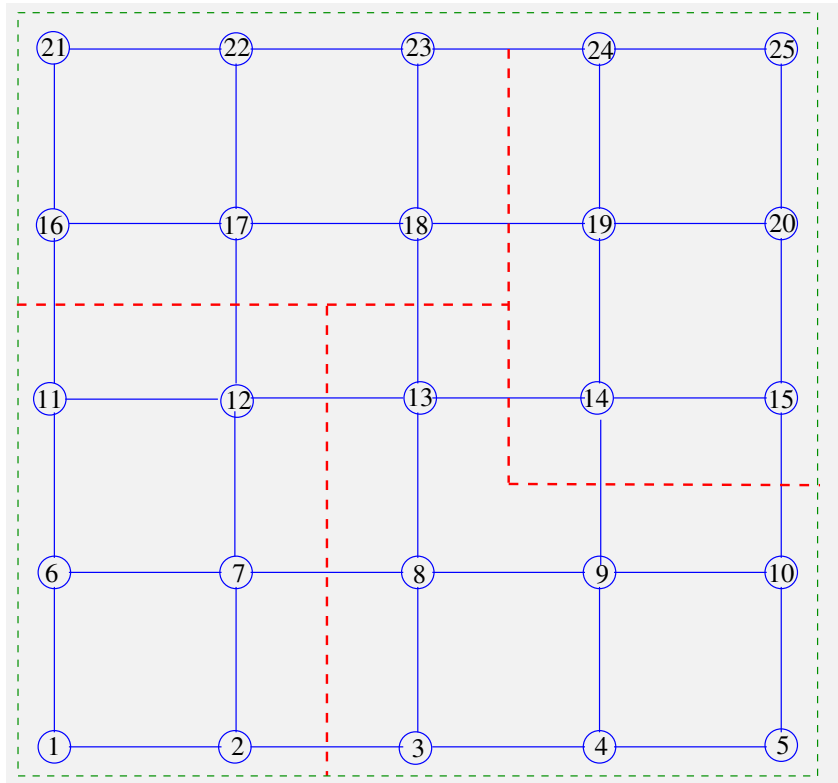
- Want a partition of the vertices of the graph so that

(1) partitions have $\sim$ the same sizes

(2) interfaces are small in size

➤ Standard dual objective: “minimize” communication + “balance” partition sizes

General Partitioning of a sparse linear system



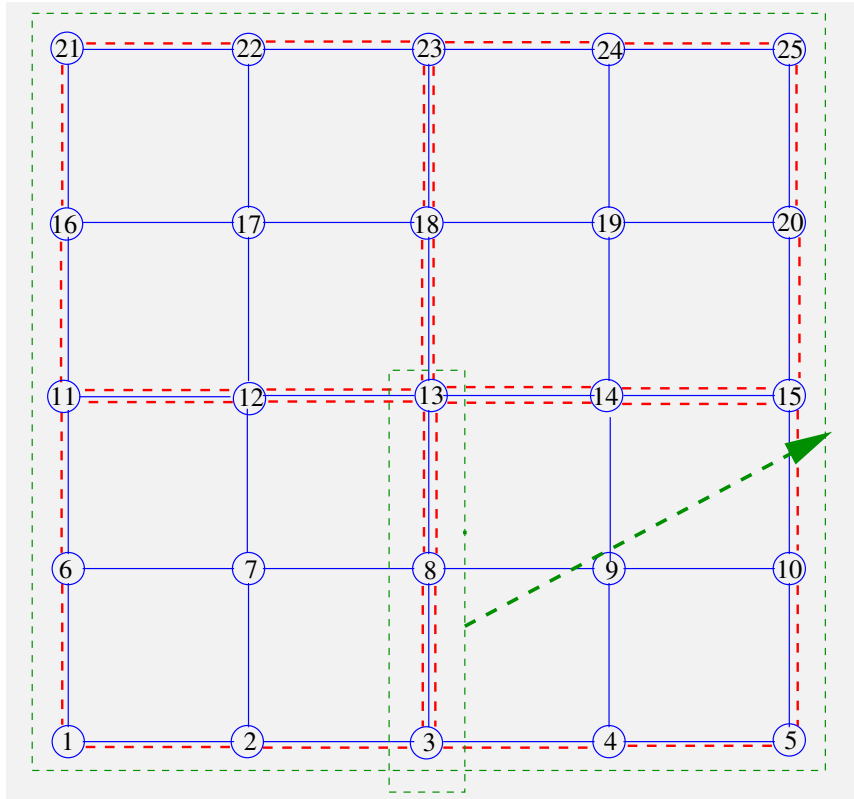
$S_1 = \{1, 2, 6, 7, 11, 12\}$: This means equations and unknowns 1, 2, 3, 6, 7, 11, 12 are assigned to Domain 1.

$S_2 = \{3, 4, 5, 8, 9, 10, 13\}$

$S_3 = \{16, 17, 18, 21, 22, 23\}$

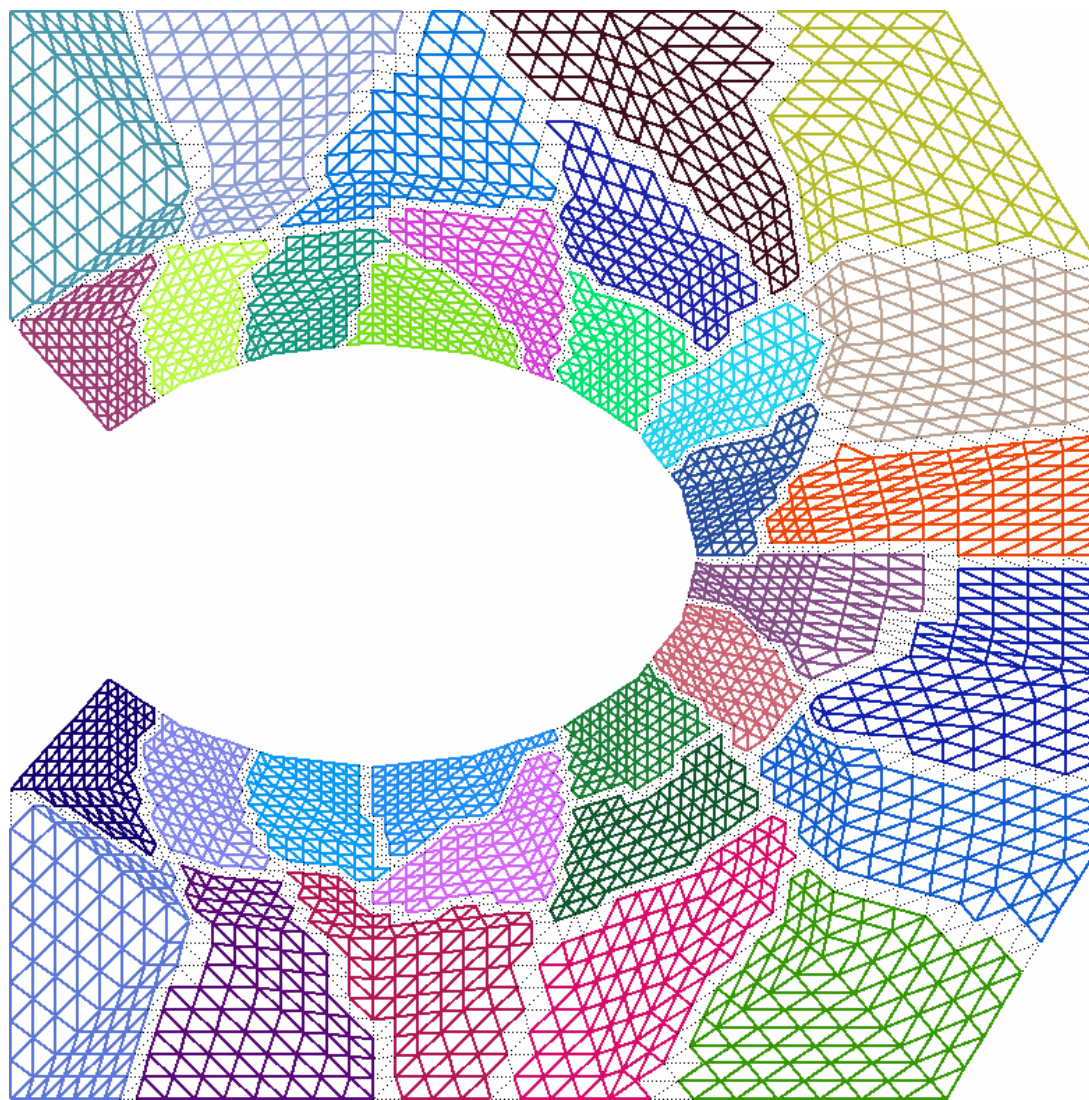
$S_4 = \{14, 15, 19, 20, 24, 25\}$

Alternative: Map elements / edges rather than vertices



Equations/unknowns 3, 8, 13 shared by 2 domains. From distributed sparse matrix viewpoint this is an overlap of one layer

► Partitioners : Metis, Chaco, Scotch, Zoltan, H-Metis, PaToH, ..



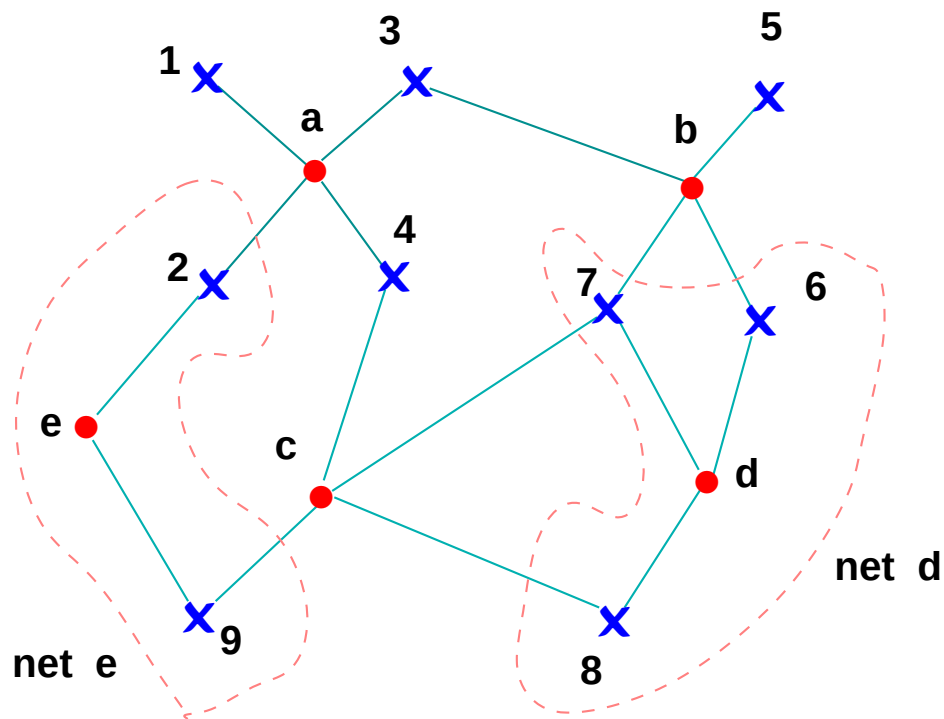
A few words about hypergraphs

- Hypergraphs are very general.. Ideas borrowed from VLSI work
- Main motivation: to better represent communication volumes when partitioning a graph. Standard models face many limitations
- Hypergraphs can better express complex graph partitioning problems and provide better solutions.
- Example: completely nonsymmetric patterns ...
- .. Even rectangular matrices

Example: $V = \{1, \dots, 9\}$ and $E = \{a, \dots, e\}$ with

$a = \{1, 2, 3, 4\}$, $b = \{3, 5, 6, 7\}$, $c = \{4, 7, 8, 9\}$,

$d = \{6, 7, 8\}$, and $e = \{2, 9\}$



Boolean matrix:

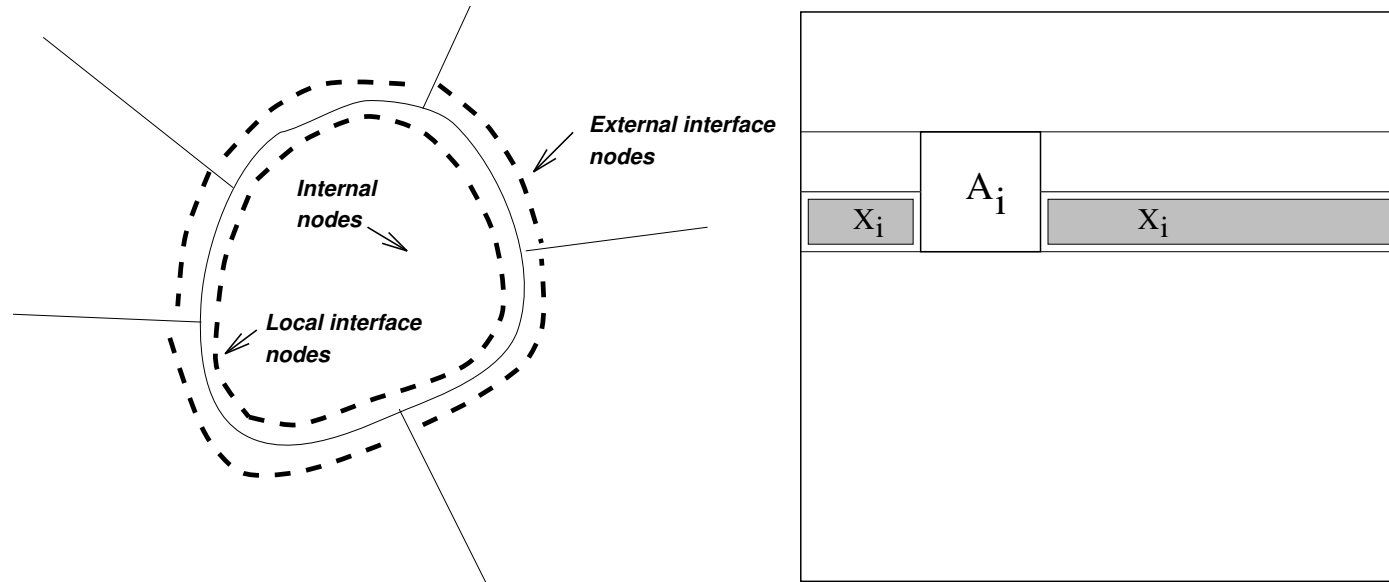
	1	2	3	4	5	6	7	8	9	
1	1	1	1	1						a
			1		1	1	1			b
				1			1	1	1	c
						1	1	1		d
	1								1	e

$A =$

Distributed Sparse matrices (continued)

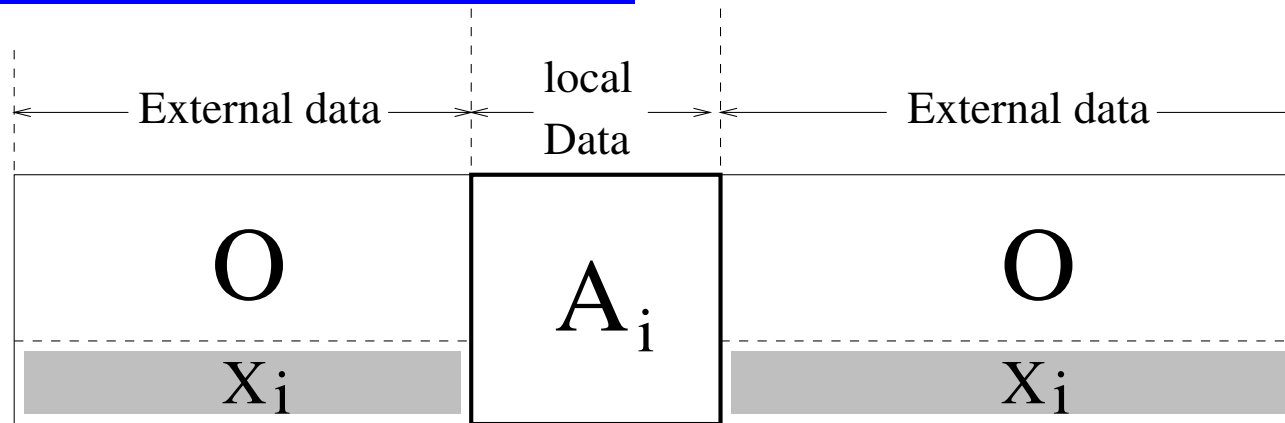
- Once a good partitioning is found, questions are:
1. How to represent this partitioning?
 2. What is a good data structure for representing distributed sparse matrices?
 3. How to set up the various “local objects” (matrices, vectors, ..)
 4. What can be done to prepare for communication that will be required during execution?

Two views of a distributed sparse matrix



- Local interface variables always ordered last.
- Need: 1) to set up the various “local objects”. 2) Preprocessing to prepare for communications needed during iteration?

Local view of distributed matrix:

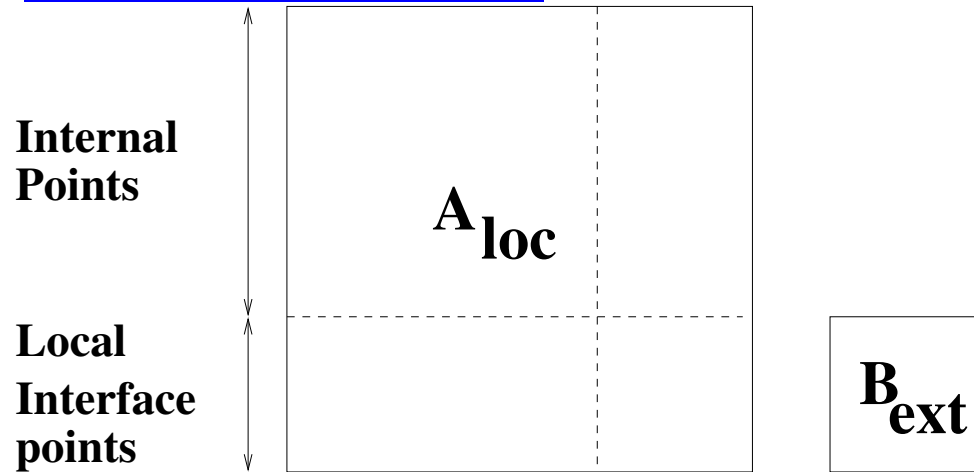


The local system:

$$\underbrace{\begin{pmatrix} B_i & F_i \\ E_i & C_i \end{pmatrix}}_{A_i} \begin{pmatrix} u_i \\ y_i \end{pmatrix} + \underbrace{\begin{pmatrix} 0 \\ \sum_{j \in N_i} E_{ij} y_j \end{pmatrix}}_{y_{ext}} = \begin{pmatrix} f_i \\ g_i \end{pmatrix}$$

► u_i : Internal variables; y_i : Interface variables

The local matrix:



The local matrix consists of 2 parts: a part (A_{loc}) which acts on local data and another (B_{ext}) which acts on remote data.

- Once the partitioning is available these parts must be identified and built locally..
- In finite elements, assembly is a local process.
- How to perform a matrix vector product? [needed by iterative schemes?]

Distributed Sparse Matrix-Vector Product Kernel

Algorithm:

1. Communicate: exchange boundary data.

Scatter x_{bound} to neighbors - Gather x_{ext} from neighbors

2. Local matrix – vector product

$$y = A_{loc}x_{loc}$$

3. External matrix – vector product

$$y = y + B_{ext}x_{ext}$$

NOTE: 1 and 2 are independent and can be overlapped.

Distributed Sparse Matrix-Vector Product

Main part of the code:

```

      call MSG_bdx_send(nloc,x,y,nproc,proc,ix,ipr,ptrn,ierr)
c
c do local matrix-vector product for local points
c
      call amux(nloc,x,y,aloc,jaloc,ialoc)
c
c receive the boundary information
c
      call MSG_bdx_receive(nloc,x,y,nproc,proc,ix,ipr,
        *      ptrn,ierr)
c
c do local matrix-vector product for external points
c
      nrow = nloc - nbnd + 1
      call amux1(nrow,x,y(nbnd),aloc,jaloc,ialoc(nloc+1))
c
      return
```

The local exchange information

- List of adjacent processors (or subdomains)
- For each of these processors, lists of boundary nodes to be sent / received to /from adj. PE's.
- The receiving processor must have a matrix ordered consistently with the order in which data is received.

Requirements

- The 'set-up' routines should handle overlapping
- Should use minimal storage (only arrays of size n_{loc} allowed).

Distributed Flexible GMRES (FGMRES)

1. **Start:** Choose x_0 and m . Let of the Krylov subspaces. Define $\bar{H}_m \in \mathbb{R}^{(m+1) \times m}$ with $\bar{H}_m \equiv 0$. and initialize all its entries $h_{i,j}$ to zero.

2. Arnoldi process:

(a) Compute $r_0 = b - Ax_0$, $\beta = \|r_0\|_2$ and $v_1 = r_0/\beta$.

(b) For $j = 1, \dots, m$ do

- Compute $z_j := M_j^{-1}v_j$; Compute $w := Az_j$;
- For $i = 1, \dots, j$, do 1. $h_{i,j} := (w, v_i)$ 2. $w := w - h_{i,j}v_i$
$$\begin{cases} h_{i,j} := (w, v_i) \\ w := w - h_{i,j}v_i \end{cases}$$
- Compute $h_{j+1,j} = \|w\|_2$ and $v_{j+1} = w/h_{j+1,j}$.

(c) Define $Z_m := [z_1, \dots, z_m]$

3. **Form the approximate solution:** Compute

$y_m = \operatorname{argmin}_y \|\beta e_1 - \bar{H}_m y\|_2$ and $x_m = x_0 + [z_1, z_2, \dots, z_m] y_m$ and $e_1 = [1, 0, \dots, 0]^T$. with $\bar{H}_m = \{h_{i,j}\}_{1 \leq i \leq j+1; 1 \leq j \leq m}$.

4. Restart: If satisfied stop, else set $x_0 \leftarrow x_m$ and goto 1.

Main Operations in (F) GMRES :

1. Saxpy's – local operation – no communication
2. Dot products – global operation
3. Matrix-vector products – local operation – local communication
4. Preconditioning operations – locality varies.

Distributed Dot Product

```
/*----- call blas1 function */  
    tloc = DDOT(n, x, incx, y, incy);  
/*----- call global reduction */  
    MPI_Allreduce(&tloc, &ro, 1, MPI_DOUBLE, MPI_SUM, comm);
```

A remark: the global viewpoint

$$\begin{pmatrix}
 B_1 & & & & \\
 & B_2 & & & \\
 & & \ddots & & \\
 & & & \ddots & \\
 & & & & B_p \\
 \hline
 E_1 & & & & \\
 & E_2 & & & \\
 & & \ddots & & \\
 & & & E_p & \\
 \hline
 & & & & F_1 & & & \\
 & & & & & F_2 & & \\
 & & & & & & \cdot & \\
 & & & & & & & \cdot \\
 & & & & & & & F_p
 \end{pmatrix}
 \begin{pmatrix}
 u_1 \\
 u_2 \\
 \vdots \\
 \vdots \\
 u_p \\
 y_1 \\
 y_2 \\
 \vdots \\
 y_p
 \end{pmatrix}
 =
 \begin{pmatrix}
 f_1 \\
 f_2 \\
 \vdots \\
 \vdots \\
 f_p \\
 g_1 \\
 g_2 \\
 \vdots \\
 g_p
 \end{pmatrix}$$

←

Interior
variables

→
←

Interface
variables

→

Example: Distributed ILU(0)

- Global view of matrix is (for 4 processors):
- A_i = local matrix restricted to internal nodes only

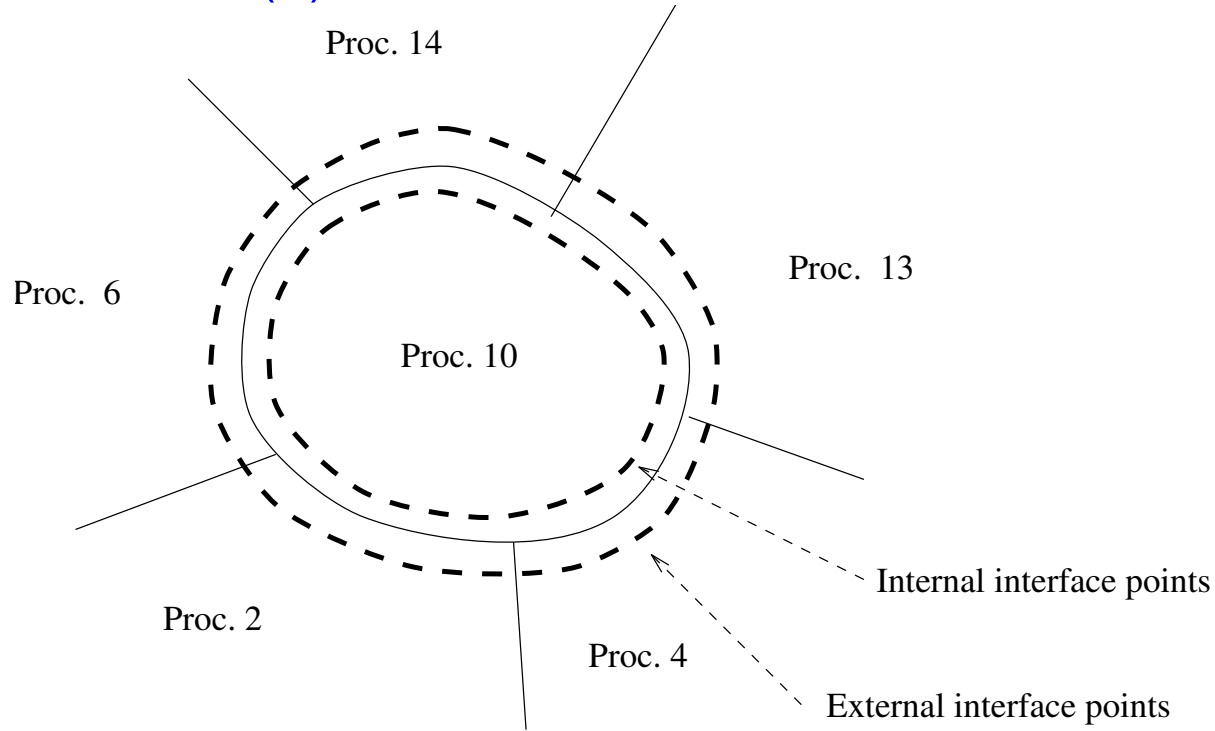
$$A = \left(\begin{array}{cccc|c} A_1 & & & & F_1 \\ & A_2 & & & F_2 \\ & & A_3 & & F_3 \\ & & & A_4 & F_4 \\ \hline E_1 & E_2 & E_3 & E_4 & D \end{array} \right)$$

- 1-st approach: Idea: ILU on this matrix – parallelism available for diagonal blocks. Define an order in which to eliminate interface unknowns.
- 2-nd approach: Multi-color, k -step SOR or SSOR preconditioners.
- 3-rd approach: Solve equations for all interface points [Schur Complement approach] – to precondition, use ideas from DD.

Example: Distributed ILU(0) – cont.

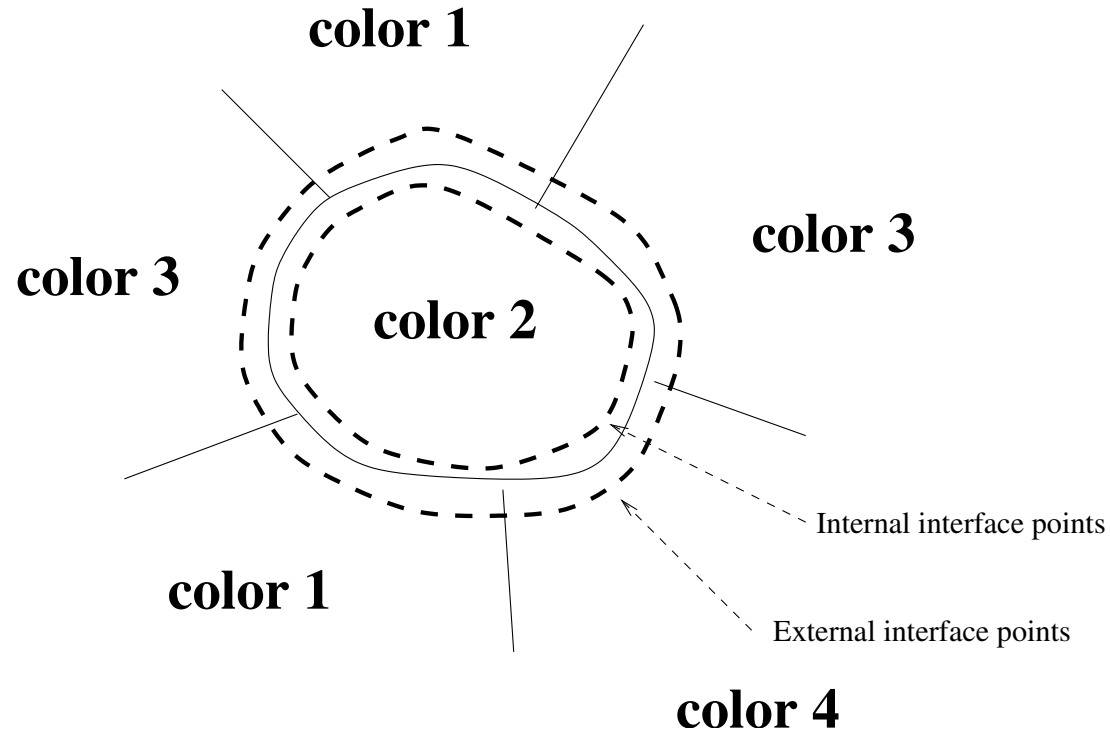
- Easy to understand from a local view of distributed matrix
- Start by selecting an order [or a “schedule”, or a “priority rule”] in which to process globally
- Then locally:
 1. Eliminate internal rows
 2. Receive rows needed to process local interface rows
 3. Process local interface rows
 4. Send local interface rows to processors needing them

A distributed view of ILU(0) – schedule based on PE numbers



Note: any schedule can be used provided neighbors have different labels.
Example: can use coloring.

A distributed view of ILU(0) – schedule based on PE coloring



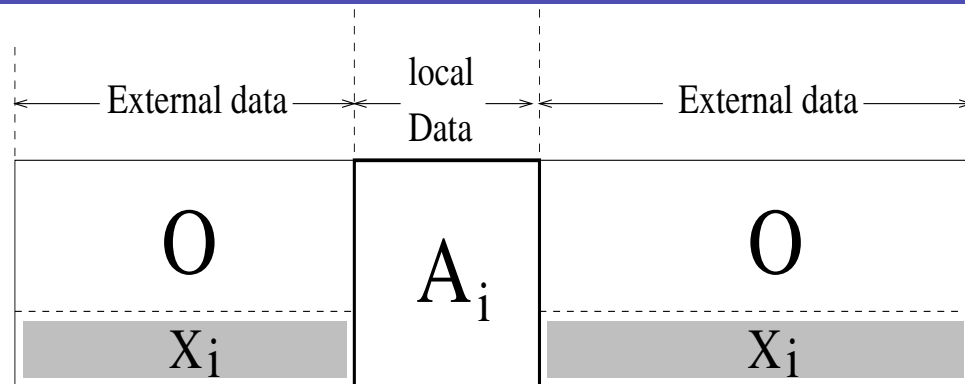
- Generalized ILU(k): D. Hysom and A. Pothen '00.

Domain Decomposition–Type preconditioners

- Schwarz Preconditioners
 - Schur-complement based Preconditioners
 - Multi-level ILU-type Preconditioners
- Observation: Often, in practical applications, Schwarz Preconditioners are used : SUB-OPTIMAL

Domain-Decomposition Preconditioners (cont.)

Local view of distributed matrix:



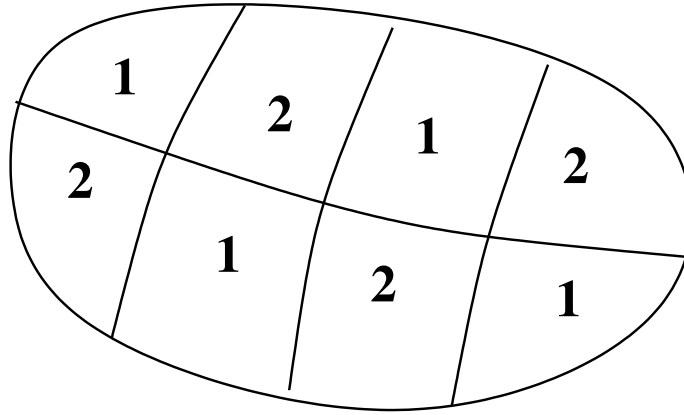
Block Jacobi Iteration (Additive Schwarz):

1. Obtain external data y_i
2. Compute (update) local residual

$$r_i = (b - Ax)_i = b_i - A_i x_i - B_i y_i$$

3. Solve $A_i \delta_i = r_i$
4. Update solution $x_i = x_i + \delta_i$

- Multiplicative Schwarz. Need a coloring of the subdomains so that:
- No two adjacent subdomains share same color



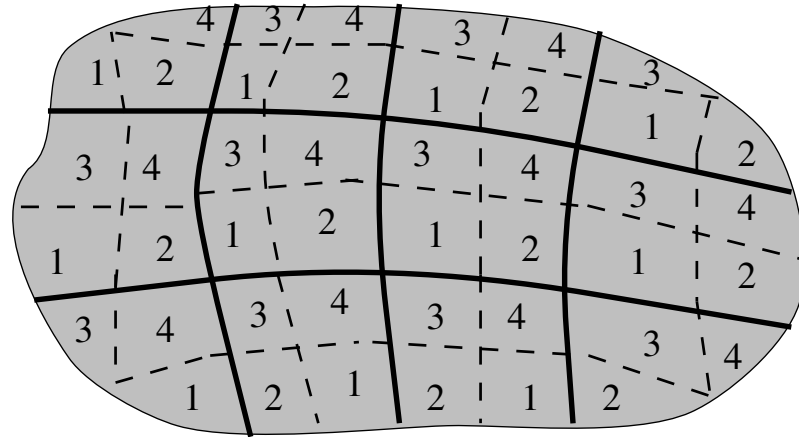
Multicolor Block SOR Iteration (Multiplicative Schwarz):

1. Do $col = 1, \dots, numcols$
2. If ($col.eq.mycol$) Then
3. Obtain external data y_i
4. Update local residual $r_i = (b - Ax)_i$
5. Solve $A_i \delta_i = r_i$
6. Update solution $x_i = x_i + \delta_i$
7. EndIf
8. EndDo

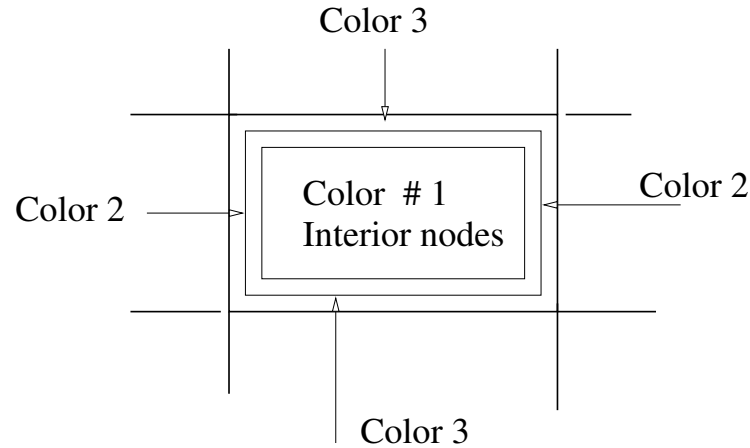
Breaking the sequential color loop

➤ “Color” loop is sequential. Can be broken in several different ways.

(1) Have a few subdomains per processors



(2) Separate interior nodes from interface nodes (2-level blocking)



(3) Use a block-GMRES algorithm - with Block-size = number of colors. SOR step targets a different color on each column of the block ➤ no idle time.

Local Solves

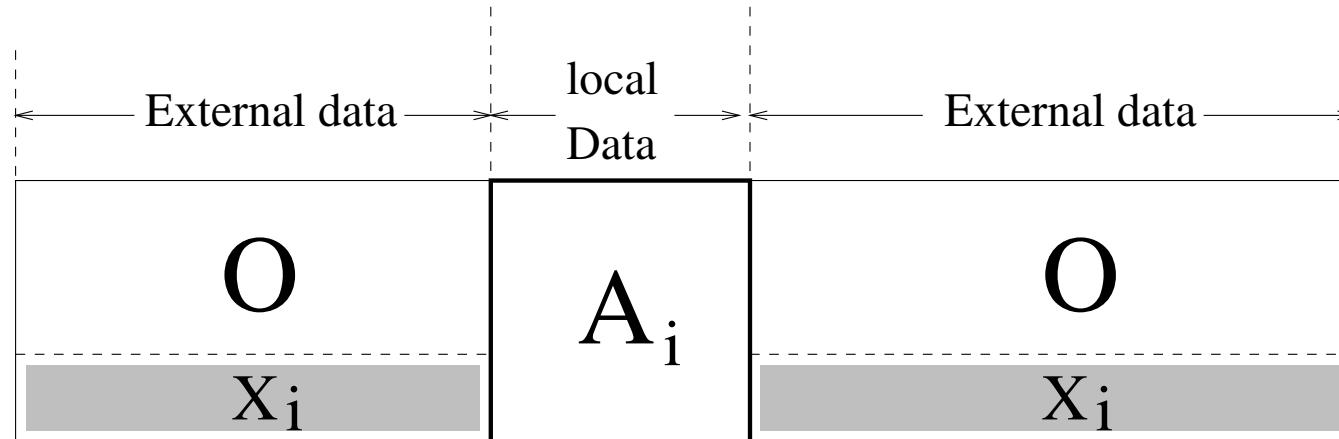
- Each local system $A_i \delta_i = r_i$ can be solved in three ways:
 1. By a (sparse) direct solver
 2. Using a standard preconditioned Krylov solver
 3. Doing a backward-forward solution associated with an accurate ILU (e.g. ILUT) preconditioner
- We only use (2) with a small number of inner steps (up to 10) or (3).

SCHUR COMPLEMENT-BASED PRECONDITIONERS

Schur complement system

Local system can be written as

$$A_i x_i + X_i y_{i,ext} = b_i. \quad (5)$$



x_i = vector of local unknowns, $y_{i,ext}$ = external interface variables, and b_i = local part of RHS.

➤ Local equations

$$\begin{pmatrix} B_i & F_i \\ E_i & C_i \end{pmatrix} \begin{pmatrix} u_i \\ y_i \end{pmatrix} + \begin{pmatrix} 0 \\ \sum_{j \in N_i} E_{ij} y_j \end{pmatrix} = \begin{pmatrix} f_i \\ g_i \end{pmatrix} \quad (6)$$

➤ eliminate u_i from the above system:

$$S_i y_i + \sum_{j \in N_i} E_{ij} y_j = g_i - E_i B_i^{-1} f_i \equiv g'_i,$$

where S_i is the “local” Schur complement

$$S_i = C_i - E_i B_i^{-1} F_i. \quad (7)$$

Structure of Schur complement system

Global Schur complement system:

$Sy = g'$ with :

$$S = \begin{pmatrix} S_1 & E_{12} & \dots & E_{1p} \\ E_{21} & S_2 & \dots & E_{2p} \\ \vdots & & \ddots & \vdots \\ E_{p1} & E_{p-1,2} & \dots & S_p \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{pmatrix} = \begin{pmatrix} g'_1 \\ g'_2 \\ \vdots \\ g'_p \end{pmatrix}.$$

- E_{ij} 's are sparse = same as in the original matrix
- Can solve global Schur complement system iteratively. Back-substitute to recover rest of variables (internal).
- Can use the procedure as a preconditioning to global system.

Simplest idea: Schur Complement Iterations

$$\begin{pmatrix} u_i \\ y_i \end{pmatrix} \begin{array}{l} \text{Internal variables} \\ \text{Interface variables} \end{array}$$

- Do a global primary iteration (e.g., block-Jacobi)
- Then accelerate only the y variables (with a Krylov method)

Still need to precondition..

Approximate Schur-LU

- Two-level method based on induced preconditioner. Global system can also be viewed as

$$\begin{pmatrix} B & F \\ E & C \end{pmatrix} \begin{pmatrix} u \\ y \end{pmatrix} = \begin{pmatrix} f \\ g \end{pmatrix}, \quad B = \left(\begin{array}{cccc|c} B_1 & & & & F_1 \\ & B_2 & & & F_2 \\ & & \ddots & & \vdots \\ & & & B_p & F_p \\ \hline E_1 & E_2 & \cdots & E_p & C \end{array} \right)$$

Block LU factorization of A :

$$\begin{pmatrix} B & F \\ E & C \end{pmatrix} = \begin{pmatrix} B & 0 \\ E & S \end{pmatrix} \begin{pmatrix} I & B^{-1}F \\ 0 & I \end{pmatrix},$$

Preconditioning:

$$L = \begin{pmatrix} B & 0 \\ E & M_S \end{pmatrix} \quad \text{and} \quad U = \begin{pmatrix} I & B^{-1}F \\ 0 & I \end{pmatrix}$$

with M_S = some approximation to S .

- Preconditioning to global system can be induced from any preconditioning on Schur complement.

Rewrite local Schur system as

$$y_i + S_i^{-1} \sum_{j \in N_i} E_{ij} y_j = S_i^{-1} [g_i - E_i B_i^{-1} f_i] .$$

- equivalent to Block-Jacobi preconditioner for Schur complement.
- Solve with, e.g., a few s (e.g., 5) of GMRES

➤ Question: How to solve with S_i ?

➤ Can use LU factorization of local matrix $A_i = \begin{pmatrix} B_i & F_i \\ E_i & C_i \end{pmatrix}$

and exploit the relation:

$$A_i = \begin{pmatrix} L_{B_i} & 0 \\ E_i U_{B_i}^{-1} & L_{S_i} \end{pmatrix} \begin{pmatrix} U_{B_i} & L_{B_i}^{-1} F_i \\ 0 & U_{S_i} \end{pmatrix} \rightarrow L_{S_i} U_{S_i} = S_i$$

➤ Need only the (L) LU factorization of the A_i [rest is already available]

➤ Very easy implementation of (parallel) Schur complement techniques for vertex-based partitioned systems : YS-Sosonkina '97; YS-Sosonkina-Zhang '99.

GRAPH COARSENING

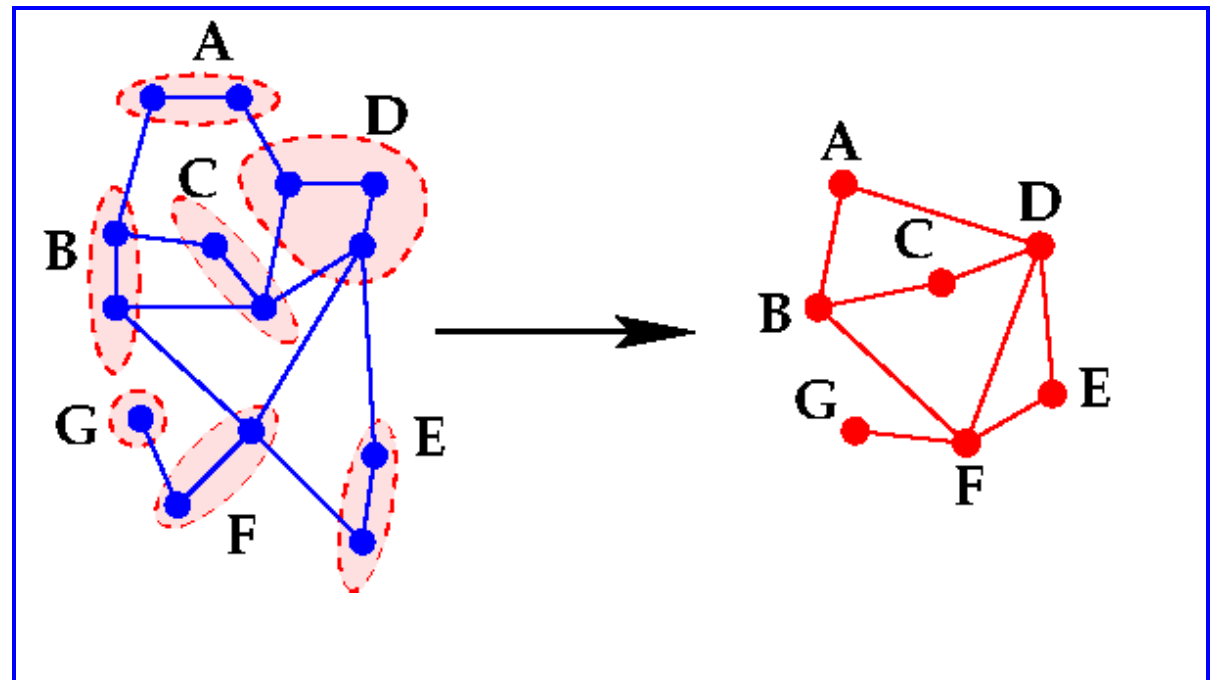
- *The general problem of coarsening*
- *Coarsening in scientific computing*
- *Multilevel methods for linear systems [in brief].*
- *Coarsening methods: aggregation, Heavy edge matching, independent sets*
- *Graph Laplaceans based coarsening: Kron*
- *Applications in machine learning*

Graph coarsening

Given a graph $G = (V, E)$, goal of graph coarsening is to find a smaller graph $G_c = (V_c, E_c)$ with n_c nodes and m_c edges, where $n_c < n$, which is a **faithful** approximation of G in some sense.

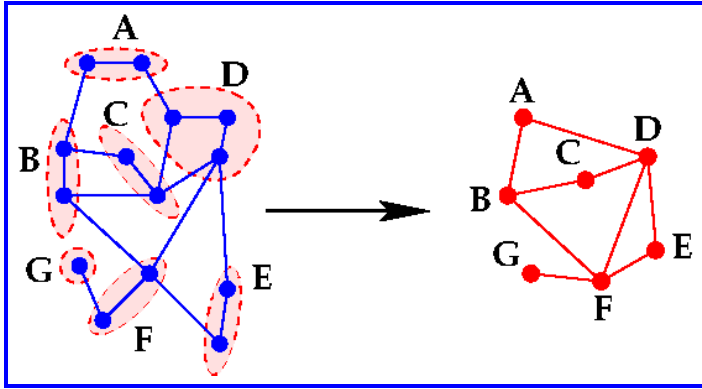
Notation:

- A_c = adjacency matrix of G_c ;
- L_c = graph Laplacian of G_c



Graph Coarsening in scientific computing

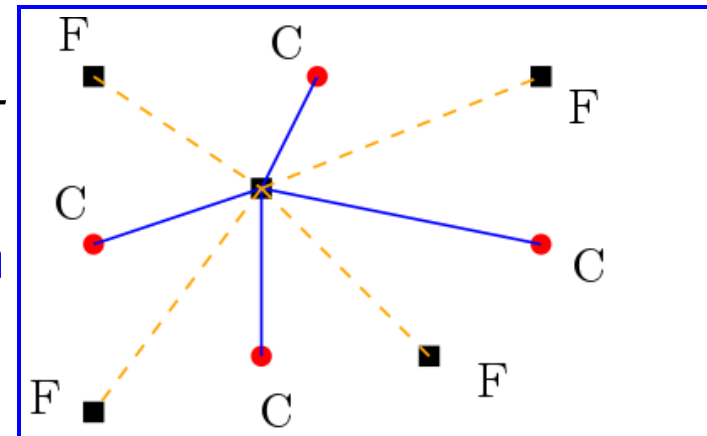
- Goal : exploit coarse representation of problem to solve linear systems



- Fewer nodes so: cheaper
- Can be used recursively

- Success story: *Multigrid, Algebraic Multigrid*

- AMG: Define coarse / fine nodes based on 'strength of coupling' →

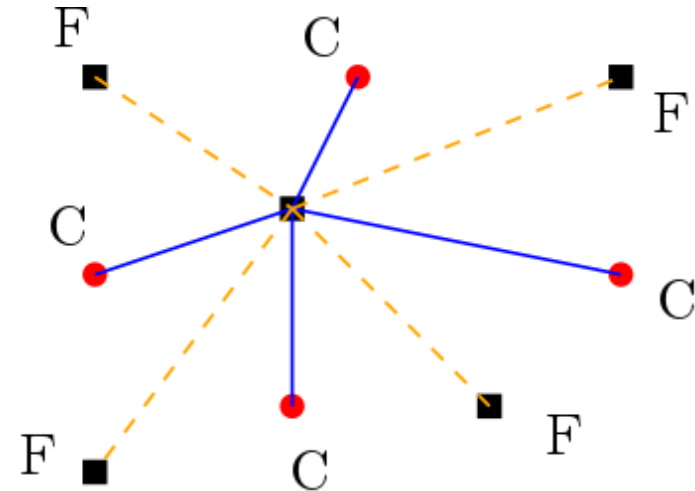


Graph coarsening in scientific computing: (A) MG

Algebraic multigrid Main idea: generalize the *interpolation* and *restriction* operations of standard MG.

- For each fine node select a set of nearest neighbors from the coarse set
- Then express a fine node i as a linear combination of a selected number of nearest neighbors that form a set C_i :

Fine nodes: ■. Coarse: ● In coarsening: central fine node is expressed as a combination of its coarse neighbors.



- Classical Ruge-Stüben strategy: selection based on ‘strong connections’ of node (i and j are strongly connected if a_{ij} has a large magnitude relative to others)
- Let C == set of coarse nodes; F == set of fine nodes
- Can define ‘interpolation operator’ P :

$$[Px]_i = \begin{cases} x_i & \text{if } i \in C, \\ \sum_{j \in C_i} p_{ij} x_j & \text{otherwise.} \end{cases}$$

- Expand into a multilevel framework by repeating the process on the graph of coarse set.

- Let $G_0 \equiv G$ (orig.) and $G_1, G_2, \dots, G_h$ be sequence of coarse graphs: $G_\ell = (V_\ell, E_\ell)$ is obtained by coarsening on $G_{\ell-1}$ for $1 \leq \ell < L$.
- Let $A^{(0)} \equiv A$ and $A^{(\ell)} \equiv$ matrix associated of ℓ -th level.
- Linear system at the ℓ -th level, can be reordered as:

$$A^{(\ell)} = \begin{bmatrix} A_{CC}^{(\ell)} & A_{CF}^{(\ell)} \\ A_{FC}^{(\ell)} & A_{FF}^{(\ell)} \end{bmatrix}, \quad f^{(\ell)} = \begin{bmatrix} f_C^{(\ell)} \\ f_F^{(\ell)} \end{bmatrix}.$$

- AMG: exploit different levels to building approximate solution. An AMG scheme depends entirely on defining a sequence of interpolation operators P_ℓ for $\ell = 0, 1, \dots$
- Once the P_ℓ 's are defined, one can design various 'cycles' : processes of going back and forth between levels

Multilevel ILU preconditioner based on coarsening

- Method: find a good ordering for ILU preconditioner
- Example: technique presented in [D. Osei-Kuffuor et al, '06]:
- Ingredient: ordering based on coarsening + apply recursively
- Matrix is ordered in block form - then $A_{22}^{(0)}$ is in turn reordered:

$$\begin{bmatrix} A_{11}^{(0)} & A_{12}^{(0)} \\ A_{21}^{(0)} & A_{22}^{(0)} \end{bmatrix} \rightarrow \left[\begin{array}{c|cc} A_{11}^{(0)} & & A_{12}^{(0)} \\ \hline & A_{11}^{(1)} & A_{12}^{(1)} \\ A_{21}^{(0)} & A_{21}^{(1)} & A_{22}^{(1)} \end{array} \right].$$

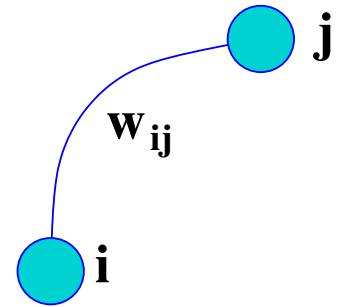
- Repeat with $A_{22}^{(1)}$ and further down for a few levels.
- Do ILU factorization of the resulting reordered system.

Example: Multilevel ILU [D. Osei-Kuffuor, R. Li, YS, '15]

Goal: Form of ILU preconditioning with improved robustness

➤ Traverse edges $(i, j) \in Nz(A)$ in decreasing order of the weights:

$$w_{ij} = \min \left\{ \frac{|a_{ij}|}{\delta_r(i)}, \frac{|a_{ij}|}{\delta_c(j)} \right\} \text{ where:}$$
$$\delta_r(i) = \frac{\|A_{i,:}\|_1}{nz(A_{i,:})} \quad \text{and} \quad \delta_c(j) = \frac{\|A_{:,j}\|_1}{nz(A_{:,j})}$$



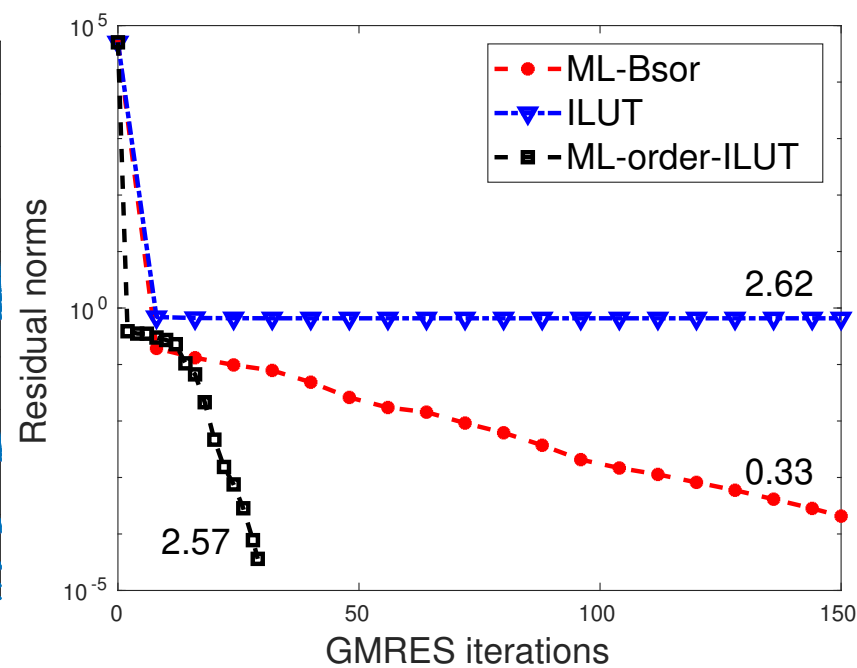
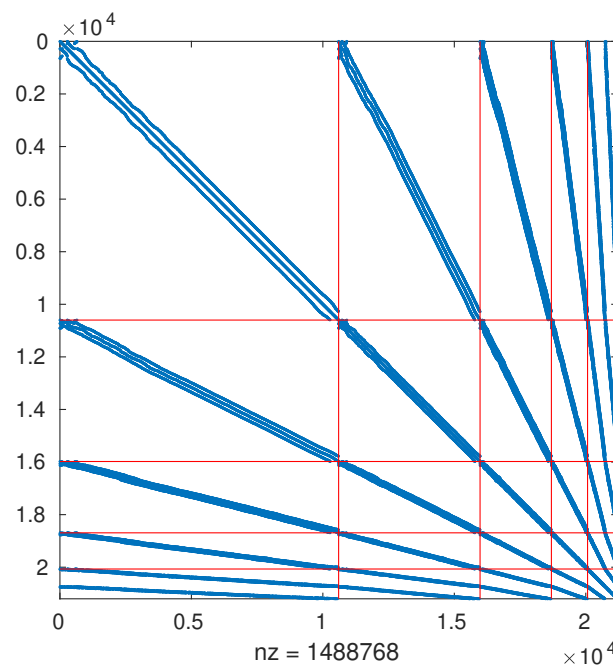
➤ Select i as 'coarse' if $\sigma_i > \sigma_j$ and j otherwise, where $\rightarrow$

$$\sigma_k = \frac{|a_{kk}|}{\delta_r(k)\delta_c(k)}$$

- Goal: to put large entries in the blocks $(A_{CF}^{(\ell)})$ and $(A_{FC}^{(\ell)})$

$$\begin{bmatrix} A_{CC}^{(\ell)} & A_{CF}^{(\ell)} \\ A_{FC}^{(\ell)} & A_{FF}^{(\ell)} \end{bmatrix}$$

- Model: very rough approximation of Gaussian Elimination.
- Next: (Matlab) Test with matrix *Raefsky3*¹
- 4 levels of coarsening. Then reorder matrix and:
- Solve with ILUT- GMRES(50) or BSOR - GMRES(50)



Left: The matrix Raefsky3 after the reordering obtained from four levels of coarsening. Right: Performance of various coarsening based preconditioners for solving a linear system with the matrix.

COARSENING APPROACHES

Coarsening by matching: Pairwise aggregation

- Strategy: coalesce (collapse) two adjacent nodes in a graph into a single node, based on some measure of nearness or similarity.
- A *matching* of a graph $G = (V, E)$ is a set of edges $\tilde{E}, \tilde{E} \subseteq E$, such that no two edges in $\tilde{E}$ have a node in common.
- Matching is **maximal** if it can't be augmented by additional edges
- Edge collapsing: usually selected using **maximal matching**
- Such **edge matching** techniques are common in AMG literature

- For each node i , build a set S_i of nodes that are ‘strongly connected’ to i
- Traverse graph nodes in a certain order of preference
- Next unmarked node in this order, say j , selected as a *coarse* node.
- Priority measure of traversal updated after each insertion of a coarse node

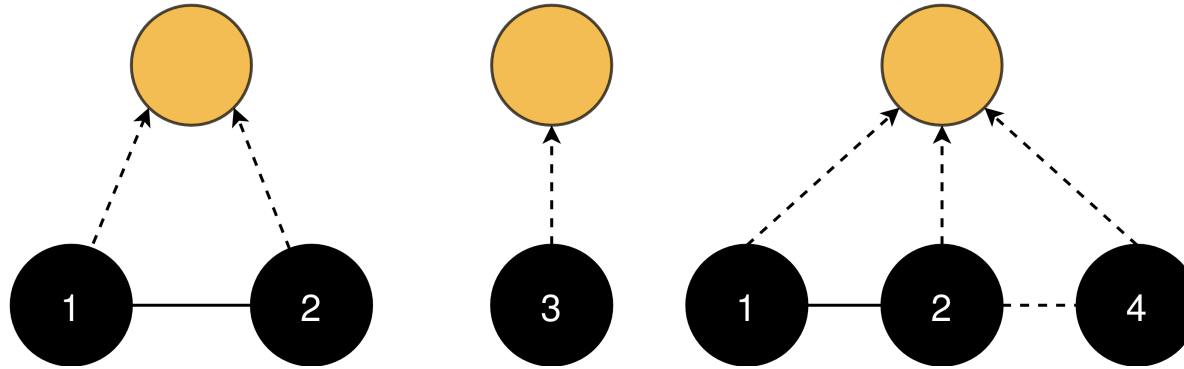
Heavy-edge matching (HEM) : matches a node i with its largest off-diagonal neighbor j_{max} ;

$$|a_{ij_{max}}| = \max_{j \in adj(i), j \neq i} |a_{ij}|$$

- There will be left-over nodes - called ‘singletons’

 7 Run *testCoars* in /coarsen

Heavy Edge Matching (HEM)



1. Visit edges (i, j) in decreasing value of their weight $w_{i,j}$
2. If both i and j have no parents yet (left), create a new coarse node ('new'). Set parents of i and j to be new .
3. At completion of traversal: deal with unassigned nodes: Either (middle) add as a coarse nodes if disconnected ("singleton") or (right) lump as a child to an existing coarse node

ALGORITHM : 21 ■ Heavy Edge Matching (HEM)

```
1: Input: Weighted graph  $G = (V, E, A)$ 
2: Output: Coarse nodes;  $Prnt$  list
3: Init:  $Prnt(i) = 0 \forall i \in V; new = 0$ 
4: for max to min edge  $(i, j)$  do
5:     if  $Prnt(i) == 0, Prnt(j) == 0$  then
6:          $new = new + 1$ 
7:          $Prnt(i) = Prnt(j) = new$ 
8:     end if
9: end for
10: for Node  $v$  with  $Prnt(v) == 0$  do
11:     if  $v$  has no neighbor then
12:          $new = new + 1; Prnt(v) = new$ 
13:     else
14:          $Prnt(v) = Prnt(j)$  where  $j = \operatorname{argmax}_i(a_{iv})$ 
15:     end if
16: end for
```

Coarsening by independent sets

Recall: Independent set: $\mathcal{S} \subseteq V$ is a set of vertices that are not adjacent to each other: $i, j \in \mathcal{S} \implies a_{ij} = 0$. It is **maximal** if it can't be augmented

➤ Can take $V_c = \mathcal{S}$ as a coarse set. Need to define edges.

➤ Let L = reordered graph Laplacian (n_c vertices of V_c listed first): (note: D_c is *diagonal*)

$$L = \begin{pmatrix} D_c & -F \\ -F^T & B \end{pmatrix}$$

➤ Replace B by $D_f = F^T \mathbb{1}$ and define $G_c = \text{graph of } S_c \rightarrow$

$$S_c = D_c - F D_f^{-1} F^T$$

Property: S_c = Graph Laplacian of coarse graph G_c

Coarsening by ‘algebraic distance’

- Motivated by “bootstrap algebraic multigrid” (BAMG) [Brandt’01]
- In BAMG notion of closeness (used for coarsening) defined from **a few steps of Gauss-Seidel for solving $Ax = 0$**
- Speed of convergence of the iterate determines an ‘algebraic distance’ between variables.
- Exploited to aggregate the unknowns and define restriction and interpolation operators. Analysis in [Chen-Safro’11]

Coarsening by 'kron' decomposition

- Kron reduction of networks proposed back in 1939 by Kron
- Revived by Dorfler and Bullo(2013) and Shuman et al. (2016)

Main idea:

- Select a coarse set V_1 : Shuman-al use eigenvectors
- Reorder matrix so that nodes of V_1 come 1st.

Laplacean becomes $\rightarrow$

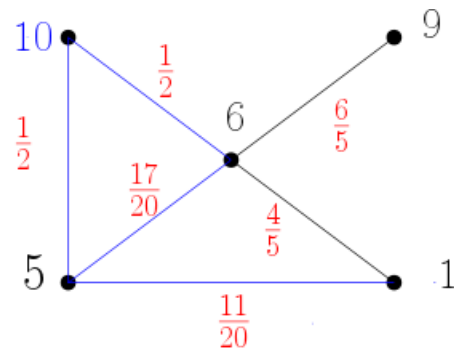
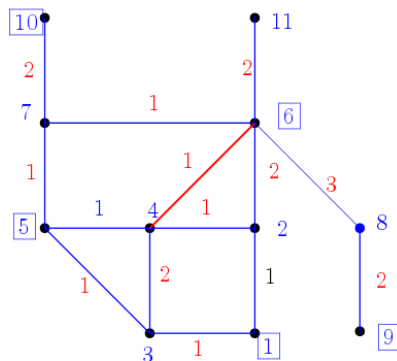
$$L = \begin{bmatrix} L_{11} & L_{12} \\ L_{12}^T & L_{22} \end{bmatrix}$$

- Kron reduction of L defined as the Schur complement:

$$L(V_1) = L_{11} - L_{12}L_{22}^{-1}L_{12}^T$$

Property $L(V_1) ==$ graph Laplacian of V_1 [Dorfler-Bullo]

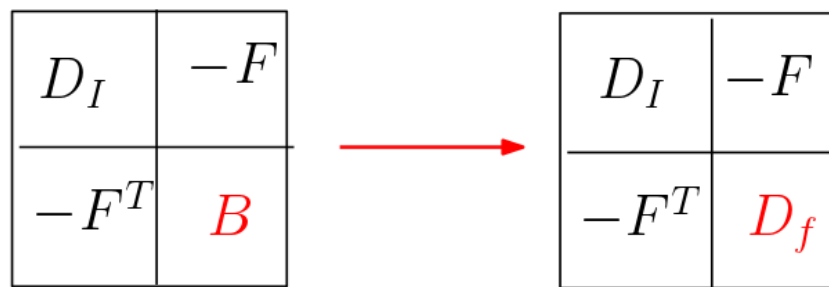
Example:



Kron coarsening

**Independent
set coarsening**

Two ways of using independent sets for coarsening.



$$L_c = D_I - FB^{-1}F^T$$

$$L_c = D_I - FD_f^{-1}F^T$$

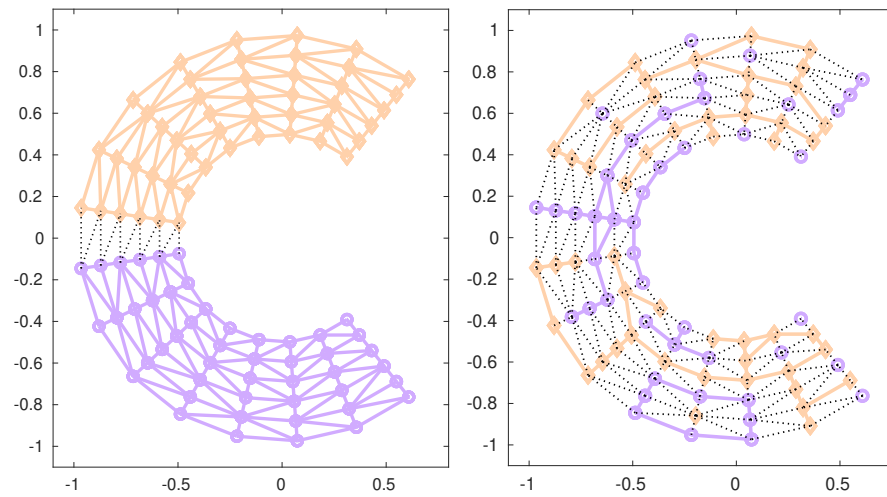
Q. 1: How to deal with 'denser' graph?

A Sparsify - using spectral sparsification

Q. 2: How to select V_1 ?

A Use signs of **largest** eigenvector of original Laplacian L

- If $u_1 = [\xi_1, \xi_2, \dots, \xi_n]^T$ = the largest eigenvector.
- Define $V_+ = \{i | \xi_i \geq 0\}$ and $V_- = \{i | \xi_i < 0\}$
- Then select one of V_+, V_- as V_1 .
- Opposite of what is done in **spectral graph partitioning**



Left side: spectral graph partitioning. Right: Coarsening with largest eigenvector

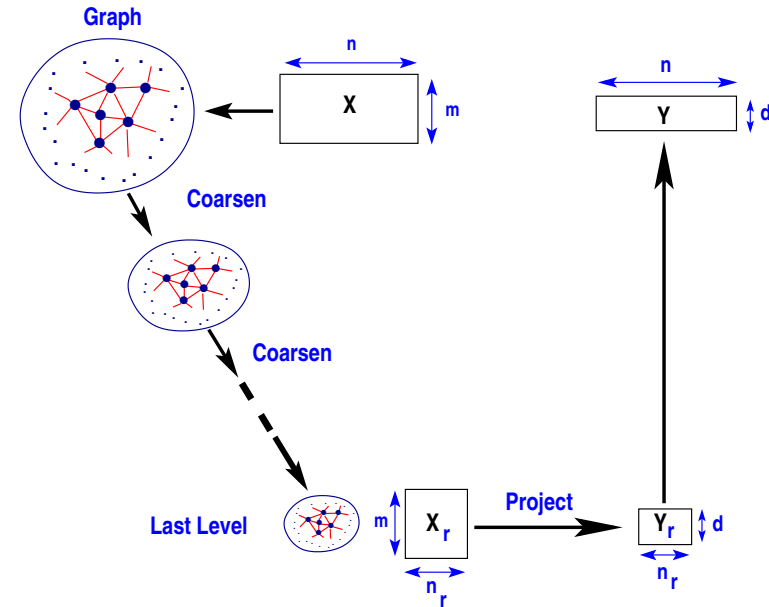
► Easy to show: (under mild condition on eigenvector) Each node of V_+ (resp. V_-) must have at least one nearest neighbor node from V_- (resp. V_+).

GRAPH COARSENING IN MACHINE LEARNING

Multilevel Dimension Reduction

Idea:

Coarsen for a few levels. Use resulting data set $\hat{X}$ to find a projector P from $\mathbb{R}^m$ to $\mathbb{R}^d$. Use this P to project data items.



- Gain: Dimension reduction is done with a much smaller set.
- Wish: not much loss compared to using whole data

Multilevel Dimension Reduction (for sparse data- e.g., text)

- Use **Hypergraph Coarsening** with *column matching* – similar to a common one used in graph partitioning
- Compute the non-zero inner product $\langle a^{(i)}, a^{(j)} \rangle$ between two vertices i and j , i.e., the i th and j th columns of A .
- Note: $\langle a^{(i)}, a^{(j)} \rangle = \|a^{(i)}\| \|a^{(j)}\| \cos \theta_{ij}$

Modif. 1: Parameter: $0 < \epsilon < 1$. Match columns i & j only if angle satisfies:

$$\tan \theta_{ij} \leq \epsilon$$

Modif. 2: Re-Scale. If i and j match and $\|a^{(i)}\|_0 \geq \|a^{(j)}\|_0$ replace $a^{(i)}$ and $a^{(j)}$ by

$$c^{(\ell)} = \left(1 + \cos^2 \theta_{ij}\right)^{\frac{1}{2}} a^{(i)}$$

- Call C the coarsened matrix obtained from A using the approach just described

Lemma: Let $C \in \mathbb{R}^{m \times c}$ be the coarsened matrix of A obtained by one level of coarsening of $A \in \mathbb{R}^{m \times n}$, with columns $a^{(i)}$ and $a^{(j)}$ matched if $\tan \theta_i \leq \epsilon$. Then

$$|x^T A A^T x - x^T C C^T x| \leq 3\epsilon \|A\|_F^2,$$

for any $x \in \mathbb{R}^m$ with $\|x\|_2 = 1$.

- Very simple bound for Rayleigh quotients for any x .
- Implies some bounds on singular values and norms - skipped.
- See details + experiments in [Ubaru-YS '19]

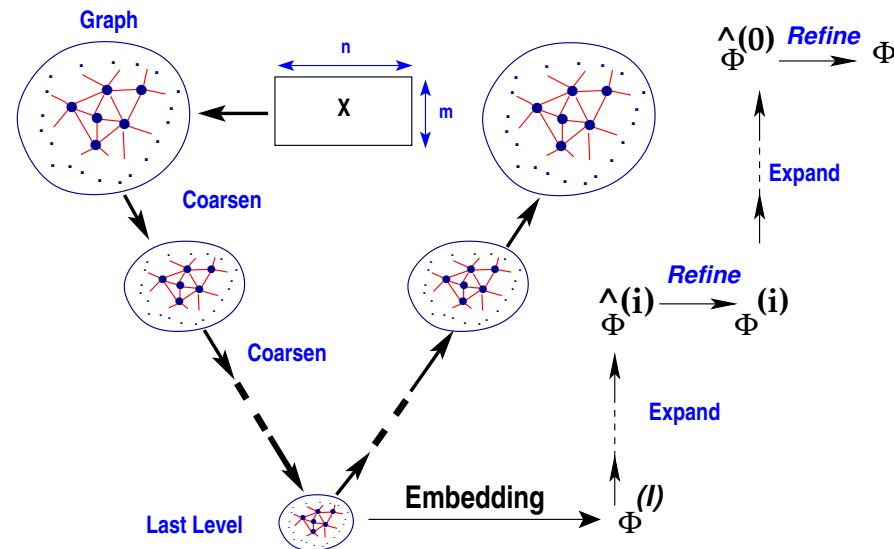
Graph coarsening for graph embeddings: HARP and MILE

- Recall Vertex embedding: Given $G = (V, E)$ find mapping Φ :

$$\Phi : v \in V \longrightarrow \Phi(v) \in \mathbb{R}^d$$

d is small: $d \ll n$

Hierarchical Representation Learning for Networks (HARP): (Chen et al. '18) coarsen for a few levels. Find embedding $\Phi^{(\ell)}$ for coarsest graph (level ℓ). Then a succession of expansions to higher level + refinement.



- Gain: Embedding done with a much smaller set.

- MILE approach [Liang et al. '18] very similar (difference in refinement).

Experiment to evaluate the effectiveness of HARP.

- Baseline. Three embedding algorithms: *DeepWalk* [Perozzi-al'14], *LINE* [Tang-al'15] and *Node2vec* [Grover-Leskovec'16]
- Combined with Coarsening methods:
 1. Heavy Edge Matching (HEM) - sketched earlier
 2. Algebraic distance (ALG) - sketched earlier
 3. Leverage Score Coarsening (LESC) – variant of HEM

Coarsening with eigenvectors

- It is possible to coarsen a graph with the goal of exactly preserving a few eigenvectors.
- This has turned out not to be too useful in practice.
- Instead we use eigenvectors to define ‘importance of nodes’ for the graph traversal

Leverage Scores

- $A = U\Sigma V^T$ ($\text{ran}(A) = \text{ran}(U)$)
- Leverage score of i -th row $\rightarrow$

$$\eta_i = \|U_{i,:}\|_2^2$$

- Used to measure importance of row i in random sampling methods [e.g. El-Aloui & Mahonney '15]

- Let A now be a graph Laplacian and $A = U\Lambda U^T$ with $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$

In *Leverage-score coarsening (LESC)* we dampen lower sing. vectors $\rightarrow$

$$\eta_i = \sum_{k=1}^r (e^{-\tau\lambda_k} U_{ik})^2$$

- Use η_i to decide order of traversal in coarsening algorithm

Note: Consider case when $r = n$ (or simply r is large)

$$\eta_i = \sum_{k=1}^n (e^{-\tau\lambda_k} U_{ik})^2 = \sum_{k=1}^n e^{-2\tau\lambda_k} |U_{ik}|^2 = e_i^T e^{-2\tau L} e_i.$$

➤ η_i equals the i -th diagonal entry of the matrix $H \equiv \exp(-2\tau L)$

- Next: visualization with 5 different coarsening methods on a graph with $n = 312$ nodes and $ne = 761$ edges